

Pro Data - Handbuch der Entwicklungsabteilung

Einleitung

Das folgende Scriptum ermöglicht einen Einblick in die Entwicklungsabteilung von Pro Data. Es werden verwendete Anwendungen aufgezeigt, Regeln erklärt und Prozesse erläutert. Dieses Handbuch soll sich ständig mitentwickeln und wird sich deshalb ebenfalls ändern.

Dokumentation

Allgemein

1. Gründe für die Dokumentation

- Neue Mitarbeiter können sich grundlegendes Know-How selbstständig aneignen.
- Neue Mitarbeiter können sich sich Anwendungen selbstständig konfigurieren.
- Wissen soll für alle zugänglich sein, d.h. wenn ein Mitarbeiter krankheitsbedingt ausfällt, sollten sich immer mehrere finden lassen, welche ihn ersetzen können.

2. Die Dokumentation muss leben

Vor allem für neue Mitglieder im Entwicklungsteam ist diese Dokumentation sehr wichtig um gut vorbereitet an die Arbeit gehen zu können. Deshalb ist es wichtig, sie ständig zu erweitern und zu verbessern.

- Wenn man beim Lesen inhaltliche oder sprachliche Fehler findet, behebt man diese und checkt die Änderungen ein.
- Wenn man etwas Neues von einem Kollegen lernt und weiß, dass es dazu noch keine Dokumentation gibt, notiert man sich am besten Stichpunkte und pflegt die Inhalte des Gelernten möglichst lückenfrei in unsere Dokumentation ein.
- Wenn man an einem neuen Prozess arbeitet, einen alten verändert oder merkt, dass die Dokumentation in einigen Punkten veraltet, kümmert man sich darum, dass die Dokumentation richtig bleibt.

3. Was gilt es zu beachten?

- Für das Verfassen unserer Doku verwenden wir die **IntelliJ-IDE** mit dem **AsciiDoc-Plugin** oder alternativ **AsciidocFX** oder **ATOM**.
- Das Repository zum Auschecken ist folgendes: [\[svn://svr-develop/staffman/ProData/trunk\]](svn://svr-develop/staffman/ProData/trunk)(<svn://svr-develop/staffman/ProData/trunk>)
- Neue Dokumente können einfach den Kopf von anderen kopieren und dann sollte das Schreiben von Dokumentationen nicht schwer fallen.
- Beim Erstellen von neuen Dateien sollten diese auch im Gesamthandbuch an einer passenden Stelle eingefügt werden. Dieser Prozess ermöglicht ein schnelles Drucken eines Handbuchs für neue Mitarbeiter oder auch Praktikanten!

- Weiters gilt: **Nur** nichts zu dokumentieren ist falsch.

4. Datei-Typ AsciiDoc

AsciiDoc ist ein Text-Dokument-Format für das Schreiben von Notizen, Dokumentation, Beiträgen, Bücher, E-Books, Slideshows, Homepages, Hilfe-Seiten und Blogs. AsciiDoc-Dateien können in viele Formate umgewandelt werden, inklusive HTML, PDF, EPUB, man page.

AsciiDoc Homepage:

```
[http://asciidoc.org/](http://asciidoc.org/)
```

AsciiDoc Syntax Quick Reference:

```
[http://powerman.name/doc/asciidoc](http://powerman.name/doc/asciidoc)
[http://asciidoctor.org/docs/asciidoc-syntax-quick-
reference/](http://asciidoctor.org/docs/asciidoc-syntax-quick-reference/)
```

AsciiDoc User Guide:

```
[http://asciidoc.org/userguide.html](http://asciidoc.org/userguide.html)
```

1. AsciiDocFX

Homepage: <http://asciidocfx.com/>

Einrichtung: `\\SVR-DC01\dfs\Software\AsciiDocFx\AsciiDocFX_Windows.zip` in einen Ordner entpacken

1. ATOM

Homepage: <https://atom.io/>

Unter "Settings / Install" können folgendes Package installiert werden:

- language-asciidoc (<https://atom.io/packages/language-asciidoc>)
- asciidoc-preview (<https://atom.io/packages/asciidoc-preview>)
- autocomplete-asciidoc (<https://atom.io/packages/autocomplete-asciidoc>)
- asciidoc-image-helper (<https://atom.io/packages/asciidoc-image-helper>)

Handbuch bearbeiten

1. Für eine einfache Bearbeitung des Handbuchs empfiehlt sich die **IntelliJ-IDE** mit dem **AsciiDoc-Plugin** mit integriertem Preview des Dokumentes.
2. Alternativ kann **AsciiDoc FX** ([\[http://asciidocfx.com\]](http://asciidocfx.com)(<http://asciidocfx.com>)) installiert werden.
 - Dieses Programm ermöglicht ebenfalls eine sofortige Preview der getätigten Änderungen.
 - Die .adoc - Quell-Dateien dieses Handbuchs können damit geöffnet werden.
3. Der "Commit" der veränderten Daten sollte stets mit einer passenden kurzen Beschreibung der Änderung kommentiert werden.

PDF des Handbuchs generieren lassen mit Asciidoctor

Info

Asciidoctor ist ein schnelles Textverarbeitungs- und veröffentlichungs-Werkzeug, das AsciiDoc-Inhalte in HTML5, DocBook 5 (oder 4.5) und anderen Formate umwandelt. Asciidoctor ist in Ruby geschrieben, als ein RubyGem gepackt und auf RubyGems.org veröffentlicht. Das gem wird in sämtlichen Linux-Distributions mitgeliefert, inklusive Fedora, Debian und Ubuntu. Asciidoctor ist Open-Source, auf GitHub gehostet und unter der MIT-Lizenz veröffentlicht.

Asciidoctor Homepage:

```
[http://asciidoctor.org/](http://asciidoctor.org/)
```

AsciiDoc Syntax Quick Reference:

```
[http://asciidoctor.org/docs/asciidoc-syntax-quick-reference/](http://asciidoctor.org/docs/asciidoc-syntax-quick-reference/)
```

Asciidoctor User Manual:

```
[http://asciidoctor.org/docs/user-manual/](http://asciidoctor.org/docs/user-manual/)
```

Asciidoctor PDF Theming Guide:

```
[http://gist.asciidoctor.org/](http://gist.asciidoctor.org/?github-asciidoctor%2Fasciidoctor-pdf%2Fdocs%2Ftheming-guide.adoc)
```

Asciidoctor Diagram:

```
[http://asciidoctor.org/docs/asciidoctor-
```

Installation

Asciidoctor braucht Ruby.

Installiere Ruby

- Homepage: [<https://www.ruby-lang.org/>](https://www.ruby-lang.org/)
- Verwende RubyInstaller: [<https://rubyinstaller.org/>](https://rubyinstaller.org/) Unter den angebotenen Installern reicht es, den aktuellsten aus der Gruppe "without Devkit" zu nehmen. Aktiviere die Option "Add Ruby to your Path". Die Option "Run 'rdik install' to set up MSYS2 and development toolchain" kann verneint werden.

Installiere RubyGems über die Kommandozeile:

- Syntax:

```
gem -h                // help
gem install <gem>      // install stable version
gem install <gem> --pre // install pre-releases
gem update <gem>       // upgrade installation
gem cleanup <gem>      // remove old versions
gem list               // display local gems
```

- Installiere Asciidoctor ([<http://asciidoc.org/>](http://asciidoc.org/)):

```
gem install asciidoctor --pre
```

- Installiere Asciidoctor PDF ([<https://github.com/asciidoctor/asciidoctor-pdf>](https://github.com/asciidoctor/asciidoctor-pdf)):

```
gem install asciidoctor-pdf --pre
```

- Installiere Source-Highlighter (coderay, rouge, ...):

```
gem install coderay
```

- Installiere Asciidoctor-Diagram ([<https://github.com/asciidoctor/asciidoctor-diagram>](https://github.com/asciidoctor/asciidoctor-diagram)):

```
gem install asciidoctor-diagram
```

Dokumente generieren

Je nach Format (HTML, PDF usw.) werden u.U. unterschiedliche Optionen benötigt. Zudem können Optionen angegeben werden, die von einigen Editoren (AsciiDocFX) nicht unterstützt werden. Es stehen cmd-Dateien mit gebräuchlichen Optionen zur Verfügung (siehe `createhtml.cmd`, `createpdf.cmd`).

AsciiDoctor aus Eingabeaufforderung starten:

PDF:

```
asciidoctor-pdf -a tabsize=4 -a source-highlighter=coderay <file>.adoc
```

PDF (mit Diagrammen):

```
asciidoctor-pdf -r asciidoctor-diagram -a tabsize=4 -a source-highlighter=coderay  
<file>.adoc
```

HTML:

```
asciidoctor -a tabsize=4 -a source-highlighter=coderay <file>.adoc
```

HTML (mit Diagrammen):

```
asciidoctor -r asciidoctor-diagram -a tabsize=4 -a source-highlighter=coderay  
<file>.adoc
```

Sichere Softwareentwicklung

Gesetze und Normen

Die Entwicklung von sicherer Software wird durch verschiedene Gesetze und Normen geregelt.

- Gesetzesdekret vom 30. Juni 2003, Nr. 196 - Maßnahmen des Sicherheitsgaranten und neue europäische Verordnung 679/2016 zum Schutz personenbezogener Daten.
- Gesetzesdekret vom 9. Jänner 2004, Nr. 4 - Förderung von barrierefreier, einfach verständlicher und einfach benutzbarer Software
- Gesetzesdekret vom 9. April 2008, Nr. 81 - Einheitstext über die Gesundheit und Sicherheit am Arbeitsplatz
- ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection — Information security management systems — Requirements.
- ISO/IEC 27017:2015 Information technology — Security techniques — Code of practice for

information security controls based on ISO/IEC 27002 for cloud services

- OWASP (Open Web Application Security Project) Guidelines.

Sicherheit in der Entwicklung

Der Quellcode der Softwareprodukte stellt einen hohen Wert für die Firma dar. Deshalb muss der Zugriff darauf beschränkt werden. Zugriff haben in erster Linie die Entwickler, die an der Weiterentwicklung der Softwareprodukte arbeiten, und deshalb ändernden Zugriff auf den Quellcode brauchen. Zugriff in Teilen zum Beispiel als "Nur Lesen" Zugriff oder Zugriff auf die Änderungslogs kann auch dem Personal gewährt werden, das für die Dokumentation wie Handbuch oder Versionshistory zuständig ist. Der Quellcode muss mit einem Versionsmanagement-System verwaltet werden, damit die durchgeführten Änderungen präzise nachvollzogen werden können. Die verwendeten Bibliotheken und Laufzeitumgebungen sind periodisch daraufhin zu prüfen ob sie dem aktuellen Sicherheitslevel entsprechen. Der Quellcode darf auf den eigenen Arbeitsplatzrechner geladen werden. Es ist nicht erlaubt Quellcode auf andere Geräte wie mobile Datenträger, Drucker, Computer zu übertragen, oder ihn Dritten zugänglich zu machen.

Sicherheit in der Auslieferungsphase

Damit die Kunden ein möglichst stabiles Produkt erhalten, wird wie folgt vorgegangen. Bereits während der Entwicklung werden Alpha-Versionen erzeugt, die auf Korrektheit und leichte Handhabbarkeit der neuen Features untersucht werden können. Am Ende des Entwicklungszyklus wird ein Releasekandidat erzeugt, der an Tester oder auch an ausgesuchte Kunden für den Echteininsatz übergeben wird. Die finale Version wird zunächst nur an einen beschränkten Kundenkreis ausgegeben, und danach in mehreren Wellen auf einen größeren Kundenkreis verteilt.

Sicherheit im Einsatz

Die erstellte Software muss garantieren, dass User authentifiziert werden, und dass sie nur Operationen durchführen können, für die sie auch autorisiert sind. Insbesondere sind Zugänge über das Internet abzusichern, und sollten gemäß den OWASP Guidelines auf die bekannten Angriffsszenarien vorbereitet sein.

Entwicklungsdatenbank

Im Normalfall wird zum Entwickeln eine Datenbank verwendet, die nur Demo-Daten enthält. Dies ist notwendig, um die Einhaltung von Privacy-Gesetzen zu garantieren. Auf diese Weise können auch firmenexterne Experten für Beratung oder auch Entwicklungsmitarbeit problemlos herangezogen werden.

Umgang mit Echtdaten des Kunden

Manchmal ist es notwendig für das Beseitigen von speziellen Fehlern, oder der Entwicklung von speziellen kundenspezifischen Features Echtdaten eines Kunden zu verwenden. Dabei sollte die Arbeit auf einem Echtsystem des Kunden nur im absoluten Ausnahmefall durchgeführt werden. Im

Regelfall wird eine Kopie der Echtdaten bei den dafür zuständigen Entwicklern installiert. Diese darf nur solange auf den Entwicklersystemen verbleiben, bis die durchgeführten Änderungen oder Fehlerbeseitigungen vom Kunden abgenommen wurden. Achtung! Es müssen alle EMail-, FTP-Http/Rest-Zugangspunkte aus den Echtdaten gelöscht werden, oder auf interne Endpunkte umgesetzt werden. Eventuelle Zertifikate müssen entfernt oder durch eigene Zertifikate ersetzt werden. Alle Dienste im Programm, insbesondere alle die eine Kommunikation nach außen vornehmen, müssen gesperrt werden. Es darf unter keinen Umständen zu Programmoperationen kommen, bei denen Dritte davon ausgehen würden, dass sie vom Kunden durchgeführt wurden.

Bibliotheken

Die für die Ausführung notwendigen Libraries (Fremdsoftware) muss periodisch auf Softwareschwachstellen (Vulnerabilities) überprüft werden. Die wichtigste Bibliothek in diesem Zusammenhang ist für uns die Java Laufzeitumgebung (JRE). Diese wird im Regelfall 4 mal im Jahr aktualisiert. Die entsprechenden Termine werden durch Oracle unter <https://www.oracle.com/security-alerts/> vorab bekanntgegeben. Wir passen uns mit der Aktualisierung im wesentlichen diesem Rhythmus an, es sei denn, dass in der Zwischenzeit schwere Sicherheitslücken in unseren verwendeten Bibliotheken bekannt werden, oder auch dass wir dringend neue Features benötigen, die in den aktuellen Bibliotheken nicht abgedeckt sind. Neben Oracle unterhalten auch weitere Bibliotheks-Anbieter Webseiten zum Thema Sicherheit. Beispiele sind:

- <https://vaadin.com/security>
- <https://jetty.org/security.html>
- <https://security.apache.org/>

Um die Bibliotheken auf Vulnerabilities zu untersuchen kann man in der von uns verwendeten Entwicklungsumgebung IntelliJ die Haupt-Konfigurationsdatei pom.xml öffnen. Dort werden Vulnerabilities in Form von Warnungen angezeigt. Eine zweite Möglichkeit ist die Log-Datei zu kontrollieren, die bei der Erstellung von neuen Programmversionen (Jars) generiert wird. Der Build-Prozess beinhaltet nämlich ein von OWASP zur Verfügung gestelltes Plugin <https://jeremylong.github.io/DependencyCheck/> . Die Interpretation dieser Angaben bedarf allerdings einer gewissen Erfahrung, da es durchaus zu falsch-positiv als auch falsch-negativ Meldungen kommen kann. Falls die Bibliotheken aktualisiert werden müssen, sind diese auch gründlich zu prüfen, dass es zu keinen Fehlern, sei es während der Kompilation, sei es während der Laufzeit kommt. Bibliotheken zu aktualisieren, kann auch notwendig werden, wenn sie neue Features mitbringen, die für die Programmentwicklung gebraucht werden. Über den aktuellen Versionsstand kann man sich über die im readme.txt angegebenen Webseiten informieren, oder in den meisten Fällen, wo die Bibliothek über Maven bezogen wird, auch über die Webseite <https://mvnrepository.com/> , oder noch besser über <https://repo.maven.apache.org/maven2/> . Diese ist zwar nicht mit einer komfortablen Bedienoberfläche versehen, dafür ist sie aber auf dem aktuellsten Stand. Durch transitorische Abhängigkeiten kann es passieren, dass eine Bibliothek nicht nur von unserem Programm aus direkt verwendet wird, sondern auch von ein oder mehreren anderen Bibliotheken, In diesem Fall ist sorgfältig zu prüfen welche Version der Bibliothek eingesetzt werden soll. In vielen Fällen sind Bibliotheken abwärtskompatibel zu ihren Vorgängerversionen, Vorsicht ist aber bei Major-Versionssprüngen angebracht. Transitorische

Deployment einer neuen JRE-Version

Im Sourcecoderepository Ombis-Hauptentwicklungszweig (Subversion/Trunk) gibt es einen Unterordner lib, der die Bibliotheken sammelt, die für die Ausführung benötigt werden. Diese werden im dort befindlichen readme.txt mit Scope runtime beschrieben. Wenn in diesem lib-Verzeichnis Bibliotheken eingefügt/geändert/gelöscht werden, dann sind folgende Schritte notwendig um diese Änderungen dann auch zu den Kunden zu bringen. Als stellvertretenden Namen für das JRE wird im folgenden das Beispiel 21.0.3_9 verwendet, als stellvertretende Programmversion die Version 24.07.

- Die Bibliothek muss im pom.xml aktualisiert werden. Anschließend Maven/Reload all Maven Projects durchführen. Das sorgt dafür, dass die Bibliothek auf dem lokalen Rechner in einem Unterordner von %userProfile%\m2 abgelegt wird, und von dort in den lib-Ordner kopiert werden kann. Komplexere Bibliotheken wie z.B. Jetty werden in OmbisDependencies gebuildet und von dort aus dem target Verzeichnis in das lib-Verzeichnis kopiert.
- Die Beschreibung der entsprechenden Bibliothek muss, falls notwendig, im readme.txt aktualisiert werden.
- Die Lizenz unter der die Bibliothek vertrieben wird, muss, falls notwendig, in THIRDPARTYLICENSEREADME.html aktualisiert werden.
- Mit Hilfe des Tools it.prodata.tools.div.ListDuplicatesInJars prüfen, dass nicht Klassen in mehreren Bibliotheken gleichzeitig vorhanden sind.
- Falls nicht bereits geschehen muss unter SupportSW\02_Runtimes ein neues jre-Verzeichnis angelegt werden. Dazu wird einfach das aktuelle JRE kopiert, und erhält einen neuen Namen wie z.B. 21.0.3_9_Ab_Version_24.07_Under_Construction. Der erste Namensbestandteil weist auf das verwendete JRE hin. Der zweite Namensbestandteil auf die aktuelle Major/Minor Version des Programmes, wie sie im pom.xml eingetragen ist. Das Under_Construction weist darauf hin, dass dieses JRE nur in Kombination mit @-Versionen verwendet werden soll, da sich im Laufe des Sprints noch Änderungen ergeben könnten.
- Die Änderungen im Trunk/lib nun auf win64\lib übertragen bzw unter win64\jdk das neueste JRE einsetzen. Gegebenenfalls auch das THIRDPARTYLICENSEREADME.html mitkopieren.
- Damit in einer Container-Umgebung die aktualisierte Runtime zur Verfügung steht, muss folgendes gemacht werden. In der Datei docker/config.json muss in der Sektion runtimes bei version die Nummer weitergedreht werden, und gegebenenfalls auch die runtime-Version angepasst werden. Nach dem Checkin in das Repository muss in unserem Jenkins-Build-System unter CLOUD das Build_Runtime angestoßen werden.
- Bei Sprintende, am Tag der Erstellung der RC-Jars wird dann vom neuen JRE in SupportSW\02_Runtimes das Suffix _Under_Construction entfernt. Die nächsten Schritte sind für Pro Data und ASA leicht unterschiedlich, zunächst die Beschreibung für Pro Data. Der win64 Ordner wird auf ein lokales Laufwerk kopiert. Die Exe-Dateien die nicht automatisch verteilt werden, oder durch die ASA verteilt werden, werden gelöscht. Im win64-Ordner werden die Zeitstempel Erstellt am, Geändert am und Letzter Zugriff auf denselben Timestamp gestellt, dazu nehme ich den aktuellen Tag, und runde die Zeit ab auf eine halbe Stunde oder Stunde, also Sekunden 00 und Minuten 00 oder 30. Anschließend wird der win64-Ordner gezippt und

umbenannt in diesem Beispiel auf jdk21.0.3.9_win64.zip. Darauf erhält die Zip-Datei dieselben Timestamps wie sie im Zip-Archiv enthaltenen Ordner gesetzt wurden. Zum Setzen der Timestamps verwende ich das Freeware-Tool Attribute Changer. Letztlich wird die Zip-Datei in die Installation des Lizenzprogrammes kopiert unter ProductCommons/2407. Für ASA gilt entsprechend, die durch Pro Data verteilten Exes werden entfernt, und anstatt ProductCommons wird das Zip in den Product-Unterordner kopiert, wo sich auch das zu verteilende Programm befindet.

Best-Practice im von uns geschriebenen Quellcode

Obwohl durch die Verwendung von Bibliotheken das Risiko von vulnerablen Code gesenkt werden kann, muss natürlich beim eigenen Programmcode auch darauf geachtet werden, dass so genannte Best-Practices eingehalten werden. Hilfestellung können Webseiten wie [\[https://docs.fluidattacks.com/\]](https://docs.fluidattacks.com/)(<https://docs.fluidattacks.com/>) bieten wo zahlreiche sicherheitsrelevante Pattern und Antipattern aufgezeigt werden.

Automatisierte Tests

Wir verfügen mittlerweile über eine voll automatisierte Deployment-Pipeline. Dort eingeklinkt sind die automatisierten Tests. Falls ein Test scheitert, wird das Deployment verhindert.

Checks durch externe spezialisierte Firmen

Zumindest bei unseren Programmen, die im Internet exponiert sind, empfiehlt sich eine periodische Kontrolle durch externe spezialisierte Dienstleister, die aktiv nach sicherheitsrelevanten Schwachstellen in unseren Programmen suchen, und uns darüber Bericht erstatten.

Zugelassene Software

Auswahlkriterien

Die Installation von Software auf einem Rechner stellt ein bestimmtes Risiko für das Unternehmen dar, da sie Malware enthalten könnte, oder auch die einwandfreie Funktionsweise des Rechners oder des Netzwerks stören könnte. Deshalb beschränken wir uns bei der Auswahl der zu installierenden Software auf Produkte von Unternehmen, die eine in dieser Hinsicht gute Reputation haben. Weiters installieren wir Software aus dem Open-Source-Bereich, mit ebenfalls anerkannter Reputation. Idealerweise ist diese Software digital signiert. In Sonderfällen kann es sich auch um Software von kleinen Firmen oder Einzelentwicklern handeln, sofern es keine Alternativen dazu gibt.

Die Reputation überprüfen wir durch das Lesen von Rezensionen in Fachzeitschriften oder fachspezifischen Web-Seiten. Wir laden die Software immer von den Original-Webseiten, achten darauf dass wir die neueste Version verwenden, aber auch keine Beta-Versionen. Die heruntergeladene Version muss geprüft virusfrei sein. Der Rechner sollte auch regelmäßig auf nicht mehr verwendete Software überprüft werden, und diese deinstalliert werden. Software die

weiterhin in Gebrauch ist sollte auf neuere Versionen hin untersucht werden. Wir installieren nur Software, bei der wir lizenzrechtlich in Ordnung sind. Diesselbe Vorgehensweise verwenden wir auch bei der Installation von Plug-Ins. Leider kommt es auch bei ansonsten seriöser Software vor, dass der Installer AD-Ware beinhaltet. Es ist sorgfältig darauf zu achten, dass diese abgewählt wird.

Es gibt eine Serie von Pflichtprogrammen die vom Unternehmen vorgeschrieben sind, und installiert werden müssen, wie zum Beispiel das Antiviren-Programm um den Rechner virenfrei zu halten, ein Programm für die Telefonie-Integration, oder die Office-Suite um den Workflow zwischen den Mitarbeitern zu garantieren.

Erlaubt ist der Einsatz von Programmen, die vom Unternehmen nach Bedarf vorgesehen sind. Das sind zum Beispiel Programme für die Fernwartung.

Erlaubt ist die Installation von unterstützender Software für eingebaute oder über den Rechner anzusteuern Hardware. Falls die entsprechende Hardware nicht mehr verfügbar ist, soll auch die unterstützende Software dazu vom Rechner wieder entfernt werden.

Zulässig ist die Installation von Tools des Betriebssystemherstellers, in unserem Fall Microsoft, empfohlen wird z.B. das Programm Microsoft Baseline Security Analyzer. Die Installation der Sysinternals-Suite ist ebenfalls erlaubt.

Zulässig ist die Installation von Tools des Hardware-Herstellers, empfohlen wird für HP-Rechner z.B. das Programm HP Performance Advisor und HP SoftPaq Download Manager.

Bei den für die Softwareentwicklung notwendigen Programmen sollte man eine Auswahl aus den angeführten Programmen treffen. Für die nicht in der Liste aufgeführten Programme ist die Erlaubnis des Abteilungsleiters einzuholen.

Liste der zulässigen Programme für die Softwareentwicklung

Name	Tags	Notiz		--	--	--		7-zip	Tool, Kompression		Adobe Reader	Pdf	
Altap Salamander	Explorer	ProData hat 3 Lizenzen gekauft (Mittlerweile Freeware)		Android Studio	IDE, Entwicklung, Editor		Angular Console	Entwicklung, Command Line	UI Tool für Angular CLI		APK-Info	Tool, Android	
AsciiDocFX	Dokumentation, Tool, Editor		AsciiDoctor	Dokumentation, Command Line		Atom	IDE, Entwicklung, Editor		Bing-Desktop	Desktop	Microsoft Software für Wechseln von Bildschirmhintergründen		CCleaner
Säuberung		Chrome	Browser, Internet		ClassFileReader - CFR	Dekompilation	Java Decompiler		Codewright		DB Browser for SQLite	SQL, Datenbanken	
Dbeaver	SQL, Datenbanken		Ditto	Tool, Zwischenablage	Verwaltet eine History der Zwischenablage		DosBox	DOS	Dos Emulator		draw.io	Tool, Visualisierung	Flowchart maker
	Eclipse IDE	IDE, Entwicklung, Editor		Eclipse MAT	Memory Analyzer	Java heap analyzer		Edge	Browser, Internet		estos ProCall	Telefonie, Chat	
FileZilla	Tool, FTP		Firebird	SQL, Datenbanken		Firefox	Browser, Internet		Flame-Robin		Gimp	Bildbearbeitung, Grafik	
Git	Versionskontrolle		Greenshot	Tool, Screenshot	Erweitert die Möglichkeiten Screenshot zu machen und zu verwalten		HxD Editor	Editor	Hexadezimal Editor		ImgBurn		Brennsoftware
Inkscape	Bildbearbeitung, Grafik	Erstellen von Vektorgrafiken		Intel Treiber- und Support-Assistent		IntelliJ IDEA (Community Edition)	IDE, Entwicklung, Editor						

| | | Irfan-View | | Viewer für Bilder | | Java Development Kit (Jdk) | Java, Entwicklung | | | Java Runtime Environment (Jre) | Java, Entwicklung | | | Jenkins | Entwicklung, Build, CI | | | KeePass | Passwort | Passwortmanager | | KeyStore Explorer | | GUI für das Java "keytool" | | Libre Office | Bürosoftware | | | LinkShellExtension | | zum Verwalten von Hard-Links | | Malwarebytes | Anti Virus | Malware Erkenner | | NetBeans IDE | IDE, Entwicklung, Editor | | | Node.js | Javascript, Entwicklung | Packagemanager | | Notepad++ | IDE, Entwicklung, Editor | | | OpenSSL | | Zum Lesen von Zertifikaten | | Opera | Browser, Internet | | | Paint.Net | Bildbearbeitung, Grafik | | | pgAdmin | SQL, Datenbanken | Development Platform for PostgreSQL | | PostgreSQL | SQL, Datenbanken | | | Postman | Rest, http | | | Radeon RAMDisk | | | | REGpro2017 | | AgriNotes Vorgänger | | Ruby | Ruby, Entwicklung | | | SOA XSD Diff | | Vergleichstool für XSDs | | SoapUI | | SOAP Schnittstellen testen | | SpeedCommander | | Pro Data hat 2 Lizenzen gekauft | | SQL-Server | SQL, Datenbanken | | | SQL-Server-Manager | SQL, Datenbanken | | | SQL2005 Service Manager | SQL, Datenbanken | zum Starten und Stoppen des SQL-Servers aus dem System Tray | | SQuirreL | SQL, Datenbanken | | | TeamViewer | Support | | | TCPMon (Apache) | | | | Thunderbird | E-Mail, Internet | | | Tortoise | Versionskontrolle | | | Visual Studio Code | IDE, Entwicklung, Editor | | | VLC Media Player | Video | | | WinMerge | | Dateien vergleichen | | Wireshark | Tool, Netzwerkanalyse | Um Netzwerkkommunikation zu analysieren | | XAMPP | Entwicklung, Web Server | | | XSD To HTML | | |

Continuous Integration - Continuous Delivery

Einleitung

Bei Continuous Integration und Continuous Delivery (CICD) geht es im Wesentlichen darum, den Zeitraum von der Äußerung eines Kundenwunsches, oder der Meldung eines Fehlers, bis zum Zeitpunkt der Auslieferung einer neuen Programm-Version so kurz wie möglich zu halten.

Hier folgt ein Überblick über unsere Umsetzung von CICD. Detaillierte Erläuterungen zu einzelnen Punkten finden sich an den entsprechenden Stellen in der Dokumentation.

Wir liefern im Schnitt mehrmals in der Woche sogenannte Service-Releases mit Fehlerbereinigungen oder neuen kurzfristig benötigten Features aus, bzw. am Ende eines Sprintzyklus ein Minor-Release das Datenbankänderungen erfordert, oder ungefähr einmal im Jahr ein Major-Release. Es ist derzeit nicht möglich diese Rhythmen insbesondere der Servicereleases weiter zu verkürzen, weil Updates derzeit nicht im laufenden Betrieb eingespielt werden können.

Es wird im folgenden vereinfachten Überblick dargestellt, wie wir in unserer Firma CICD umgesetzt haben.

{align=center}

Beispielhaft wird im folgenden das Deployment einer Servicerelease erklärt. Andere Releasetype laufen durchaus vergleichbar ab, mit nur geringen Abweichungen.

Deployment einer Service-Release

Die Notwendigkeit zur Erstellung einer Service-Release wird entweder von einem Supporter an das Entwicklungsteam herangetragen, oder das Entwicklungsteam stellt selbst diese Notwendigkeit zB. beim Daily-Meeting fest. Der für das Deployment zuständige Mitarbeiter startet im Firmenchat eine Ankündigung an die Entwickler, dass die Erstellung einer Service-Release geplant ist, so dass den anderen Entwicklern noch Möglichkeit zu einer Stellungnahme gegeben wird, und noch Änderungen in das Source-Code-Repository eingecheckt werden können.

Falls kein Veto mehr vorliegt, startet der Mitarbeiter den Build-Prozess über die Web-Oberfläche des Jenkins-Servers, gegebenenfalls unter Angabe der notwendigen Parameter. Der Jenkins-Server holt sich nun die Sourcen aus dem Source-Code-Repository, kompiliert sie, stellt sie zu einem Jar zusammen und führt erste Checks durch. Anschließend wird eine Serie von automatisierten Tests mit der neu erzeugten Jar-Programmdatei durchgeführt.

Wenn alle diese Schritte erfolgreich verlaufen sind, wird die Jar-Programmdatei in verschiedene Ordner kopiert, unter anderem in einen Unterordner des Lizenzprogrammes, um diesem das Deployment zu ermöglichen. Es wird außerdem mit Hilfe des Ombis-Verwaltungsprogrammes eine E-Mail erzeugt, die eine Versionhistory zur neuen Programmversion enthält und an ausgewählte Empfänger verteilt wird. Wenn der Build scheitert, werden ausgewählte Entwickler über E-Mail informiert, und können nach Fehlerbeseitigung den Build-Prozess erneut starten.

Die Kundeninstallationen kontaktieren das Lizenzprogramm einmal am Tag, standardmäßig um ca. 10 Uhr abends. Das Lizenzprogramm prüft ob das Kundenprogramm Anrecht auf eine neue Programmversion hat, und bietet diesem die Möglichkeit sie herunterzuladen.

Falls das Kundenprogramm Updates heruntergeladen hat, werden diese automatisiert um 2:00 morgens eingespielt, und stehen damit den Kunden am nächsten Tag zur Verfügung.

Scrumban Regeln

Die Entwicklungsabteilung von Pro Data organisiert sich nach dem Scrum-Vorgehensmodell, welches mit Hilfe des Kanban-Vorgehensmodell ergänzt und laufend weiterentwickelt wird. Wo die beiden Vorgehensmodelle in Konflikt zueinander stehen, hat Scrum Vorrang. Die Softwareentwicklung verläuft in geplanten Sprints mit einer regulären Dauer von 4 Wochen. Bei Bedarf kann ein Sprint nach Absprache aller Beteiligten verkürzt oder verlängert werden.

Regulärer Sprint-Ablauf:

Beginn: **Donnerstag** Dauer: 4 Wochen (Planning-Day + 19 Arbeitstage) Ende: **Mittwoch**

Jars

Während eines Sprints werden laufend verschiedene Jars erstellt. Alpha und RC-Jars dienen dazu vorab im Haus von Supportern getestet zu werden, so können Einwände oder Fehler frühzeitig erkannt und überarbeitet werden.

Um die übrigen Mitarbeiter über ein neues Jar zu informieren wird eine Sprint-Mail mit den in der

Version enthaltenen Punkten versendet. Dazu wird der **Druck "Sprint"** in der Tabelle des Sprints ausgeführt, im Druckdialog unter Parameter die Version angegeben und mittels Mail an die eingetragenen Empfänger versendet.

Alpha

Während des Sprints jeweils am **Montag** wird eine Alpha-Version unter „SupportSW\01_Jars\04_AlphaJars“ zum Testen bereitgestellt. Damit die Alpha-Versionen untereinander unterschieden werden können werden sie nach Woche nummeriert und fortlaufend mit Buchstaben versetzt.

Beispiel:

- Alpha-Version in der ersten Woche des Sprints: 15.03@1 wird im Laufe der Woche eine weitere Alpha-Version erstellt um bestimmte Punkte zu testen so heißt die nächste 15.03@1a usw.
- Erste Alpha-Version in der zweiten Woche nach Sprintbeginn 15.03@2 usw.

Service-Release

Werden Fehler in der letzten finalen Version entdeckt, so kann ein Service-Release erstellt werden, welches **nur die korrigierten Fehler** und keine neuen Funktionalitäten enthält. Service-Releases werden immer aus dem Branch generiert, daher muss jeder Programmierer:

- Seine korrigierten Fehler selbst auf den **Branch portieren**
- Jeden Fehler **separat einchecken**
- Die **Portierung** im entsprechenden Backlog festhalten, damit die Sprint-Mail korrekt generiert wird.

RC

Am Ende jeden Sprints wird am **letzten Mittwoch** eine RC-Version erstellt, diese sollte von den Supportern bis zum Freitag ausgiebig getestet werden.

Final

Die definitive Version wird nach Sprintende am **Freitag** erstellt und wird meist am Wochenende an die Kunden verteilt und eingesetzt.

Nach dem Erstellen der finalen Version:

- Inhalt des **Alpha-Jars-Ordners** löschen
- Versionsnummer ab Fakt auf **Alpha** weiterdrehen
- **Produkt-Versionen** im Task Tracking pflegen

Task Tracking

Das Task Tracking im Ombis wurde wie folgt an den Scrumprozess angepasst:

Sprint

Ein Sprint hat ein definiertes Enddatum, bis zu welchem alle zugewiesenen Punkte, erledigt werden müssen. Der aktuelle Sprint ist mit einem Kennzeichen ausgewählt. In der Beschreibung werden Informationen notiert, welche in der Sprint-Mail aufscheinen, in der Bemerkung kann die Anzahl der Urlaubstage pro Mitarbeiter vor der Planning notiert werden, um den kommenden Sprint besser planen zu können. Anhand der letzten Sprint und der voraussichtlichen Arbeitstage wird in einer Excel-Datei, welche beim Sprint hinterlegt ist die geplanten zur Verfügung stehenden Stunden pro Team und Sprint berechnet.

Burndown - Chart

Das Burndown-Chart visualisiert die tägliche Arbeit, die Verfolgung des Zieles alle Backlogs termingerecht abzuliefern und zeigt Verzögerungen schnell auf. Anhand dieser Werte können die Zeiten für nachfolgende Sprints besser geplant werden. Um jene Backlogs zu kennzeichnen, welche in der Planungssitzung besprochen, geschätzt und für den Sprint geplant wurden gibt es das Feld "Geplant". Backlogs, welche erst im Laufe des Sprint unvorhergesehen dazukommen, werden als nicht geplant eingestuft und im Burndown-Chart in einer separaten Linie dargestellt. Beide Linien im Burndown-Chart zeigen die idealen (geplanten) Zeiten, welche pro Tag im Sprint vom jeweiligen Team abgearbeitet werden soll. In den Daily-Zeiten, werden täglich jene Zeiten eingetragen, die von den geplanten abgezogen werden können. Bei nicht geplanten Backlogs werden die effektiven Produktionszeiten verwendet.

Kanban-Board

Das Kanban-Bord visualisiert die Backlogs des jeweiligen Teams im aktuellen Sprint die einzelnen Arbeitsschritte werden so gut wie möglich nachgebildet und dient dazu die Arbeit und deren Verteilung sichtbar zu machen. Ursprünglich handelte es sich um eine Magnet-Pinnwand auf welcher Post-it's fixiert wurden. Mittlerweile, bedingt durch Homeworking bzw. durch die Verteilung der Mitglieder eines Teams auf verschiedene Standorte wurde das physische Kanbanboard durch ein virtuelles Kanbanboard in unserem Ombis-Verwaltungsprogramm ersetzt. Die damaligen Post-it's entsprechen den Backlogs im Sprint mit dessen Nummer, Titel und dem geschätzte Zeitaufwand vom Planning Poker. Die Farbe der Einträge richtet sich nach der Serviceklasse.

Backlog

Der Backlog (BL) stellt eine Anforderung für die Software dar, dies kann ein Fehler eine Änderung oder eine Erweiterung sein. Meist wird er von einem Supporter im Auftrag eines Kunden erstellt, kann aber auch von jedem anderen erfasst werden. Alle bekannten Informationen werden im BL festgehalten und die Problembeschreibung muss klar und für Jeden verständlich formuliert werden. Bemerkungen, Fragen, Antworten und neue Erkenntnisse müssen fortlaufend bei der Problembeschreibung mit dem eigenen Namens-Kürzel und aktuellen Datum festgehalten werden. Sollte die Änderung Kundenspezifisch sein und keinen Checkin/Commit zur Folge haben, so ist dieser als Serviceanfrage und nicht als Backlog zu dokumentieren. Die SVN Nummer des letzten Commits muss im BL vermerkt werden, jeder BL wird alleinstehend eingecheckt. BLs welche sich auf OMBIS und Webapplikationen beziehen sollten weitgehend getrennt erfasst werden und über verknüpfte BL miteinander verlinkt werden. Prinzipiell wird kein zuständiger Programmierer in

den BL eingetragen, jedoch eine interne Ansprechperson, an welche sich jeder der Fragen zu diesen BL hat, wenden kann.

Scrum-Team

Die Programmiererabteilung wird in Teams aufgeteilt, welche sich auf verschiedene Bereiche konzentrieren. Alle Teams arbeiten am selben Sprint und an der selben Version, organisieren sich aber ansonsten in verschiedenen Sitzungen und Meetings.

Planning Poker

Der Zeitaufwand jedes BLs wird vom Programmiererteam im vorhinein geschätzt und im Feld Planning Poker notiert, so kann abgeschätzt werden, welche Punkte innerhalb des Sprints abgearbeitet werden können.

Produktionszeiten

Wird an einem BL gearbeitet, so werden die aufgewendeten Stunden auch dort festgehalten, am Ende des Tages werden diese als Arbeitszeiten übernommen.

Backlog Task

Die *Backlog Tasks* dienen dazu große BL in kleinere Arbeitsschritte zu unterteilen, um Zeiten besser abschätzen zu können und/oder einzelne Tasks anderen Personen zuzuteilen. Sowohl Produktionszeiten als auch SVN Nummern können pro Task hinterlegt werden, beispielsweise für Teilcheckins/-commits.

Backlog Status

Der *Status* muss fortlaufend vom Programmierer gepflegt werden. Die meisten sind selbsterklärend, bei Folgenden jedoch gibt es genauere Vereinbarungen:

- *Testen*: Der BL **MUSS** bis zum Ende des Sprints getestet werden, ansonsten wird kein finales Jar erstellt! Dieser Status wird für heikle Umbauten oder Änderungen verwendet, welche unter Umständen schwerwiegende Folgen nach sich tragen können. Zum Testen können Alpha- oder Beta-Jars erstellt werden. Es ist die Aufgabe des Programmierers sich darum zu kümmern, ein Feedback einer weiteren Person einzuholen. Um besser zu verdeutlichen, dass keine finale Version erstellt werden kann, solange sich ein BL im Status "Testen" befindet, werden diese im Versions-Mail rot gekennzeichnet. Der Status ist auf **zwei** BLs pro Programmierer limitiert.
- *Abzunehmen*: Sobald ein BL eingecheckt wurde, **SOLLTE** er von einem anderen Programmierer bis zum Ende des Sprints abgenommen werden! Bei der Daily-Sitzung wird jeder abzunehmende BL einem anderen Programmierer zugeteilt für ein Code-Review. Hat ein Programmierer Schwierigkeiten einen BL zu Reviewen kann er sich gerne Hilfe bei einem weiteren Programmierer holen und den BL danach auf erledigt stellen.
- *Unklar*: Der BL **MUSS** nochmals auf seine Anforderung oder klare Formulierung hin überprüft werden, ansonsten wird er nicht in den Sprint aufgenommen. Den Status Unklar kann sowohl ein Programmierer, als auch ein Supporter setzen, wenn er der Meinung ist, der Punkt könnte Probleme machen oder sollte überdenkt werden.

- *In Bearbeitung*: Jeder Programmierer darf nur **drei** BL-Einträge zeitgleich bearbeiten.
- *Erledigt*: Ein BL gilt als erledigt, sobald dieser getestet eingecheckt und reviewt wurde, es muss daher eine **SVN Nummer** und einen Text im Feld "**Lösung**" aufweisen, welche als Referenz dient um später eine Versionshistorie für den Kunden zu erstellen. Daher sollte die Lösung alle wichtigen Aspekte beinhalten und auch von Personen ohne technischen Wissen verstanden werden.

Backlog Serviceklasse

- **AGREEDDATE (Blau)**: Backlogs welche ab dem verbindlichen Termin erhebliche Kosten verursachen, sollte dieser nicht eingehalten werden können. Diese Punkte müssen im Sprint erledigt werden und haben ein verbindliches Lieferdatum, welche unter Umständen bereits einem Kunden mitgeteilt wurde.
- **EXPEDITE (Rot)**: Fehler, mit sehr hohen Priorität, oder Implementieranfragen die umgehend eingebaut werden müssen. Meist handelt es sich um Anforderungen, die nachträglich und ohne Planung, während des Sprints aufgenommen wurden.
- **LOW (Grün)**: Backlogs mit geringer Priorität, welche kaum Verzögerungskosten verursachen.
- **STANDARD (Orange)**: Normale Anforderungen, welche regulär geplant und eingetacktet werden sollten. Die Kosten verhalten sich linear zur Zeit.

Blockiergrund

Der Blockiergrund dient zusätzlich zum Status Blokiert die Information festhalten, warum momentan nicht am Backlog gearbeitet werden kann. Handelt es sich nur um kurzfristige Verzögerungen oder Ähnliches, sollte der Grund wieder herausgelöscht werden, oder in der Problembeschreibung vermerkt werden. Nur solche Einträge welche von historischer Relevanz sind sollten als Blockiergründe auch weiterhin ersichtlich bleiben, um evtl. später Auswertungen darauf aufzusetzen. Außerdem kann ein solcher Blockiergrund auch in einem BL vermerkt werden bevor dieser in einen Sprint eingetaktet wird um zu verdeutlichen, dass mit diesen Punkt nicht ohne weiteres begonnen werden kann, solange der Blockiergrund und dazugehörige Status bestehen.

Sitzungen und Meetings

Wer nicht zu einer der Sitzungen oder Meetings kommen kann, muss sich im vorhinein abmelden.

Daily Meeting

- Termin: je nach Team verschieden, bei den meisten gegen 16:00 Uhr.
- Time-Box: 15 Minuten
- Teilnehmer: Scrum-Team und Interessierte
- Kanban-Board wird aktualisiert und die übrigen Programmierer werden darüber informiert, woran am Vortag gearbeitet wurde, was aktuell in Bearbeitung ist und mit welchem BL im Laufe des Tages noch begonnen wird. Nachträglich eingeschobene BLs müssen in diesem Meeting von allen genehmigt werden.

WARNING

Eine Falle beim Daily Meeting ist, dass dabei gerne begonnen wird, Probleme zu diskutieren!

Sprint Planning

- Termin: Am ersten Donnerstag des Sprints um 8:00 Uhr
- Time-Box: 8 Stunden
- Teilnehmer: Scrum-Team und Produkt-Owner
- Der Produkt-Owner stellt jeden Backlog-Punkt vor und das Programmierer-Team entscheidet darüber ob der Punkt im geplanten Sprint umgesetzt wird und schätzt mittels Planning-Poker die Zeit zur Umsetzung des BLs. Da es immer wieder BL gibt, welche im Laufe des Sprints eingeschoben werden müssen, werden 15% der Zeit für solche Fälle reserviert und weitere 5% für die Optimierung des Quellcodes. Die restlichen 80% der Produktionszeit wird für die geplanten BL aufgewendet, davon sollten 10% die Serviceklasse LOW haben.
- Nach der Planning Sitzung wird das Kanban-Board und das Burndownchart erneuert.

Technische Sitzung

- Termin: Bei Bedarf
- Time-Box: 1 Stunde
- Teilnehmer: Alle Interessierten
- Besprechung von komplexeren technischen Themen, welche vorher im Firmenkalender beim entsprechenden Termin eingetragen werden, damit jeder individuell entscheiden kann, ob er an der Sitzung teilnehmen möchte.

Sprint Retrospektive

- Termin: Bei Bedarf
- Time-Box: 1 Stunde
- Teilnehmer: Scrum-Team und Produkt-Owner
- Rückblick auf den Sprint, Verbesserungsvorschläge, aktuelle Problematiken aufdecken und nach dessen Lösungen suchen, die Themen werden in der Retro-Box gesammelt.

Sprint Review

- Termin: Auf Anfrage der Supporter
- Time-Box: 1,5 Stunden
- Teilnehmer: Scrum-Team, Supporter und Produkt-Owner
- Es werden jene Backlogs präsentiert, welche mit "REview-Call" markiert wurden. Alle Anwesenden darunter auch Supporter können Fragen stellen, Verbesserungen vorschlagen oder Ergänzungen als BL für den nächsten Sprint erfassen. Diese Sitzung dient dazu allen die wichtigsten Neuerungen und Änderungen der Version mitzuteilen ohne, dass jeder die gesamte Sprint-Mail durchlesen muss.

Eine Ordnerstruktur für den Anfang

Vorlage

Die folgende Ordnerstruktur gilt lediglich als vage Vorlage, sie soll neuen Mitentwicklern eine einigermaßen stabile Grundlage bieten. Sie kann mit der Zeit natürlich verändert und individuell angepasst werden.

~ C:/ |- jDevelopment |- Documentations |* OmbisDokumentationen |* Staff Manual |- jClasses |- jdk8 |- win32 |- win64 |- jFirebird |- jODBC |- jSource |* Ombis |- workspace1

Z:/ |- testDBs ~

Legende

| - normaler Ordner |* Ordner mit Versionskontrolle (SVN), die benötigten Links findet man im Manual, bzw. durch das Durchstöbern des SVN-Archivs.

Ersteinrichtung

Projekte auschecken

- **Tortoise SVN** installieren (\\SVR-DC01\dfs\Software\Subversion)
- Hierbei ist zu beachten, dass **alle** Features des Programms installiert werden sollten.
- Optional: Deutsches Sprachpaket für TortoiseSVN in der korrekten Produktversion installieren (\\SVR-DC01\dfs\Software\Subversion)
- Die folgenden Projekte können anschließend über TortoiseSVN ausgecheckt werden:
- **Ombis:** svn://svr-develop/develop/Ombis/trunk
- **OmbisAndroid:** [svn://svr-develop/develop/OmbisAndroid/trunk](svn://svr-develop/develop/develop/OmbisAndroid/trunk)
- **OmbisMobile:** [svn://svr-develop/Develop/OmbisMobile/trunk](svn://svr-develop/Develop/Develop/OmbisMobile/trunk)
- **StaffMan:** svn://svr-develop/staffman/ProData/trunk

NOTE

Die benötigten Zugangsdaten für Subversion (Benutzer + Passwort) können beim Abteilungsleiter angefordert werden.

JDK

- Im Development-Verzeichnis den Ordner **Java** anlegen und dort das aktuellste ZIP für 64Bit von `\\SVR-DC01\dfs\Software\Adoptium` entpacken.

WARNING

Darauf achten, dass das JDK und nicht das JRE verwendet wird.

Firebird Einrichtung

Firebird installieren

- `\\SVR-DC01\dfs\Software\firebird`

Verknüpfung und Start

- Verknüpfung von `/bin/fbserver.exe` auf Desktop (oder sonst wo) einrichten.
- Stand April 2017 sollte Firebird in der Version 2.5.x verwendet werden (kann sich zukünftig ändern, eventuell einen Mitarbeiter fragen).
- Rechtsklick auf die Verknüpfung → Eigenschaften
- Im Textfeld von "Ziel:" folgende Parameter hinter Zielpfad anfügen. (Achtung: Zielpfad muss erhalten bleiben): `-a`
- Doppelklick auf die Verknüpfung um Firebird zu starten.

Einrichten von Flamerobin (Optional)

- Die neueste Version von Flamerobin kann in `\\SVR-DC01\dfs\Software\Tools\PD_PortableApps\apps` gefunden werden.
- Sobald man es lokal öffnet, kann man in der Menüleiste über "Server" - "Register Server" einen Server eintragen. ~ Display name: svr-develop Hostname: svr-develop ~
- Sobald man mit dem Server verbunden ist, kann man die Master-Datenbanken hinzufügen. Rechtsklick auf den Server "svr-develop" und dann "Register existing database" auswählen.
- Die Datenbanken sollten zuvor in ein lokales Verzeichnis kopiert werden.
- Passwort bitte bei der Abteilungsleitung nachfragen ~ Display name: MasterFirm0000 Database path: D:\develop\DB\Firm0000.fdb Authentication: Use saved user name and password Username: sysdba Charset: UTF-8 Role: ~ ~ Display name: MasterSystem Database path: D:\develop\DB\System.fdb Authentication: Use saved user name and password Username: sysdba Charset: UTF-8 Role: ~

SQL Server Einrichtung

Microsoft SQL Server Installation

- SQL Server 2017 Developer[...].iso mounten Zu finden unter (\\SVR-DC01\Software\Microsoft\SQL 2017)
- **Setup.exe** ausführen.
- Unter Installation den Punkt "Neue eigenständige SQL Server-Installation oder Hinzufügen von Funktionen zu einer vorhandenen Installation" auswählen.
- In der Installationsroutine folgendermaßen vorgehen:
 - **Produktkey** → Developer auswählen - Weiter
 - **Lizenzbedingungen** → Akzeptieren - Weiter
 - **Produktupdates** → Weiter
 - **Setupdateien installieren** → Weiter
 - **Installationsregeln** → Weiter
 - **Installationstyp** → SQL Server 2016 neu installieren - Weiter
 - **Setuprolle** → Weiter
 - **Funktionsauswahl** → Folgende Funktionen auswählen
 - Database Engine Services (Datenbankmoduldienste)
 - SQL Client Connectivity SDK
 - **Instanzkonfiguration** → Standardinstanz auswählen - Weiter
 - **Serverkonfiguration** → Weiter
 - **Datenbankmodulkonfiguration** →
 - Gemischter Modus (...) auswählen
 - Passwort für SA-Konto eingeben und merken
 - Aktuellen Benutzer hinzufügen
 - **Funktionskonfigurationsregeln** → wird automatisch übersprungen, sonst weiter
 - **Installationsbereit** → Installieren
- Schließen
- Zusätzlich muss noch ein Dienst korrekt eingestellt werden.
- **Dienste** öffnen (z. B. im Startmenü "Dienste" eintippen und bestätigen)
- Rechtsklick auf den Dienst **SQL Server (MSSQLSERVER)** → Eigenschaften
- Tab: **Anmelden** -> **Lokales Systemkonto** als Option auswählen
- Das Feld **Datenaustausch zwischen Dienst und Desktop zulassen** soll auf aktiv gesetzt werden.

TCP/IP Protokoll aktivieren

- SQL Server Konfigurations Manager öffnen (engl. Server Configuration Manager)

- SQL Netzwerkkonfiguration aufklappen
- Protokolle für MSSQLSERVER anklicken
- Rechtsklick auf TCP/IP um es zu aktivieren

TCP/IP-Protokoll aktivieren

{} .TCP Port anzeigen {}

Benutzerkonto SYSDBA

Nun soll das Benutzerkonto "SYSDBA", welches anschließend von unseren Programmen verwendet wird, angelegt werden. Das kann entweder über das SQL Server Management Studio oder über die Eingabeaufforderung erledigt werden. Für das zu verwendende Passwort bitte bei der Abteilungsleitung nachfragen.

- **SQL Server Management Studio:** Sicherheit → Anmeldungen → Rechtsklick: Neue Anmeldung...

{}

- **Eingabeaufforderung** (als Admin ausführen und Pfad evt. anpassen): `cd "C:\Program Files\Microsoft SQL Server\130\Tools\Binn" osql.exe -U sa -P saPassword CREATE LOGIN SYSDBA WITH PASSWORD = 'Beispielpasswort', CHECK_POLICY = OFF go sp_grantdbaccess 'SYSDBA' go sp_addsrvrolemember 'SYSDBA', 'sysadmin' go quit`

Servereigenschaften anpassen

SQL Server Management Studio öffnen.

- Server Authentifizierung aktivieren. Localhost → Rechtsklick: Eigenschaften und folgende Einstellung vornehmen.

{}

- Sicherung komprimieren unter Servereigenschaften/Datenbankeinstellungen aktivieren

{}

{}

- Wiederherstellungsmodell auf einfach setzen (keine Protokollsicherungen) und Einstellungen anpassen. Datenbanken → Systemdatenbanken → model → Rechtsklick: Eigenschaften

{}

{}

- Automatische Vergrößerung der Dateien auf 10 Prozent setzen.

{}

- Benutzerrechte der Datenbankordner der Testdatenbanken anpassen. Zunächst können mit Rechtsklick auf den SQL Server Dienst in den Diensten, dessen Eigenschaften überprüft werden. Der hier verwendete Name ist später wichtig für die Rechte- und Zugriffsverwaltung im Ordnersystem.

{}

- Dann muss er bei den gewünschten Ordnern als Benutzer hinzugefügt werden. (Mindestens die DATA Ordner der einzelnen Testdatenbanken.) Rechtsklick auf den gewünschten Ordner: Eigenschaften → Sicherheit → Bearbeiten → Hinzufügen Suchpfad: eigener PC Objektname: NT Service\MSSQLSERVER Berechtigungen: Vollzugriff aktivieren

{}

{}

- Optional
- Ombis Einrichtungsdialog → Datenbank: Alternativ zur bisherigen Einstellung mit Port kann bei den einzelnen Testinstallationen auch der Name der Instanz verwendet werden. (Hinweis: Hierfür muss der Dienst "SQL Server-Browser" ausgeführt werden.)

{}

PostgreSQL

Links

Home: [<https://www.postgresql.org/>](https://www.postgresql.org/) Docs: [<https://www.postgresql.org/docs/>](https://www.postgresql.org/docs/) Wiki: [https://wiki.postgresql.org/wiki/Main_Page](https://wiki.postgresql.org/wiki/Main_Page) Tutorial: [<http://www.postgresqtutorial.com>](http://www.postgresqtutorial.com)

JDBC-Treiber: Home: [<https://jdbc.postgresql.org/>](https://jdbc.postgresql.org/) Docs: [<https://jdbc.postgresql.org/documentation/head/index.html>](https://jdbc.postgresql.org/documentation/head/index.html) Tutorial: [<http://www.postgresqtutorial.com/postgresql-jdbc/>](http://www.postgresqtutorial.com/postgresql-jdbc/)

pgAdmin (Verwaltungstool): Home: [<https://www.pgadmin.org/>](https://www.pgadmin.org/) Download: [<https://www.pgadmin.org/download/>](https://www.pgadmin.org/download/)

Bücherliste: [<http://www.allitebooks.com/>](http://www.allitebooks.com/?s=postgresql)

Kurzinfo zu PostgreSQL

- Siehe [<https://www.postgresql.org/about/>](https://www.postgresql.org/about/)
- Umfangreiche SQL-Unterstützung
- Umfangreiche Unterstützung von Datentypen

- Erweiterbar (Typen, Funktionen)
- Standardzeichensatz ist UTF-8
- Verschiedene Index-Typen für spezielle Zwecke (B-tree, Hash, GiST, SP-GiST, GIN and BRIN)
- Viele Tools und umfangreiche Dokumentation erhältlich
- Hinweis: bei Recherchen findet man häufig Informationen zu älteren Versionen, die für die aktuelle nicht mehr gelten. Es ist daher entsprechende Vorsicht geboten.

Besonderheiten

- Der **Datenbankname** bei Verbindungs-URL ist **case sensitive**.
- Die Datenbank arbeitet prinzipiell **case sensitive** (WHERE, ORDER BY, ...). Es gibt eine "case insensitive" Lösung mit Hilfe des Datentyps **citext**, dazu später mehr.
- PostgreSQL benutzt normalerweise die **Standard-Collation** (Sortierung von Zeichenketten) des Betriebssystems und kann sich daher auf den verschiedenen Plattformen unterschiedlich verhalten.
- Für Zeichenketten stehen die Datentypen **varchar** (mit Angabe einer maximalen Länge) und **text** (ohne Längenbeschränkung) zur Verfügung. Abgesehen von einer zusätzlichen Längenüberprüfung im Falle von varchar werden beide Typen intern gleich behandelt (bei Speicherung, Indizierung usw. werden String- und Memo-Felder gleich behandelt). Strings werden je nach Länge unterschiedlich gespeichert. Längere Strings werden komprimiert, die maximale Speichergröße ist 1 GB.
- **Large Objects** in PostgreSQL sind ein spezielles Feature und nicht mit BLOBs oder CLOBs anderer Datenbanken zu vergleichen. Es handelt sich hierbei um Daten, auf die mittels Streaming-API zugegriffen wird und die bis zu 4 TB Speicher beanspruchen können. Sie werden normalerweise nicht benötigt. Für **BLOBs** steht der Datentyp **bytea**, für **CLOBs** der Datentyp **text** zur Verfügung (beide mit max. 1 GB Speicher).
- **Null-Sortierung** ist **hoch**.
- **Schema** ist nicht **Owner**: für PostgreSQL sind das unterschiedliche Konzepte (im Gegensatz zu MS-SQL-Server).
- **JDBC-Treiber**:
 - Connection-URL: jdbc:postgresql://<HOST_NAME>:<PORT>/<DB_ALIAS>[?<PARAMS>]
 - Umfangreiche Connection-Parameter, z.B.:
 - currentSchema=<SCHEMA>: Schema für nicht qualifizierte Objektnamen (z.B. Tabellen, für "schema based multi tenancy").
 - loggerLevel=TRACE
 - loggerFile=pgjdbc.log
 - Statement/ResultSet.setFetchSize: muss für Server-Cursoren größer 0 gesetzt werden, ansonsten wird die gesamte Ergebnismenge in den Speicher geladen. Über die Connection-URL kann ein Defaultwert vorgegeben werden:

jdbc:postgresql://<HOST_NAME>:<PORT>/<DB_ALIAS>?defaultRowFetchSize=10

- **Backup/Restore** werden über die (Kommandozeilen-) Tools **pg_dump** und **pg_restore** ausgeführt. Hierfür benötigen wir eine entsprechende **PATH**-Einstellung, z.B. unter Windows: `PATH="C:\Program Files\PostgreSQL\<VERSION>\bin";%PATH%` (<VERSION> entspricht der jeweiligen Version, z.B. 10)
- Es gibt **keine Möglichkeit**, einen **exklusiven Zugriff** (single user mode) im laufenden Betrieb anzufordern.
- **Datenbanken** werden **in Ordnern** gespeichert. Diese werden von PostgreSQL benannt, sind fest mit anderen Systemordnern verknüpft und können nicht isoliert verwaltet werden (direktes Umkopieren einzelner Datenbankordner ist nicht möglich). Alle Daten werden in einem Datenordner verwaltet. Unter Windows befindet sich dieser Ordner nach der Installation unter "C:\Program Files\PostgreSQL\<VERSION>\data". Dieser kann (sollte) verschoben werden. Dies wird weiter unten behandelt.

Installation

Unter [\[https://www.postgresql.org/download/\]](https://www.postgresql.org/download/)(<https://www.postgresql.org/download/>) stehen Downloads zur Verfügung. Von PostgreSQL selbst werden nur die Quellen angeboten, die Installer-Pakete werden von Drittanbietern zur Verfügung gestellt. Diese packen z.T. zusätzliche Tools dazu, die man nicht unbedingt benötigt. Der Standard-Ordner unter Windows ist "C:\Program Files\PostgreSQL\<Version>". Der Ordner für die physischen Daten kann gesondert angegeben werden.

Konfiguration

[\[https://www.postgresql.org/docs/current/static/config-setting.html\]](https://www.postgresql.org/docs/current/static/config-setting.html)(<https://www.postgresql.org/docs/current/static/config-setting.html>) [\[https://www.postgresql.org/docs/current/static/runtime-config-resource.html\]](https://www.postgresql.org/docs/current/static/runtime-config-resource.html)(<https://www.postgresql.org/docs/current/static/runtime-config-resource.html>)

- **postgresql.conf** ist die zentrale Konfigurationsdatei. Sie befindet sich im Datenordner. Die wichtigsten Einstellungen sind:
- `shared_buffers`: gemeinsam genutzter Speicher. Empfehlung: 25% des verfügbaren Speichers.
- `work_mem`: interner Arbeitsspeicher für Sort-Operationen usw. Die Defaulteinstellung ist 4MB. In besonderen Fällen könnte mehr benötigt werden. Diese Einstellung kann auch pro Session oder pro (Datenbank-) User mit "set" gesetzt werden. Es gilt folgende Empfehlung: `max_connections * work_mem = 50% RAM`
- `search_path`: Bestimmt die Reihenfolge in der nach Schemata für nicht qualifizierte Namesangaben (z.B. Tabellen) gesucht werden soll. Die Defaulteinstellung ist `''$user', public'`. Könnte für spezielle Anwendungen ausgenutzt werden.
- **pg_hba.conf**: Konfiguration der Netzwerck ACLs (HBA stands for host-based authentication). Damit können beispielsweise Backup/Replikations-Rechte auf bestimmte Hosts eingeschränkt werden.
- Über Umgebungsvariablen können Standardparameter für Tools festgelegt werden (siehe [\[https://www.postgresql.org/docs/current/static/libpq-envvars.html\]](https://www.postgresql.org/docs/current/static/libpq-envvars.html)(<https://www.postgresql.org/docs/current/static/libpq-envvars.html>)).

Administration / Operating

PostgreSQL ist generell kommandozeilenorientiert. Für alle Aufgaben finden sich (Kommandozeilen-) Tools, die üblicherweise in Skripts benutzt werden.

pgAdmin

<https://www.pgadmin.org/> Ist laut Eigenwerbung "The most popular and feature rich Open Source administration and development platform for PostgreSQL". Die Oberfläche wird in einem Browser ausgeführt. Es stehen grafische Tools für verschiedene Aufgaben zur Verfügung. Das Query Tool ist SQL-Kommando-orientiert. So müssen beispielsweise COMMIT und ROLLBACK als (eingetippte) Kommandos abgesetzt werden.

Datenbankserver (Dienst) starten/stoppen

Der Datenbankserver wird mit **pg_ctl** verwaltet:

```
pg_ctl [-D <PGDATA>] [-l <Log-File>] start
pg_ctl [-D <PGDATA>] stop
```

Wenn der Parameter -D für den Datenordner weggelassen wird, wird der Wert der Umgebungsvariablen **PGDATA** benutzt.

Upgrades

Der Datenordner (mit allen Datenbanken) ist versionsabhängig und muss bei Upgrades auf höhere PostgreSQL Versionen migriert werden. Es gibt hierzu zwei Möglichkeiten:

- binär mit **pg_upgrade** (ist schnell).
- logisch über Backup/Restore (**pg_dump/pg_restore**). Ist bei größeren Datenmengen deutlich langsamer, dafür aber sehr robust und "clean".

Backup/Restore

Es gibt grundsätzlich drei Möglichkeiten, Daten zu sichern:

- **Cold backup of PGDATA:** Datenbankserver stoppen, Datenordner (vollständig mit allen Datenbanken) kopieren, Datenbankserver wieder starten.
- **Hot backup of PGDATA:** Mittels **pg_basebackup** kann der gesamte Datenordner im laufenden Betrieb gesichert werden. Es wird eine binäre Kopie der Daten erstellt. Das Standardformat kopiert die Daten so um, dass der Zielordner anschließend als Datenordner benutzt werden kann. Die Sicherung benutzt das Replikations-Protokoll über eine reguläre Verbindung zum Server und kann daher auch übers Netzwerk durchgeführt werden (entsprechende Rechte vorausgesetzt, siehe auch `pg_hba.conf`).

Beispiel:

```
pg_basebackup -h localhost -U postgres -X stream -P -D pgdata-backup
```

- **Logisch:** Die Datenbanken können im laufenden Betrieb einzeln mit **pg_dump** gesichert und mittels **pg_restore** wieder restauriert werden. Über Optionen kann das Format und der Umfang der zu sichernden Daten bestimmt werden.

```
pg_dump -F c -b -d <URI> -f <file>  
pg_restore -d <URI> -c <file>
```

URI hat folgendes Format:

```
postgres://[user[:password]@][netloc][:port][,...][/dbname][?param1=value1&...]
```

Globale Objekte (de facto sind das die Rollen) werden nicht gesichert. Hierfür steht **pg_dumpall** zur Verfügung. Mit dem Parameter **-g** können diese gesichert werden, z.B.:

```
pg_dumpall -U postgres -g > role-backup.dump
```

Diese Daten können mit **psql** (PostgreSQL interactive terminal) restauriert werden:

```
psql -U postgres < role-backup.dump
```

Hinweis: Beim Restore mittels **pg_restore** werden die Indizes neu erstellt und die Constraints überwacht. Daher benötigt der Vorgang entsprechend Zeit.

Incremental Backup

Es gibt in PostgreSQL kein eingebautes "incremental backup". PostgreSQL selbst kann **PITR** (point in time recovery). PostgreSQL verwaltet einen **WAL** (write ahead log), welcher dazu benutzt werden kann, die Datenbank auf einen beliebigen Zeitpunkt zurückzusetzen. Näheres dazu findet man unter [\[https://www.postgresql.org/docs/current/static/continuous-archiving.html\]](https://www.postgresql.org/docs/current/static/continuous-archiving.html)(<https://www.postgresql.org/docs/current/static/continuous-archiving.html>.)

Backup-Skripts

Als Anregung für eigene Backup-Skripts kann die Anleitung unter [\[https://wiki.postgresql.org/wiki/Automated_Backup_on_Windows\]](https://wiki.postgresql.org/wiki/Automated_Backup_on_Windows)(https://wiki.postgresql.org/wiki/Automated_Backup_on_Windows) benutzt werden.

pgBackRest

[\[https://pgbackrest.org/\]](https://pgbackrest.org/)(<https://pgbackrest.org/>) Ein vielversprechendes frei benutzbares Tool eines Drittanbieters für Datensicherung mit vielen Optionen.

VACUUM

Mit VACUUM wird die Operation bezeichnet, mit welcher ungültig gewordene (gelöschte) Speicherbereiche als frei zur Wiederverwendung markiert und in die Free Space Map (FSM) eingetragen werden.

Seit PostgreSQL 8.1 gibt es einen Dienst, **autovacuum**, der die Bereinigung der Datenbanken automatisch ausführt und in der postgresql.conf aktiviert werden muss. Dieser Daemon

- prüft periodisch alle Tabellen
- markiert dead tuples
- trägt wiederverwendbare Speicherbereiche in die FSM ein
- führt das Kommando **ANALYZE** aus

Der **autovacuum-Daemon** ist nicht standardmäßig eingeschaltet. Um ihn zu nutzen, müssen in der postgresql.conf die folgenden Optionen aktiviert sein.

- autovacuum = on
- stats_start_collector = on
- stats_row_level = on

Monitoring

Im **Dashboard von pgAdmin** erhält man einen schnellen Überblick über die Datenbank-Aktivitäten.

Alternativ/zusätzlich erhält man auch über folgende **Abfragen** interessante Informationen:

```
select * from pg_stat_activity
select * from pg_stat_user_tables      // vacuum info
select * from pg_stat_user_indexes    // welche Indizes werden benötigt
select * from pg_locks                // Sperren
```

Ein nützliches Feature ist das **slow query logging**: über die Einstellung **log_min_duration_statement** in **postgresql.conf** kann die automatische Aufzeichnung von Abfragen aktiviert werden, die länger als die angegebenen Millisekunden dauern.

Für einen Datenbank-Administrator ist selbstverständlich die Überwachung der Festplatte äußerst wichtig. Ein **Disk Full Failure** muss unter allen Umständen vermieden werden.

Daten-Ordner verschieben

Quelle: How to Migrate your PostgreSQL Data Directory in Windows <https://radumas.info/blog/tutorial/2016/08/08/Migrating-PostgreSQL-Data-Directory-Windows.html>

Folgende Schritte sollten durchgeführt werden, um unter Windows den Datenordner zu

verschieben:

- Dienst stoppen
- Datenordner kopieren
- NTFS-Berechtigungen ändern:
- Datenordner: "Network Service" (Netzwerkdienst): Vollzugriff
- Elternverzeichnisse: "Network Service" (Netzwerkdienst): Lesen, Ausführen, Ordnerinhalt anzeigen
- Startkommando des Dienstes ändern (hier für Version 10):
- Eingabeaufforderung (Administrator) öffnen und folgende Kommandos ausführen:

```
sc qc postgresql-x64-10
```

Den aufgelisteten Pfad anpassen (für <PGDATA> muss der entsprechende Ordner eingesetzt werden):

```
sc config postgresql-x64-10 binPath="\"C:\Program Files\PostgreSQL\10\bin\pg_ctl.exe\"  
runservice -N \"postgresql-x64-10\" -D \"<PGDATA>\" -w"
```

- Dienste aktualisieren, PostgreSQL-Dienst neu starten

Integration

Einige Informationen zur Integration von PostgreSQL in unser Framework:

PostgreSQLManager

Das **Erstellen**, **Umbenennen**, **Löschen** und **Verdichten** von Datenbanken erfolgt über **SQL-Kommandos**. Dazu wird eine Verbindung zur (System-) Datenbank "postgres" hergestellt. Wegen des Problems der "case sensitiven" Interpretation des Datenbanknamens beim Öffnen von Verbindungen wird hier der Name immer zwischen doppelten Hochkommas angegeben, damit die Groß-/Kleinschreibung beibehalten wird. Andernfalls würde der Name in Kleinbuchstaben umgewandelt und beim Öffnen müsste man ihn dann ebenfalls in Kleinbuchstaben angeben. **Backup/Restore** wird mit Hilfe der Tools **pg_dump** und **pg_restore** durchgeführt. Wie bereits eingangs erwähnt muss hierfür der PATH entsprechend eingestellt sein.

citext

(<https://www.postgresql.org/docs/current/static/citext.html>) Für den "**case insensitive**" Vergleich (Filter und Sortierung) von Zeichenketten wird der Datentyp **citext** benutzt. Dieser wandelt intern die Werte vor dem Vergleich in Kleinbuchstaben um, verhält sich ansonsten aber wie der Standardtyp **text**. Das bedeutet gleichzeitig, dass es keine Längenbeschränkung für String-Felder gibt. Sie werden de facto gleich wie Memo-Felder behandelt.

JDBC URL

```
jdbc:postgresql://<HOST_NAME>:<PORT>/<DB_ALIAS>?stringtype=unspecified&defaultRowFetchSize=10
```

- `stringtype=unspecified` wird benötigt, damit String-Parameter für Felder vom Typ `citext` "case insensitive" behandelt werden. Andernfalls würden sie als `varchar` Typ und somit "case sensitive" behandelt.
- `defaultRowFetchSize=10` wird benötigt, damit Abfrageergebnisse nicht vollständig in den Speicher eingelesen werden. Es ist ein Wert größer 0 erforderlich, unser Standardwert ist 10.

Weitere Informationen

Namenskonvention

- The standard names for indexes in PostgreSQL are: {tablename}_{columnname(s)}_{suffix}

where the suffix is one of the following:

```
pkey for a Primary Key constraint
key for a Unique constraint
excl for an Exclusion constraint
idx for any other kind of index
fkey for a Foreign key
check for a Check constraint
```

- Standard suffix for sequences is

```
seq for all sequences
```

Migration der Datenbank zwischen Versionen

Die Migration von einer PostgreSQL Datenbank eine neuere Version kann mit *pg_upgrade* bewerkstelligt werden. Bei der Migration auf PostgreSQL Version 17 ist es auf Windows notwendig manuell auf der Datenbank die Erweiterung für *pgadmin3* zu deinstallieren.

```
..shell DROP EXTENSION adminpack CASCADE; ..
```

Das Kommando muss auf der master Datenbank *postgresql* ausgeführt werden. Hierbei handelt es sich um einen bekannten Bug (Issue #18635). Mit der Deinstallation lässt sich der Upgrade problemlos durchführen.

Für das Upgrade müssen beide Datenbank Server (Ursprungs- und Zielversion) gestoppt werden. Ausserdem muss darauf geachtet werden, dass der Benutzer zu beiden Ordnern, in welchen die Daten liegen, sowie zum BIN Ordern der neuen Version Zugriffsrechte hat. Das Upgrade wird

immer mit *pg_upgrade.exe* der Ziel Version ausgeführt.

Bevor das Upgrade durchgeführt werden kann muss in der Powershell noch das PostgreSQL Password als Umgebungsvariable definiert werden:

```
..shell $env:PGPASSWORD = "XXXXXXX" ..
```

Anschliessend wird ein "dry run" durchgeführt, um zu verifizieren dass die Migration möglich ist. Als Parameter werden dies Quell- und Ziel_Ordner der Daten sowie der Binär-Ordner angegeben.

```
..shell .\pg_upgrade.exe -r -d 'C:\Develop\pgdata\13\' -D 'C:\Develop\pgdata\17\' -b 'C:\PostgreSQL\13\bin\' -B 'C:\PostgreSQL\17\bin\' -U postgres --check ..
```

Der Check sollte folgendes Resultat liefern.

```
..shell Führe Konsistenzprüfungen durch
```

```
Checking cluster versions                                ok
Checking database user is the install user               ok
Checking database connection settings                    ok
Checking for prepared transactions                       ok
Checking for contrib/isn with bigint-passing mismatch   ok
Checking data type usage                                 ok
Checking for user-defined encoding conversions          ok
Checking for user-defined postfix operators              ok
Checking for incompatible polymorphic functions         ok
Checking for presence of required libraries              ok
Checking database user is the install user               ok
Checking for prepared transactions                       ok
Checking for new cluster tablespace directories         ok
```

```
*Cluster sind kompatibel*
```

```
~~~
```

Nachdem der Trockenlauf korrekt durchgeführt wurde, kann das Kommando ohne die Option ``--check`` ausgeführt werden um die Operation definitiv durchzuführen.

Eclipse Einrichtung (Optional)

Einsatzhinweis

Die Eclipse-IDE die über viele Jahre das Standardwerkzeug der Entwicklungsabteilung war, ist derzeit durch die IntelliJ-IDE abgelöst worden. Dieser Schritt wurde notwendig durch den Wechsel des Build-Systems von Ant auf Maven, die in der Eclipse-IDE zu erheblichen Performance-Problemen geführt hat. Die Eclipse-IDE wird weiterhin für das Projekt "OmbisAndroid" und vom Support zur Bearbeitung von Html-Dokumenten eingesetzt.

Herunterladen und installieren

Die Eclipse IDE kann von der

<https://www.eclipse.org/downloads/packages/>[offiziellen Webseite] heruntergeladen werden.

Es stehen verschiedene Packages zur Verfügung:

- Eclipse IDE for Java Developers:
Paket für Java-Entwickler. Beinhaltet nur die notwendigen Plugins.
- Eclipse IDE for Enterprise Java Developers:
Beinhaltet zusätzliche Plugins für die Web-Entwicklung.

Da man benötigte Plugins auch nachträglich installieren kann, wird das erste Paket empfohlen.

Nach der Installation können über die `_eclipse.ini_` Datei, welche sich im Installationsverzeichnis befindet, die Startparameter angepasst werden. Hier empfiehlt es sich den maximalen Arbeitsspeicher anhand des Parameters `'-Xmx'` zu erhöhen (z.B. `'-Xmx3072m'`).

Plugins installieren

Nachdem das gewünschte Paket heruntergeladen und installiert wurde, müssen die zusätzlichen Plugins installiert werden.

Dazu kann man in `_Window -> Preferences_` unter `_Install/Update -> Available Software Sites_` die Updateseiten der entsprechenden Plugins eintragen.

Folgende Plugins werden von uns genutzt:

- AnyEdit Tool -

<https://raw.githubusercontent.com/iloveeclipse/plugins/latest/>:

Verschiedene Editor Funktionen, wir benötigen hauptsächlich das Konvertieren von Leerzeichen zu Tabs.

- AutoDeriv -

<http://nodj.github.io/AutoDeriv/update>:

Ermöglicht das automatisierte Setzen des `'derived'-Flags`, um generierte Dateien von der Suche u.s.w. auszuschließen.

- Eclipse Encoding - <https://cypher256.github.io/eclipse-encoding-plugin/>:

Zeigt das Encoding der aktuellen Datei an.

- Eclipse External Annotations M2E - <http://www.lastnpe.org/eclipse-external-annotations-m2e-plugin-p2-site/>:

Plugin für die Maven-Integration der Eclipse External Annotations, welche für die erweiterte Null-Analyse benötigt werden.

- Spotbugs -

<https://spotbugs.github.io/eclipse/>:

Analysiert den Code und meldet potentielle Fehler.

- Subclipse -

<https://dl.bintray.com/subclipse/releases/subclipse/latest/>:

Subversion integration für Eclipse.

In den aufgelisteten Features sollten folgende installiert werden:

- Core SVNKit Library: Alle
- Subclipse: Alle außer `_Integration for Mylyn_`
- JavaHL 1.11: Alle
- Maven: Alle
- TestNG - <http://dl.bintray.com/testng-team/testng-eclipse-release/>: Eclipse Plugin für das TestNG testing framework.

Einstellungen

Nachdem alle Plugins installiert und Eclipse neu gestartet wurde, können die Einstellungen angepasst werden.

Folgende Einstellungen sollten vorgenommen werden:

- General -> Editors -> AnyEdit Tools:
 - Im Tab `_Auto - Convert_` das Flag `'Convert tabs <--> spaces'` aktivieren und darunter `'Spaces to Tabs'` auswählen.
- General -> Editors -> Text Editors:
 - Einstellung `'Show Print Margin'` aktivieren und `'Print Margin Column'` auf `'120'` setzen.
- General -> Workspace:
 - Einstellung `'Text file encoding'` auf `'_Other: UTF-8_'` setzen.
- AutoDeriv:
 - Einstellung `'Add markers in .derived file (per-rule info or warn)'` deaktivieren.
- Java -> Code Style -> Code Templates:
 - Über `'Import...'` die Datei `_settings/codetemplates.xml_` importieren und unter `'Comments -> Types'` den Author anpassen.
- Java -> Code Style -> Formatter:
 - Über `'Import...'` die Datei `_settings/formatter.xml_` importieren und Profil `'Pro Data'` auswählen.
- Java -> Code Style -> Organize Imports:
 - Importe in folgender Reihenfolge: `_android_, _com_, _it_, _java_, _javax_, _org_`
 - Number of imports/static imports auf `'50'`.
- Java -> Compiler -> Errors/Warnings:
 - Code style
 - Resource not managed via try-with resource: ****Warning****
 - Method can be static: ****Warning****
 - Potential programming problems
 - Possible accidental boolean assignment: ****Warning****
 - Boxing and unboxing conversions: ****Ignore****
 - Using a char array in string concatenation: ****Warning****
 - 'switch' case fall-through: ****Warning****
 - Dead code: ****Ignore****
 - Resource leak: ****Ignore****
 - Serializable class without serialVersionUID: ****Ignore****
 - Missing synchronized modifier on inherited method: ****Warning****
 - Class overrides 'equals()' but not 'hashCode()': ****Warning****
 - Deprecated and restricted API
 - Forbidden reference: ****Warning****
 - Unnecessary code

- Unnecessary cast or 'instanceof' operation: ****Warning****
- Redundant super interface: ****Warning****
- Generic types
 - Redundant type arguments: ****Warning****
- Annotations
 - Missing '@Override' annotation: ****Warning****
 - Include implementation of interface methods (1.6 or higher): aktiviert
 - Missing '@Deprecated' annotation: ****Warning****
- Null analysis
 - Redundant null check: ****Warning****
 - Enable annotation-based null analysis: aktiviert (Achtung! Popup nach dieser Einstellung mit 'No' quittieren, da es sonst Einstellungen ändert!)
 - Violation of null specification: ****Warning****
 - Conflict between null annotations and null inference: ****Ignore****
 - Unsafe conversion of annotated parameterized type to less-annotated type: ****Ignore****
 - Unchecked conversion from non-annotated type to @NonNull type: ****Ignore****
 - Problems detected by pessimistic analysis for free type variables: ****Ignore****
 - Unsafe '@NonNull' interpretation of free type variable from library: ****Ignore****
 - Redundant null annotation: ****Warning****
 - '@NonNull' parameter not annotated in overriding method: ****Warning****
 - Missing '@NonNullByDefault' annotation on package: ****Ignore****
- Java -> Compiler -> Javadoc:
 - Malformed Javadoc comments: ****Warning****
 - Only consider members as visible as: ****Private****
 - Validate tag arguments: ****Aktiv****
 - Report non visible references: ****Aktiv****
 - Report deprecated references: ****Aktiv****
 - Missing tag description: ****Validate @return tags****
 - Missing Javadoc tags: ****Ignore****
 - Missing Javadoc comments: ****Ignore****
- Java -> Compiler -> Task Tags:

Setzt Tags, welche in der View _Tasks_ angezeigt werden.

Folgende Task-Tags müssen angelegt werden:

 - 'Provisorisch (VN)' -> Mit eigenen Initialien um eigenen Code für spätere Überarbeitung zu markieren.
 - 'Provisorisch (*)' -> für alle, markiert Code der von einem beliebigen Programmieren überarbeitet werden muss (Bsp. die '<=' im Updatercode).
- Maven:
 - Hide folders of physically nested modules: ****Aktiv****
- Java -> Installed JREs:

Zum kompilieren und starten der Projekte sollte nicht das JRE des Systems sondern eigene JREs verwendet werden.

Dadurch ist es möglich die JRE-Version des Systems und jene unserer Projekte unabhängig zu verwalten. Zudem müssen die Projekte in Eclipse oft mit unterschiedlichen JREs getestet werden.

Die zu verwendenden JREs können folgendermaßen hinzugefügt und konfiguriert werden:

 - Mit _Add..._ den Wizard für das Anlegen eines neuen JREs öffnen.

- Eintrag `_Standard VM_` auswählen
- Ordner des JDKs auswählen (z.B. `_C:\Apps\JDKs\jdk1.8.0_232x64_`)
- Namen gegebenenfalls anpassen
- Default VM arguments auf `'-ea -esa'` setzen, wodurch Assertions aktiviert werden.

- Für Java 1.8 JDKs: Libraries unter `_lib\ext_` entfernen.

- Java -> SpotBugs:

- mit `_Restore Defaults_` Standard-Einstellungen setzen.
- Unter `_Detector configuration_` `'FindSqlInjection'` deaktivieren.

- Run/Debug -> String Substitution:

Variablen für die Run/Debug-Konfigurationen. Ermöglichen es die selben Konfigurationen in mehreren Workspaces wiederzuverwenden. Lediglich die Werte der Variablen müssen geändert werden.

- `'ombis_dir'`:

Verzeichnis des Ombis-Projektes dieses Workspaces (z.B. `_C:\JDev1\Ombis_`)

- `'test_dir'`:

Verzeichnis der Test-Installationen (z.B. `_D:\TestDBs_`)

- `'odbc_dir'`:

Verzeichnis, in welchem sich z.B. das `'jspeed.jar'` befindet (z.B. `_C:\jODBC_`)

- `'patch_jar_loc'`:

Pfad zum Patch.jar (z.B. `_C:\jODBC\patch_latest.jar_`), welches bis Java 8 benötigt wird.

Weitere Einstellungen

- Package Explorer:

Damit im `_Package Explorer_` die Ordner der Maven Projekte im Parent-Projekt nicht angezeigt werden, kann der Filter `_Maven Modules_` aktiviert werden.

Dazu die Optionen des `_Package Explorers_` über die 3 Punkte ('⋮') öffnen und den Eintrag `_Filters..._` wählen. In der Liste der Filter kann anschließend der Eintrag `_Maven Modules_` aktiviert werden.

- Problems View:

Da wir die Bundles der Webapplikationen aus dem Projekt `_Web_` in das Jar der jeweiligen Applikation kopieren müssen, warnt uns Maven, dass wir auf Ressourcen außerhalb des eigenen Projektes zugreifen. Da dies in unserem Projekt derzeit nicht anders möglich ist sollten die Warnungen rausgefiltert werden.

Dazu kann in der `_Problems View_` über das Filter-Symbol der Dialog der Filter geöffnet werden. Dort kann ein neuer Filter angelegt werden, in welchem man den Eintrag `'Maven Problems -> Maven Build Participant Problem'` deaktiviert. Anschließend muss dieser Filter als aktiver Filter ausgewählt werden.

NOTE: Dadurch werden die Warnungen lediglich in der `_Problems View_` ausgeblendet, im `_Package Explorer_` und in der betroffenen Datei werden sie weiterhin angezeigt.

Projekt importieren

NOTE: Es empfiehlt sich das Flag `_Build automatically_` zunächst zu deaktivieren.

In `_File -> Import_` den Eintrag `_Maven -> Existing Maven Projects_` auswählen.

Im nächsten Fenster den Ordner der Ombis-Sourcen (z.B. `_C:\JDev1\Ombis_`) auswählen und

alle angezeigten Projekte auswählen. Mit 'finish' bestätigen und warten bis der Import abgeschlossen ist (dauert ca. 20 min., der Fortschrittsbalken verändert isch dabei jedoch nicht).

Anschließend muss in jedem Projekt 'SpotBugs' aktiviert werden. Dazu muss in den _Properties_ unter _SportBugs_ das Flag 'Run automatically' aktiviert werden.

Über folgende Vorgehensweise kann das Ganze bei unserer Anzahl von Projekten relativ schnell erledigt werden:

1. Erstes Projekt markieren
2. Mit 'Alt' + 'Enter' die Properties öffnen.
3. Im Menüpunkt und 'SpotBugs' die Maus über die Checkbox 'Run automatically' bewegen.
4. 'Run automatically' aktivieren und den Dialog mit 'Strg' + 'Enter' schließen.
Die Maus sollte dabei nicht bewegt werden.
5. Mit '↓' das nächste Projekt auswählen und mit 'Alt' + 'Enter' die Properties öffnen.
6. Punkt 4 + 5 für alle Java-Projekte wiederholen.

Nachdem die Projekte importiert und konfiguriert wurden, kann 'Build automatically' wieder aktiviert werden.

Launch Configurations

In _File -> Import_ können über den Eintrag _Run/Debug -> Launch Configurations_ die Launch Configurations aus _settings/launches_ importiert werden.

Wenn die 'String Substitutions' korrekt gesetzt wurden, können sie so bereits verwendet werden und als Vorlage für zusätzliche Launch Configurations dienen.

NOTE: Für Java >= 9 müssen unter _Dependencies>Classpath Entries_ noch zusätzlich die _jakarta_-Jars (_extlib\jakarta*.jar_) hinzugefügt werden, da die darin enthaltenen Klassen nicht mehr Teil des JREs sind.

WARNING: In den Launch Configurations, welche verwendet werden um reale Testinstallationen aufzurufen ****muss**** der Startparameter 'StartDefaultServices' gesetzt sein.

Dieser verhindert, dass die eingerichteten Dienste automatisch beim Programmstart gestartet werden.

Weitere Workspaces einrichten

Um weitere Workspaces einzurichten kann der bereits eingerichtete Workspace als Vorlage verwendet und die Einstellungen in den neuen Workspace kopiert werden.

Dazu kann über _File -> Switch Workspace -> Other..._ ein neuer Workspace angelegt werden.

Im Dialog können unter 'Copy Settings' die Flags 'Workbench layout' und 'Preferences' aktiviert werden. Das Flag 'Working sets' muss nicht aktiviert werden.

Im neuen Workspace muss anschließend das Projekt (siehe oben) und die Launch Configurations (siehe ebenfalls oben) importiert werden und die 'String Substitutions' angepasst (Pfade!) werden.

IntelliJ IDEA Einrichtung

Herunterladen und installieren

Die Community Edition von IntelliJ IDEA kann von der <https://www.jetbrains.com/de-de/idea/download/>#section=windows[offiziellen Webseite] kostenlos heruntergeladen werden.

Nachdem IntelliJ installiert wurde muss es beim ersten Start konfiguriert werden. Dabei wird man zunächst gefragt, ob man anonyme Usage-Statistics zur Verbesserung der IDE senden will.

Anschließend können nicht benötigte Standard-Plugins deaktiviert werden. Dabei kann 'Android', 'Swing Designer' und 'Plugin Development' deaktiviert werden.

Der nächste Punkt ('Featured Plugins') kann übersprungen werden.

Einstellungen

Eine Vorlage der Einstellungen befindet sich unter './Settings'.

Wurde noch kein Projekt geöffnet, so können diese Einstellungen über __Configure -> Import Settings__ importiert werden. Dabei muss lediglich der 'Settings'-Ordner ausgewählt werden und IntelliJ importiert automatisch alle Einstellungen.

Ist bereits ein Projekt geöffnet, können die Einstellungen über __File -> Manage IDE Settings -> Import Settings__ importiert werden.

Die Einstellungen können anschließend über __Configure -> Settings__ bzw. __File -> Settings__ angepasst werden. Dabei müssen zumindest die 'TODO'-Pattern (__Editor -> TODO__) und das entsprechende 'Live Template' (__Editor -> Live Templates -> Java -> xx (Provisorisch)__) angepasst werden.

NOTE: Einstellungen, welche den Code betreffen (Code Style, Inspections etc.) sollten nur nach Rücksprache geändert und auch im './Settings'-Ordner aktualisiert und eingecheckt werden.

Zusätzlich zu den Importierten Settings sollten auch einige VM-Settings angepasst werden:

- Über __Help -> Change Memory Settings__ kann der zur Verfügung stehende Arbeitsspeicher angepasst werden.
- Über __Help -> Edit Custom VM Options__ können zusätzliche VM-Parameter angepasst werden.

Hier sollte mit dem Parameter '-Duser.name' der Benutzername eingestellt werden.

Bsp.: '-Duser.name=Robert Pattis'.

Über '-Dide.usages.page.size' kann die maximale Treffer-Anzahl bei der Suche u.s.w. eingestellt werden (Default: 100).

Um nur Teile der Settings zu importieren müssen diese manuell kopiert werden.

Der Ordner mit den Settings befindet sich unter __%appdata%/jetbrains__, im Ordner __Idea*__ (Der Name hängt von der Version ab).

Die gewünschten Settings können einfach in diesen Ordner oder den entsprechenden Unterordner kopiert werden.

IMPORTANT: Die Settings 'codestyles' und 'inspection' sollten immer aktualisiert werden, damit alle Entwickler die gleichen Einstellungen haben.

Zudem sollte sichergestellt werden, dass das unter __Settings -> Editor -> Code

Style__ und unter __Settings -> Editor -> Inspections__ das Profil 'Default IDE' gewählt ist.

NOTE: Für die 'Inspections' kann gegebenenfalls auch das Profil 'Strict IDE' aktiviert werden, um schärfere Kontrollen durchzuführen (vor allem bei neuen Dateien).

Projekt importieren

Um das Projekt zu Importieren kann 'Open or Import' gewählt werden. Ist bereits ein Projekt geöffnet, kann über __File -> Open__ ein weiteres geöffnet werden.

Im anschließenden Dialog kann der Root-Ordner ('Ombis') ausgewählt werden. Wurde dieser Ordner vorher bereits in Eclipse als Projekt importiert, erscheint die Auswahl zwischen 'Eclipse project' und 'Maven project'. Hier sollte 'Maven project' ausgewählt werden.

IntelliJ beginnt nun mit dem Import des Projektes.

Nach ca. 5 Minuten hängt die IDE kurz und ist anschließend bereit.

Falls nach dem Importieren Maven-Fehler ('Failed to load dependency' oder ähnliches) auftreten, kann über das Aktualisieren-Symbol im Maven-Fenster (rechte Seitenleiste) ein erneutes Laden der Maven-Module forciert werden.

Projekt Einstellungen

Nachdem das Projekt importiert wurde, und der Build-Prozess einmal durchgelaufen ist (eventuell muss bei einer Meldung 'Non-managed pom.xml file found' die Auswahl 'Add as a Maven Project' getätigt werden), kann unter 'File -> Project Structure' das Projekt konfiguriert werden.

Hier muss zunächst das JDK ('Project SDK') hinterlegt und das Language Level konfiguriert werden (sollte dem JDK entsprechen).

Damit die Importe nicht immer händisch organisiert werden müssen, kann unter 'File -> Settings -> Editor -> General -> Auto Import' das Flag 'Optimize imports on the fly' aktiviert werden. Auch diese Einstellung ist Projekt-Abhängig und muss daher für jedes Projekt eingestellt werden.

Unter 'File -> Settings -> Build, Execution, Deployment -> Build Tools -> Maven -> Importing' kann unter 'Automatically download' ausgewählt werden, ob die Sourcen, die Dokumentation und die Annotations von Maven-Bibliotheken automatisch heruntergeladen werden sollen.

Launch Configurations

Die Launch Configurations unter './runConfigurations' können in IntelliJ importiert werden, indem der gesamte Ordner in den '.idea' Ordner des Projektes kopiert wird (z.B. 'Ombis/.idea').

Über __Run -> Edit Configuration__ können die Importierten Konfigurationen angepasst und neue Konfigurationen erstellt werden.

NOTE: Die Folder-Prompt wird beim Ausführen einer Launch Configuration zwei mal angezeigt. Erst das Ergebnis der zweiten Abfrage ist für die Ausführung von Ombis relevant.

Um Vaadin im Entwickler-Modus starten zu können, müssen außerdem die Frontend-Dateien vorbereitet werden.

Dazu kann in der 'Maven' View mittels 'Execute Maven Goal' ('m'-Symbol)

folgendes Kommando ausgeführt werden:

~~~

```
mvn prepare-package -DskipDefaultBuild -DskipWeb -DskipTests -DskipDependencyCheck  
-Pvaadin-dev
```

~~~

Nach der ersten Ausführung werden vorherige Kommandos bereits vorgeschlagen,

es muss also nur einmal eingegeben werden.

Alternativ kann die Run-Configuration "Vaadin prepare-frontend" importiert und genutzt werden.

Arbeiten mit IntelliJ

Errors / Warnings

Anders als in Eclipse werden in IntelliJ nicht alle Errors / Warnings des gesamten Projektes, sondern nur jene der aktuell geöffneten Datei angezeigt. Zudem werden Dateien automatisch gespeichert, aber nicht automatisch kompiliert.

Das automatische Kompilieren kann zwar aktiviert werden (__File → Settings → Build, Execution, Deployment → Compiler -> Build project automatically__) jedoch funktioniert es nur wenn kein Programm gestartet wurde (Run / Debug).

Wenn ein Programm läuft muss das Recompile hingegen manuell gestartet werden. Dafür wurde in den Settings ein Macro hinterlegt (__Save and Recompile__), welches alle Dateien speichert und die Änderungen kompiliert. Das Macro wurde auf die Keys `Strg + S` und `Strg + Shift + S` gelegt.

Durch das Kompilieren werden eventuelle Fehler in anderen Dateien im Build-Fenster angezeigt und IntelliJ springt automatisch zum ersten Fehler.

Vor dem Commit sollten alle geänderten Dateien auf Warnings überprüft werden.

Dazu kann über `Analyze -> Inspect Code` eine Code Analyse gestartet werden. Im folgenden Dialog kann `Uncommitted Files` `(All)` gewählt werden um alle geänderten Dateien zu überprüfen.

Nachdem die Code Analyse fertig ist, wird ein Fenster mit den Fehlern und Warnungen angezeigt. Diese sollten vor dem Commit korrigiert werden. Wurden keine Fehler / Warnungen gefunden, erscheint lediglich eine Notification.

Falls der Commit in IntelliJ ausgeführt wird, wird die Code Analyse automatisch durchgeführt, falls die Checkbox __Perform code analysis__ aktiv ist.

Über die Checkbox __Check TODO__ kann zusätzlich überprüft werden, ob neue bzw. veränderte TODOs / Provisorisch-Anmerkungen vorhanden sind.

Es kann unter Umständen passieren, dass einige Class-Files korrupt sind und IntelliJ unbegründet Fehler meldet. Wenn das der Fall ist kann unter __Build -> Rebuild Project__ das Projekt komplett neu gebaut werden.

In einigen Fällen muss der Cache von IntelliJ geleert und neu erstellt werden. Dazu kann unter __File -> Invalidate Cache / Restart__ die Aktion __Invalidate and Restart__ gewählt werden.

Nach dem Neustart initialisiert IntelliJ die Dateien neu und baut den Cache neu auf. Manchmal ist es auch notwendig die Maven Projekte neu zu laden. Das kann über das Refresh-Icon in der Maven-Leiste (am rechten Rand) gemacht werden.

Es kam auch häufiger vor, dass nach dem Update einer Bibliothek über das __pom.xml__

der Fehler `'package xx does not exist'` auftrat. Diesen kann man mit folgenden Schritten lösen:

1. Unter `'File' -> 'Invalidate caches...'` die Checkboxes `'Clear file system cache and Local History'` und `'Clear downloaded shared indexes'` auswählen und mit `'Invalidate and restart'` bestätigen.
2. `'Maven'` (rechter Bildschirmrand) -> `'Execute Maven Goal'` (kursives `__m__`) -> `'mvn clean'`
3. `'Maven'` -> `'Reload All Maven Projects'` (Refresh Symbol)
4. `'Build'` -> `'Rebuild Project'`

Debugging

IntelliJ erlaubt es bei Breakpoints alle Threads (Default) oder nur den aktuellen Thread zu stoppen.

Um das umzustellen kann über rechtecklick auf den Breakpoint die Einstellung des Breakpoints geöffnet werden.

Hier kann neben der Checkbox `__Suspend__` entweder `__All__` oder `__Thread__` gewählt werden.

Es ist auch möglich Logpoints zu erstellen, welche anstatt eines `System.out.println` verwendet werden können.

Dazu kann in den Breakpointeinstellungen unter `__Evaluate and log__` (eventuell zunächst Checkbox `__Suspend__` deaktivieren) der gewünschte Ausdruck hinterlegt werden.

Der Debugger in IntelliJ wertet standardmäßig die `'toString'` Methode aller Variablen mit jedem Step aus.

Wenn in der `'toString'`-Methode also Variablen verändert werden, verhält sich das Programm beim Debuggen u.U. anders.

Bestes Beispiel ist die Verwendung von `'ResMgr.getCalendar'`. Falls innerhalb der `'toString'` Methode der Calendar verändert wird, beeinflusst dies u.U. den weiteren Programmfluss.

Eine Möglichkeit, dieses Problem zu umgehen ist es, die Einstellung `'enable "toString" object view'` unter `__File -> Settings -> Build, Execution, Deployment -> Debugger -> Data Views -> Java__` zu deaktivieren. Dadurch wird die `'toString'` Methode erst durch Klick in der `'Variables'`-View ausgewertet.

Es ist auch möglich Jar-Files über IntelliJ zu debuggen.

Dazu kann eine neue Run-Configuration vom Typ `__Remote JVM Debug__` erstellt werden. Die Default-Einstellungen können so belassen werden, wenn gewünscht kann jedoch ein anderer `'Port'` vergeben werden (z.B. wenn mehrere Jars gleichzeitig debugt werden sollen).

In der Auswahl `'Use module classpath'` sollte das Root-Projekt `'Ombis'` ausgewählt werden.

Im Feld `'Command line arguments for remote JVM'` werden die Kommandozeilen Argumente angezeigt, welche beim Starten des Jars angegeben werden müssen.

Nachdem das Jar mit den entsprechenden VM-Argumenten (`'Command line arguments for remote JVM'`) gestartet wurde, kann auch die Run-Configuration gestartet werden.

NOTE: Das Skript zum Starten eines unobfuskierten Jars im Debug-Modus (`'startUnobfuscatedJar.cmd'`) findet ihr im Ombis-Projekt, im `'bin'` Ordner.

Sonstiges

Es ist auch möglich externe Tools zu registrieren, welche z.B. über das Kontextmenü aufgerufen werden können.

Dadurch können z.B. Dateien mit Notepad{plus}{plus}, Visual Studio Code oder anderen Programmen geöffnet werden.

Die externen Tools können unter `__File -> Settings -> Tools -> External Tools__` eingerichtet werden.

Für Visual Studio Code und Notepad{plus}{plus} können folgende Einstellungen verwendet werden:

- `'Program'`: Pfad zur *.exe (z.B. `C:\Program Files\Notepad++\notepad++.exe`)
- `'Arguments'`: `'$FilePath$'` (Pfad der aktuellen Datei, welche dann extern geöffnet wird)
- `'Working directory'`: `'$FileDir$'` (Pfad des Parent-Ordners der aktuellen Datei)

Android Studio Einrichtung (Optional)

Installation

Zunächst muss die aktuellste Version von Android Studio heruntergeladen werden.

Diese findet man unter

<https://developer.android.com/studio/index.html>.

Ist bereits **Eclipse mit ADT (Android Development Tools)** installiert, so sollten die neuen SDK-Pakete nicht in den selben SDK-Ordner sondern in einen gesonderten gespeichert werden.

Die optionale Komponente **AndroidVirtualDevice** sollte installiert werden, da es für den Android Emulator benötigt wird.

SDK Manager Pakete

Nachdem Android Studio installiert wurde, müssen die notwendigen SDKs installiert werden. Dazu kann man Android Studio öffnen und über `'Configure'` den `'SDK Manager'` öffnen.

Im **SDK Manager** können unter **SDK Platforms** nun die gewünschten Android Versionen installiert werden.

Dabei muss mindestens Android 8.0 (API 26) installiert werden, da dieses die aktuelle Target-API ist.

Unter **SDK Tools** können zusätzlich Tools, wie der Emulator, Dokumentationen u.s.w. installiert werden.

Hier sollten folgende Tools installiert werden:

- Android Emulator
- Android SDK Platform-Tools
- Android SDK Tools
- Documentation for Android SDK
- Google USB Driver
- Intel x86 Emulator Accelerator
- Support Repository

Einige der Pakete sind möglicherweise bereits vorinstalliert.

Settings

Die Einstellungen können in Android Studio unter **File -> Settings** angepasst werden. Auf dem Startbildschirm Android Studios können die Einstellungen unter **Configure -> Settings** erreicht werden. Hier können z.B. Shortcuts angepasst, Templates erstellt und der Code-Style verändert werden.

Eine Vorlage der Settings, welche z.B. die gewohnten Eclipse-Shortcuts, Settings für den Code-Style u.s.w enthält befindet sich unter **Settings** im Ordner dieses Dokumentes.

Um diese Settings zu importieren, muss der gesamte Inhalt in eine ZIP-Datei verpackt werden. Anschließend kann sie in Android Studio unter **File -> Manage IDE Settings -> Import Settings...** ausgewählt und importiert werden. Die Einstellungsmöglichkeiten im Import-Wizard müssen dabei nicht verändert werden.

Anschließend müssen lediglich die Templates für die **Provisorisch**-Anmerkung (z.B. **_Provisorisch (RP)_**) angepasst werden:

- Unter **Editor -> TODO** das Pattern **Provisorisch \(\)** anpassen (z.B. **Provisorisch \(\RP\)**)
- Unter **Editor -> Live Templates** im Eintrag **Android** das Template **xx** anpassen.
- Unter **Editor -> Live Templates** im Eintrag **AndroidComments** das Template **xx** anpassen.
- Unter **Editor -> File and Code Templates**
 - Files: **Java File Header** anpassen
 - Includes: **File Header** und **Types** anpassen

Anschließend sollte Android Studio neu gestartet werden.

Falls Änderungen an den Settings vorgenommen werden, welche für alle Entwickler nützlich sein könnten (z.B. Ergänzung der Shortcuts) sollten diese auch im Repository aktualisiert werden.

Dazu können die Settings über **File -> Export Settings** exportiert und die veränderte(n) Datei(en) aus dem exportierten JAR in den entsprechenden Ordner im Repository kopiert werden.

Projekt OmbisMobile einrichten

Das Projekt **OmbisMobile**, welches unter **svr-develop/develop/OmbisMobile/trunk** zu finden ist, muss zunächst über SVN ausgecheckt werden.

Um das Projekt in Android Studio zu öffnen kann **Open an existing Android Studio project** im Startfenster gewählt werden.

Nachdem der Ordner ausgewählt wurde öffnet Android Studio das Projekt.

Anschließend können die Einstellungen unter **File > Project Structure** überprüft werden. Dabei sollte bei allen Modulen **API 26** als **Compile Sdk Version** eingestellt sein. Zudem sollte **Source Compatibility** und **Target Compatibility** auf 1.8 eingestellt sein.

Nachdem diese Einstellungen abgeschlossen sind sollte ein **Gradle Sync** gestartet werden (**File -> Sync Project with Gradle Files**).

Emulator einrichten

Falls die Apps über einen Android Emulator getestet werden sollen, muss zunächst ein **Android Virtual Device** (AVD) eingerichtet werden.

Dazu muss man den **Android Virtual Device Manager** über **Tools->Android->AVD Manager** öffnen. Hier können neue AVDs angelegt bzw. bestehende bearbeitet oder gelöscht werden.

Um ein neues AVD zu erstellen muss zunächst das Gerät ausgewählt werden. In Android Studio stehen dazu zahlreiche reelle Geräte (wie z.B: Nexus 9) sowie generische Geräte (z.B: 7" WSVGA) zur Verfügung. Zudem ist es möglich eigene Hardware Profile zu erstellen bzw. zu importieren.

Als nächstes kann ein System Image gewählt werden. Dabei sollte eines mit x86-Architektur verwendet werden, da Emulatoren mit dieser Architektur deutlich performanter sind.

Um diese allerdings überhaupt erst starten zu können muss

- ein Intel-Prozessor verwendet werden.
- im BIOS die "Virtualization Technology (VTx/VTd)" aktiviert werden.
 - Da das BIOS je nach PC unterschiedlich erreichbar ist (z.B. Betätigen der ESC-Taste während des Bootvorgangs) und auch die Einstellung leicht verändert benannt werden könnte, sollte zunächst nachgeforscht bzw. nachgefragt werden, wie man das BIOS erreicht und wo die Virtualisierung aktiviert werden kann.
 - Für die HP Z230 Workstation PCs muss z.B. im BIOS unter Security -> System Security -> **Virtualization Technology (VTx/VTd)** auf Enabled gestellt werden.
 - Wurde die Virtualisierung erfolgreich aktiviert sollte nun im Windows-Task-Manager unter dem Tab **Leistung**, der Punkt Virtualisierung als aktiviert aufscheinen.
- der **Intel Hardware Accelerated Execution Manager (HAXM)** (Link: <https://software.intel.com/en-us/android/articles/intel-hardware-accelerated-execution-manager>)(<https://software.intel.com/en-us/android/articles/intel-hardware-accelerated-execution-manager>)) installiert werden.
- das Windows-Feature **Hyper-V** deaktiviert werden.

Im nächsten Schritt erhält man eine Übersicht über das AVD und kann zusätzliche Einstellungen anpassen. Um z.B. das Hardware Keyboard zu verwenden kann man unter **Advanced Settings** das Flag **Enable keyboard input** setzen.

Anschließend kann das AVD gestartet und verwendet werden.

WARNING: Das AVD muss bei Code-Änderungen nicht neu gestartet werden, es reicht die Launch-Configuration bzw. **Instant Run** erneut zu starten.

Android Gerät einrichten

Falls über ein Android Gerät getestet werden soll muss auf diesem zunächst USB-Debugging aktiviert werden. Dies kann man unter der Entwickleroptionen aktivieren. Falls die Entwickleroptionen noch versteckt sind, kann man diese aktivieren, indem man unter **Geräteinformationen** bzw. unter **Über das Telefon/Tablet** mehrfach auf die **BauNummer** klickt.

Sobald USB-Debugging eingeschaltet ist kann man das Gerät über USB mit dem PC verbinden. Die Abfrage, ob USB-Debugging von diesem PC zugelassen werden soll kann man bestätigen.

Launch Configuration

Die Launch Configuration für die App-Module sollten bereits automatisch erstellt worden sein.

Falls dies nicht der Fall ist, kann über **Edit Configurations** neben den Run- bzw.

Debug-Buttons in der Toolbar eine neue Android Run/Debug Configuration erstellt werden.

Hier muss zunächst das Modul (z.B. ****Stock****) angegeben werden. Zudem kann ausgewählt werden, ob ein USB-Device oder ein AVD verwendet werden soll.

Testing

In Android stehen mehrere Arten von Tests zur Verfügung:

- Unit Tests
- Instrumented Tests

Unit Tests

Bei den Unit Tests handelt es sich um normale JUnit Tests, welche einzelne Logik-Einheiten, unabhängig von UI oder dem Android Framework, testen sollen.

Sie werden typischerweise in der lokalen JVM ausgeführt.

Instrumented Tests

Die Instrumented Tests hingegen werden im Emulator ausgeführt und testen eine installierte APK. Sie können verwendet werden um Logik , welche vom Android-Framework abhäng (z.B. einen Context benötigt), oder die UI zu testen.

Da das Testen von Benutzer-Interaktionen sehr aufwendig sein kann, bietet Android Studio die Möglichkeit Tests aufzuzeichnen. Die Aufzeichnung kann unter ****Run -> Record Espresso Test**** gestartet werden.

Nachdem der Test aufgezeichnet wurde kann die Klasse ausgewählt werden, in welche die Tests gespeichert werden.

Diese Klasse kann anschließend bearbeitet werden um die Tests anzupassen und zu vervollständigen.

Mocking

Da die Tests in einer undefinierten Umgebung laufen, müssen Dinge wie z.B. Http-Requests gemockt werden.

Im Projekt OmbisMobile werden Klassen wie z.B. der ****RestClient**** daher nicht über den Konstruktor erzeugt, sondern über den globalen ****Provider**** geholt:

~~~

```
Provider.get(RestClient.class)
```

~~~

Um den RestClient in den Tests zu mocken, muss dem Provider mitgeteilt werden, dass eine andere Klasse verwendet werden soll:

~~~

```
Provider.provide(RestClient.class, MockRestclient.class)
```

~~~

WARNING: Der Provider im OmbisMobile-Projekt wurde selbst geschrieben, um die Tests schnellstmöglich zum Laufen zu bringen und die Abhängigkeiten im zu testenden Code zu reduzieren.

Es kann daher sein, dass sich die hier beschriebene API in Zukunft ändert, oder sogar auf eine Bibliothek wie ****Dagger 2**** ausgewichen wird.

APK erstellen:

****TODO****

Visual Studio Code Einrichtung (Optional)

Installation

Die Installationsdatei für Visual Studio Code befindet sich auf dem Server unter `_Software\Microsoft\VSCode_`. Alternativ kann die aktuellste Version unter [\[https://code.visualstudio.com\]\(https://code.visualstudio.com\)](https://code.visualstudio.com) heruntergeladen werden. Visual Studio Code existiert als Stable und Insider Version. Beide können parallel und unabhängig voneinander installiert werden.

Die Insider-Version bekommt fast täglich Updates, wodurch neue Features früher genutzt werden können. Allerdings kommt es in seltenen Fällen zu fehlerhaften Versionen.

Die Stable-Version hingegen wird seltener aktualisiert, neue Features sind erst später verfügbar. Dafür ist diese Version sehr stabil, da alle Features bereits in den Insider-Versionen getestet wurden.

TIP: Da beide Versionen unabhängig voneinander installiert werden können, empfehle ich im Normalfall mit der Insider-Version zu arbeiten. Bei eventuellen Problemen kann auf die Stable-Version gewechselt werden.

Erweiterungen

Die Funktionalität von Visual Studio Code kann durch das installieren von Plug-ins erweitert werden. Um Erweiterungen zu Installieren und zu Verwalten, kann die Erweiterungs-Ansicht geöffnet werden. Diese befindet sich in der linken Seiten-Leiste (das letzte Symbol der oberen Gruppe). Ansonsten lässt sie sich über ****Anzeigen -> Extensions**** öffnen.

Folgende Erweiterungen sind unter anderem für die Entwicklung von Webseiten mit Angular nützlich:

- ****Angular Language Service**** (Angular):
Plug-In, welches Code-Completion, Validierungen usw. für Angular-Syntax in HTML-Files zur Verfügung stellt.
- ****angular2-inline**** (Nate Wallace):
Syntax highlighting von inline Templates und Styles.
- ****Eclipse Keymap**** (Alphabot Security):
Ersetzt einige Shortcuts durch jene von Eclipse.
- ****ESLint**** (Dirk Baeumer):
Plug-in, welches die Verwendung von ESLint ermöglicht. Dazu gehören Validierungen des Code-Styles und zugehörige Quick-Fixes.
- ****goto-last-edit-location**** (Christian Oetterli):
Ermöglicht es anhand eines Shortcuts zur letzten Änderung zu springen.
- ****TODO Highlight**** (Wayou Liu):
Ermöglicht die Suche und das Hervorheben von (selbst definierten) TODO-Tasks, ähnlich der `_Provisorisch_`-Anmerkung, welche wir in Java verwenden.
- ****vscode-icons**** (Roberto Huertas):
Icon-Paket für Visual Studio Code, welches verschiedenen Dateien (z.B. Angular Komponenten, Angular Services, package.json usw.) eigene Icons zuweist.
- ****German Language Pack for Visual Studio Code**** (Microsoft):
Das deutsche Sprackpaket bietet eine lokalisierte Benutzeroberfläche für Visual

Studio Code.

Settings

Die Einstellungen können in Visual Studio Code unter **Datei -> Einstellungen** angepasst werden. Hier können z.B. Shortcuts angepasst, Templates erstellt oder Einstellungen der verschiedenen Erweiterungen verändert werden.

Eine Vorlage der Settings, welche z.B. die gewohnten Eclipse-Shortcuts, einige Code-Templates u.s.w enthält befindet sich unter **Settings** im Ordner dieses Dokumentes.

NOTE: Falls Änderungen an den Settings vorgenommen werden, welche für alle Entwickler nützlich sein könnten (z.B. Ergänzung der Shortcuts) sollten diese auch im Repository aktualisiert werden.

Einstellungen

Im Untermenü **Einstellungen** befinden sich allgemeine Einstellungen des Editors und der Erweiterungen.

Es wird hier unterschieden zwischen Benutzereinstellungen und Arbeitsbereicheinstellungen, welche vom geöffneten Projekt abhängig sind.

Um die Einstellungen zu bearbeiten kann entweder der entsprechende Eintrag über den Bleistift am linken Rand bearbeitet werden, oder die JSON-Datei auf der rechten Seite angepasst werden.

Um die Vorlage (`_settings.json`) zu importieren, kann der Inhalt der JSON-Ansicht einfach mit jenem der Datei überschrieben werden.

Die JSON-Ansicht kann man mithilfe des Symbols *****{ }***** rechts oben geöffnet werden. Angepasst werden muss in dieser Datei lediglich die Einstellung `_todohighlight.keywords_`, welche das "Provisorisch"-Tag enthält.

Tastenkombinationen

Hier können die Tastenkombinationen für nahezu alle Kommandos angepasst werden. In der Suchleiste kann nach Kommandos (z.B. "Datei öffnen") oder nach Tastenkombinationen (z.B. "strg+shift+r") gesucht werden. Um die Tastenkombination eines Kommandos zu ändern kann auf den Bleistift am linken Rand geklickt werden, um eine Tastenkombination zu entfernen kann im Kontextmenü `_Tastenkombination entfernen_` gewählt werden.

Alternativ kann auch hier wieder die zugehörige JSON-Datei angepasst werden, welche man über das Symbols *****{ }***** rechts oben öffnen kann.

Dadurch öffnen sich zwei Editoren, wobei der linke die Standard-Einstellungen und der Rechte die angepassten Einstellungen anzeigt. In den rechten Editor kann der Inhalt der Datei `_keybindings.json` eingefügt werden, falls die Vorlage angewandt werden soll.

Benutzercodeausschnitte

Über die Benutzercodeausschnitte können Completion-Templates für verschiedene Sprachen festgelegt werden. Dazu wählt man im Dialog zunächst eine Sprache (z.B. `_TypeScript_`), wodurch eine Datei (z.B. `_typescript.json`) erstellt wird.

In diese Datei können anschließend eigene Code-Snippets als JSON definiert werden.

Die Vorlagen können wieder einfach übernommen werden, indem der Inhalt der entsprechenden Dateien in den Editor kopiert werden.

Folgende Vorlagen stehen zur Verfügung:

- TypeScript: `_typescript.json`

- HTML: `_html.json_`

Projekt "Web" einrichten

Die Web-Projekte befinden sich im Ordner `**web**` im Ombis-Projekt. Um das Projekt zu öffnen kann in Visual Studio Code der Ordner über `**Datei -> Ordner öffnen**` geöffnet werden.

Falls die Bibliotheken (`_node_modules_`) noch nicht installiert wurden, sollten diese im Terminal (`**strg+ö**`) über den Befehl `**npm install**` installiert werden. Wichtig ist, dass das Kommando im Root-Verzeichnis des Projektes (`_Ombis\web_`) ausgeführt wird.

Anschließend sollte die TypeScript-Version auf die im Arbeitsbereich installierte Version gesetzt werden. Dazu kann eine beliebige TypeScript-Datei im Arbeitsbereich geöffnet werden. In der Fuß-Leiste sollten auf der rechten Seite nun `_TypeScript_` und die Version (z.B. `_2.6.2_`) angezeigt werden. Durch einen Klick auf die Version öffnet sich ein Auswahl-Dialog, in welchem `**Use Workspace Version**` ausgewählt werden sollte.

Anschließend können die Apps über die Angular-CLI Kommandos ausgeführt (`_ng serve_`), gebuildet (`_ng build_`) und getestet werden (`_ng test_`):

WARNING: Um die Angular-CLI Kommandos ausführen zu können, sollte Angular CLI global installiert werden. Dazu kann einfach `**npm install -g @angular/cli**` in der Kommandozeile ausgeführt werden.

Tasks

Ähnlich den Run-/Debug-Konfigurationen in Eclipse, können in Visual Studio Code die `**Tasks**` verwaltet werden. Einige Tasks werden von Visual Studio Code automatisch erkannt (z.B. npm-Scripts), es ist aber möglich eigene Tasks zu definieren.

Um eigene Tasks zu definieren kann über `**Aufgaben -> Aufgaben konfigurieren**` eine Vorlage (z.B. `_npm start_`) ausgewählt werden. Es wird dadurch die Datei `_tasks.json_` mit der Vorlage für das npm-Script "start" erstellt. Diese Vorlage kann mit den eigenen Tasks ersetzt werden.

Ein Task besteht aus einem Label (`_label_`), welcher den Namen des Tasks definiert, einem Typ (`_type_`), welcher den Typ (z.B. Npm, Shell u.s.w) definiert, einer Gruppe (Test, Build oder None) und einer Liste von Problem-Matchern, welche den Output des Tasks verarbeiten können. Zudem stehen, abhängig vom Typ, zusätzliche Eigenschaften zur Verfügung.

Im Falle von Shell-Scripts kann über die Eigenschaft `_command_` das auszuführende Kommando (z.B. `_ng serve_`) angegeben werden.

Eine Vorlage mit Tasks für das Ausführen, Builden und Testen der Web-Projekte befindet sich unter `**Settings**` im Ordner dieses Dokumentes (`_tasks.json_`).

Diese Vorlage beinhaltet unter anderem den Task `**Lint all**`, welcher das gesamte Projekt mit ESLint überprüft und Fehler/Warnungen in der Problems-View anzeigt.

Über den Task `**Compile all**` werden alle TypeScript-Dateien im Projekt kompiliert und eventuelle Probleme in der Problems-View angezeigt. Dies ist sehr nützlich, wenn das gesamte Projekt (z.B. nach einem Refactoring) geprüft werden soll.

Zudem befinden sich in der Vorlage einige Tasks, welche zusätzliche Parameter über die Kommandozeile einlesen, um z.B. das zu startende Projekt zu definieren.

Proxy

Während der Entwicklung wird die Webseite vom Webpack-Server, welcher über `_ng serve_` gestartet wird, bereitgestellt. Die Daten und einige Konfigurationsdateien müssen aber

vom Ombis-Server geholt werden. Der Webpack-Server ermöglicht es, Proxies zu konfigurieren, wodurch es möglich ist, Requests auf gewisse URLs weiterzuleiten. Dafür muss zunächst eine Konfigurations-Datei (`_proxy.conf.json`) im Root-Ordner der Web-Projekte angelegt werden. Diese wurde bereits in die Ignorier-Liste von SVN eingetragen und wird daher nicht eingchecked. Um diese Proxy-Konfiguration zu verwenden, muss das Kommando `_ng serve_` um den Parameter `_proxy-config_` erweitert werden (z.B. `_ng serve --project agent-web --host 10.150.242.83 --proxy-config proxy.conf.json_`).

Im Ordner dieses Dokumentes befindet sich unter **Settings** eine Vorlage der Proxy-Konfiguration für Web-Portale (`_proxy.conf.json`) sowie eine weitere Vorlage für Web-Clients (`_proxy.conf.webclient.json`). Die Vorlage für Web-Clients fügt zu einigen Pfaden den Präfix ``/webclient/demo/auswertung`` hinzu und ist damit für einen Betrieb ``demo`` und App ``auswertung`` konfiguriert. Um andere Apps zu testen, muss überall ``auswertung`` mit dem entsprechenden Verzeichnisnamen ersetzt werden. Analog dazu muss bei anderen Betrieben der Code ausgetauscht werden.

Tests

Für Unit Tests wird Jest verwendet. Sie lassen sich über die Kommandozeile ausführen:

```
~~~
npx ng test <project>
~~~
```

Für weitere Parameter siehe [<https://jestjs.io/docs/cli>](https://jestjs.io/docs/cli.) Außerdem befindet sich unter **Settings** eine Vorlage für `_launch.json`, welche das Starten der Tests von VS Code aus ermöglicht.

Einrichtung Angular Projekt

Angular-CLI

Für die Entwicklung und das Deployment der Angular Projekte wird **Angular-CLI** (siehe [<https://cli.angular.io>](https://cli.angular.io)) verwendet. Um Angular-CLI verwenden zu können muss das zugehörige NodeJs-Package installiert werden. Dafür kann das Kommando ``npm install -g @angular/cli`` ausgeführt werden.

WARNING: Um ``npm`` ausführen zu können muss NodeJs und der Node Package Manager (npm) installiert sein. Dies sollte aber bereits bei allen der Fall sein.

Aufbau eines Projektes

Ein Angular-CLI Projekt besteht aus folgenden Kern-Dateien und Ordnern:

- **angular.json**: Konfigurationsdatei für Angular-CLI. Hier werden die verschiedenen Apps dieses Projektes registriert, die E2E- und Unit-

Tests konfiguriert und verschiedene Default-Werte festgelegt.

Eine genaue Beschreibung dieser Datei und der einzelnen Properties findet sich unter [\[https://angular.io/guide/workspace-config\]](https://angular.io/guide/workspace-config)(<https://angular.io/guide/workspace-config>)

- ****package.json****:

Konfigurations-Datei für 'npm'. Definiert die verwendeten Bibliotheken (inkl. Version), sowie verschiedene Scripts.

- ****package-lock.json****:

Automatisch generierte Datei, welche die verwendeten Bibliotheken und deren Versionen "sperrt". Wird verwendet um reproduzierbare Builds zu garantieren.

- ****tsconfig.json****:

TypeScript-Konfigurationsdatei für die IDE-Integration. Diese Datei wird nicht vom Angular-CLI verwendet, das Build wird also nicht beeinflusst.

Lediglich das Verhalten des TypeScript-Plugins der IDE kann dadurch gesteuert werden.

- ****eslinttrc.json****:

Konfigurationsdatei für ESLint.

Hier können verschiedene Formatierungsrichtlinien festgelegt werden (z.B. Indent mit Spaces oder Tabs, Double- oder Single-Quotes etc.), deren Einhaltung anschließend validiert wird.

- ****nx.json****:

Konfigurationsdatei der Angular-CLI Erweiterung 'nrwl/nx'.

- ****node_modules**** (Ordner):

Ordner mit den installierten npm-Packages.

Diese werden über 'npm install' installiert/aktualisiert.

- ****apps**** (Ordner):

Die verschiedenen Applikationen dieses Projektes.

- ****libs**** (Ordner):

Die eigenen Bibliotheken, welche gemeinsamen Code mehrere Applikationen beinhalten.

- ****tools**** (Ordner):

Ordner mit verschiedenen Tools für das Projekt.

Beinhaltet verschiedene Build-Scripts, sowie die Schematics, welche das Erstellen neuer Komponenten u.s.w. erleichtern.

Die Ordner der einzelnen Applikationen enthalten folgende Dateien und Ordner:

- ****eslinttrc.json****:

Angepasste ESLint-Konfiguration für diese Applikation. Erbt meist die Konfiguration vom Projekt.

- ****tsconfig.app.json**/**tsconfig.spec.json****: Angepasste TypeScript-Konfiguration für die Applikation bzw. für die Unit-Tests.

- ****karma.conf.json****: Konfiguration für Karma (Unit-Tests)

- ****src**** (Ordner):

Ordner mit den Dateien für die Web-Applikation.

Beinhaltet folgende Dateien/Ordner:

- ****app**** (Ordner): Die Dateien für die konkrete Angular-Applikation (Komponenten, Services u.s.w.)

- ****assets**** (Ordner): Assets wie Bilder, Übersetzungen u.s.w. Diese werden beim Build unverändert in den entsprechenden dist-Ordner kopiert.

- ****environments**** (Ordner):

Ordner mit den verschiedenen Umgebungen/Konfigurationen dieser Applikation.

Beinhaltet typischerweise eine `_environment.ts`-Datei, welche im Build durch die entsprechende Konfigurations-Abhängigkeit `_environment.*.ts`-Datei ersetzt wird.

- `**styles**` (Ordner) (Optional):

Ordner für globale, Komponenten-übergreifende Styles. Häufig wird auch eine einzelne Datei anstatt eines ganzen Ordners verwendet.

- `**index.html**`: Index-Datei, welche die benötigten Scripts lädt und die Angular-Applikation anzeigt.

- `**main.ts**`: TypeScript-Datei, welche die Angular-Applikation startet.

- `**polyfills.ts**`: Polyfills-Datei, welche neuere JavaScript-Funktionen für ältere Browser zur Verfügung stellt.

- `**test.ts**`: TypeScript-Datei, welche die Unit-Tests dieser Applikation ausführt.

Erstellung eines neuen Projektes

1. Projekt anlegen:

```
~~~  
ng g application [Projektname]  
~~~
```

Während der Installation folgende Eingaben machen:

```
~~~  
Which stylesheet format would you like to use?  
  SASS(.scss) [ http://sass-lang.com ]  
Would you like to configure routing for this application?  
  Yes  
~~~
```

1. Projektordner im Format "[Projektname]-e2e" löschen

2. Anpassungen in Projekt-Ordner

- ``.browserslistrc`` löschen
- ``.eslintrc.json`` ändern (siehe vorhandene Projekte)
- Copyright-Kommentar in ``src/main.ts`` einfügen
- ``src/polyfills.ts``: in neuen Projekten muss nur die Zeile ``/** eslint-disable */`` eingefügt werden. Siehe Projekt `**service-web**`.
- Datei ``.gitkeep`` unter ``src/assets`` löschen
- Ordner ``src/app/i18n`` erstellen. In diesem Ordner die Sprachdateien anlegen (bspw. ``de.json``, ``it.json``)
- Ordner ``src/assets/images`` erstellen, wo Bilder eingefügt werden können.
- Ordner ``src/styles`` anlegen und dort folgende Dateien von vorhandenen Projekt übernehmen: ``_variables.scss``, ``libs.scss``. Datei ``theme.scss`` neu erstellen. Datei ``styles.scss`` von ``src/styles.scss`` nach ``src/styles/styles.scss`` verschieben.

3. Anpassungen in Hauptordner "apps"

- ``angular.json`` ändern
 1. SVN-Änderungen rückgängig machen
 2. Vorhandenen Projekt-Knoten kopieren (wegen den korrekten Einstellungen)

beachten ob Projekt ein PWA-Projekt ist) und Projektnamen mit neuen Projektnamen ersetzen

3. In allen "styles"-Knoten des Projektes der 'angular.json' müssen folgende Dateien eingebunden sein: 'styles.scss', 'theme.scss', 'libs.scss'.

- SVN-Änderungen in 'package.json' und '.eslintrc.json' rückgängig machen

4. Service worker einrichten (optional)

Für Projekte welche Service worker verwenden müssen folgende Schritte befolgt werden:

- Befehl 'ng add @angular/pwa --project **Projektname**' ausführen, damit Service worker eingerichtet werden
- Datei 'manifest.webmanifest' wie in anderen Projekten anpassen
- Datei 'ngsw-config.json' wie in anderen Projekten anpassen
- Die App-Farbe wird von den Browsern von zwei unterschiedlichen Orten gelesen:
 1. von der Property 'theme_color' in der Datei 'manifest.webmanifest'
 2. vom Meta-Tag 'theme-color' in der Datei 'index.html'

Code-Konventionen

Subscription-Management

Ein wichtiger Bestandteil von Angular ist RxJS. RxJS wird z.B. für Http-Abfragen, Events von Formularen u.s.w. verwendet.

Dabei ist es wichtig die sogenannten Subscriptions korrekt freizugeben.

Typischerweise merkt man sich dafür die Subscription und ruft 'unsubscribe' auf, sobald sie nicht mehr benötigt wird.

RxJS bietet dabei auch die Möglichkeit Subscriptions anhand der 'add'-Methode zu verknüpfen. Dadurch können beliebig viele Subscriptions über eine einzelne verwaltet werden.

RxJS bietet zudem Operatoren, über welche das Ganze deklarativ gelöst werden kann:

- 'first': Liefert den ersten Wert und beendet das 'Observable' mit 'complete'.
- 'first(predicate)': Liefert den ersten Wert, welcher dem 'predicate' entspricht und beendet das 'Observable' mit 'complete'.
- 'take(n)': Liefert die ersten 'n' Werte und beendet das 'Observable' mit 'complete'.
- 'takeWhile(predicate)': Liefert alle Werte, bis das angegebene 'predicate' false liefert und beendet das 'Observable' mit 'complete'.
- 'takeUntil(notifier)': Liefert alle Werte, bis der angegebene 'notifier' (ein 'Observable') benachrichtigt wird und beendet das 'Observable' mit 'complete'. +
!!n

Quellen:

[<https://medium.com/>](<https://medium.com/@xtianjohns/managing-subscriptions-in-angular-8c18ef992f3f>)[Managing Subscriptions in Angular]

<https://blog.angularindepth.com/the-best-way-to-unsubscribe-rxjs-observable-in-the-angular-applications-d8f9aa42f6a0>[The Best Way To Unsubscribe RxJS Observables In The Angular Applications]

!!

Services

Generell gilt, dass komplexe Logik und RestAbfragen in Services ausgelagert werden sollen.

Die Komponenten selbst sollten sich lediglich um die Darstellung der Daten kümmern.

Services, welche von vielen, voneinander unabhängigen Komponenten verwendet werden können, sollen dabei in `core` platziert werden. Services für einen bestimmten Teilbereich einer Applikation sollten im Modul dieses Teilbereiches erstellt werden, Services welche nur für eine einzige Komponente verwendet werden, können auch im Ordner der entsprechenden Komponente erstellt werden.

Verwendung von Arrow-Funktionen

Generell gilt, dass innerhalb von Arrow-Funktionen lokale Variablen nicht verändert werden sollen. Bei asynchronem Code (z.B. HTTP-Abfragen) kann das zu schwer nachvollziehbaren Fehlern führen, weshalb dieses Pattern prinzipiell vermieden werden soll.

Im Falle von `Arrays` wird `forEach` häufig falsch verwendet, um z.B. die Werte zu summieren. Für die meisten Fälle gibt es dabei eine passende Array-Funktion (z.B. `reduce`), andernfalls sollten hier normale `for`-Schleifen bevorzugt werden.

Beispiele:

~~~javascript

```
const arr = [ 1, 2, 3, 4 ];
```

```
// Summieren
```

```
// Vorher
```

```
let sum = 0;
```

```
arr.forEach(num => sum += num);
```

```
// Nachher
```

```
const sum = arr.reduce((s, num) => s + num, 0)
```

```
// =====
```

```
// Filtern
```

```
// Vorher
```

```
let result = [];
```

```
arr.forEach(num => {
```

```
    if (num > 2)
```

```
        filtered.push(num);
```

```
});
```

```
// Nachher
```

```
const result = arr.filter(num => num > 2);
```

```
~~~
```

## ## Einrichtung Android Projekt (Optional)

### ### Eclipse einrichten

Zunächst muss das **Android SDK** heruntergeladen und installiert werden. Der dazugehörige Windows-Installer wird nicht mehr aktiv weiterentwickelt, weshalb die letzte verfügbare Version r24.4.1 verwendet werden muss (Link: [\[https://dl.google.com/android/installer\\_r24.4.1-](https://dl.google.com/android/installer_r24.4.1-)

windows.exe](https://dl.google.com/android/installer\_r24.4.1-windows.exe) ). Mit dieser ist es aber auch möglich, neuere Android Plattformen auszuwählen.

WARNING: Die letzte unterstützte Eclipse-Version ist **Eclipse 2020.06**. Mit neueren Versionen kompiliert die APP nicht mehr, da diese zwingend Java 11 voraussetzen.

Anschließend muss man das **ADT-Plugin** für Eclipse installieren. Dazu geht man in Eclipse im Tab "Help" auf **Install new Software**. Dort kann man über die URL [https://dl-ssl.google.com/android/eclipse/](https://dl-ssl.google.com/android/eclipse/) die **Developer Tools** herunterladen und installieren.

Nachdem man Eclipse neugestartet hat erscheint ein Dialog zur Einrichtung von ADT. Hier muss man nun das Installationsverzeichnis des **Android SDK** angeben.

### ### SDK Manager Pakete

In Eclipse über **Window->Android SDK Manager** den SDK Manager öffnen. Über diesen kann man die gewünschten Pakete herunterladen bzw. aktualisieren.

Installiert werden müssen die jeweils neueste Version der **SDK Tools** und **SDK Platform Tools**, sowie die Version Rev. 25.0.2 der **SDK Build Tools**.

Zudem müssen die gewünschten Android-Versionen ausgewählt werden. Da die App nur mit Android **Version 4.4.2 (API 19)** oder höher kompatibel ist, muss keine ältere Version installiert werden.

Nicht benötigte Pakete der Android-Versionen können ausgeschlossen werden. Dazu zählen z.B. Android Wear, Android TV oder das Glass Development Kit.

### ### Projekt einrichten

Das Android Projekt, welches unter **svr-develop/develop/OmbisAndroid/trunk** zu finden ist, muss zunächst über SVN ausgecheckt werden.

Das Android Projekt greift auf Bibliotheken mit **.so**-Endung zurück. Damit diese Bibliotheken nicht von der Versionskontrolle ausgeschlossen werden muss man die Endung **.so** aus dem globalen Ignoriermuster von SVN entfernen. Diese Einstellung findet man unter **SVN->Einstellungen->Globale Ignoriermuster**.

Das Projekt kann in Eclipse über **File->Import** importiert werden. Dazu kann im Dialog **Existing Android Code into Workspace** gewählt werden. Anschließend muss das Root-Verzeichnis des Android-Projektes ausgewählt werden. In der Liste der zu importierenden Projekte muss das Projekt **appcompat\_v7** nicht ausgewählt werden. Anschließend muss das Android Target unter **Project->Properties->Android** auf Android Version **7.1.1 (API 25)** oder höher eingestellt werden. Gegebenenfalls muss dazu mit dem Android SDK Manager die gewünschte Package-Version nachinstalliert werden.

Das **Java Compiler compliance level** muss zudem auf **1.7** eingestellt werden. Dies kann unter **Project->Properties->Java Compiler** gemacht werden.

### ### Class-Files auslagern

Die Class-Files können eventuell ausgelagert werden, damit sie nicht in der Arbeitskopie landen. Dazu muss man unter **Project->Properties->Java Build Path->Source** über **Browse** den Dialog öffnen.

Über **Create New Folder** kann ein neuer Ordner angelegt werden. Anschließend kann man unter **Advanced** einen Ordner aus dem File System auswählen.

Der neu erstellte Ordner muss dann als **Default output folder** festgelegt werden. Siehe dazu auch das Kapitel **Eclipse Einrichtung** -> **Ombis Projekt anlegen**.

### ### Emulator einrichten

Falls über einen Android Emulator getestet werden soll muss zunächst ein **Android Virtual Device** (AVD) eingerichtet werden.

Dazu muss man den **Android Virtual Device Manager** über **Window->Android Virtual Device Manager** öffnen. Hier können neue AVDs angelegt bzw. bestehende bearbeitet oder gelöscht werden.

Um ein neues AVD zu erstellen muss zunächst das Gerät (z.B. "10.1 WXGA (Tablet) (1280 x 800: mdpi)") und die Android Version (5.1.1 oder höher) ausgewählt werden.

Als CPU sollte falls möglich eine **x86** Variante gewählt werden da diese deutlich schneller ist als **ARM**. Um diese allerdings überhaupt erst starten zu können muss

- ein Intel-Prozessor verwendet werden.
- im BIOS die "Virtualization Technology (VTx/VTd)" aktiviert werden.
  - Da das BIOS je nach PC unterschiedlich erreichbar ist (z.B. Betätigen der ESC-Taste während des Bootvorgangs) und auch die Einstellung leicht verändert benannt werden könnte, sollte zunächst nachgeforscht bzw. nachgefragt werden, wie man das BIOS erreicht und wo die Virtualisierung aktiviert werden kann.
  - Für die HP Z230 Workstation PCs muss z.B. im BIOS unter Security -> System Security -> **Virtualization Technology (VTx/VTd)** auf Enabled gestellt werden.
  - Wurde die Virtualisierung erfolgreich aktiviert sollte nun im Windows-Task-Manager unter dem Tab **Leistung**, der Punkt Virtualisierung als aktiviert aufscheinen.
- der **Intel Hardware Accelerated Execution Manager (HAXM)** (Link: <https://software.intel.com/en-us/android/articles/intel-hardware-accelerated-execution-manager>) (<https://software.intel.com/en-us/android/articles/intel-hardware-accelerated-execution-manager>)) installiert werden.
- das Windows-Feature **Hyper-V** deaktiviert werden.

Über das Flag **Hardware Keyboard present** kann man bestimmen ob die eigene Tastatur oder ein On-Screen-Keybaord verwendet werden soll.

Unter **Skin** sollte ein zur Auflösung passendes Skin ausgewählt werden (z.B. WXGA 800 für das oben genannte WXGA Tablet).

WARNING: Das Flag **Use Host GPU** muss ausgewählt werden, ansonsten startet der Emulator scheinbar nicht.

### ### Android Gerät einrichten

Falls über ein Android Gerät getestet werden soll muss auf diesem zunächst USB-Debugging aktiviert werden. Dies kann man unter der Entwickleroptionen aktivieren.

Falls die Entwickleroptionen noch versteckt sind, kann man diese aktivieren, indem man unter **Geräteinformationen** bzw. unter **Über das Telefon/Tablet** mehrfach auf die **Baukastennummer** klickt.

Sobald USB-Debugging eingeschaltet ist kann man das Gerät über USB mit dem PC verbinden. Die Abfrage, ob USB-Debugging von diesem PC zugelassen werden soll kann man bestätigen.

### ### Launch Configuration

Den Dialog der Run bzw. Debug Configurations öffnen. Dort unter **Android Application** eine neue Debug Configuration erstellen.

Im Tab **Android** kann die Launch Action **Launch Default Activity** gewählt werden.

Zudem kann im Tab **Target** das gewünschte Gerät ausgewählt werden. Alternativ kann man auch "Always prompt" wählen.

### ### AVD starten

Beim Starten sollte die **LogCat-View** aktiv sein, da über diese ersichtlich ist, ob der Emulator noch lädt oder es beim Start zu einem Fehler kam.

Das AVD kann über den Android Virtual Device Manager gestartet werden. Im Start-Dialog sollte **Scale display to real size** ausgewählt werden, da das Fenster sonst den Bildschirm eventuell überschreitet.

Um die richtige Größe zu finden kann die Bildschirmgröße in Zoll angegeben und über "?" die DPI berechnet werden.

Das Starten des Emulators kann einige Minuten dauern, wobei in der LogCat View ersichtlich ist, ob der Emulator überhaupt noch lädt. Sobald der Emulator geladen ist erscheint der normale Android Screen anstelle des "Android" Schriftzuges.

WARNING: Das AVD muss bei Code-Änderungen nicht neu gestartet werden, es reicht die Launch-Configuration erneut zu starten.

### ### Login

Für eine erste Synchronisation und einem Login in die APP muss in einer lokalen Ombis Installation zusätzlich ein neuer Dienst "REST Web Services (Vollzugriff)" erstellt und gestartet werden. Die zugehörigen Einstellungen weichen je nach Test-Installation und Dienst-Einstellungen ab, folgendes Beispiel dient also nur zur Orientierung:

- Http-Ports: **8070** (ein verfügbarer Port)
- Benutzer(in): **admin** (ein beliebiger Benutzer mit ausreichenden Rechten)
- Authentifizierung: **Mitarbeiter Webzugang**
- Betrieb: **01 Demo** (der gewünschte Betrieb)
- Aufzählungen: **Name**

Ist dieser Dienst gestartet können beim erstmaligen Start der APP nun als Einstellung folgende Werte eingegeben werden:

- Zugangspunkt:
  - Https: **Deaktiviert**
  - Host: **10.0.2.2**
  - Port: **8070** (der zuvor ausgewählte Port)
  - Kode Benutzer: **ADMIN** (Einrichtung -> System -> Benutzer(innen), das Feld "Kode" des ausgewählten Benutzers, in diesem Fall WebZugriff)
  - Kode Betrieb: **DEMO** (unter Einrichtung -> Sytem -> Betriebe, das Feld "Kode" des ausgewählten Betriebs, in diesem Fall Mortec)
- Allgemein:
  - Geschäftsjahr: Falls leer, wird automatisch das aktuelle Kalenderjahr verwendet, ansonsten kann es auf ein vorhandenes Geschäftsjahr der Test.Installation fixiert werden.

Auf der Anmeldungs-Seite der APP muss nun der Kode des gewünschten Mitarbeiters, samt zugehörigem Passwort, eingetippt werden. Die Benutzerdaten sind in Ombis unter Allgemein -> Stammdaten -> Mitarbeiter -> Tab: Allgemein erreichbar- und editierbar.

### ### APK erstellen:

Um eine APK zu erstellen muss zunächst der automatische Build Prozess ausgeschaltet werden, da dieser das Flag **DEBUG** der automatisch generierten Klasse **BuildConfig** auf true setzt. Dazu kann unter **Project->Build Automatically** ausgeschaltet werden.

Anschließend muss das Projekt über **Project->Clean** gesäubert werden, sodass der Build-Prozess sicher erst beim Erzeugen der APK erfolgt und somit das Flag auf false gesetzt wird.

Anschließend kann über einen Rechtsklick auf das Android Projekt das Contextmenü geöffnet werden und dort unter **Android Tools** der Eintrag **Export Signed Application Package** gewählt werden.

Im Dialog muss nun zunächst das Projekt ausgewählt werden. Anschließend muss der Keystore **build->OmbisKeystore.jks** ausgewählt werden.

Das Passwort des Keystores kann bei der Abteilungsleitung nachgefragt werden.

Im nächsten Schritt muss der Alias **ombiskey** ausgewählt und das Passwort erneut eingegeben werden. Ist in der Auswahl kein Alias vorhanden, so wurde das Passwort falsch eingegeben und der vorherige Schritt muss wiederholt werden.

Anschließend muss noch die gewünschte Export-Location angegeben werden.

### ### Notizen zum Programmieren

- Wenn eine Änderung für eine der beiden Apps Stock oder Stockhelper durchgeführt wird, dann muss in der entsprechenden build.gradle Datei die Variable 'releaseThisVersion' auf true gesetzt werden. Nur dann wird die Build-Routine entsprechende APKs erstellen und verteilen.

### ### Sonstiges

- Die Android JVM **Dalvik** unterstützt **Hot Code Replace** nicht. Code Änderungen werden somit erst beim erneuten Starten der App übertragen. Beim Speichern der Änderungen erscheint eine Warnung, dass **Hot Code Replace** nicht unterstützt wird. Diese Meldung kann unter **Window->Preferences->Java->Debug** ausgeschaltet werden, indem man das Flag **Show Error when hot code replace is not supported** ausschaltet.
- Die XML-Warnungen können gegebenenfalls unter **Project->Properties->Validation->XML Syntax** ausgeschaltet werden.

## ## Schnittstelle Android App

### ### Übersicht

Die Android App kann über das Ombis-Protokoll (`[_ombis://_](_ombis://_)`) aufgerufen werden. Angesprochen werden können die Listenansicht eines Objekt Typs oder die Formularansicht eines Objekts. Momentan steht nur die erste Hierarchie-Ebene der Tabellen zur Verfügung d.h. das nur Tabellen/Objekte welche direkt von der Startseite aus zur Verfügung stehen aufgerufen werden können.

Die Url stellt sich wie folgt zusammen:

~~~

```
ombis://host/[object_id][?param_name=param_value]
[¶m_name=param_value...]
```

~~~

### ### Listenansicht

Eine Liste einer bestimmten Klasse wird mittels der Angabe des `_host_` ermittelt. Der Host steht für den Klassennamen der gewünschten Tabelle.

Falls die (gefilterte) Liste nur ein Objekt enthält, wird automatisch die Formularseite dieses Objektes geöffnet.

Folgende Parameter stehen zur Verfügung:

- `$_$filter_[FIELD_NAME]$_$` die Filterwerte für das angegebene Feld/Referenz. Mehrere Werte müssen mit einem `,` getrennt werden. Um Referenz zu ermitteln werden die Feldnamen mit `_` getrennt

Beispiel:

~~~

```
ombis://WeingutQualitaetskontrolle?filter_mitglied_nummer=1001&filter_wiese_nummer=2
```

~~~

### ### Formularansicht

Ein Formular eines bestimmten Objekts wird mittels der Angabe des `_host_` und der `_object_id_` ermittelt. Der `_host_` steht für den Klassennamen und die `_object_id_` steht für die Ombis-ID des gewünschten Objekts.

Zudem wird die Formularseite geöffnet, wenn der Aufruf der Listenseite eine Liste mit nur einem Objekt liefern würde.

Es stehen keine Parameter zur Verfügung.

Beispiel:

~~~

```
ombis://WeingutQualitaetskontrolle/3668
```

~~~

## ## JDK und JRE austauschen

### ### Download

#### #### Im Vorfeld

1. Eventuell müssen Bibliotheken die für Ombis verwendet werden auf den neuesten Stand



gebracht werden.

2. In der Systemsteuerung unter Programme sollten eventuell installierte JDKs und JREs deinstalliert werden. Das führt zu einer bereinigten Registry und ermöglicht später ein einfacheres Wechseln zwischen der neusten und einer älteren Version.

#### #### Download

1. Nachdem auf

[<http://www.oracle.com/technetwork/java/javase/downloads/index.html>](<http://www.oracle.com/technetwork/java/javase/downloads/index.html>)[Oracle] die **\*\*ReleaseNotes** einer neuen Java Version gründlich gecheckt\*\* wurden, werden schlussendlich folgende Downloads durchgeführt:

1. jdk für Win32 x86
2. jdk für Win64 x64
3. jre für Win32 x86

#### ### Installation

Die folgenden Schritte sind zuerst für die Win32- und dann für die Win64-Version des JDK von Anfang bis zum Ende durchzuführen. Das darf also nicht parallel geschehen.

1. Beim Installieren ist darauf Acht zu geben, dass der Installationspfad nicht auf die Programme zeigt, sondern auf einen Ordner im eigenen Development Verzeichnis. Dort soll es jeweils einen separaten Ordner für installierte Win32 JDKs und einen für Win64 JDKs geben.

Vorgeschlagene Namensgebung für die Installationsordner: `jdk1.8.0:112` + `jre1.8.0:112`

Anpassungen an die jeweilige Version sind vorzunehmen. So baut sich eine übersichtliche Sammlung von JDKs auf.

2. Nach der Installation ist eine Kopie des neuen JDK-Ordners zu erstellen und vorerst einfach abzulegen.

3. In der Systemsteuerung unter Programme die eben installierten Packages, das JDK und die JRE, deinstallieren. Das löscht die Verweise in der Registry.

4. Zurück zum Installationsordner, hier sind die automatisch erstellten Ordner durch die Deinstallation wieder gelöscht worden. Jetzt sollen die erstellten Kopien deren Platz einnehmen. Dazu einfach das Suffix "- Kopie" aus dem Ordernamen entfernen. Hiermit wird eine sog. private Installation erstellt.

#### #### Ein erster Test

In der Verknüpfung von Eclipse kann als Startparameter nun der Pfad des neuen JDK übergeben werden. Sobald man es startet, sollte man unter `Help - About Eclipse - Installation details -> Tab: Configurations` den Pfad zum neuen JDK finden.

#### ### Umgebungsvariablen

Pfad in JAVA\_HOME aktualisieren und an neue Installation anpassen.

#### ### Anpassungen

1. Den Ordner Server unter `jdk.../jre/bin` nach `jre.../bin` kopieren. Dieses JRE-Verzeichnis wird später an den Kunden ausgeliefert und braucht diese und andere

Anpassungen deshalb. (Sollte nur bei Win32 zutreffen.)

2. Folgende Einstellungen werden nur noch für Installationen auf Systemen vor Windows 10 benötigt. In Windows 10 kann für hochauflösende Monitore in den Kompatibilitätseinstellungen eine Einstellung vorgenommen werden.

Für java.exe Dateien muss hierzu jeweils eine Anpassung mit XN Resource Editor vorgenommen werden. Diese Anpassung ist wichtig, damit Ombis auch auf hochauflösenden Monitoren leserlich bleibt.

1. Standard -> `java.exe` bleibt erhalten.

2. Kopie -> `java.exe.dpiawareoff`

3. java.exe.dpiawareoff mit XN Resource Editor öffnen.

4. Im Manifest den Knoten **\*\*dpiaware\*\*** auf false setzen.

5. Folgende sind die betroffenen Dateien, die bearbeitet werden müssen und nach der Modifikation etwas kleiner sein sollten:

- win32/jdk.../jre/bin/java.exe

- win64/jdk.../jre/bin/java.exe

6. [silver]#Folgende Files müssen nicht mehr bearbeitet werden.#

- [silver]#win32/jre.../bin/java.exe#

- [silver]#win32/jdk.../bin/java.exe#

- [silver]#win64/jre.../bin/java.exe#

- [silver]#win64/jdk.../bin/java.exe#

### ### Verteilung

1. Win32 JDK zippen: `jdk...-win32`

2. Win64 JDK zippen: `jdk...-win64`

3. Zip-Ordner auf Server schieben: DFS/Software/Oracle

### ### Patchen

Hier werden die Java-Sourcen gepatcht, zu jedem Major-Java-Release gibt es jeweils ein zweigeteiltes Repository. Das ermöglicht einen fließenden Übergang zwischen zwei Major-Java-Releases.

1. Im Win32 JDK den src.zip entpacken. (Eventuell Rechtsklick - Eigenschaften - Erweitert - Komprimieren um Platz zu sparen)

2. Die beiden Repositories in einen "Patch-Ordner" im Development-Ordner klonen.

- `[svn://svr-develop/develop/JPatch18/vendor](svn://svr-develop/develop/JPatch18/vendor)`

- `[svn://svr-develop/develop/JPatch18/trunk](svn://svr-develop/develop/JPatch18/trunk)`

3. Die vorhandenen Source-Dateien im Vendor-Repo mit den entsprechenden aus des neuen JDK ersetzen. So erkennt man mit Hilfe von SVN ob es Änderungen in betroffenen Dateien gab.

Falls es Änderungen gab, ist in den Patches (trunk-Repo) nachzuarbeiten. Patches sind dort mit `PATCH` gekennzeichnet.

4. Trunk-Repo: `MakePatch.cmd` mit einem Editor öffnen und Versionsnummern anpassen (2x, siehe ältere Commits zum Vergleich).

5. MakePatch ausführen. Hiermit wird ein patch.jar im `JAR\_HOME` abgelegt.

(**\*\*Achtung\*\*** in MakePatch werden `JAVA\_HOME` verdeckt, entweder richtig setzen oder im File gewünschtes JDK eintragen.)

### ### Tests mit dem JDK in Eclipse

1. In Eclipse das neue JDK in den \_Installed JREs\_ hinzufügen und konfigurieren.
2. Die Lauchkonfigurationen gegebenenfalls anpassen, sodass diese mit dem gewünschten JDK gestartet werden.

### ### Launcher neu erstellen

Nicht immer notwendig.

### #### Aufbereitung

1. Im File `src/launchres.rc` Copyright Angabe überprüfen.
2. Evt. alle Dateien des `bin`-Verzeichnis' öffnen und Major-Versionsnummer austauschen.

```

/ Major Versionsnummer
| / Minor Versionsnummer
| | / Minimaler Speicherbedarf
| | | / Maximaler Speicherbedarf
| | | |
18 00 640 960
```

3. Einchecken: Mit alten Logs vergleichen und kontrollieren.

### #### Neuerstellung

1. `Launcher2/bin/makeAll.cmd` starten um die neuen Launcher in den Ordnern win32 und win64 im `JAR-HOME` Ordner abzulegen.

### ### Verteilung

1. Auf dem Server Ordner `DFS/SupportSW/Runtimes` den letzten Versionsordner duplizieren und die neue Versionsnummer mit Suffix "\_Zum\_Testen" als Name verwenden.
2. Falls neue Launcher erstellt wurden: exe-Dateien (Launcher des jeweiligen Systems) aus `JAR\_HOME` in den entsprechenden Ordner kopieren und ersetzen.
3. patch....jar aus `JAR\_HOME` ebenfalls in beide Ordner kopieren, Versionsnummer nochmal kontrollieren, dann altes löschen und neues auf patch.jar umbenennen.
4. Ordner jre löschen und neuen jre-Ordner aus lokalem jdk-Ordner kopieren.  
\*\*Kontrolle ob der Ordner Server\*\* im bin-Verzeichnis vorhanden ist. Dann neuen jre-Ordner umbenennen auf jre.
5. Im lib Ordner die vorhandenen Libraries mit Libraries aus dem Source-Verzeichnis Ombis/lib ersetzen. Nicht vorhandene sollen nicht hinzugefügt werden. (Ausnahmen sind Libraries welche zu neuen Features gehören, das sollte aber bekannt sein.)

### ### Mit lokaler Installation testen

1. Zuerst erstellt man sich einen lokalen Ordner nach Belieben.
2. In diesen Ordner kopiert man sich vom Server (\\Svr-pdsw01\pd\Ombis) das fakt.jar und evt. das licence.jar (falls Zugriff vorhanden).
3. Weiters kopiert man die zwei Ordner Win32 und Win64 aus dem gewünschten jdk-Ordner

vom Server.

4. Aus einem der beiden übertragenen Ordner kopiert man sich die den Jars entsprechenden .exe Dateien und entfernt das "Ombis" aus deren Dateinamen.
5. Für diese .exe Dateien legt man Verknüfungen mit folgenden Parametern an:
  - Ziel: \_'pfadZurExe'\_ -J-DTestMode
  - Ausführen in: \\SVR-PDSW01\PD\Ombis
6. Wenn das Ausführen klappt sollte in Ombis unter 'Programm -> Hilfe -> Systeminformationen -> Java' die neue JDK (Version der Laufzeitumgebung) eingetragen sein.

## ## Jars Erstellen

### ### Einrichtung

#### #### Umgebungsvariablen

1. (Alte) Systemsteuerung öffnen: z.B. Start-Knopf anklicken, "control" eintippen und bestätigen
2. Dort nach "Umgebung" suchen
3. Systemumgebungsvariablen bearbeiten: im Tab "Erweitert" die Schaltfläche "Umgebungsvariablen..." anklicken

Alternativ:

4. Rechtsklick auf Startknopf -> "System" -> "Erweiterte Systemeinstellungen" -> "Umgebungsvariablen..."

#### ##### Dialog "Umgebungsvariablen"

Unter der Box Systemvariablen, die Schaltfläche "Neu..." betätigen.  
Es kann eventuell nötig sein zunächst in den Windows-Ordneroptionen unter "Ansicht > Erweiterte Einstellungen" die Einstellung "Versteckte Dateien und Ordner" auf "Ausgeblendete Dateien, Ordner und Laufwerke anzeigen" zu wechseln, um alle benötigten Pfade sichtbar zu machen.

!!w

Pfade müssen evt. angepasst werden!!

!!

\*\*\*\*

- Name: \*\*JAVA\_HOME\*\*
- Wert: C:\Development\jdk1.8x64\jre

\*\*\*\*

\*\*\*\*

- Name: \*\*SUPPORTSW\_HOME\*\*
- Wert: P:\SupportSW

\*\*\*\*

\*\*\*\*

- Name: \*\*UPDATE\_HOME\*\*

- Wert: \\svr-pdsw02\OmbisDrive\Licence\Product

\*\*\*\*

\*\*\*\*

- Name: \*\*NODE\_HOME\*\*

- Wert: C:\Program Files\nodejs

\*\*\*\*

\*\*\*\*

- Name: \*\*TEST\_HOME\*\*

- Wert: C:\Development\TestDBs

\*\*\*\*

#### #### DAT-Dateien

Über die DAT-Dateien ("Umgebung") werden die lizenzrelevanten Einstellungen (z.B. Module) sowie die Umgebung (Datenbank-Einstellungen u.s.w) einer Installation festgelegt.

Vorlage-Dateien für die Testdaten befinden sich im 'StaffMan' Projekt unter 'Testing\Testdatenbanken\TestFolder'.

#### ### Test Ombis JAR Datei

Um zu testen ob alle Einstellungen in Ordnung sind, können folgende Befehle ausgeführt werden.

1. Eingabeaufforderung im Ordner 'Ombis\bin' öffnen
2. Kommando: [white black-background]'\*\*build -help\*\*' (optional) ausführen, um zu sehen welche Möglichkeiten das Script anbietet.
3. Kommando: [white black-background]'\*\*build FA\*\*'
4. Kommando: [white black-background]'\*\*build OP\*\*'
5. Sollten beide build-Kommandos erfolgreich sein, sollten alle Einstellung korrekt sein.

Die erstellten JARs ('fakt.jar' & 'obstpool.jar') befinden sich im jeweiligen Modul (z.B. 'Ombis\fakt') im Ordner 'target'.

#### ### Übersicht Ombis Versionen

Ombisversionen haben folgende Form \*\*XX.YYz\*\* (Bsp: 15.13a)

##### #### Masterrelease (XX)

- Änderungsrythmus: ca. 1x jährlich
- Datenbank: Update notwendig
- Kunden müssen neue Lizenz erwerben, um ein solches Release benutzen zu können.

##### #### Versionsnummer (YY)

- Änderungsrythmus: ca. 3 Wochenzyklus, im Einklang mit den Sprints
- Datenbank: Update notwendig
- Kunden bekommen diesen Programmstand, im Zuge der bezahlten Wartung, ohne weitere Ausgaben

##### #### Revisionsnummer (z)

- Änderungsrythmus: kann innerhalb eines Sprints, bis zu täglich erhöht werden
- Datenbank: kein Update notwendig
- Kunden bekommen solche Versionen nur in Notfällen.
- **\*\*Alpha (@)\*\***
  - @-Jars stammen aus der laufenden Entwicklung und sind deshalb nur für interne Testzwecke und keinesfalls für Kunden bestimmt! Alpha-Versionen werden wöschentlich im Laufe des Sprints erstellt, normalerweise ab dem 2ten Montag, da am ersten Montag noch zu wenig Backlogs abgearbeitet wurden. Um bestimmte Punkte zu Testen können eigene Alpha-Versionen erstellt werden.
- **\*\*Beta (a)\*\***
  - Fehlerbeseitigungen & Verbesserungen bei unverändertem Datenmodell und darf in dringenden Fällen bei Kunden eingesetzt werden.
- **\*\*Release Candidate (RC)\*\***
  - Wird selektiv bei Kunden eingesetzt, um letzte Testdaten mit Echtdateien zu erheben.
- **\*\*Final\*\***
  - Sollte der RC keine Fehler mehr aufweisen, wird eine Final-Version kompiliert und damit die Versionsnummer erhöht.

### ### Unser Build-Script

Um einen "Production-Build" zu starten verwenden wir im Normalfall das Script buildPD im bin Ordner des Ombis Quellverzeichnis.

Das Skript funktioniert folgendermaßen:

- Als Parameter wird einmal **\*\*Daily, Alpha, Beta, RC oder Final\*\*** angegeben. Falls angegeben wird das Jar in den entsprechenden Ordner ('SUPPORT\_SW') auf dem Server verschoben.

Nach einem erfolgreichen RC/Final Build werden die Jars zusätzlich in die dafür vorgesehenen Ordner für automatische Updates ('UPDATE\_HOME') kopiert. Bei RC in den Ordner **\*\*early\*\*** und bei FINAL in den Ordner **\*\*fast\*\***.

Falls nichts angegeben wird bleibt das Jar nur lokal. Groß-/Kleinschreibung wird nicht beachtet.

- Der nächste Parameter ist die Anwendung, für die die Jars erstellt werden sollen. Diese Angabe kann fehlen, dann wird für alle Anwendungen das Jar erstellt, oder es kann eine bzw. mehrere Anwendungen angegeben werden. Die Reihenfolge bestimmt den Ablauf! Die Angabe erfolgt über die folgende Kürzel: **\*\*FA, SI, OB, OP, WE, AD, AN\*\***.
- Die Angabe der Version ist nicht mehr nötig, diese wird aus der Datei prog\_ver.txt im entsprechenden Ordner ausgelesen und beim Kopieren des Jars auf dem Server so benannt. Lokal bleibt das Jar immer ohne Version.
- Vor dem Erstellen des Jars werden eventuell lokale Jars gelöscht, damit diese alten JARs nicht auf den Server kopiert werden.
- Bei einem Fehler steht in der prog\_ver.txt u.U. eine alte Version.

**\*\*Beispiel-Aufrufe\*\*:**

- ``buildPD Final FA OB``:

Es wird das Jar für Fakt und Obst erstellt und auf dem Server untern NewJars mit den Namen fakt%ver%.jar (fakt1804a) bzw. obst%ver%.jar (obst1804c) gestellt.

- `'buildPD RC'`

Es werden alle Jars erstellt und auf dem Server unter NewJars mit den Namen z.B. `fakt%ver%.jar` (fakt1805RC) gestellt.

- `'buildPD OP WE'`

Es wird das Jar für ObstPool und Wein erstellt. Die Jars bleiben nur lokal mit den Namen `obstPool.jar` und `wein.jar`

- `'buildPD Daily'`

Daily funktioniert ähnlich wie Alpha mit dem Unterschied, dass keine Produktversion erstellt, keine Mail verschickt und die wöchentliche Standard-Alpha nicht ersetzt wird. Die Jars aus einem Daily Build landen im Alpha Ordner mit dem Suffix "daily".

Dieser Parameter wurde eingeführt, damit die Daily Builds auf dem Build-Server fehlschlagen und die Entwickler benachrichtigt werden. Bisher wurde dort `buildPD` ohne weitere Angabe aufgerufen und dabei werden Warnungen ignoriert.

- `'buildPD'`

Es werden alle Jars erstellt. Die Jars bleiben nur lokal

Das `buildPD`-Skript erstellt außerdem die Produktversionen in OMBIS. Dies erfolgt, falls das `buildPD` mit den Angaben FINAL, ALPHA, BETA oder RC gestartet wird.

Wichtig ist, dass der entsprechende Entwicklungszweig in OMBIS eingetragen ist.

Damit das Skript weiß wo OMBIS gestartet werden soll, muss die Umgebungsvariable `OMBIS_HOME` definiert werden, dort wird das Installationsverzeichnis von OMBIS am Server erwartet (`'\\svr-pdsw01\OmbisDrive\Ombis'`).

Des Weiteren versendet unser `buildPD` Skript automatisch die zu den erstellten Jars zugehörige E-Mail. Der Absender der E-Mail ist `develop@prodata.it` eingetragen.

Um das Senden der E-Mail zu verhindern, kann der Parameter `NO_MAIL` (Groß-/Kleinschreibung irrelevant) angegeben werden.

**\*\*Alternativen\*\*:**

- Das Kommando `'build'` ermöglicht es ein einzelnes Jar oder alle Jars zu erstellen.

Im Unterschied zum `'buildPD'` werden dabei weder die Produkte in Ombis erstellt, noch eine Mail verschickt. Daher sollte dieses Kommando nur für kurze Tests, aber nicht für offizielle Jars verwendet werden.

- Das Kommando `'buildLocal'` ermöglicht es das Build in einer anderen Arbeitskopie auszuführen. Dadurch ist es möglich in der aktuellen Arbeitskopie weiter zu arbeiten, ohne das Build zu stören.

Das Script benötigt folgende Argumente:

- **\*\*workingcopy\*\***: Die Ziel-Arbeitskopie, in welcher das Build ausgeführt werden soll.

Diese muss auf den selben Branch zeigen und darf keine lokalen Änderungen haben. Sie wird automatisch auf die Revision der aktuellen Arbeitskopie aktualisiert und die lokalen Änderungen werden angewandt.

Nach dem Build werden die Änderungen an dieser Arbeitskopie wieder rückgängig gemacht.

- **\*\*command\*\***: Das auszuführende Kommando inklusive Parameter (z.B. `_build FA_`)

### "Release Candidate" (RC) erstellen

**\*\*Achtung\*\***: Beim Erstellen bitte unbedingt das Dokument Checklisten benutzen.\_

1. Projekt auf dem Server und lokal auf den aktuellsten Stand bringen.  
Das bedeutet für den Programmierer, dass er auch alle anderen Programmierer nach noch ausstehenden fertigen Commits fragt.
2. Wenn jeder Programmierer seine letzten abgeschlossenen Änderungen mit einem Commit auf den Server geladen hat, muss der Programmierer seine ausgecheckten Projekte mit Tortoise aktualisieren.
3. Nach dem Aktualisieren im Windows Explorer, sollen die Quell-Dateien auch im Eclipse aktualisiert werden. Hierfür markiert man die eingebundenen Projekte im Eclipse und drückt F5.
4. Immer im Eclipse dürfen in der Problemansicht keine weiteren Probleme aufscheinen.
5. Im Eclipse noch die eigenen Tasks kontrollieren.
6. WARMUP BUILD (CMD auf 5000 Zeilen einstellen, damit man gesamten Bericht lesen kann!)
  1. 'build op' - um umfangreichsten Build zu testen (Für einen effizienteren Build, können im Windows Defender die Development Ordner ausgeschlossen werden, damit dieser nicht jede .class Datei überprüft!)
  2. Log durchschauen und auf Unregelmäßigkeiten, Warungen und Fehler überprüfen.
7. Versionsnummer erhöhen
  1. Im Eclipse die Klasse 'FaktConstants' öffnen '(Strg + Shift + T)'
  2. In der Klasse 'FaktConstants' den Bezeichner 'MODULE\_REVISION' markieren und nach anderen Deklarationen dieses Feldes suchen. Einfach 'Strg + G' drücken.
  3. Alle '\*Constants'-Klassen unter 'it.prodata.gest' öffnen und dort, mit Rücksicht auf die folgenden Ausnahmen, den Wert von 'MODULE\_REVISION' auf RC stellen.
    - Ausnahmen:
    - 'FibuConstants' wird selten weitergedreht
    - 'BaseConstants', 'LicenceConstants', 'TestConstants' wird sehr selten weitergedreht
    - Standardmäßig wird die Revision in den folgenden Klassen auf RC gestellt: 'AgriBaseConstants', 'AgriCatConstants', 'AgroQSConstants', 'AgroDiaryConstants', 'FaktConstants', 'ObstConstants', 'ObstPoolConstants', 'ProdConstants' und 'WeinConstants'.
8. Modul ObstPool (OP) in Eclipse starten.
  - Sollte der Dialog zum Updaten der Mastertabelle angezeigt werden, soll man einem solchen Update nicht zustimmen.

**\*\*Lösung:\*\*** In Eclipse den Launcher updateDevelopDababaseVersions starten.  
Falls es diesen noch nicht gibt, kann er über Rechtsklick auf das Ombis Projekt - Run - Run Configurations eingerichtet werden. Eine neue Konfiguration anlegen:

Name: updateDevelopDababaseVersions  
Main class: it.prodata.tools.div.DevelopDatabaseVersionsUpdater  
Unter Tab Arguments in Working directory - Other: den Workspace eintragen.

**\*\*Alternative:\*\*** Mit dem Programm Flamerobin (Installation und Einrichtung im Dokument: FirebirdInstallation) kann man die Versionen der Datenbanken erhöhen. Dazu verbindet man sich mit den beiden Master-Datenbanken Firm0000 und System und sucht sich jeweils die Tabelle 'DBVERINFO' heraus. Rechtsklick darauf - Browse Data. Dann wählt man das INFO-Feld des einzigen Tupel aus, wählt im Kontextmenü "Edit BLOB" und passt die Versionsnummern an die Programmversionen an.



9. Projekt nochmal einchecken: "Versionsnummer XX.YY auf RC weitergedreht.
10. Dann nochmal aktualisieren (Refresh)!
11. Eingabeaufforderung im Ordner 'Ombis\bin' öffnen und 'buildPD' für Doku ausführen.
12. Nächstes Kommando: 'buildPD RC'
13. Kontrolle im Ordner SupportSW\01\_Jars der verschiedenen Module:
  - Größe des aktuellsten JARs: sollte im Normalfall ein kleines bisschen größer als sein Vorgänger sein!
  - Rechtsklick - 7Zip - Öffnen - 'META-INF' - 'Manifest.mf' - Versions- und Revisionsnummer checken.
14. Abschließend wird eine Rundmail von Ombis an die Programmierer und Supporter geschickt. Dazu wird in Ombis in der Tabelle Sprint der aktuelle Sprint ausgewählt und das Einzeldruckkommando "Sprint" ausgeführt. Durch genaue Angaben zur Version (Versionsnummer und betroffene Module) wird eine leserliche PDF-Datei erstellt und an Supporter und Entwickler versendet. Der Mail Betreff und PDF-Name wird im Druckdialog Feld "Version" festgelegt. Um zu kennzeichnen, dass die Mail für alle Produkte gilt, muss "OMBIS" und dem Namen des @-Jars (Beispiel: "OMBIS 15.03@2") eingetragen werden.

Ansonsten müssen die einzelnen Produkte mit gleicher Versionsnummer aufgezählt werden und bei auseinandergehenden Versionsnummern separate Mails versendet werden.
15. Von diesem Commit aus soll ein Branch eingerichtet werden, aus dem das Final-Jar erstellt wird und auf dem später Portierungen folgen können. Name: Release-VXXYY
16. Abschließend sind noch die Versionsnummern im Ombis-'trunk' auf die nächsthöhere Minor-Version zu drehen. Hierbei werden auch die Revisionsnummern auf "@" gestellt.

### ### FINAL Version erstellen

\_\*\*Achtung\*\*\_: Beim Erstellen bitte unbedingt das Dokument Checklisten benutzen.\_

Die Final-Versionen sollten i.d.R. aus dem jeweiligen Branch erstellt werden.

1. Läuft bis Punkt 9 gleich ab wie das Erstellen einer RC-Version.

'MODULE\_REVISION = null' steht für Final

Commit: "Revisionsnummern von ... auf Final weitergedreht"
2. Zusätzlich sind die neuen JARs in .exe Dateien umzuwandeln.
3. Die .exe werden, anschließend mit Filezilla auf den Fileserver in den Ordner 'AUTO\_DISTRIBUTE' geladen.
  - Fakt.exe -> Fakt
  - Obst.exe -> ObstStd
  - ObstPool.exe -> Obst
  - Wein.exe -> Wein
4. Zugangsdaten für Filezilla:

Server: 78.46.221.97  
Benutzername: prodata  
Passwort: Bitte bei der Abteilungsleitung nachfragen  
Port: 21

5. Anschließend wird wie immer eine Mail verschickt (siehe <<"Release Candidate" (RC) erstellen>> Punkt 14)!

### ### Portieren von Commits für Service-Releases

Wir befinden uns in folgender Situation:

Normalerweise werden nur Minor-Releases an Kunden verteilt, diese können auch Strukturänderungen (d.h. Datenbankupdates) mit sich bringen. Während eines Sprints kann es aber vorkommen, dass ein oder mehrere Kunden dringend eine Fehlerbehebung benötigen, für diesen Fall wird ein Service-Release erstellt. Ein solches Service-Release darf aber keine Datenbankupdates auslösen und darf deshalb nicht alle Commits seit dem letzten Minor-Release beinhalten.

In genau solch einer Situation müssen wir einzelne Commits (wie z.B. den der Fehlerbehebung)

auf das letzte Minor-Release portieren und ein Service-Release erstellen!

Wie funktioniert das also?

1. Für Portierungen legt man sich ein beliebigen Ordner (Bsp: OmbisPortierungen) an, welcher bei der nächsten Portierung überschrieben werden kann.
2. In diesen Ordner wird jetzt der benötigte Release-Ordner ausgecheckt!  
In unserem Repository gibt es zu jedem Minor-Release einen Branch.  
Diese Branches befinden sich unter .../Ombis/branches/.  
Tipp: Mit dem Button "..." hinter "URL des Projektarchivs" kann man das Projektarchiv durchstöbern.
3. Nach dem Auschecken wählt man im Kontextmenü unter TortoiseSVN die Option "Zusammenführen..." aus. Im Dialog sollte bereits die Option "Einen Revisionsbereich" ausgewählt sein, also klickt man auf "Weiter".
4. Im nächsten Fenster muss im Textfeld zu "Zusammenführen aus URL" der Quellordner ausgewählt werden: Das ist so gut wie immer der `Ombis/trunk`!  
Neben der ausgewählten Option "Bestimmter Bereich" kann mit dem Button die Log der Commits angezeigt werden. Dort können zu portierende Commits mit Checkboxes ausgewählt werden. Bereits portierte werden grau präsentiert.  
Danach können mit "Weiter" und "Start" die Portierungen durchgeführt werden!
5. Wenn die Portierung erfolgreich war ist der Ordner "OmbisPortierungen" einzuchecken!  
Hierbei findet man als eine automatisch generierte Commit-Meldung in den "Letzten Meldungen", die 1. Meldung in der Liste sollte hier zur Portierung passen.
6. Sobald eingchecked ist, muss man die Portierung in den betreffenden Backlogs eintragen!
7. Falls man jetzt noch ein Service-Release erstellen möchte, dreht man im Ordner "OmbisPortierungen" die entsprechenden Versionsnummern (Constans) weiter, überträgt (Commit)  
diese und aktualisiert den Ordner ein letztes Mal. Bevor die Jars aus "OmbisPortierungen" erzeugt werden, dürfen lokal keine Änderungen mehr vorhanden sein!
8. Jetzt können die benötigten Jars aus "OmbisPortierungen" generiert werden.
9. Anschließend wird wie immer eine Mail verschickt (siehe <<"Release Candidate" (RC) erstellen>> Punkt 14)!

## Versionierung Web-Projekte

### ### Einrichtung

Für eine korrekte Verteilung der ZIPs, der Erstellung der Produktversionen in Ombis und dem automatischen Versand der Mail müssen außerdem folgende Umgebungsvariablen gesetzt werden:

!!w

Pfade müssen evt. angepasst werden!!

!!

\*\*\*\*

- Name: **\*\*OMBIS\_HOME\*\***
- Wert: ``\\svr-pdsw01\OmbisDrive\Ombis``
- Info: Installationsverzeichnis von OMBIS, um die Produktversionen zu erstellen und die Mail zu versenden.

\*\*\*\*

\*\*\*\*

- Name: **\*\*SUPPORTSW\_HOME\*\***
- Wert: ``P:\SupportSW``
- Info: Root-Zielverzeichnis für die fertigen Zip-Files.

\*\*\*\*

\*\*\*\*

- Name: **\*\*UPDATE\_HOME\*\***
- Wert: ``\\svr-pdsw02\OmbisDrive\Licence\Product``
- Info: Root-Zielverzeichnis für die fertigen Zip-Files für die automatisierte Verteilung.

\*\*\*\*

### ### buildPDWeb

Um die auszuliefernden ZIPs der diversen Web-Applikationen zu erstellen, existiert im Ordner **\*\*web/tools\*\*** der Ombis-Quelldateien das Tool **\*\*buildPDWeb\*\***. Dieses kann über die Kommandozeile im angegebenen Ordner ausgeführt werden.

Hierbei können verschiedene Parameter gesetzt werden, je nachdem welche Version(en) man ausführen will:

#### Hilfe zum Skript

`all_apps` optional, bestimmt, dass alle Web-Projekte gebuildet werden sollen. Alternative:

`project1 project2 ...` notwendig, falls `all_apps` nicht angegeben, mehrere Angaben erlaubt, bestimmt welche Web-Projekte gebuildet werden sollen

`ci_build` optionale Angabe, falls angegeben werden erstellte Apks je nach Version verteilt.

Falls nicht angegeben, wird ein lokaler Build durchgeführt.

`no_mail` optional, die gibt, an dass keine automatisierte Mail versendet werden soll.

no\_ombis                      Effekt nur bei Alpha, RC, Final.  
optional, gibt an ob OMBIS nicht gestartet werden soll. Keine  
Produktversion in OMBIS und  
keine automatisierte Mail.  
Effekt nur bei Alpha, RC, Final

Folgende Umgebungsvariablen müssen gesetzt sein:

SUPPORTSW\_HOME            Root-Zielverzeichnis für die fertigen Zip-Files  
UPDATE\_HOME               Root-Zielverzeichnis für die fertigen Zip-Files für die  
automatisierte Verteilung  
OMBIS\_HOME                Installationsverzeichnis von OMBIS, um die Produktversionen zu  
erstellen und die Mail zu versenden.

Versionstypen:

local                       Es werden nur die Zips erstellt und im "dist" - Ordner  
abgelegt  
daily                       ...@daily#svn#.zip Ordner werden nach  
SupportSW\01\_Jars\04\_AlphaJars\WepPortale kopiert  
rc                           ...XXYYRC.zip Ordner werden nach  
SupportSW\01\_Jars\02\_RCJars\WepPortale und in den  
ProductCommons Ordner auf dem Lizenzserver kopiert  
final                       ...XXYY[z].zip Ordner werden nach  
SupportSW\01\_Jars\01\_NewJars\WepPortale und in den  
ProductCommons Ordner auf dem Lizenzserver kopiert

Verfügbare Web-Applikationen (Stand September 2021):

agent-web arbeitszeiten-teams auswertung betriebsheft betriebsmappe bio-members-  
management bl-konto-admin check-in check-mobile-clients check-ogs foradori guest-  
station helpdesk home housekeeping licence-manager obst-pool-mobile qs-nachreichung  
service-web smart-invoice sqk-controls tasks warenvermittlung zeiterfassung

Um zum Beispiel die Final-Version der Helpdesk-ZIPs zu erstellen, und sie automatisch  
auf dem Server zu verteilen, ohne allerdings eine Mail zu versenden, muss der folgende  
Befehl eingegeben werden:

```
buildPDWeb ci_build helpdesk no_mail
```

Diese Hilfe zur Benutzung von buildPDWeb kann auch mithilfe des Parameters **\*\*help\*\***  
aufgerufen werden:

```
buildPDWeb help
```

```
productInfo.json
```

Damit die verschiedenen Web-Applikationen auch korrekt verteilt werden, und es nicht  
etwa zu inkompatiblen Versionen kommt, muss weiters die Datei **\*\*productInfo.json\*\*** im  
**\*\*src/environments\*\*** des jeweiligen Web-Projektes existieren und gepflegt werden. Ein  
Beispiel für eine solche Inkompatibilität wäre, dass die aktuelle Version einer Web-  
Applikation ein Feld aus Ombis benötigt, welches in Ombis 19.10 eingeführt wurde,  
jedoch auf einer Ombis-Installation der Version 19.09 eingesetzt wird. Solch ein Fall

kann (und wird in den meisten Fällen) zu einem ungewollten fehlerhaften Verhalten der Applikation führen.

!!s

Die Produkt-Version (`ombisVersion` + `revision`) muss gleich geführt werden wie jene der Ombis Applikationen. Wenn in einem Webportal während eines Sprints dann aber keine Änderungen gemacht werden, muss dieses Webportal beim Kunden auch nicht aktualisiert werden. Dafür gibt es den Parameter `releaseThisVersion`, dieser wird im Trunk bei der Entwicklung **auf true gesetzt, sobald es Änderungen** für das entsprechende Webportal gibt.

!!

Als Beispiel dient hier folgend die productInfo-JSON der qs-Nachreichung Web-Applikation:

```
{
 "ombisVersion": "1909",
 "revision": "",
 "releaseThisVersion": false,
 "ombisProductCode": "W_NACHR",
 "configurations": {
 "obst": {
 "products": ["Obst"],
 "zipName": "qs-nachreichung.o"
 },
 "obst-pool": {
 "products": ["ObstPool"],
 "zipName": "qs-nachreichung.op"
 }
 }
}
```

Folgende Parameter können hierbei gesetzt werden:

- **ombisVersion**: die Ombis-Version, für welche diese ZIP eingesetzt werden kann [1909, 1911 ...] (sollte lediglich im trunk geändert werden)
- **revision**: die Revision der ZIP ["@", "RC", "", "a", "b" ...]: wird eine neue Version einer ZIP benötigt, so muss die Revision im ReleaseBranch höher geschaltet werden, damit sie von der vorherigen Version unterschieden werden kann
- **releaseThisVersion**: Wird vom Entwickler bei Änderungen eines Webportals auf true gesetzt, damit spätestens am Ende des Sprints eine neue Version released wird. Mit einem neuen Sprint sollte dieses Feld immer wieder auf false gesetzt werden.
- **ombisProductCode**: der in OMBIS vorgegebene Produkt-Code der Web-Applikation (zu finden in der Tabelle **Produkte** in der jeweiligen Referenz **Anwendung** im Feld **Kode**)
- **configurations**: die Konfiguration der jeweiligen ZIP (die möglichen Konfigurationen der einzelnen Anwendungen sind in der Datei **angular.json** im Ordner **web** zu finden)
- **products**: für welche Ombis-Varianten diese ZIP-Konfiguration verteilt werden soll [Fakt, Obst, ObstPool, Wein ...]; hierbei können auch mehrere Varianten getrennt durch ein Komma angegeben werden ["Obst","ObstPool"] falls

- **zipName**: der Name

## ## JSpeed

### ### Was ist JSpeed und was macht es?

JSpeed ist ein, von Dietmar Hechensteiner, selbstgeschriebenes Programm um unseren Source Code zu optimieren und zu obfusieren.

Kurz zusammengefasst hat es folgende Aufgaben:

- Ausschließen von nicht verwendeten Klassen und Methoden -> Verkleinern und verkürzen der Sourcen
- Verschlüsseln/Obfusieren und kürzen -> Klassen und Methoden werden refactored und mit unlesbaren kurzen Namen (Bsp.: `ruex()` statt `getAllTheInterestingInformation()`)
- Files werden durch zusätzliches Inlining von kurzen Methoden und Blöcken weiter verkleinert.
- Klassen und Methoden welche zur Runtime unbenutzt bleiben werden ebenfalls ausgeschlossen.

## ## Visual Studio Build Tools

### ### Information

Die Visual Studio Build Tools enthalten den C-Compiler und die weiteren Tools, um unsere Launcher (exe-Dateien) zu erzeugen. Es werden nur die minimal benötigten Tools installiert, um die Launcher von der Kommandozeile aus erzeugen zu können.

### ### Installation

Die Build Tools können über die Visual Studio Homepage unter [\[https://visualstudio.microsoft.com/\]](https://visualstudio.microsoft.com/) (<https://visualstudio.microsoft.com/>) heruntergeladen werden. Man findet sie unter Downloads / Tools für Visual Studio / Buildtools für

Visual Studio. Der derzeitige direkte Link ist

[\[https://visualstudio.microsoft.com/de/downloads/\]](https://visualstudio.microsoft.com/de/downloads/) (<https://visualstudio.microsoft.com/de/downloads/>)#build-tools-for-visual-studio-2022

.Download der Tools für Visual Studio

{}

- Die heruntergeladene Datei **vs\_buildtools\_\*.exe** (Visual Studio Installer) ausführen.
- Die folgenden Optionen auswählen:

.Auswahl der C++-Buildtools

{width=540}

.Einzelne Optionen eventuell abwählen (z.B. AddressSanitizer)  
{width=420}

.Downloadcache eventuell deaktivieren  
{}

- Installation durchführen

### ### Konfiguration

Es werden u.a. folgende Einträge im Windows-Startmenü erzeugt:

- x64 Native Tools Command Prompt for VS
- x86 Native Tools Command Prompt for VS

Sie stellen eine für die Benutzung der Tools vorkonfigurierte Eingabeaufforderung zur Verfügung. Über den Eigenschaften-Dialog sieht man, dass sie jeweils die Batch-Dateien **\*\*vcvars64.bat\*\*** und **\*\*vcvars32.bat\*\*** im Ordner **\*\*C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\VC\Auxiliary\Build\*\*** ausführen, welche die erforderlichen Einstellungen vornehmen.

Diese beiden Batch-Dateien werden auch von unseren Batch-Dateien (**\*\*make<App>.cmd\*\***, **\*\*makeAll.cmd\*\***) zur Erzeugung der Launcher benötigt. Da sie unqualifiziert aufgerufen werden, muss man den Windows-Suchpfad für ausführbare Dateien (PATH) um den obigen Ordner erweitern.

### ### Build und digitale Signatur

Nach Änderungen am Quellcode (launcher.c bzw launchres.rc) oder anderen relevanten Änderungen die im Versionskontroll-Repository launcherX gehalten werden, muss ein Aufruf von makeAll.cmd erfolgen. Dieser erzeugt alle notwendigen .exe Dateien im durch die JAR\_HOME-Environmentvariable definierten Verzeichnis in den Unterverzeichnissen win32 bzw. win64. Diese .exe Dateien müssen anschließend noch digital signiert werden. Dies geschieht durch die Batchdatei signPD.cmd bzw signAsa.cmd. In bin befindet sich eine entsprechende Vorlage namens signPD\_Template.cmd. Damit diese funktionsfähig wird, muss dort noch der Name Zertifikat.pfx durch den Namen des echten Zertifikates ersetzt werden, und der Text Passwort durch das echte Passwort. Aus Sicherheitsgründen wird diese Operation auf dem Rechner des Abteilungsleiters in einem geschützten Verzeichnis durchgeführt, wo sich das Zertifikat zum Signieren von ausführbaren Dateien und die Batchdatei signPd.cmd bzw. signAsa.cmd befinden.

### ## Einrichtung der Qualitätsdatenbank für das Betriebsheft

Damit die Kommunikation zwischen Webseite, Ombis und QDB funktioniert, benötigt es eine AgroDiary Installation (betriebsheft.net) und ein ObstPool-Ombis mit Variante ObstVerband (VOG QDB) mit jeweils folgende Einstellungen:

### Ombis AgroDiary: betriebsheft.net by ProData

#### #### Dienste

- **\*\*QDB-Portal-Konto\*\***
  - Typ: QDB-Portal-Konto
  - Arbeitsplatz: [PC-Benutzername]
  - Betrieb: betriebsheft.net by ProData
  - Kennzeichen bei:
    - Verifizierung der Konten
    - Aktualisierung Anbaudaten
    - Abgabe Betriebsheft
    - Aktualisierung Nachreichungen
    - Status überprüfen
- **\*\*Qualitätssicherung\*\***
  - Typ: Qualitätssicherung
  - Arbeitsplatz [PC-Benutzername]
  - Betrieb: betriebsheft.net by ProData
  - Importe / Aktualisierungen können deaktiviert werden
- **\*\*Allgemeiner Betriebsheft-Dienst: Web-Portal\*\***
  - Typ: Web-Portal
  - Arbeitsplatz: <Master>
  - Internes Web-Portal: Betriebsheft.net
  - Benutzer: betnetPortal
  - Gastbenutzer: GAST
  - Betrieb: betriebsheft.net by ProData
  - Http-Ports: 80

#### #### QDB-Portale

- **\*\*folgenden Eintrag anpassen:\*\***
  - Name: VOG Verband
  - Typ: VOG QDB
  - Basis URL: [http://localhost](http://localhost):8071/rest/betnet/vogqdb
  - Benutzer: betnet
  - Passwort: selbes Passwort wie Benutzer betnet im VOG QDB Kommunikationsserver
  - Basis URL (Konten): [http://localhost](http://localhost):8071/rest/betnet/vogqdb
- **\*\*restliche Einträge kommentieren:\*\***
  - Basis URL und Basis URL (Konten) kommentieren. Bspw mit "XXX":  
  
[http://XXXqdb.vog.it](http://XXXqdb.vog.it):8090/rest/betnet/vogqdbtest

#### #### QDB-Portal-Konto



Im Betriebsleiter unter QDB-Portal-Konten für den Benutzer das selbe Passwort eingeben wie für VOG-QDB-Agrar-Logins.

### Ombis ObstVerband: VOG QDB Kommunikationsserver

#### Dienste

- \*\*Betriebsleiter Datenübertragung formale Prüfung\*\*
  - Typ: Betriebsleiter Datenübertragung formale Prüfung
  - Arbeitsplatz: <Master>
  - Betrieb: VOG QDB Kommunikationsserver
- \*\*Betriebsleiter Datenübertragung übernehmen\*\*
  - Typ: Betriebsleiter Datenübertragung übernehmen
  - Arbeitsplatz: <Master>
  - Betrieb: VOG QDB Kommunikationsserver
- \*\*REST Web Services (Vollzugriff)\*\*
  - Typ: REST Web Services (Vollzugriff)
  - Arbeitsplatz: <Master>
  - Bemerkung: Datenübertragung
  - Http-Ports: 8071
  - Benutzer: betnet
  - Benutzertyp: Standard
  - Auth. Algorithmus: Digest
  - Authentifizierung: Agrar Login Webzugang
  - Betrieb: VOG QDB Kommunikationsserver
  - Aufzählungen: Name
  - (restliche Einträge leer)

#### VOG-QDB Agrar Logins

Passwort des Betriebsleiter für QDB-Portale setzen

## Automatische Ombis-Updates bei Kunden

### Einleitung

Das Updaten mehrerer 1000 Kunden ist sehr aufwändig und läuft oft sehr mechanisch ab. Wir haben uns deshalb entschieden diesen Prozess iterativ immer weiter zu automatisieren.

Jeden Tag zwischen 20:00 und 21:00 Uhr melden sich die Kunden-Installationen beim Lizenz-Server von Pro Data um aktuelle Versionen und zusätzlich benötigte Dateien

(Jar, Runtime, Hilfe-Datei usw.) herunterzuladen.

Die Bereitstellung dieser Dateien/Ordner erfolgt über das Dateisystem im Lizenz-Programmordner unter Product ('\\svr-pdsw02\OmbisDrive\Licence\Product'). Die Ordner Struktur dort ist folgende: 'Name der Anwendung/Minorversion/Priorität'

Hier unterscheiden wir 4 Prioritäten:

- early: ist für RC's gedacht
- fast: für schnelle Auslieferung
- norm: normale Auslieferung
- slow: langsame Auslieferung

Die Priorität kann in jeder Kundeninstallation separat im Umgebungsdialog eingestellt werden.

- ---: Keine Updates
- Vorab: Der early-Kanal
- Schnell: Der fast-Kanal
- Normal: Der norm-Kanal
- Langsam: Der slow-Kanal

Im Lizenzprogramm wiederum, bei den Lizenzen, lässt auf Basis des Wartungsvertrages hinterlegen, wie lange die Kundeninstallation mit Versionsupdates versorgt werden soll. Dabei lässt sich ein Bis-Datum oder eine Bis-Version definieren. Falls beide Einträge leer sind, wird die Kundeninstallation mit Minor-Updates versorgt. Bei Angabe eines Bis-Datums gibt es Major-Updates bis zum angegebenen Stichtag, danach nur noch Revisions-Updates. Falls eine Bis-Version definiert ist, also ungleich 0, limitiert das die vom Lizenzprogramm zur Verfügung gestellte Programmversion. Bei einem Wechsel der Major-Version wird vom Kundenprogramm eine automatische Neuregistrierung beim Lizenzprogramm vorgenommen.

### ### Bereitstellen der JARs

- Alle Jars (Alpha, RC, Final) werden weiterhin unter SupportSW für die manuelle Weiterverarbeitung abgelegt.
- RC-Jars werden direkt nach dem Buildprozess in die 'early'-Ordner der entsprechenden Anwendungen und Versionen gestellt.
- Final-Jars (Minor- und Service-Releases) werden direkt nach dem Buildprozess in die 'fast'-Ordner der entsprechenden Anwendungen und Versionen gestellt.

### ## SVN Auto Update

### ### Batch Datei um alle Repositories in einem Rutsch zu aktualisieren

Erstelle die Batchdatei "updateSVN.bat" und füge den folgenden Code ein. Bearbeite die lokalen Pfade und führe es wie jedes Programm aus oder lege es in den Autostart-Ordner.

**\*\*CODE\*\***

```
@echo off
FOR %%A IN (
 "E:\home\ozh\wordpress\"
 "E:\svn\someplugin\"
 "E:\svn\someproject\"
) DO START TortoiseProc.exe /command:update /path:%%A /closeonend:0
```

### ### Autostart der SVN Updates beim Login

1. In Windows 10+ kopiere die Links zu den Programmen, die nach der Anmeldung gestartet werden sollen, in den Autostart-Ordner
2. Navigiere über den Windows-Explorer zum folgenden Ziel und wähle anstelle von USERNAME deinen Benutzernamen.

C:\Users\USERNAME\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup

3. Lege einfach die Links/Aliase der Programme, die automatisch gestartet werden sollen, in diesen Ordner.

### #### Weitere Kommentare:

- Die `FOR %%A` -Schleife enthält die Pfade zu den Projekten, die Sie aktualisieren möchten.
- Das `START` -Bit bedeutet "asynchron starten, d. h. nicht auf das Ende der vorherigen Aufgabe warten, um die nächste zu starten", so dass alle Aktualisierungsfenster gleichzeitig erscheinen.
- Das `/closeonend:0` bedeutet, "das Update-Fenster nicht schließen, wenn es fertig ist", so dass Sie tatsächlich sehen können, was aktualisiert wurde.
- In den seltenen Fällen, in denen Sie ein Repository nicht aktualisieren wollen, können Sie jederzeit ein Rollback durchführen.

## ## Neue Pipeline in Jenkins erstellen

### ### Erstellen der Pipeline

Sobald man sich in Jenkins angemeldet hat, kann man unter "Element anlegen", einen passenden Namen eingeben, zusätzlich muss man die Option "Pipeline" auswählen und mittels "OK" bestätigen.

Anschließend kommt man direkt in die Konfiguration der Pipeline, hier kann man verschiedene Einstellungen und Parameter festlegen.

### ### Wie ist das Skript aufgebaut?

```
pipeline {
 agent any
 stages {
 stage('Build') {
 steps{
 //
```

```

 }
 }
 stage('Test') {
 steps {
 //
 }
 }
 stage('Deploy') {
 steps {
 //
 }
 }
}
}

```

### 1. **\*\*Pipeline\*\***

Eine Pipeline ist ein benutzerdefiniertes Modell für eine CD-Pipeline. Der Code einer Pipeline definiert ihren gesamten Build-Prozess, der in der Regel Phasen für die Erstellung einer Anwendung und die anschließende Bereitstellung umfasst.

### 2. **\*\*Stage\*\***

Ein Stage-Block definiert eine konzeptionell klar abgegrenzte Teilmenge von Aufgaben, die über die gesamte Pipeline hinweg ausgeführt werden (z.B. die Phasen "Build", "Test" und "Deploy") und wird von vielen Plugins zur Visualisierung oder Darstellung des Status/Fortschritts der Pipeline verwendet.

### 3. **\*\*Step\*\***

Eine einzelne Aufgabe. Grundsätzlich sagt ein Step Jenkins, was zu einem bestimmten Zeitpunkt (oder "step" im Prozess) zu tun ist. Um z.B. den Shell-Befehl make auszuführen, verwendet man "sh make"; falls Jenkins auf einer Windows-Umgebung läuft, muss das Kommando "bat make" verwendet werden. Wenn ein Plugin die Pipeline-DSL erweitert, bedeutet das normalerweise, dass das Plugin einen neuen Step implementiert hat.

## ### Wichtige Funktionen

### 1. **\*\*Checkout\*\***

```

checkout([
 $class: 'SubversionSCM',
 additionalCredentials: [],
 excludedCommitMessages: '',
 excludedRegions: '',
 excludedRevprop: '',
 excludedUsers: '',
 filterChangelog: false,
 ignoreDirPropChanges: false,
 includedRegions: '',
 locations: [[
 cancelProcessOnExternalsFail: true.
 credentialsId: '1a2702fa-3f5e-45e0-823a-6dd8c92dfa81',

```

```

 depthOption: 'infinity',
 ignoreExternalsOption: true,
 local: '.',
 remote: '[svn://svr-develop/develop/Ombis/trunk](svn://svr-
develop/develop/Ombis/trunk)'
]],
 quietOperation: true,
 workspaceUpdater: [
 $class: 'UpdateWithCleanUpdater'
]
})

```

- **\*\*\$class: 'SubversionSCM'** Checkt den Quellcode aus dem Subversion-Repository aus.

- **\*\*additionalCredentials: []** Falls für ein vollständiges Checkout zusätzliche Anmeldeinformationen erforderlich sind, können diese hier angegeben werden.

- **\*\*excludedCommitMessages: ''** Wenn diese Option gesetzt ist und Jenkins eingestellt ist, um auf Änderungen zu reagieren, ignoriert Jenkins alle Commit-Nachrichten, die einen der angegebenen regulären Ausdrücke enthalten, wenn es feststellt, ob ein Build ausgelöst werden muss.

- **\*\*excludedRegions: ''** Wenn diese Option gesetzt ist und Jenkins eingestellt ist, um auf Änderungen zu reagieren, ignoriert Jenkins alle Dateien und/oder Ordner in dieser Liste, wenn es feststellt, ob ein Build ausgelöst werden muss. Jeder Ausschluss verwendet den Mustervergleich mit regex Ausdrücken und muss durch eine neue Zeile getrennt werden.

```

/trunk/myapp/src/main/web/*.html
/trunk/myapp/src/main/web/*.jpeg
/trunk/myapp/src/main/web/*.gif

```

Das obige Beispiel zeigt, dass ein Build nicht zustande kommt, wenn nur html/jpeg/gif-Dateien an den SCM übergeben wurden.

- **\*\*excludedRevprop: ''** Wenn diese Option gesetzt ist und Jenkins eingestellt ist, um auf Änderungen zu reagieren, ignoriert Jenkins alle Revisionen, die mit der angegebenen Revisionseigenschaft (revprop) markiert sind, wenn es feststellt, ob ein build ausgelöst werden muss. Dies kann verwendet werden, um Commits, die durch den Build selbst durchgeführt wurden, von der Auslösung eines weiteren Builds auszuschließen, vorausgesetzt, der Build-Server committet die Änderung mit der richtigen revprop. Diese Art des Ausschlusses funktioniert nur mit Subversion 1.5 Servern und neueren Versionen.

- **\*\*excludedUsers: ''** Wenn diese Option gesetzt ist und Jenkins eingestellt ist, um auf Änderungen zu reagieren, ignoriert Jenkins alle Revisionen, die von Benutzern in dieser Liste vorgenommen wurden, wenn es feststellt, ob ein Build ausgelöst werden muss. Dies kann verwendet werden, um Commits, die durch den Build selbst durchgeführt wurden, von der Auslösung eines weiteren Builds auszuschließen, vorausgesetzt, der Build-Server nimmt die Änderung mit einem anderen SCM-Benutzer vor. Jeder Ausschluss verwendet einen wörtlichen Mustervergleich und muss durch eine neue Zeile getrennt werden.

Das obige Beispiel zeigt, dass ein Build nicht zustande kommt, wenn nur die Revisionen von "auto\_build\_user" in den SCM übertragen wurden.

- **filterChangelog: false** Wenn diese Option aktiviert ist, wendet Jenkins bei der Anzeige von Änderungsprotokolleinträgen die gleichen Ein- und Ausschlussmuster an wie bei der Abfrage von Änderungen. Wenn dies deaktiviert ist, werden Änderungen, die von der Abfrage ausgeschlossen sind, weiterhin im Änderungsprotokoll angezeigt.

- **ignoreDirPropChanges: false** Wenn diese Option gesetzt ist, ignoriert Jenkins nur svn-property-Änderungen von Verzeichnissen. Diese Änderungen werden bei der Entscheidung, ob ein Build ausgelöst werden soll, ignoriert und werden aus dem Changeset eines Builds entfernt. Hauptanwendung dieser Eigenschaft ist das Ignorieren von svn:mergeinfo-Änderungen (die sonst z.B. zu einem kompletten Rebuild eines Maven-Projekts führen würden, trotz inkrementeller Build-Option).

- **includedRegions: ''** Wenn diese Option gesetzt ist und Jenkins eingestellt ist, um auf Änderungen zu reagieren, ignoriert Jenkins alle Dateien und/oder Ordner, die nicht in dieser Liste enthalten sind, wenn es feststellt, ob ein Build ausgelöst werden muss. Für jeden Einschluss wird ein Mustervergleich mit regex Ausdrücken verwendet, und jeder Einschluss muss durch eine neue Zeile getrennt werden. Dies ist nützlich, wenn Sie eine gesamte Ressource für einen Build auschecken müssen, aber den Build nur durchführen wollen, wenn sich ein bestimmter Teil geändert hat.

```
/trunk/myapp/c/library1/*
/trunk/myapp/c/library2/*
```

Ein Build wird in diesem Falle nur ausgeführt, falls sich etwas oder in "c/library1" oder "c/library2" ändert.

- **locations**

- **cancelProcessOnExternalsFail** Legt fest, ob der Prozess abgebrochen werden soll, wenn Checkout/Update svn:externals fehlgeschlagen ist. Funktioniert nur, wenn das Kästchen "Ignore externals" nicht gesetzt ist. Standardmäßig wird der Prozess abgebrochen.

- **credentialsId** Legt die Id des Benutzers fest, mit dem der Checkout durchgeführt wird.

- **depthOption** Setzt die --depth option des Checkouts und update-Kommandos, der Standardwert ist "infinity"

- "empty" Schließt nur das unmittelbare Ziel der Operation ein, nicht aber die untergeordneten Dateien oder Verzeichnisse.

- "files" Umfasst das unmittelbare Ziel der Operation und alle seine unmittelbaren Dateien.

- "immediates" Schließt das unmittelbare Ziel der Operation und alle seine unmittelbaren Dateien oder Verzeichnisse ein. Die untergeordneten Verzeichnisse selbst sind leer.

- "infinity" umfasst das unmittelbare Ziel, die untergeordneten Dateien und Verzeichnisse, die untergeordneten Dateien der untergeordneten Verzeichnisse und so weiter bis zur vollständigen Rekursion.

- "as-it-is" Übernimmt die Tiefe aus der aktuellen Arbeitskopie und ermöglicht die manuelle Einstellung der Aktualisierungstiefe mit der Option --set-depth.

- **ignoreExternalsOption** Die Option "--ignore-externals" wird mit den Befehlen svn checkout und svn update verwendet, um die Verarbeitung externer Definitionen zu deaktivieren.

- **local** Hier kann ein lokales Verzeichnis (relativ zum Stammverzeichnis des Arbeitsbereichs) angegeben werden, in dem dieses Modul ausgecheckt werden soll. Bleibt es leer, wird der letzte Teil der URL als Standard verwendet, genau wie bei der svn CLI. Ein einzelner Punkt (.) kann verwendet werden, um das Projekt direkt in den Arbeitsbereich und nicht in ein Unterverzeichnis auszuchecken.

- **remote** Definiert den Pfad, von welchem die Sourcen ausgecheckt werden sollen.

- **quietOperation: true** Ahmt den Subversion-Befehlszeilenparameter --quiet für checkout / update nach, um die Ausgabe kürzer zu halten und gibt nichts oder nur zusammenfassende Informationen aus.

## 2. **deleteDir()**

Diese Funktion kann verwendet werden um zwischen einem Checkout und dem nächsten den Workspace zu leeren um so die Dauer des zweiten Checkouts zu optimieren.

## 3. **dir('')**

Diese Funktion wird verwendet um in ein bestimmtes verzeichnis zu wechseln. Ein Beispiel könnte sein:

```
pipeline {
 agent any
 stages {
 stage('Build') {
 steps{
 dir('tools') {
 //do something in the tools folder
 }
 }
 }
 }
}
```

## 4. **sh / bat**

Diese Funktionen werden verwendet um Kommandos auszuführen. "sh" wird auf Linux-Servern verwendet und "bat" auf Windows-Servern.

## ## Superbuilds

### ### RC\_and\_new\_Branch

Dieses Skript wird am letzten Dienstag des Sprints ausgeführt. Vor dem Starten muss kontrolliert werden, ob

es noch Punkte mit dem Status TEST gibt, diese müssen abgeklärt werden, denn ansonsten dürfen keine

RCs gestartet werden. Anschließend informiert man mit einer kurzen Nachricht die Programmierer und

startet das Skript, welches folgenderweise vorgeht: Jenkins checkt Ombis frisch aus und erstellt einen neuen Branch, auf welchem dann anschließend die RCs erstellt werden. Anschließend ersetzt es in den productInfo.json und der pom.xml die Revision von @ auf RC und aktualisiert zudem die Versionsnummer. Sobald er die Variablen angepasst hat startet er das Build für die Jars und Zips. Nach dem Build wird das Ganze, für OmbisMobile wiederholt. Dort wird wiederum die Revision auf RC gesetzt und anschließend die major und minor aktualisiert. Falls ein Fehler auftritt wird das Build abgebrochen und eine Rundmail gesendet. Zudem muss der Benutzer im TaskTracking den neuen Release-Branch unter den Entwicklungszweigen eintragen und beim Entwicklungszweig den Sprint eintragen. Dieser muss nur einmal eingetragen werden, weil die ReleaseBranches von Ombis und OmbisMobile denselben Namen haben. Zusätzlich müssen die Konstanten der Masterdatenbank weitergedreht werden, dies kann mit der IntelliJ Run-Configuration DevelopDatabaseVersionsUpdater erledigt werden. Anschließend sollte man kontrollieren, dass die Jars und Zips erfolgreich erstellt wurden.

Inputs::

- **\*\*Neue Major Version\*\***... Die Major-Version, welche in build.gradle festgehalten wird und in Zusammensetzung mit der Minor die Versionsnummer erstellt.
- **\*\*Neue Minor Version\*\***... Die Minor-Version, welche in build.gradle festgehalten wird und in Zusammensetzung mit der Major die Versionsnummer erstellt.
- **\*\*E-Mail-Benachrichtigung\*\***... Mit diesem Parameter kann man angeben, wer eine E-Mail bekommt wenn der Build failt.

Als Standardwert sind Andreas Rabensteiner, Robert Pattis und Marco Sciacovelli vermerkt.

### RC\_to\_Final

Dieses Skript wird am letzten Freitag des Sprints ausgeführt. Vor dem Starten muss überprüft werden, ob alle Backlogs des Sprints erledigt wurden und für die Final portiert wurden. Falls dies nicht der Fall ist erscheint der Backlog nicht in der Mail, welche nach erfolgreichem Erstellen der Finals automatisch gesendet wird. Anschließend sollten die Programmierer kurz informiert werden und dann kann das Skript gestartet werden, welches folgenderweise vorgeht: Jenkins checkt Ombis frisch aus. Anschließend ersetzt er in den productInfo.json und der pom.xml die Revision von RC auf einen leeren String. Sobald er die Variablen angepasst hat startet er das Build für die Jars und Zips. Nach dem Build wird das Ganze, für OmbisMobile wiederholt. Dort wird wiederum die Revision auf einen leeren



String gesetzt. Falls ein Fehler auftritt wird das Build abgebrochen und eine Rundmail gesendet.

Inputs::

- **\*\*Branch\*\***... Bestimmt den Branch auf dem die Finals ausgeführt werden sollen.
- **\*\*E Mail-Benachrichtigung\*\***... Mit diesem Parameter kann man angeben, wer eine E-Mail bekommt wenn der Build failt.

Als Standardwert sind Andreas Rabensteiner, Robert Pattis und Marco Sciacovelli vermerkt.

### Service-Release

Dieses Skript wird verwendet, falls Fehler in der letzten finale Version, oder im letzten Release-Candidate entdeckt werden, welches nur die korrigierten Fehler und keine neuen Funktionalitäten enthält. Im Falle einer Final version wird die revision des gewünschten Jars um erhöht. Bei einem Release-Candidate wird das gleiche getan, der unterschied, liegt darin, dass bei RCs eine Zahl angehängt wird und bei Finals ein Buchstabe eingefügt wird.

Inputs::

- **\*\*Branch\*\***... Bestimmt den Branch auf dem die Finals ausgeführt werden sollen.
- **\*\*Jars\*\***... Bestimmt welche Jars erstellt werden.
- **\*\*E Mail-Benachrichtigung\*\***... Mit diesem Parameter kann man angeben, wer eine E-Mail bekommt wenn der Build failt.

Als Standardwert sind Andreas Rabensteiner, Robert Pattis und Marco Sciacovelli vermerkt.

## Zertifikat erstellen

### Allgemein

Einige Möglichkeiten von PWAs können nur in einer sicheren Umgebung (gültige Https-Verbindung oder `localhost`) verwendet werden.

Um diese Funktionen auch während der Entwicklung auf mobilen Geräten testen zu können, muss ein selbst signiertes Https-Zertifikat erstellt und das entsprechende CA-Zertifikat auf dem mobilen Gerät installiert werden.

NOTE: Nicht alle Browser scheinen selbst importierte CAs als vertrauenswürdig zu akzeptieren.

Auf Android-Geräten kann aber zumindest mit Google-Chrome getestet werden, dieser akzeptiert die selbst importierten CAs.

### Zertifikate erstellen

Zertifikate können mithilfe der Batch-Datei `create-cert.cmd` erstellt werden. Diese verwendet `keytool` um alle notwendigen Dateien in Working Directory zu erzeugen.

Das Kommando fragt dabei nach folgenden Angaben:

- **\_\_Domain Name\_\_**: Der Domain Name für welchen das Zertifikat erstellt werden soll.

Bsp.: `pc-robert.g14.local`.

- Der eigene Domain Name kann unter Windows z.B. mit Hilfe des Kommandozeilen-Befehls `ipconfig /all` ermittelt werden. Er setzt sich aus dem angezeigten Wert für `Hostname` und (optional) `Primäres DNS-Suffix` zusammen.
- `\_\_Password\_\_`: Das Passwort für die Schlüsselpaare und Zertifikate. Defaultwert: `password`.
- `\_\_Output file\_\_`: Der Dateiname des erstellten Server-Zertifikates.

Dabei werden folgende Dateien erstellt:

- `root.jks`: Privates Schlüsselpaar für das Root-Zertifikat.
- `ca.jks`: Privates Schlüsselpaar für das CA-Zertifikat.
- `root.crt`: Öffentliches Root-Zertifikat (Public Key)
- `ca.crt`: Öffentliches CA-Zertifikat (Public Key)
- `.jks`: Privates Schlüsselpaar des Server-Zertifikates. Der Dateiname hängt von der Benutzereingabe ab.
- `server.crt`: Öffentliches Server-Zertifikat (Public Key).
- ``: Privates Server-Zertifikat im `\_\_PKCS#12\_\_`-Format.

WARNING: Die privaten Schlüsselpaare / Zertifikate (\*\*.jks\*\* und \*.p12\*\*) sollten nicht veröffentlicht werden.

Sie können verwendet werden um beliebige Https-Zertifikate zu erstellen, welche dann von deinem mobilen Gerät als vertrauenswürdig angesehen werden.

### ### Server-Zertifikat in Ombis einrichten

Um das Zertifikat in Ombis zu verwenden muss es zunächst unter `Umgebung -> Zertifikate` importiert werden.

Dazu kann über den Punkt `PKCS#12 Datei importieren` die erstellte `\*.p12`-Datei ausgewählt und das angegebene Passwort in den zwei Eingabefeldern angegeben werden. Ombis muss anschließend neu gestartet werden und wird anschließend dieses Zertifikat für Https-Verbindungen verwenden.

### ### Client-Zertifikat in Ombis einrichten

Da das Serverzertifikat nicht vertrauenswürdig ist (es wurde nicht von einer offiziellen Zertifizierungsstelle signiert), muss für eine "Ombis zu Ombis"-Kommunikation das zugehörige Clientzertifikat im Schlüsselspeicher der Ombis-Installation importiert werden.

Dazu kann unter `\_Einrichtung > System > Umgebung\_` im Tab `\_Schlüsselspeicher\_` die erstellte `\*.cer`-Datei importiert werden.

Das importierte Client-Zertifikat gilt nun als vertrauenswürdig und es kann mit dem vorhin eingerichteten Server über Https kommuniziert werden.

### ### Test über Browser

Das Server-Zertifikat kann gegebenenfalls über einen Browser getestet werden. Dazu einfach die Https-URI (z.B. [https://localhost](https://localhost):443) im Browser eingeben. Der Browser sollte melden, dass das Zertifikat nicht vertrauenswürdig ist und die Möglichkeit bieten dieses dennoch zu akzeptieren.

Ist dies der Fall, so ist das selbstsignierte Zertifikat und die Einstellungen am Https-Server gültig.

### ### CA-Zertifikat in Android importieren

Damit Android das verwendete Server-Zertifikat als vertrauenswürdig einstuft muss das verwendete CA-Zertifikat als vertrauenswürdige CA importiert werden.

Dazu kann die Datei 'ca.crt' auf das Gerät kopiert und dort geöffnet werden.

Android fragt nun zunächst nach dem Pin/Passwort deines Gerätes.

Anschließend muss ein Name für das Zertifikat vergeben und der Verwendungszweck ('VPN und Apps') ausgewählt werden.

Nachdem das Zertifikat installiert wurde sollte das Gerät neugestartet werden.

### ### CA-Zertifikat in Android entfernen

Falls das selbst erstellte CA-Zertifikat nicht mehr benötigt wird, sollte es auch vom Android-Gerät entfernt werden.

Dazu kann in den Einstellungen unter 'Sicherheit & Datenschutz->Zusätzliche Einstellungen->Verschlüsselung und Anmeldedaten->Vertrauenswürdige Anmeldedaten' im Tab 'Nutzer' das entsprechende Zertifikat gelöscht werden.

NOTE: Der genaue Menüpunkt kann, abhängig vom Geräte-Hersteller leicht abweichen. Über die Suche (Suchbegriff 'Anmeldedaten') sollte der Punkt jedoch leicht zu finden sein.

## ## Datenbankfelder anlegen

### ### Feld Business und Manipulatorklasse hinzufügen

1. **\*\*Eventuell eine Vorlage heraussuchen. In 99% erleichtert einem eine Vorlage die Arbeit erheblich.\*\*** (Stichworte: Patterns, Das Rad nicht neu erfinden)

2. In Businessklasse ('BSClass')

1. Konstanten für Feldnamen anlegen (persistent, fiktiv, Referenzen oder Collections).

2. Übersetzungen in den ResBundles einpflegen. Dabei ist darauf zu achten, dass bei mehrfachen Einträgen alle nachfolgenden unter Kommentar gesetzt werden! Nicht gelöscht!

3. 'SCRIPT\_FN\_'-Konstanten anlegen, falls nötig. (= Wenn der Inhalt der Konstante nach außen hin nicht verständlich ist.)

4. Business Objekte anlegen ('BOA...' oder 'Bsf...').

5. Objekte für fiktive Felder ('transient') anlegen.

6. In der Methode 'static void initialize...Guide(BsfGuide g)' Felder registrieren. **\*\*Immer\*\*** auf die Reihenfolge achten.

7. In der Methode 'void init()' Felder initialisieren.

8. Getter und Setter nachbauen.

3. In der Manipulatorklasse ('MnClass')

1. Reihenfolge ist sehr wichtig!

2. Offsets einpflegen (Collection, Referenzen und Felder). Blockmarkierung hilft hier beim Weiterschieben der Nummerierung!

3. Felder im Konstruktor registrieren.

4. Felder in der Methode `'updateAccessImpl'` einfügen.
  5. Felder in der Methode `'getFieldValue(int field, boolean fastOutput)'` einfügen.
  6. Felder in der Methode `'setFieldValue(int field, Object value)'` einfügen.
  7. Für eventuelle weitere Implementierungen kurze Rücksprache.
4. Debugging von Ombis
1. `/updateDB` als Startparameter eingeben um Änderungen zu testen.

### ### Feld/er in entsprechenden Formularen anlegen

1. Betroffene Produktversion im Debugmodus starten.
2. Änderungsformular in der betroffenen Tabelle öffnen und durch exzessives Vergrößern und Verkleinern die Eigenschaften feststellen.
3. Button "Formular" anklicken ( `[Umschalt]+[F2]` ) und Formular mit Kode "COMPL" bearbeiten.
4. Im Tab Zusatz überprüfen, wie viele Produktversionen davon betroffen sind. (Formular ist für alle anzupassen!)
5. Im Tab Daten das Formular anpassen -> Button "Übernehmen" -> Testen durch exzessives Vergrößern und Verkleinern. Guidelines zur Anpassung von Formularen sind in der Dokumentation `Forms.doc` zu finden. (`'[svn://svr-develop/documentation/Ombis/trunk/Divers](svn://svr-develop/documentation/Ombis/trunk/Divers)'`)

### ### Feld/er in entsprechenden Tabellenansichten anlegen

1. Betroffene Produktversion im Debugmodus starten.
2. Button "Tabelle" anklicken ( `[Umschalt]+[F5]` ) und Tabellenansicht mit Kode "STD" bearbeiten. Eventuell auch andere überprüfen.
3. Im Tab Zusatz überprüfen, wie viele Produktversionen davon betroffen sind. (Formular ist für alle anzupassen!)
4. Im Tab Daten das Formular anpassen -> Button "Übernehmen" -> "OK" (`'[svn://svr-develop/documentation/Ombis/trunk/Divers](svn://svr-develop/documentation/Ombis/trunk/Divers)'`)

### ### Updatercode schreiben

Wenn in Ombis Strukturänderungen vorgenommen werden, muss oftmals die Datenbank angepasst werden. Hierfür gibt es viele verschiedene `**Updater.java` Klassen, welche je nach Version (Minor Releases erlauben Strukturänderungen) verschiedene Updates einspielen. So wird sichergestellt, dass sich die Datenbank bei einem Kunden nach einem Programmupdate, automatisch in den angeforderten Zustand updated. `._(-> Filesearch **updater.java` zeigt alle Updater-Klassen an)`_`  
 Diese Updater-Klassen bestehen hauptsächlich aus folgenden nacheinander ablaufenden Methoden:

#### 1. `**updateTables(BsfDatabaseUpdater)**`

Aktualisiert die Struktur aller Tabellen. Falls eine Tabelle nicht existiert, wird sie neu erstellt. Andernfalls werden evtl. fehlende Felder hinzugefügt.  
 Diese Methode kann überschrieben werden, um explizit Tabellen und Felder umzubenennen (vor dem Aufruf von `super.updateTables()`!) oder nicht mehr benötigte zu löschen. Daten, Indizes und Referenzen werden an dieser Stelle noch nicht überprüft (siehe `updateData` und `updateIndicesAndReferences`).

## 2. **\*\*updateData(BsfDatabaseUpdater)\*\***

Diese Methode wird unmittelbar nach updateTables zur Aktualisierung der Daten nach der Umstrukturierung der Tabellen aufgerufen. Zu diesem Zeitpunkt haben alle Tabellen das neue gewünschte Format. Hier können beispielsweise neue Felder initialisiert und zwischengespeicherte alte Daten im neuen Format zurückgeschrieben werden. Die Daten müssen so weit in Ordnung gebracht werden, dass anschließend alle Indizes und Referenzen erstellt werden können.

## 3. **\*\*postUpdateData(BsfDatabaseUpdater)\*\***

Diese Methode wird unmittelbar nach updateData aber noch vor updateVersions und updateIndicesAndReferences aufgerufen. Zu diesem Zeitpunkt sind alle Daten aktualisiert. Diese Methode wird in der Regel dazu benutzt, nicht mehr benötigte Tabellen und/oder Felder zu löschen. Außerdem könnten alle Indizes und/oder Referenzen gelöscht werden, um sie anschließend neu aufzubauen. @return true falls das Update fortgesetzt werden kann.

**\*\*Achtung:\*\*** Meistens wird während des Sprints ein "kleiner gleich" Vergleich (ver \<= xxyy) eingesetzt, um den Updater beim Debuggen zu testen. Vor dem Erstellen des finalen Release müssen diese Stellen überarbeitet werden und müssen deshalb mit folgendem Kommentar gekennzeichnet werden: ``// Provisorisch (*) <``

**\*\*Richtlinie\*\*** Stellen mit ``ver \<= xxyy`` müssen **\*\*immer\*\*** von einem Kommentar begleitet werden: entweder von ``// Provisorisch (*) <`` um den Ausbau vorzumerken oder von einem Kommentar der den beabsichtigten Verbleib des ``\<=`` im Code dokumentiert!

**\*\*Beispielanwendung:\*\*** Mit dem Code in der if-Anweisung ( ``if (result && weinVer < 1518)`` ) werden bei einem Update, von Version 15.17 auf Version 15.18, Werte für Produkte mit Saldo=0 auf NULL korrigiert.

Die Variable result, checkt jeweils ab ob das vorherige Update erfolgreich durchgeführt wurde, ansonsten, können kumulative Updates nicht durchgeführt werden.

~~~java

```
protected boolean updateData(BsfDatabaseUpdater dbUpdater) {
 boolean result = super.updateData(dbUpdater);
 int weinVer = dbUpdater.getVersion(WeinConstants.MODULE_NAME);
 if (result && weinVer > 0 && dbUpdater.isSystemDatabase()) {
 }
 if (result && weinVer > 0 && dbUpdater.isFirmDatabase()) {
 SqlDatabaseMetaData sqlMD = dbUpdater.getSqlMetaData();
 // ... other updates ...
 // Check version:
 // for release: < version
 // for testing: <= version with comment!
 if (result && weinVer < 1518) {
 result = dbUpdater.execute("UPDATE wvWeinRegProd SET
SaldoBEaktuell=NULL WHERE SaldoBEaktuell=0")
 }
 if (result && weinVer <= 1519) { // Provisorisch (*) <
 result = dbUpdater.execute("UPDATE wvWeinReg SET xxx=NULL WHERE
yyy=0")
 }
 }
}
```

```
 return result;
 }
    ~~~
```

### ### Vor dem Commit MasterDatenbank updaten

1. cmd öffnen und in den betroffenen `...\Ombis\bin` Ordner navigieren.
2. Das Kommando `build DB` ausführen. Output durchschauen.
  - Sollte die Ausführung mit einer Fehlermeldung abbrechen, so kann dies an der fehlenden Datei `**svnversion.exe**` liegen. Es sollte bei einem Mitarbeiter nach dieser gefragt, und in den korrekten Ordner (z.B. `TortoiseSVN\bin` bei einer Installation von TortoiseSVN) geschoben werden.
3. Die neu erstellte Datei Firm.xml auf neu angelegte Felder überprüfen. (Rechtsklick auf Ombis Ordner -> TortoiseSVN -> Prüfe auf Änderungen -> Firm.xml Datei sollte gelistet sein.)
4. Im Eclipse die Run Konfiguration "UpdateDevelopDatabases" laufen lassen. Output durchschauen.

### ## Testdatenbanken

#### ### Einleitung

##### #### Ziel

- jeder Entwickler soll auf eine relativ einfache Art und Weise jede unserer Applikationen inkl. der Programmvarianten starten können und dort in jeder Tabelle Daten vorfinden, um testen zu können.
- verschiedene Testcases sollen einfach durchgeführt werden können und nicht ein langes Einrichten/Konfigurieren benötigen.
  - z.B. abhängige Installationen wie ObstPool Verband mit ObstPool Standard und dort Datenverteilung und Austausch der Produzentendaten und der Dokumentdaten.
  - Abgabe eines Betriebsheftes in die Qualitätsdatenbank
- Verwendung von Kundendatenbanken ist umständlich, benötigt viel Speicherplatz und bewirkt ein großes Risiko z.B. Daten an Live-Umgebungen zu senden.
- Kundendatenbanken zu Test-Zwecken zu verwenden ist aber vor allem nicht in Ordnung. Außer wenn man ein spezifisches Problem nachstellen muss, anschließend sollte sie aber wieder entfernt werden.

##### #### Anforderung

- einfaches Handling
- Testdaten sollten für alle von uns unterstützten Datenbankserver verwendet werden können
- sie sollen für jeden beliebigen Programmstand nutzbar sein (Branch, bestimmte Subversion Revision).
- deshalb sollten die Testdaten im Versionskontrollsystem der Sourcen committed werden
  - da eignen sich Textdateien
  - Änderungen an Daten können da auch gut verglichen werden
- aus diesen Dateien soll dann die Datenbank erstellt werden können

- eine Datenbank soll in diese Dateien exportiert werden können
- Testdaten sollen für automatisierte Integration Tests verwendet werden können

----

Die Erstellung von Testdaten für Ihre automatisierten Tests ist langweilig, aber Sie sollten Zeit und Mühe in die Erstellung gut strukturierter Daten investieren. Wenn Sie über gute Testdaten verfügen, wird das Schreiben automatisierter Tests viel einfacher. Je früher Sie Daten von guter Qualität erstellen, desto einfacher ist es, bestehende automatisierte Tests zusammen mit der Entwicklung der Anwendung zu erweitern.

----

### ### Allgemein

Die Testdaten befinden sich im Ombis-Projekt im Ordner testdata. Dort gibt es pro Datenbank und Applikation einen Ordner mit den zugehörigen Testdaten. Pro Tabelle eine Textdatei. Die Felder sind dort mit einem Tabulator getrennt.

### ### Export

Um den Export der Datenbank in die Testdaten frei zu schalten, muss das Programm mit dem Startparameter "TestDataMode" gestartet werden.

Die Speicherung der Testdaten erfolgt im Programm mit dem Kommando "Testdaten exportieren" im Menüpunkt Service oder mit dem Startparameter testDataExport.

### #### Technische Details

Tabellen wie OID, Verteilungstabellen, Nachrichten usw. werden nicht exportiert ('TestData.NO').

Daten können mit dem referenzierten Objekt exportiert werden ('TestData.WITH\_REFERENCED\_OBJECT')

Das ist bei Memo- und Image-Referenzen aktiviert und kann auch bei anderen nicht sharable Contained-Referenzen gesetzt werden

Enum für die Bestimmung der Behandlung der Testdaten im Guide.

~~~java

```
public enum TestData {
    UNKNOWN,           // noch nicht bekannt
    APP_MODUL,          // Modul der Applikation (Default)
    GUIDE_MODUL,        // Modul des Guides
    SUPER_GUIDE_MODUL,  // Erstes Super-Modul des Guides
    WITH_REFERENCED_OBJECT, // Daten werden mit dem referenzierten Objekt behandelt
    NO;                 // keine Testdaten

    /** Gibt an, ob Testdaten verwaltet werden sollen. */
    public boolean isTestData() {
        return switch (this) {
            case APP_MODUL, GUIDE_MODUL, SUPER_GUIDE_MODUL -> true;
            default -> false;
        };
    }
}
```

```

    };
}
}
~~~

```

Falls das Guide das nicht benennt, wird der Default-Wert beim ersten Zugriff ermittelt (TestData.APP_MODUL).

Mit den Werten Guide-Modul und Super-Guide-Modul wurde eine Logik für Ableitungen der Testdaten implementiert, die sich aber in der Praxis als sehr schierig darstellte und deshalb zur Zeit nicht verwendet wird.

Auch aufgrund des Ansatzes von Ableitungen wird das ParentGuide befragt.

Import

Um das Programm mit den Testdaten zu starten, muss wie beim Export der Startparameter ``/TestDataMode`` angegeben sein.

Der Import der Daten startet beim DB-Update automatisch, wenn die Datenbanken leer sind.

Um eine bestehende Datenbank neu mit den Daten zu initialisieren oder wenn die Datenbanken noch nicht vorhanden sind, kann die Neuerstellung mit dem Startparameter ``/recreateDB`` getriggert werden.

Dabei werden die eventuell vorhandenen Datenbanken gelöscht und neu erstellt.

Anschließend werden die Strukturen erstellt und die Tabellen mit den Testdaten gefüllt.

Mit dem Startparameter ``/NotUseSystemTestData`` kann gesteuert werden, dass die System-Datenbank nicht neu mit Testdaten gefüllt wird, außer sie existiert noch nicht.

Aus den Testdaten können Datenbanken für jeden Datenbankserver (Firebird, SQL-Server, PostgreSQL) generiert werden.

!!s

bestehende Datenbanken werden mit ``/recreateDB`` gelöscht, daher sollten die Einstellungen zur Datenbank und Datenbank-Dateien im Umgebungsdialog geprüft werden.

Die Zusatzdatenbanken nicht vergessen!

!!

Testdaten-Varianten

Um für eine Applikation auch mehrere Testdaten verwalten zu können, wurde der Startparameter ``/TestDataVariant`` eingeführt. Mit diesem kann ein spezieller Name für den Datenbank-Ordner der Testdaten angegeben werden.

z.B. ObstPoolWaage, das ein Waagen-Notfallsystem simuliert.

Testdatenbanken aktuell halten!!

!!s

Testdatenbanken aktuell halten!!

!!

Falls neue Tabellen und Felder implementiert werden, sollten die Testdatenbanken mit Daten gefüllt werden.

Auch eventuelle Update-Codes sollten darüber laufen.
Dabei könnte es sinnvoll sein, vor allem wenn Felder gelöscht werden und deren Daten in ein anderes Feld geschrieben werden, die Testdatenbanken in einem sauberen Workspace zu generieren und das DB-Update dann im aktuellen Workspace durchzuführen. Die Testdaten dann wieder exportieren und mit dem Commit der Sourcen oder gleich nach diesem Commit ins Repository zu committen.

Operationen der Testdatenbanken im Batch-Betrieb

Mit den Skripts `testDataBatch` und `testDataBatchIDE` können die Operationen für `RecreateDB`, `UpdateDB` und `Export` der Testdaten aller Applikationen im Batch-Betrieb ausgeführt werden.

- `recreateDB` -> Neuerstellung der Datenbanken und Import der Testdaten
- `updateDB` -> Aktualisierung der Datenbankstrukturen
- `export` -> Export der Testdaten -> Startparameter ist `testDataExport`

Notwendig ist die Einrichtung aller Testdatenbank-Installationen und die Umgebungsvariable `'TEST_HOME'`, mit dem Basis-Ordner der Testdaten-Installationen.

`testDataBatch` setzt auf maven auf

~~~

```
mvn clean prepare-package -pl fakt,obstpool,wein,agrodiary,hotel,cloudmanager -am
-DrecreateDBTestData -DupdateDBTestData -DexportTestData -DskipWeb -DskipDefaultBuild
-DskipTests %args%
```

~~~

Das Skript `testDataBatchIDE` verwendet die kompilierten IntelliJ-Klassen. Deshalb ist es notwendig, dass alle Klassen kompiliert sind.

Vorlagen

Im Ordner `'TestFolder'` stehen Unterordner für alle Applikationen zur Verfügung. Die Datenbankeneinstellungen sind für Firebird ausgelegt und müssen für Tests mit anderen Datenbankservern entsprechend angepasst werden. Die Einstellungen für die Ordner der Datenbanken benutzen die Variable `%TEST_HOME%`, die dynamisch durch die entsprechende Umgebungsvariable ersetzt wird, sodass diese Einstellung nicht individuell angepasst werden muss.

Mit den Argumenten `SysDB` und `FirmDB` werden jeweils eigene Alias-Namen für die Datenbanken definiert, damit sie im Falle von SQL-Server und PostgreSQL nicht in Konflikt mit anderen stehen.

!!s

Zur Sicherheit sollten die Einstellungen zur Datenbank und zu den Zusatzdatenbanken kontrolliert und bei Bedarf angepasst werden.

!!

Testdaten gibt es aktuell für

- AgroDiary:

- Installations Ordner = TestAgroDiary
- Fakt:
 - Installations Ordner = TestFakt
 - dat-Datei = Fakt.dat
- SoftDev:
 - Installations Ordner = TestSoftDev
 - Programm-Variante = SoftwareDevelopment
- TWG - Trinkwassergenossenschaft
 - Installations Ordner = TestTWG
 - Programm-Variante = PotableWaterSupply
- Obst:
 - Installations Ordner = TestObst
- Obst Agentur:
 - Installations Ordner = TestObstAgentur
- ObstPool - VAB:
 - Installations Ordner = TestVAB
- ObstPool - VAB2:
 - Installations Ordner = TestVAB2
- ObstPool - Waage:
 - Installations Ordner = TestVABWaage
 - TestDataVariant = ObstPoolWaage
- ObstVerband:
 - Installations Ordner = TestVerband
 - Programm-Variante = FruitAssociation
- ObstVerbandProd:
 - Installations Ordner = TestVerbandProd
- Wein:
 - Installations Ordner = TestWein
- Hotel:
 - Installations Ordner = TestHotel
- CloudManager:
 - Installations Ordner = TestCloudManager

Belegung Ports bei Testdaten

- 8070 - TestVerband - Replikation
- 8071 - TestVAB001 - Replikation
- 8072 - TestVAB001Waage - Replikation
- 8073 - TestVerbandProd - Replikation
- 8074 - TestVAB002 - Replikation
- 8077 - TestVermarkt - Replikation
- 8081 - TestVerband - REST-Dienste Agrarsoftwaren
- 8083 - TestAgrodiary - REST-Dienste Agrar
- 8085 - TestSoftDev - WEB-Bedienoberfläche

Hinweise

- Die Testdatenbanken sollten ohne /startDefaultServices gestartet werden, damit der Datenverteilungs-Dienst gestartet wird.
- Für PostgreSQL und SQL-Server kann nur ein DB-Name vergeben werden, deshalb muss bei einer zweiten Installation der DB-Name über die Argumente im Umgebungsdialog geändert werden.

Zusammenfassung der Schritte für die Benutzung von Testdaten

1. Alle oder einzelne Unterordner des TestFolder-Ordners in den gewünschten Basisordner (TEST_HOME) für Tests kopieren.

1. Die Namen der Ordner sind vorgegeben, damit Skripts ausgeführt werden können (siehe "Abschnitt Testdaten gibt es aktuell für" und den jeweiligen Installations-Ordner).

2. Wenn ein Ordner neu erstellt wird, dann darauf achten, dass alle notwendigen Ordner der Datenbanken und für Sicherungen erstellt (Data, DB, OLB, Div, ExtDB, Backup) sind.

2. Run/Debug Configuration mit Parameter /TestDataMode (siehe auch 'Run/Debug Configurations' im nächsten Abschnitt)

3. Startparameter je nach Ausführung: '/env /recreateDB /updateDB /TestDataExport'

1. '/env' Bei erster Ausführung einer Applikation (Fakt, Obst...), um die Einstellungen zu kontrollieren und bei Bedarf die Datenbankpfade (auch Zusatzdatenbanken!) anzupassen.

2. '/recreateDB' beim Prompt für Startparameter eingeben, wenn man die Datenbanken neu erstellen möchte. Ein sauberer Workspace vermeidet hier Probleme.

3. '/updateDB' Um einen Update-Code auf bestehende Testdaten anzuwenden und sie später zu exportieren.

4. Menüpunkt ****Service -> Testdaten exportieren****: für den Export der aktualisierten Testdaten (durch Update-Codes oder Neuerfassung)

5. Kontrolle und Commit der aktualisierten Testdaten mit neuen Strukturen

Run/Debug Configurations

Im Ordner 'runConfigurations' stehen die IntelliJ Run/Debug Configurations für alle Applikationen zur Verfügung. Die Arbeitsordner benutzen die IntelliJ Pfadvariable \$TEST_HOME\$, welche unter 'Appearance & Behavior > Path Variables' einmalig mit dem entsprechenden Wert anzulegen ist.

Dokumentation für automatisches Testen in Java

Allgemein

Automatisierte Tests sind wichtig in der Software-Entwicklung, um auf geänderte Funktionalitäten aufmerksam zu machen, wenn diese eine Fehlersituation hervorrufen oder nicht mehr das erwartete Ergebnis liefern.

Nicht nur interne Code-Änderungen sind davon betroffen, sondern auch Änderungen in der Java-Runtime und den von uns verwendeten Libraries wie z.B. Vaadin, Datenbank-Treiber usw.

Beim Umstieg von Java 8 auf Java 11 gab es Änderungen im Default-Calendar in Bezug auf die Locale, wodurch

als erster Wochentag Sonntag und nicht mehr Montag geliefert wurde.

Dadurch wurden Daten, die sich auf den Wochentag bezogen, falsch ermittelt. Der Fehler ist erst nach einiger Zeit einem Kunden aufgefallen.

Das wäre ein typischer Fall, den man mit einem ganz einfachen Unit-Test gleich festgestellt hätte.

Jetzt werden für bestimmte Programmteile von einem Supporter regelmäßig bestimmte Testabläufe manuell ausgeführt.

Das Problem dabei ist:

1. Er macht diese, wenn er glaubt, dass es Änderungen in diesen Programmteilen gab. Das kann er aber nicht immer wissen.
2. Es ist sehr zeitintensiv und dadurch teuer.
3. Der manuelle Tester muss anwesend sein, ansonsten wird der Test nicht durchgeführt.

Genau solche wiederkehrende Tests eignen sich perfekt für eine Automatisierung.

Automatische Tests sind auch ein wichtiger Bestandteil in der Continuous Integration / Continuous Delivery Pipeline.

Continuous Integration and Continuous Delivery (CI/CD)

Continuous Integration ist ein Ansatz, bei dem Entwickler ihren Code mehrmals täglich in ein gemeinsames Repository einbringen. Zur Überprüfung des integrierten Codes werden automatisierte Tests und Builds durchgeführt.

Continuous Delivery ist eine Strategie, bei der die Entwicklungsteams sicherstellen, dass die Software jederzeit zuverlässig freigegeben werden kann. Bei jedem Commit durchläuft die Software den automatisierten Testprozess. Wenn sie die Tests erfolgreich durchläuft, ist sie bereit für die Freigabe zur Produktion.

{align=center}

Testpyramide

Die Testpyramide veranschaulicht von unten nach oben den Zeitaufwand und die Kosten der Tests.

An unterster Stelle, also am schnellsten und mit den niedrigsten Kosten sind die Unit-Test und an oberster Stelle sind die manuellen Tests.

{align=center}

Test Typen

Die automatischen Tests können in drei Hauptgruppen unterteilt werden.

Unit-Tests

Unit-Tests testen die kleinsten Einheiten (Units) einer Software auf ihre Funktionalität.

Ziel ist es, jede einzelne Komponente unabhängig und isoliert von anderen Daten und externen Einflüssen zu prüfen.

z.B. Funktionen in Dates -> DatesTest

Integrationstests

Das Ziel von Integrationstests besteht darin, Fehler aufzudecken, die ein Unit-Test nicht finden kann, da man von anderen Einflüssen wie Daten aus Datenquellen abhängig ist.

Bei uns sind das u.a die Tests, die von Daten einer Datenbank abhängen, also unser Business-Code.

End-to-End-Tests (E2E) oder UI-Tests

Mit E2E-Tests wird das gesamte Softwareprodukt von Anfang bis Ende getestet, um sicherzustellen, dass sich der Anwendungsfluss wie erwartet verhält.

Der Hauptzweck der E2E-Tests besteht darin, anhand der Erfahrungen des Endbenutzers zu testen, das reale Benutzerszenario zu simulieren und das zu testende System und seine Komponenten auf Integration und Datenintegrität zu überprüfen.

Test Framework - TestNG

Als Test Framework verwenden wir TestNG.

TestNG ist ein Test-Framework, das von JUnit und NUnit inspiriert ist, aber einige neue Funktionen einführt, die es leistungsfähiger und einfacher zu benutzen machen, wie z. B.:

- Annotationen
- Ausführung der Tests in beliebig großen Thread-Pools mit verschiedenen Richtlinien (alle Methoden in einem eigenen Thread, ein Thread pro Testklasse, etc...)
- Tests, ob Code multithread-sicher ist
- Flexible Testkonfiguration
- Unterstützung für datengesteuerte Tests (mit @DataProvider)
- Unterstützung für Parameter
- Leistungsstarkes Ausführungsmodell (keine TestSuite mehr)
- Unterstützung durch eine Vielzahl von Tools und Plug-Ins (Eclipse, IDEA, Maven, etc...)
- Einbettung der BeanShell für mehr Flexibilität
- Standard-JDK-Funktionen für Laufzeit und Protokollierung (keine Abhängigkeiten)
- Abhängige Methoden für das Testen von Anwendungsservern

TestNG ist so konzipiert, dass es alle Kategorien von Tests abdeckt: Unit-, Funktions-, Integrations-, End-to-End-Tests

Dokumentation:

<https://testng.org/doc/documentation-main.html>

Suite

Ein Test startet immer mit einer Suite. Eine Suite kann einen oder mehrere Tests beinhalten.

Eine Suite kann mit einer XML-Datei (alternativ mit einer YAML-Datei) beschrieben werden.

Wird ein Test ohne diese XML-Datei ausgeführt, ermittelt IntelliJ bzw. TestNG die Suite automatisch. Dabei werden aber die Test-Klassen in beliebiger Reihenfolge ermittelt (nicht pro Package oder sortiert nach Klassennamen) und ausgeführt.

Vor allem bei Integrationstests, wo wir auf Datenbanken aufsetzen und

Applikationswechsel vornehmen, muss eine beschriebene Suite erstellt werden.

z.B. kann in einer Suite eine Test-Gruppe pro Package definiert werden, die jeweils eine eigene Datenbank verwendet.

```
~~~xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="All Test Suite">
  <test verbose="2" preserve-order="true" name="ObstPool Standard">
    <packages>
      <package name="it.prodata.gest.obst.pool.data"/>
    </packages>
  </test>
  <test verbose="2" preserve-order="true" name="ObstPol Verband">
    <packages>
      <package name="it.prodata.gest.obst.pool.verband"/>
    </packages>
  </test>
</suite>
~~~
```

- verbose: definiert den Umfang des Logging
- preserve-order: soll die Ausführung in der angegebenen Anordnung ausgeführt werden (default=true)

Auch bei einem Build kann pro Maven-Modul auf eine testng.xml-Datei zurückgegriffen werden,

indem man beim Surefire-Plugin die Konfiguration angibt.

Der Kontext, in der die Datei gesucht wird, ist der Modul-Ordner (z.B. ObstPool).

```
~~~xml
<suiteXmlFiles>
  <suiteXmlFile>testng.xml</suiteXmlFile>
</suiteXmlFiles>
~~~
```

Da noch nicht alle Module Tests haben, muss diese Konfiguration in der POM des Moduls angegeben werden und noch nicht im Root-POM.

Beim Schreiben einer neuen Testklasse muss diese dann in der testng.xml-Datei des Moduls (falls vorhanden) und in der testng-all.xml-Datei auf Root-Ebene eingefügt werden.

Maven

Maven integriert die Ausführung der Tests im Build und aktiviert diese standardmäßig. Um sie zu deaktivieren muss die Option `'-DskipTests'` angegeben sein, was wir bis jetzt immer gemacht haben.

Um nur die Tests auszuführen:

```
~~~  
mvn test  
~~~
```

Mit der Option `'-Dmaven.test.failure.ignore=true'` kann gesteuert werden, dass ein Test-Fehler den Build nicht als fehlerhaft sieht.

!!s

Wenn bei einem Maven-Modul Tests für mehrere Applikationen vorhanden sind (Programmvarianten), muss für das entsprechende Modul die Test-Suite mit der `testng.xml`-Datei definiert und diese in der `pom`-Datei eingetragen werden.

Dies ist notwendig damit man die Reihenfolge der Ausführung der Tests bestimmen und diese pro Applikation gruppieren kann.

Ansonsten wird eine Applikation öfters gestartet und Testdatenbanken dazu neu erstellt.

!!

IntelliJ

Unter Run/Edit Configurations können für TestNG die Launchers definiert werden. Es gibt mehrere Möglichkeiten, die auszuführenden Tests zu bestimmen.

- All in Package: Alle Tests im angegebenen Package
- Class: Eine bestimmte Klasse
- Method: Eine bestimmte Methode
- Group: Eine Gruppe z.B. integration oder unit
- Suite: Eine Suite, die mit einer XML- oder YAML-Datei definiert wird
- Pattern: Tests, die einem bestimmten Pattern entsprechen

Ein Test kann auch auf Klassen- oder Methoden-Ebene direkt aus dem Quellcode gestartet werden, dabei wird als Arbeitsverzeichnis `$MODULE_WORKING_DIR$` verwendet und das ist der Root-Ordner jedes Moduls. D.h. es werden dort u.U. Dateien angelegt und diese sollten nicht committed werden.

Wenn man den Test öfters braucht, kann man den Launcher ändern und das Arbeitsverzeichnis anpassen.

Um die Suite-XML-Datei (`testng.xml`) zu generieren gibt es in IntelliJ das Plugin `Create TestNG XML`, das über das Kontext-Menü eines Moduls oder auf Root-Ebene ausgeführt werden kann.

Die generierte `testng.xml`-Datei landet im Root-Ordner.

Bei einer Ausführung auf Root-Ebene wird IntelliJ ca. 5 Minuten komplett blockiert.

Die erstellte Datei ist nicht formatiert, dies kann mittels `Code/Reformat Code` gemacht werden.

In der Datei werden Fehler angezeigt, da auf Root-Ebene Modul-Abhängigkeiten nicht

gegeben sind. Diese Fehler können ignoriert werden.

Im Ordner runConfigurations befinden sich Vorlagen.

Tests schreiben

Namenskonvention

Test-Klassen sollten mit Test aufhören und Methoden mit test anfangen. Grund dafür ist u.a. die Namenskonvention von Clean-Coding, Klassen als Nomen und Methoden als Verb zu benennen.

z.B.

- Klasse: WebTest
- Methode: testLogin

Annotation @Test

Die Annotation '@Test' markiert eine Klasse oder eine Methode als Teil des Tests. Sie wird üblicherweise bei den Test-Methoden deklariert.

```
~~~java
public class WebTest extends UnitTestCase {
    @Test()
    public static void testLogin() {
    }
}
~~~
```

Bei der Klasse müsste man die Annotation '@Test' nicht anführen. Wir machen das aber, da wir in einer Klasse nicht Unit- und Integration-Tests gemischt haben können und deshalb bei der Klasse bereits die Gruppe definieren.

```
~~~java
@Test(groups = { "unit" })
public class WebTest extends UnitTestCase {
}
~~~
```

Ein Test kann auch mehreren Gruppen zugeordnet werden.

```
~~~java
@Test(groups = { "unit", "web" })
public class WebTest extends UnitTestCase {
}
~~~
```



~~~

#### #### Tests ausschließen

Wenn ein Test über eine beschriebene Suite ausgeführt wird, kann der Test dort einfach nicht angegeben werden.

Ansonsten kann man den Test mit der Annotation `@Ignore` bei der Klasse ausschließen. Die Annotation `@Test` darf dann nicht bei der Klasse angegeben werden. Diese kann in Kommentar gesetzt werden.

Die `@Test` Annotation bei den Methoden kann aber bleiben.

```
~~~java
@Ignore
//@Test(groups = { "unit" })
public class SchemaTest extends UnitTestCase {

}
~~~
```

Die `@Ignore` Annotation kann auch auf Package-Ebene gesetzt werden.

```
~~~java
@Ignore
package it.prodata.service;
~~~
```

#### #### Tests überspringen

Test können auch übersprungen werden, wenn man anhand von Abhängigkeiten z.B. eine grafische Oberfläche wird vorausgesetzt.

Das kann über eine `SkipException` realisiert werden.

z.B.:

```
~~~java
@BeforeClass
void ensureNotHeadless() {
 if (GraphicsEnvironment.isHeadless())
 throw new SkipException("Skipping test because is running in headless mode");
}
~~~
```

#### #### Assert

Die Prüfung, dass der ermittelte Wert einem erwarteten Wert entspricht, wird mit den `assert`-Methoden der Klasse `org.testng.Assert` durchgeführt.

Dort gibt es verschiedene Methoden wie z.B. `'assertTrue'`, `'assertEquals'`, `'assertNotEquals'`, `'assertNotNull'`, `'assertNull'`, `'fail'` usw.

Bei allen Varianten kann auch ein Text angegeben werden, der ausgegeben wird, wenn das Assert scheitert.

Das ist ziemlich hilfreich, um den Fehler schnell zu finden.

Bezüglich Sprache wird empfohlen, ab dem Base-Package in Deutsch zu schreiben und vorher in Englisch.

!!s

Es muss die Klasse `org.testng.Assert` verwendet werden und nicht `org.testng.AssertJUnit`  
!!

#### #### Abhängigkeiten von Methoden

Testmethoden können von anderen Testmethoden abhängig sein, welche vorher ausgeführt werden müssen.

Man kann die Priorität der Ausführung der Methoden in einer Klasse mit `'priority'` bestimmen.

```
~~~java
public class WebTest {

 @Test (priority=1)
 public void testOpenWebUI() {
 }

 @Test (priority=2)
 public void testLogin() {
 }
}
~~~
```

Die bessere Variante ist aber die Definition von Abhängigkeiten, das funktioniert mit `'dependsOnMethods'`.

Scheitert der Test der abhängigen Methode, dann wird der Test der Methode übersprungen.

```
~~~java
public class WebTest {

 @Test
 public void testOpenWebUI() {
 }

 @Test(dependsOnMethods = {"testOpenWebUI"})
 public void testLogin() {
 }
}
~~~
```

#### #### Given-When-Then

{align=center}

#### #### AbstractTestCase

Die Tests werden in einer Virtual Machine ausgeführt und da wir alle Tests in einer Ausführung durchlaufen möchten, müssen wir einen Applikationswechsel durchführen können.

Dieser Applikationswechsel ist in der Basisklasse AbstractTestCase implementiert. Bei diesem Wechsel müssen die statischen Ressourcen abgegeben werden und alle concreteClasses müssen zurückgesetzt werden.

Die konkreten Business- und Manipulator-Klassen werden automatisch über die Guides zurückgesetzt. Andere betroffene Stellen müssen durch die Registrierung von "Aufräumen" bei der Applikation bereinigt werden (siehe App.registerTestRunCleaner(Runnable)), z.B. in der Klasse BsExtraField:

```
~~~java
/** @see App#cleanUpTestRun() */
private static void cleanUpTestRun() {
 specialFieldTypes = null;
 maxCount = -1;
}

/** Registriert einen speziellen Feldtyp. */
public static synchronized void registerSpecialFieldType(SpecialFieldType sft) {
 if (specialFieldTypes == null) {
 specialFieldTypes = new SortedList<>(Fun.getIgnoreCaseNameComparator());
 App.registerTestRunCleaner(BsExtraField::cleanUpTestRun);
 }
 specialFieldTypes.addIfAbsent(sft);
}
~~~
```

#### #### Unit Tests (Basisklasse UnitTestCase)

Die Unit Tests sollten von UnitTestCase abgeleitet werden, damit sie in einer Applikation ausgeführt werden und eine Session benutzen können.

Der Test sollte der Gruppe 'unit' zugeordnet werden.

```
~~~java
@Test(groups = { "unit" })
public class DatesTest extends UnitTestCase {
}
~~~
```

#### #### Integrationstests (Basisklasse IntegrationTestCase)

Integrationstests erfordern eine bestimmte Applikation. Im Konstruktor der Test-Klasse

wird die dazu erforderliche konkrete ApplicationFactory angegeben. Es stehen bereits Factory-Klassen für alle Applikationen zur Verfügung, z.B. FaktApplicationFactory.

```
~~~java
public BsArtikelTest() {
 super(new FaktApplicationFactory());
}
~~~
```

Je nach Bedarf des Integrationstests stehen weitere Basisklassen zur Verfügung:

- BsTestCase: verwaltet Zugriff auf Datenbanken, Abfrage von Objekten und Überprüfung von Abfrageergebnissen.
- BaseTestCase: verwaltet Zugriff auf Betriebsdaten.

Ab dem Package bsf müssen wir auf Daten zugreifen können. Sobald dies erforderlich ist, brauchen wir Testdaten.

#### ##### Allgemein

Als Testdaten werden die im Ombis-Projekt im Ordner testdata zur Verfügung gestellten Daten verwendet (siehe Dokumentation zu Testdatenbanken).

#### ##### Konfiguration

Es wird die Environment Variable 'TEST\_HOME' benötigt, welche den Startordner definiert. Die Integrationstests werden im Unterordner 'Integration' verwaltet:

#### ##### TEST\_HOME

z.B. TEST\_HOME=C:\Develop\TestDir -> Ordner für  
Integrationstests=C:\Develop\TestDir\Integration

In diesem Ordner werden die Properties-Dateien gesucht, mit denen die Datenbank und die Argumente definiert werden. Diese kann jeder für sich individuell anpassen. Muster-Dateien befinden sich im Projekt-Ordner unter testing/properties. Es werden auch Umgebungsvariablen unterstützt, um beispielsweise Ordnerangaben in Abhängigkeit der TEST\_HOME-Einstellung verwalten zu können.

TIP: Die Dokumentation zu den Testdaten enthält auch einen Muster-Ordner für Integrationstests mit allen erforderlich Dateien.

#### - db.properties

Konfiguration der Datenbank

z.B.:

~~~

```
JDBC_DRIVER=com.microsoft.sqlserver.jdbc.SQLServerDriver
JDBC_URL=jdbc:sqlserver://<HOST_NAME>:<PORT>;DatabaseName=<DB_ALIAS>;SelectMethod=curs
or
HOST_NAME=localhost
PORT=1433
DB_PATHNAME=%TEST_HOME%\Integration\Data\DB
DB_EXTENSION=.mdf
USER=SYSDBA
```

```
PASSWORD=masterkey
```

```
~~~
```

```
- arg.properties
```

Die Argumente, die in die Umgebung (Environment) geladen werden, z.B. der Name der System-Datenbank und Präfix der Firmendatenbank.

z.B.:

```
~~~
```

```
SysDb=IntegrationTestSystem
```

```
FirmDb=IntegrationTestFirm
```

```
~~~
```

Test schreiben

Ein Integrationstest mit Datenbankzugriff muss von `BsfTestCase` oder im Falle von Betriebsdaten von `BaseTestCase` abgeleitet sein. Die Basisklassen kümmern sich automatisch um die Erstellung und Befüllung der Datenbanken mit den Testdaten. Dies wird nur bei einem Wechsel der konkreten Applikation (mit Berücksichtigung der Variante) durchgeführt, um unnötige Operationen zu vermeiden. Daher ist es auch wichtig, dass die Test-Klassen in der entsprechenden Reihenfolge ausgeführt werden (siehe Angaben zur Test-Suite weiter oben).

Der Test sollte der Gruppe `integration` zugeordnet werden

Auch muss `'singleThreaded=true'` angegeben sein, damit alles in einen Thread ausgeführt wird.

```
~~~java
```

```
@Test(groups = {"integration"}, singleThreaded=true)
```

```
public class BsArtikelTest extends BaseTestCase<FaktApp> {
```

```
    public BsArtikelTest() {
        super(new FaktApplicationFactory());
    }
}
```

```
~~~
```

Tests für mehrere konkrete Klassen nutzen

Um Tests in einem Basis-Package zu schreiben und dann in den konkreten Applikationen nutzen zu können, kann man sich mit einem `TestHelper`-Objekt behelfen.

Dieses Objekt ist nicht selbst ein Test, hat also keine Test-Annotations.

Im Package `fakt.data` gibt es z.B. die Klasse `BsArtikelTestHelper`. Dort werden Tests implementiert,

die dann in `BsArtikelTest` und `BsObstPoolArtikelTest` verwendet werden können.

Für das `TestHelper`-Objekt wurden die Basis-Klassen `ObjectTestHelper` und `BaseObjectTestHelper` implementiert,

die Hilfsmethoden für ein Business-Objekt enthalten.

Diese Klassen können auch in einem Test als Hilfsobjekt verwendet werden.

Entwicklung von Tests

Die Entwicklung von Tests läuft meistens in Schleifen ab:

Test schreiben, Test ausführen, Test ergänzen/korrigieren, Test ausführen, ...

Bei jeder dieser Ausführungen werden die Datenbanken neu erstellt (je nach Applikation auch mehrere Firmendatenbanken), was ziemlich zeitaufwendig ist.

Hier kann der Startparameter `/SkipRecreateDB` nützlich sein, um bei einer wiederholten Ausführung eines Tests die Neuerstellung der Datenbanken zu unterdrücken.

Es gilt Folgendes zu beachten:

- Der Parameter wird nur im Entwickler-Modus (nicht obfuskiert) und beim Start der Applikation als Test ausgewertet.

Da Tests indirekt gestartet werden, muss der Parameter mittels Umgebungsvariable übergeben werden: `APP_ARGS=/SkipRecreateDB`

- Alternativ kann auch die Implementierung der Methode `BsfTestCase.getInitArgs` temporär geändert werden (Parameter ist als Kommentarzeile vorbereitet).

!!s

Die Datenbank muss klarerweise bei der ersten Ausführung der Tests erstellt werden, d.h. eine erste Ausführung

muss ohne diesen Parameter erfolgen. Zudem muss berücksichtigt werden, dass Tests auf Basis derselben Daten nur dann wiederholbar sind, wenn sie keine Daten ändern bzw. wenn sie die Daten nach Abschluss der Tests wieder auf den Ausgangszustand zurückversetzen.

!!

WSL2 & Docker für die Entwicklung

Windows Subsystem 4 Linux

Das Windows-Subsystem für Linux, kurz WSL (Windows Subsystem 4 Linux), ist eine in Windows 10 und Nachfolgeversionen

integrierte Kompatibilitätsschicht zum Ausführen von Linux-Executables im ELF-Format.

Der Nachfolger WSL 2 setzt stattdessen auf einen Virtualisierungsansatz mittels Hyper-V. Dies bietet neben einer besseren

Performance auch eine vollständige Kompatibilität von Systemaufrufen, da hier ein vollständiger Linux-Kernel zum Einsatz kommt.

Das Windows-Subsystem für Linux (WSL) unterstützt jetzt die Ausführung von Linux-GUI-Anwendungen (X11 und Wayland) unter

Windows in einer vollständig integrierten Desktopumgebung. Hierfür müssen die entsprechenden Grafik Treiber installiert werden

link:[<https://learn.microsoft.com/de-de/windows/wsl/tutorials/gui->

apps](https://learn.microsoft.com/de-de/windows/wsl/tutorials/gui-apps)[Linux GUI-Apps]

Installation von WSL 2

Ab Windows 11 wird standardmäßig das Subsystem in der Version 2 installiert.

```
~~~shell
wsl --install
~~~
```

Die aktuell installierte Version lässt sich jederzeit abfragen

```
~~~shell
wsl --version
~~~
```

Bei der Installation wird die **Default** Linux version installiert (Ubuntu). Über ein entsprechendes Kommando lassen sich die verfügbaren Versionen abfragen und eine spezifische Version mit dem Parameter **-d** installieren:

```
~~~shell
wsl --list --online
```

Nachstehend finden Sie eine Liste der gültigen Distributionen, die installiert werden können.

Führen Sie die Installation mithilfe des Befehls `wsl.exe --install <Distro>` aus.

| NAME | FRIENDLY NAME |
|------------------------------|------------------------------|
| Ubuntu | Ubuntu |
| Debian | Debian GNU/Linux |
| kali-linux | Kali Linux Rolling |
| Ubuntu-18.04 | Ubuntu 18.04 LTS |
| Ubuntu-20.04 | Ubuntu 20.04 LTS |
| Ubuntu-22.04 | Ubuntu 22.04 LTS |
| Ubuntu-24.04 | Ubuntu 24.04 LTS |
| OracleLinux_7_9 | Oracle Linux 7.9 |
| OracleLinux_8_7 | Oracle Linux 8.7 |
| OracleLinux_9_1 | Oracle Linux 9.1 |
| openSUSE-Leap-15.6 | openSUSE Leap 15.6 |
| SUSE-Linux-Enterprise-15-SP5 | SUSE Linux Enterprise 15 SP5 |
| SUSE-Linux-Enterprise-15-SP6 | SUSE Linux Enterprise 15 SP6 |
| openSUSE-Tumbleweed | openSUSE Tumbleweed |

```
wsl --install -d Ubuntu-24.04
~~~
```

Einmal installiert, lässt sich sowohl von Windows auf das Linux Subsystem als auch von Linux auf

die Umgebung von Windows zugreifen. Da Windows **C:** Dateisystem ist unter Linux automatisch auf **/mnt/c** angebunden.

!!w

Der Zugriff von Windows auf Linux und umgekehrt reicht für einfache Operationen aus, ist aber nicht

geeignet permanent auf von einem System auf dem anderen zuzugreifen.

!!

{align=center}

Das installierte Subsystem wird von Windows wie eine eigene Applikations behandelt.

Zum entfernen des Subsystems

kann das Subsystem (z.B. Ubuntu) aus den Apps von Windows deinstalliert werden.

Docker auf dem Subsystem

Es bietet sich an Docker direkt im Subsystem zu installieren anstatt auf dem Windows Host System. Für Docker auf Windows ist die Installation von Docker Desktop notwendig, welches nur für den persönlichen Gebrauch kostenlos ist.

Die Installation von Docker auf dem Subsystem kann direkt über das Windows Terminal erfolgen, indem man in das Subsystem einsteigt und über die Shell entsprechende Kommandos ausführt.

~~~shell

wsl

# Aktualisieren der Umgebung und Installation der Basis Bibliotheken

sudo apt-get update

sudo apt-get install -y apt-transport-https ca-certificates curl software-properties-common libssl-dev libffi-dev git wget nano

# Erstellen der docker Gruppe. Der aktuelle Benutzer muss dieser Gruppe zugewiesen werden.

sudo groupadd docker

sudo usermod -aG docker \${USER}

# Docker key and repository für den Bezug der Docker Installation

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -

sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu \$(lsb\_release -cs) stable"

# Aktualisieren des Paketmanagers

sudo apt-get update

# Upgrade der Umgebung

sudo apt-get upgrade -y

sudo apt-get autoremove -y



```
# Installieren von docker
sudo apt-get install -y docker-ce containerd.io

# (optional) Installation von Docker Compose
sudo curl -sSL https://github.com/docker/compose/releases/download/`curl -s
https://github.com/docker/compose/tags | \
grep "compose/releases/tag" | sed -r 's|.*([0-9]+\.[0-9]+\.[0-9]+).*\|1|p' | head -n
1`/docker-compose-`uname -s`-`uname -m` \
-o /usr/local/bin/docker-compose && sudo chmod +x /usr/local/bin/docker-compose

# Registrieren für den Start im standardprofile des Benutzers
echo "sudo service docker start" >> ~/.profile

# exit and then restart WSL
exit
~~~
```

Damit alle Einstellungen greifen, sollten die Linux Instanz nochmals gestartet werden.

Oder es wird VSCode verwendet um auf dem Subsystem zu arbeiten. Hierfür muss die VSCode Extension **\*\*WSL\*\*** installiert werden und anschliessend VSCode mit dem Subsystem über WSL verbunden werden.

```
{align=center}
```

```
{align=center}
```

```
!!w
```

VSCode arbeitet ab diesem Zeitpunkt auf dem Subsystem anstatt auf Windows.

```
!!
```

Die Installation von Docker kann nun im Terminal von VSCode oder im Windows Terminal über WSL erfolgen

```
~~~shell
```

```
# Aktualisieren des Paket repositories auf Ubuntu als superuser (sudo)
```

```
sudo apt update
```

```
# Installieren der Docker und Docker-Compose
```

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
```

```
# Verifizieren ob der Docker Service als Dienst läuft (Output muss 'active' sein)
```

```
sudo systemctl is-active docker
```

```
# Den aktuellen Benutzer (User) in die Grupper der Docker user hinzufügen
```

```
sudo usermod -aG docker ${USER}
```

```
~~~
```

Ab diesen Zeitpunkt können die Images & Container direkt aus der VSCode Umgebung verwaltet werden.

{align=center}

Eine Docker Umgebung kann so eingerichtet werden, dass die Daten im Docker Container gehalten werden oder dass sie im Filesystem des Subsystem gespeichert werden. Die Daten bleiben auch dem löschen des Containers erhalten. z.B. einen Ordner **\*/data\*** auf dem Subsystem welches für die unterschiedlichen Container als **volume** verwendet wird.

Eine Docker Instanz kann mittels eines **Dockerfile** oder mittels **docker-compose.yml** definiert werden. Docker compose eignet sich besonders wenn 2 oder Docker Images als Einheit verwaltet werden muss.

Im Substem wird eine Ordner für die Docker Umgebung eingerichtet mit einer Docker Compose Datei **docker-compose.yml**. Die Image lässt sich direkt über VSCode installieren und der Container kann direkt gestartet werden.

{align=center}

In der VSCode Docker Sicht lässt sich die Docker Container verwalten und die **Shell** des containers öffnen.

{align=center}

Wenn auf die Integration bzw. den direkten Zugriff von VSCode auf das Subsystem verzichtet werden kann, ist es auch möglich als Alternative IntelliJ zu verwenden um Docker Container zu verwalten.

{}

### ### Docker Datenbanken

Die Verwendung von Datendatenbanken in einem Dockersystem ermöglicht das schnelle Einrichten einer Testumgebung für die Entwicklung ohne die Software direkt im Host System zu installieren und konfigurieren zu müssen. Die Datenbank kann jederzeit in Sekunden in den Ursprungszustand zurückversetzt werden.

### #### Firebird unter Docker

Um Firebird in Docker auszuführen, reicht die Definition einer **docker-compose.yml** Datei.

```
~~~text
services:
```

```

firebird:
  image: firebirdsql/firebird
  restart: always
  environment:
    - FIREBIRD_ROOT_PASSWORD=masterkey
    - FIREBIRD_USER=SYSDBA
    - FIREBIRD_PASSWORD=masterkey
    - FIREBIRD_DATABASE_DEFAULT_CHARSET=UTF8
  ports:
    - 3050:3050
#   volumes:
#     - /data/firebird:/var/lib/firebird/data
~~~

```

Die Firebird Image hält die Daten immer im Ordner **\*\*/var/lib/firebird/data\*\***. Optional können die Daten auf einem persistenten volume auf dem Subsystem ausgelagert werden. In der Konfiguration von Ombis muss allerdings immer der lokale Ordner angegeben werden.

```
{align=center}
```

#### #### SQLServer unter Docker

Die Erstellung eines SQL Server containers ist analog zu jener von Firebird

```

~~~text
services:
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2017-latest
    environment:
      - ACCEPT_EULA=Y
      - MSSQL_PID=Developer
      - MSSQL_SA_PASSWORD=masterkey
    user: root
    ports:
      - 1433:1433
#   volumes:
#     - /data/mssql:/var/opt/mssql
    restart: always
~~~

```

Bei SQLServer muss zusätzlich der Benutzer und die Datenbank erstellt werden.

```

~~~shell
/opt/mssql-tools/bin/sqlcmd -S localhost -U sa -P Masterkey123

CREATE LOGIN SYSDBA WITH PASSWORD = 'masterkey', CHECK_POLICY = OFF
go
sp_grantdbaccess 'SYSDBA'

```

```

go
sp_addsrvrolemember 'SYSDBA', 'sysadmin'
go
CREATE DATABASE SoftDevTestSystem
go
quit
~~~

```

Die MS SQLServer Image hält die Daten immer im Ordner **\*\*/var/opt/mssql\*\***. Diese Angabe ist in Ombis nicht nötig.

```
{align=center}
```

#### #### PostgeSQL unter Docker

```

~~~text
services:
  postgres:
    image: postgres
    restart: always
    # set shared memory limit when using docker-compose
    shm_size: 128mb
    # or set shared memory limit when deploy via swarm stack
    #volumes:
    # - type: tmpfs
    #   target: /dev/shm
    #   tmpfs:
    #     size: 134217728 # 128*2^20 bytes = 128Mb
    environment:
      POSTGRES_USER: SYSDBA
      POSTGRES_PASSWORD: masterkey
    ports:
      - 5432:5432
#   volumes:
#     - /data/pgdata:/var/lib/postgresql/data
~~~

```

Die PostgreSQL Image hält die Daten immer im Ordner **\*\*/var/lib/postgresql/data\*\***. Diese Angabe ist in Ombis nicht nötig.

```
{align=center}
```

// NB: tabsize und source-highlighter:coderay werden von AsciiDocFX nicht unterstützt. Sie sollten daher beim Generieren der Dokumente (PDF, HTML) als Kommandozeilen-Parameter angegeben werden.  
 //:tabsize: 4

```
//:source-highlighter: coderay
```

## ## Java Code Style

### ### Operatoren

Unäre und binäre Operatoren:

```
~~~java
```

```
boolean success = !error;
```

```
int sum = a + b;
```

```
~~~
```

Zuweisungs-Operatoren:

```
~~~java
```

```
i++;
```

```
sum += val;
```

```
~~~
```

Umbrüche:

```
~~~java
```

```
String info = "Bei Umbrüchen in Ausdrücken kommt der Operator"  
    + " in die nächste Zeile, sowohl bei Strings als auch bei Zahlen.";
```

```
~~~
```

Bei verschachtelten Operationen kann Punkt vor Strich vorausgesetzt werden:

```
~~~java
```

```
y = 2 * x + c;
```

```
res = (a + b) * (a - b);
```

```
~~~
```

Bei verschiedenen logischen Operationen sollte immer explizit geklammert werden:

```
~~~java
```

```
boolean p = (a && b) || (b && c);
```

```
~~~
```

Beim ternären Operator werden kurze Ausdrücke in einer Zeile geschrieben und längere bei den Operatoren umgebrochen:

```
~~~java
```

```
x = a ? b : c;
```

```
wrapped = longerCondition
```

```
    ? longerValue1
```

```
    : longerValue2;
```

```
~~~
```

Es gibt auch Fälle, wo sich andere Schreibweisen anbieten, die an entsprechende `if-else` Muster angelehnt sind.

Bei Behandlung von Spezialfällen kann die Spezialbedingung und der entsprechende Wert

zusammen in einer Zeile stehen,  
der "normale" Wert folgt dann in der nächsten (Anlehnung an Guard-Clauses).

```
~~~java
wrapped = specialCondition ? specialValue
      : normalValue;
~~~
```

Verschachtelte ternäre Ausdrücke können leicht unübersichtlich werden. Um dem entgegenzuwirken, kann eine Bedingung mit dem jeweiligen Wert immer in der selben Zeile stehen, in der nächsten Zeile folgt nach `:` das nächste Bedingung-Wert-Paar usw. bis zum Default-Wert in der letzten Zeile (Anlehnung an `if-elseif-else`).

```
~~~java
nested = condition1 ? value1
      : condition2 ? value2
      : condition3 : value3
      : defaultValue;
~~~
```

### if, else, while, for

Keine geschweiften Klammern bei einzeliliger Bedingung und einzeliligem Statement.

```
~~~java
if (shortCondition)
    shortStatement();
else if (shortCondition)
    shortStatement();
else (shortCondition)
    shortStatement();

for (int i = 0; i < 10; i++)
    sum += i;

for (String name : names)
    System.out.println(name);
~~~
```

Geschweifte Klammern bei mehrzeiligem Body

```
~~~java
if (shortCondition) {
    statement1();
    statement2();
}
else if (shortCondition) {
    singleLongStatement(
        spanning, multiple, lines
    );
}
```

```

else if (shortCondition)
    shortStatement();
else (shortCondition) {
    statement1();
    statement2();
}
~~~

```

Geschweifte Klammern bei mehrzeiligen Bedingungen.

```

~~~java
while (complex || (condition && spanning
    && multiple && lines))
{
    shortStatement();
}
~~~

```

Immer mit geschweiften Klammern bei `do-while`.

```

~~~java
do {
    statement();
} while (condition);
~~~

```

### switch

Herkömmliches `switch`:

```

~~~java
switch (legacySwitch) {
    case SIMPLE:
        handleSimpleCase();
        break;
    case FALL_THROUGH1:
    case FALL_THROUGH2:
        handleFallThrough(type);
        break;
    case WITH_SCOPED_VAR: {
        var scoped = computeTempValue(type);
        use(scoped);
        break;
    }
    default:
        handleDefault();
        break;
}
~~~

```

Enhanced switch, wird bevorzugt. Wenn möglich als Expression und ohne `default`:

```

~~~java
return switch (simpleExpression) {
    case ZERO -> 0;

```

```

    case ONE -> 1;
};
~~~

```

Umbrüche in enhanced switch ohne geschwungene Klammern:

```

~~~java
switch (wrappingWithoutBraces) {
    case WRAPPED_BODY -> wrapNormally(
        "as you do outside switch"
    );
    case CASE_AND_BODY_ONE_LINE_EACH ->
        lineBreakAfterArrow();
    case MANY,
        DIFFERENT_CASES,
        IN_MULTIPLE_LINES ->
        additionalIndentationAfterArrow();
}
~~~

```

Umbrüche in enhanced switch mit geschwungenen Klammern:

```

~~~java
switch (wrappingWithBraces) {
    case SIMPLE -> {
        openingBraceInSameLine();
        closingBraceOnLevelOfCase();
    }
    case MANY,
        DIFFERENT_CASES,
        IN_MULTIPLE_LINES ->
    {
        lineBreakAfterArrow();
        bracesOnLevelOfCase();
    }
    default -> {}
}
~~~

```

Der Übersicht halber sollten Konstanten innerhalb desselben Cases auf getrennten Zeilen stehen und mit einem zusätzlichen Space nach der Einrückung so ausgerichtet werden, dass alle auf der gleichen Ebene beginnen.

Das erleichtert die Kontrolle, ob alle Fälle korrekt berücksichtigt wurden. Zusammengehörige Gruppen können in einer Zeile zusammengefasst werden.

Hier ein konkretes Beispiel:

```

~~~java
return switch (op) {
    case OP_NULLBELEG,                                // eigenständig
        OP_INBETRIEBNAHME, OP_AUSSERBETRIEBNAHME,    //
}

```



```

zusammengehörige Gruppe
    OP_TAGESABSCHLUSS, OP_MONATSABSCHLUSS, OP_JAHRESABSCHLUSS, //
zusammengehörige Gruppe (Abschlüsse)
    OP_NACHERFASSUNG -> true;                                // eigenständig
    default -> super.isOperationSupported(op);
};
~~~

```

`OP\_INBETRIEBNAHME` bildet mit `OP\_AUSSERBETRIEBNAHME` eine zusammengehörige Gruppe, wie auch `OP\_TAGESABSCHLUSS` mit `OP\_MONATSABSCHLUSS` und `OP\_JAHRESABSCHLUSS`. Daher befinden sich diese Konstanten jeweils in einer Zeile. `OP\_NULLBELEG` und `OP\_NACHERFASSUNG` hingegen repräsentieren einen eigenständigen Fall, der nur gleich wie Fehlerstatus behandelt wird, und befindet sich daher in einer eigenen Zeile innerhalb desselben Cases.

```

try

Try-catch-finally:
~~~java
try {
    operation();
}
catch (SomeException e) {
    handle(e);
}
catch (OtherException | YetAnotherException e) {
    handle(e);
}
finally {
    cleanup();
}
~~~

```

```

Try-with-ressources:
~~~java
try (var resource = open()) {
    use(resource);
}

try (var resource1 = open();
    var resource2 = open())
{
    use(resource1);
    use(resource2);
}
~~~

```

```

var

```

`var` sollte mit Bedacht eingesetzt werden. Es bietet sich an, wenn der Typ klar ist:

```
~~~java
var list = new ArrayList<String>();
~~~
```

\... der Typ uninteressant ist:

```
~~~java
try (var tx = db.transaction()) {
    ...
    tx.commit();
}
~~~
```

\... oder für Typen mit langem Namen oder komplexen Generics:

```
~~~java
Stream<List<String>> sentences = ...
var sentencesByLength = lists.collect(groupingBy(List::size));

for (var entry : sentencesByLength.entrySet()) {
    ...
}
~~~
```

Für primitive Typen verwenden wir nie `var`. Auch für kurze und viel benutzte Typen wie `String` oder `Date` bietet `var` wenig Vorteile.

Zudem ist zu beachten, dass `var` automatisch den Typ des Initialisierungs-Ausdrucks annimmt. Wenn also dort eine Methode aufgerufen wird, und deren Rückgabebetyp ändert sich in Zukunft, dann ändert sich auch der Typ der Variable. Wo das zu Problemen führt, die der Compiler nicht erkennen kann, sollte `var` vermieden werden.

### ### Lambdas und Methoden Referenzen

Einfache Lambdas:

```
~~~java
value -> 2 * value

(key, val) -> method(
    key, val, otherParams
)

() -> {
    simpleBlock();
    whenNecessary();
}
~~~
```

Komplexe Lambdas sollten vermieden werden, außer bei Wrapper-Methoden, welche nur etwas Boilerplate verbergen:

```
~~~java
var result = runInWaitState(() -> {
    var local = computeValue();
    // ... Komplexe Berechnung
    return res;
});
~~~
```

Ansonsten sollen komplexe Lambdas als private Funktionen ausgelagert und wenn möglich zu Methodenreferenzen umgebaut werden:

```
~~~java
values.stream()
    .filter(this::complexFilter)
    .map(v -> transform(v, otherParam))
    .forEach(this::handleTransformedValues);
~~~
```

### ### Methoden

Mehrere `return`s sind erlaubt, z.B. für Guard-Clauses oder als Abbruch aus einer Schleife.

Man sollte damit aber nicht übertreiben, vor allem bei komplexen Methoden mit vielen verschachtelten Kontrollstrukturen.

```
~~~java
String findEmail(User user, String searchterm) {
    if (user == null)
        return null;

    for (String email : user.getEmailAddresses()) {
        if (matches(email, searchterm))
            return email;
    }
    return null;
}
~~~
```

Guard-Clauses sind nützlich zum Behandeln von Spezialfällen, ohne den Normalfall einrücken zu müssen.

Bei "gleichberechtigten" Bedingungen jedoch sollte trotz `return` das `else` stehenbleiben, sodass beide Fälle gleich eingerückt werden:

```
~~~java
boolean hasPermissionFor(User user, Operation operation) {
    if (user.getRoles().contains(Role.ADMIN))
        return hasAdminUserPermissionFor(user, operation);
}
```

```

    else
        return hasOrdinaryUserPermissionFor(user, operation);
}
~~~

Umbrüche in Methodensignaturen:
~~~java
void simpleMethod(String param) {
    ...
}

void wrappedParams(String param1, String param2,
    String param3, String param4)
{
    ...
}
~~~

Umbrüche in Aufrufen:
~~~java
method(
    param1, param2, param3,
    param4, nestedMethod(
        nestedParam1, nestedParam2,
        nestedParam3
    )
);
~~~

Verkettete Aufrufe:
~~~java
var result = simple().chained().call();

var complex = suchAs.stream()
    .filter(elem -> passesFilter(elem))
    .map(this::method)
    .flatMap(this::otherMethod)
    .toList();
~~~

Klassen

Standard Klassen-Struktur:
~~~java
public class MyClass {

    public enum SmallEnum {
        CASE1,
        CASE2
    }
}

```

```

public static final int A_CONSTANT          = 1;
public static final int ANOTHER_CONSTANT    = 2;

private static final String PRIVATE_CONSTANT = "x";

static {
    ...
}

private final ArrayList<String> list;      // Member final falls möglich
private SmallEnum member;

public MyClass(SmallEnum param) {
    this.member = param;
    list = new ArrayList<>();
}

void method() {
    ...
}

private static class InnerClass {

    ...
}
}
~~~

Kommentare

Datei-Header:
~~~java
/*
 * MyClass  2000-12-31
 *
 * Copyright (c) Pro Data GmbH & ASA KG. All rights reserved.
 */

package it.prodata.example;
~~~

Zeilen-Kommentare:
~~~java
// Kommentar in ganzer Zeile.

// Auch für mehrzeilige Kommentare
// verwenden wir "//".

private int x;      // Kommentare können auch am Ende einer Zeile stehen

//private boolean commentedOut;

```

~~~

JavaDoc

Klassen-Header:

~~~java

/\*

\* MyClass 2000-12-31

\*

\* Copyright (c) Pro Data GmbH & ASA KG. All rights reserved.

\*/

package it.prodata.example;

import ...

/\*\*

\* Beschreibung des Konzepts, welches die Klasse repräsentiert, welche Rolle die Klasse im System spielt,

\* wie sie benutzt wird, welche Garantien sie bietet usw. (Immutability, Thread-Safety, Invarianten ...)

\*

\* @param <T> Beschreibung des Typ-Parameters.

\*

\* @author Max Mustermann

\* @since 2000-12-31

\*/

public class MyClass<T> {

~~~

Bei top-level Records sollten nicht triviale Komponenten dokumentiert werden:

~~~java

/\*\*

\* Beschreibung wie bei Klassen.

\*

\* @param count Die Anzahl der Reviews.

\* @param average Die durchschnittliche Bewertung.

\*

\* @author Max Mustermann

\* @since 2000-12-31

\*/

public record ReviewStats(int count, double average) {

}

~~~

Methoden:

~~~java

/\*\* Einzeilige Beschreibung einer Methode. \*/

public void simpleMethod() {

...

}

```

/**
 * Mehrzeilige Beschreibung einer Methode.
 * @param param1 Beschreibung eines Parameters.
 * @param param2 Beschreibung eines weiteren Parameters.
 *      Mehrzeilige Beschreibungen von Parametern werden am linken Rand des
 *      Namens ausgerichtet.
 * @return Beschreibung des Rückgabewerts.
 * @throws CustomException In welchem Fall wird die Exception geworfen?
 * @see RelatedClass#relatedMethod
 */
public String complexMethod(String param1, String param2) throws CustomException {
    ...
}
~~~

```

Inner-Classes:

```

~~~java
/** Einzeilige Beschreibung einer Inner-Class. */
private static class InnerClass {
    ...
}

/**
 * Inner-Classes können auch mehrzeilige Beschreibungen haben, brauchen aber keinen
 * Author und kein Erstelldatum.
 */
private static class InnerClass {
    ...
}
~~~

```

## ## Module

### ### Was ist ein Modul?

Ein **Modul** ist ein Bundle von bestimmten Ombis-Features eines Bereiches und dient dazu diese spezifisch pro Installation zu- oder wegzuschalten. Ist ein Modul deaktiviert so können die Klassen und/oder Funktionalitäten dieses Moduls nicht genutzt werden.

Die derzeit aktiven Module können entweder beim Start des Programmes mit dem Parameter **/env** oder in Ombis selbst unter **Einrichtung** -> **System** -> **Umgebung** eingesehen und geändert werden.

### ### Wie erstelle ich ein Modul?

In allen Ombis-Basis-Varianten (**Fakt**, **Obst**, **ObstPool**, **Prod** oder **Wein**) für die das neue Modul erstellt wurde, muss in der jeweiligen

**EnvironmentInfo-Klasse** zunächst ein neuer Schlüssel erstellt werden. Der Wert dieses Schlüssels kann mit Hilfe von **EdiDat** ermittelt werden.

#### #### EdiDat

**EdiDat** ist ein Zusatzprogramm, welches - ähnlich den verschiedenen Ombis-Varianten selbst - über eine Launch-Konfiguration in Eclipse gestartet werden kann. Dazu wird eine ebensolche Konfiguration mit dem **Project: Ombis** und der **Main class: it.prodata.gest.licence.edidat.Edidat** erstellt und ausgeführt. In dem nun offenen Tool kann über den Button **Hinzufügen** der Name des neuen Moduls eingegeben werden, von welchem direkt der Schlüssel angezeigt wird. Das Feld für **Wert** und das Häkchen **Für die Berechnung des Schlüssels verwenden?** kann hierbei freigelassen werden.

Der so erhaltene Wert kann mit einer kleinen Änderung nun als Schlüsselwert in die jeweilige EnvironmentInfo-Klasse eingetragen werden. Die Änderung besteht darin, dass vor jedem Block von vier Zeichen **\u** eingefügt wird. Der Wert besteht hierdurch nun aus Unicode-Zeichen.

Die Prozedere wird wiederholt für den Schlüssel der Bemerkung des erstellten Moduls. Der Name ist hierbei **[ModulName]Rem** (z.B. KundenkarteRem).

#### #### Check-Liste

Für die weitere Vorgehensweise wird empfohlen sich ein bestehendes Modul als Anhaltspunkt zu nehmen (gegebenenfalls einen Kollegen nach einem passenden Beispiel fragen), und ähnlich diesem vorzugehen. Als Gedächtnisstütze kann hierbei die nun folgende (potentiell unvollständige) Check-Liste dienen:

1. mit EdiDat die Schlüsselwerte für den Modulnamen und ModulnamenRem erstellen
2. die Schlüssel registrieren: **[AppVariante]EnvironmentInfo**-Klasse
3. die Abfrage des aktivierten Moduls registrieren: **[AppVariante]Licence**-Klasse ( **hasModule()** )
4. die Schlüssel an der Oberfläche registrieren und anzeigen:  
**[AppVariante]LicenceEditor**-Klasse und zuständige ResourceBundles
5. die betreffende(n) Klasse(n) im Startbaum registrieren und wegschalten, falls das Modul nicht aktiviert ist: **[AppVariante]App**-Klasse
6. Schritte 2 bis 5 für die betreffenden App-Varianten wiederholen

// NB: tabsize und source-highlighter:coderay werden von AsciiDocFX nicht unterstützt. Sie sollten daher beim Generieren der Dokumente (PDF, HTML) als Kommandozeilen-Parameter angegeben werden.

#### ## Persistence Framework

#### ### Business-, Manipulator-, Navigator- und Guide-Objekte

```
~~~uml{align=center}
class BsfDatabasePool {
    +startupPool(): void
```



```

+shutdownPool(): void
+clearPool(): void
+getDatabase(..): BsfDatabase
+releaseAll(BsfDatabaseID): void
+<..>Changed(..): void
+getCacheStatisticsInfo(): String
}
class BsfDatabase {
+open(): void
+close(): void
+beginUse(): void
+endUse(): void
+beginTransaction(): void
+commitTransaction(): void
+abortTransaction(): void
+transaction(): BsfTransaction
+runInTransaction(Callable<T>): T
+getObjectCollection(Class<?>): BsfCollection
}
class BsfCollection {
+getGuide(): Guide
+createNavigator(): BsfNavigator
+createManipulator(): BsfManipulator
+create(): Object
+add(Object): int
+update(Object): int
+remove(Object): int
+reload(Object): int
+addAll(Collection): int
+updateAll(Collection): int
+removeAll(Collection): int
+reloadAll(Collection): int
+getAll(boolean): Object[]
+getArray(CfgFilter, CfgOrder): Object[]
+getEnumeration(CfgFilter, CfgOrder): BsfEnumeration
+lookup(Object): Object
+getObject(CfgFilter, CfgOrder): Object
+getUniqueObject(CfgFilter, CfgOrder): Object
+isEmpty(): boolean
+getCount(CfgFilter): long
+exists(CfgFilter): boolean
+discard(Object): void
+discardAll(): void
+discardTree(): void
}
class BsfReference {
+getReferenceInfo(): ReferenceInfo
+getValue(): Object
+setValue(Object): void
+get(): Object
+getOrThrow(): Object

```

```

+set(Object): void
+isNull(): boolean
+isEmpty(): boolean
+create(): Object
#preUpdate(int): int
#postUpdate(int): int
#postCommit(int): void
}
class Navigator {
    +getGuide(): Guide
    +open(): boolean
    +close(): boolean
    +setFilterAndOrder(CfgFilter, CfgOrder): int
    +setFilter(CfgFilter): int
    +setOrder(CfgOrder): int
    +get(): Object
    +first(): boolean
    +last(): boolean
    +next(): boolean
    +previous(): boolean
    +skip(int): boolean
    +setPosition(long): boolean
    +size(): long
    +count(): long
    +refresh(Object): boolean
    +restart(Object, int): boolean
    +reload(): boolean
    +addNew(): boolean
    +edit(): boolean
    +cancelUpdate(): boolean
    +update(): boolean
    +delete(): boolean
}
class Manipulator {
    +getGuide(): Guide
    +getName(): String
    +attachObject(Object): void
    +checkValidity(MessageContext, MnValidation): boolean
    +getConfig(..): Cfg
    +getConfigs(..): Cfg[]
    +updateComponents(): void
    +getAccess(): int
    +setAccess(int): void
    #updateAccess(): void
    #getFieldValue(int): Object
    #setFieldValue(int, Object): void
    #createReferenceNavigator(int, boolean): Navigator
    #createCollectionNavigator(int): Navigator
}
class BsObject {
    +isEmpty(): boolean

```

```

+getCollection(Class, String): BsfCollection
+getReferenceCollection(Class, String): BsfCollection
+updateShallowly(): int
+undoAllChanges(boolean, boolean): void
+prepareForEditing(): boolean
#readOldValuesImpl(): void
#cleanUp(int): void
#postCreate(Object): void
#postReadImpl(): void
#postRefreshImpl(boolean, boolean): boolean
#preUpdateImpl(int, boolean, boolean): int
#postUpdateImpl(int, boolean, boolean): int
#preCommitImpl(int): int
#postCommitImpl(int): int
#transactionCompletedImpl(int): void
#transactionAbortedImpl(int, boolean): void
#referenceChanged(..): void
#collectionChanged(..): void
}

class Guide {
    +getName(): String
    +getBusinessClass(): Class
    +createManipulator(Session): Manipulator
    +getField(String): FieldInfo
    +getReference(name): ReferenceInfo
    +getCollection(name): CollectionInfo
    +getSearchFields(): List
    +getCaching(): int
    +getFetching(): int
}

```

```

BsfDatabasePool -[hidden]r- BsfDatabase
BsfDatabase     -[hidden]r- Navigator
BsfDatabasePool -d-> BsfDatabase : liefert
BsfDatabase     -d-> BsfCollection : liefert
BsfCollection   -u-> Navigator     : erzeugt
BsfCollection   -r-> Manipulator   : erzeugt
BsfCollection   -d-> BsObject      : verwaltet Persistenz
Navigator       -d-> Manipulator   : benutzt
Manipulator     -d-> BsObject      : manipuliert
Manipulator     -d-> Guide         : benutzt
Guide           -l-> BsObject      : beschreibt
BsObject "1"    -u-> "*" BsfCollection : referenziert
BsObject "1"    -l-> "*" BsfReference : enthält
~~~

```

#### #### Business-Objekte

Business-Objekte implementieren die Geschäftsregeln einer Anwendung. Sie weisen folgende Eigenschaften auf:

- Sie sind persistent: Wie diese Persistenz implementiert ist (RDBS mit Hilfe von jOOQ, VBSF, OODBS usw.) ist ein Implementierungsdetail, das nach außen hin möglichst verborgen bleiben soll. Die Persistenz wird mit Hilfe von Collection- und Navigations-Objekten verwaltet.
- Die Persistenz wird über (Bsf-)Collections verwaltet. Die Business-Objekte bekommen über entsprechende Methoden jeden Schritt der Verarbeitung mit und können so speziell darauf reagieren. So können beispielsweise Dokumente, die Lieferungen von Artikel verwalten, automatisch deren Bestand aktualisieren.
- Auf alle Business-Objekte kann mit Hilfe von entsprechenden Manipulator-Objekten auf eine generelle Art und Weise zugegriffen werden (lesen und ändern). Bei Änderungen ist es prinzipiell unerheblich, ob sie vom Benutzer innerhalb einer grafischen Desktop- oder einer Web-Oberfläche, über Datenimport oder über REST-Schnittstelle erfolgen.
- Alle persistenten Business-Klassen beginnen mit dem einheitlichen Präfix "`Bs`" (unabhängig von ihrer technischen Implementierung!).
- Nicht persistente (Speicher-) Business-Klassen haben in der Regel das Präfix "`Mem`".

```

~~~uml{align=center}
class BsBetrieb {
    -adresse: BsReference
    -kurzname: BOAString
    -plKopfzeilen: BOAString[]
    -slKopfzeilen: BOAString[]
    -colorAsString: BOAString
    -geschaeftsjahresbeginn: BOADate
    -geschaeftsjahre: BsfMemberCollection
    +getAdresse(): BsAdresse
    +getKurzname(): String
    +getKopfzeile(boolean, int): String
    +getColorAsString(): String
    +getColor(): Color
    +getGeschaeftsjahresbeginn(): Date
    +getGeschaeftsjahre(): BsCollection
}
~~~

```

#### #### Manipulator-Objekte

Manipulatoren sind Hilfsobjekte für UI-Komponenten, Navigations-Objekte und verschiedene weitere Tools. Sie implementieren eine Schnittstelle zum generellen Zugriff auf die Business-Objekte. Pro Business-Klasse gibt es eine entsprechende Manipulator-Klasse.

Das Business-Objekt, auf welches über Manipulator zugegriffen werden soll, wird über `attachObject(Object)` an den Manipulator angebunden. Mit einer einzelnen Manipulator-Instanz kann man also auf beliebig viele Objekte derselben Business-Klasse zugreifen.

Für die unterschiedlichen Elemente (Felder, Collections usw.) stehen eigene von `MnElement` abgeleitete Klassen zur Verfügung. Allen gemeinsam ist, dass Zugriffsrechte verwaltet werden können und dass sie datengebundene Komponenten für ein Formular

liefern können.

```
~~~uml{align=center}
class Manipulator {
    +getGuide(): Guide
    +getField(int): MnField
    +getReference(int): MnReference
    +getCollection(int): MnCollection
    +getComponent(int): MnComponent
    +getCommand(int): MnCommand
    +attachObject(Object): void
    +collectionChanged(Object source, int, Object): void
    +workingNotification(int, Object): void
    +startEditing(Object, boolean): void
    +cancelEditing(): void
    +editingSucceeded(): void
    +initializeDefaultValues(boolean, boolean): void
    +completeDefaults(): void
    +copyFrom(Object, boolean): void
    +checkValidity(Object, MnValidation): boolean
    +updateComponents(): void
    +getMaxAccess(): int
    +getAccess(): int
}
class MnElement {
    +getType(): String
    +getName(): String
    +getFullName(): String
    +getDisplayName(): String
    +create[Web]Component(): Component
    +detach[Web]Component(Component): void
    +updateComponents(): void
    +getMaxAccess(): int
    +getAccess(): int
}
class MnField extends MnElement {
    +getInfo(): FieldInfo
    +getReference(): MnReference
    +create[Web]CellRenderer(): DataRenderer
    +create[Web]CellEditor(): DataEditor
    +getFillConstraint(): int
    +isEmpty(): boolean
    +getValue(): Object
    +setValue(Object): void
    +getDataUnit(): String
    +getDataHandler(): DataHandler
    +checkValidity(Manipulator, MnValidation): void
}
class MnReference extends MnElement {
```

```

+getInfo(): ReferenceInfo
+getGuide(): Guide
+getFields(): MnField[]
+createReferenceManipulator(): Manipulator
+getReferenceManipulator(): Manipulator
+createNavigator(boolean): Navigator
+isNull(): boolean
+isEmpty(): boolean
+get(): Object
+getReferenceValue(): Object
}
class MnCollection extends MnElement {
+getInfo(): CollectionInfo
+getGuide(): Guide
+createCollectionManipulator(): Manipulator
+getCollectionManipulator(): Manipulator
+createNavigator(): Navigator
+getNavigator(): Navigator
}
class MnCommand extends MnElement {
+execute(): void
+onExecute(ExecuteFn)
+onDownload(ExecuteFn, DownloadConfig)
+onUpload(ExecuteFn, UploadConfig)
}
class MnComponent extends MnElement {
}

Manipulator "1" -d-> "*" MnField : enthält
Manipulator "1" -d-> "*" MnReference : enthält
Manipulator "1" -d-> "*" MnCollection : enthält
Manipulator "1" -d-> "*" MnCommand : enthält
MnField "*" -r-> "1" MnReference : referenziert
MnReference -[hidden]r- MnCollection
MnCollection -[hidden]r- MnCommand
~~~

```

- `MnElement`: Basisklasse für alle Manipulator-Elemente.
- `MnField`: ein Feld. Im Falle eines Referenzfeldes liefert `getReference()` das Element für die Referenz.
- `MnReference`: eine Referenz. `getFields()` liefert alle mit der Referenz verbundenen Felder (mit alternativen Suchbegriffen).
- `MnCollection`: eine Collection (Untersammlung).
- `MnComponent`: Element für spezielle Komponenten zur Bearbeitung von Objekten.

Für verschiedene Konfigurationen gibt es eigene Elemente, über welche die entsprechende Funktionalität in der Oberfläche zur Verfügung gestellt werden kann:

- `Mn[Web]Form`: Element für Unterformulare, die in andere direkt oder über Link-Buttons eingebettet werden können.

- ``MnPrint``: Element für Berichte, das eine entsprechende Schaltfläche für die Ausgabe des Berichts zur Verfügung stellt.
- ``MnExport``: Element für Exportdefinitionen, das eine entsprechende Schaltfläche zum Export von Daten zur Verfügung stellt.
- ``MnImport``: Element für Importdefinitionen, das eine entsprechende Schaltfläche zum Import von Daten zur Verfügung stellt.

#### #### Navigator-Objekte

Navigatoren bieten eine einheitliche Schnittstelle zur Verwaltung der Business-Objekte. Sie verbergen technische Implementierungsdetails und bieten z.T. Möglichkeiten, die eigentlich gar nicht zur Verfügung stehen würden (z.B. bidirektionales Scrolling, Suche nach Schlüsselwerten, Keyset und Offset Pagination usw.).

Die Navigator-Klasse orientiert sich sowohl im Konzept (Cursor) wie auch in der Namensgebung der Methoden am `java.sql.ResultSet` Interface. Gleichnamige Methoden verhalten sich äquivalent und sind im Kommentar gekennzeichnet. Ein wichtiger Unterschied ist aber die Positionsangabe, die beim Navigator von 0 startet, beim `ResultSet` hingegen bei 1. Ein weiterer Unterschied ist der, dass keine der Navigator-Methoden Exceptions auslöst. Sie liefern in der Regel einen logischen Wert, mit welchem der Erfolg der Ausführung angegeben wird. Außerdem steht die Methode `getLastError` zur Verfügung, welche den Fehlercode des letzten Aufrufs liefert. Somit können auch für Methoden wie `next`, `skip` usw. Informationen über evtl. aufgetretene Fehler eingeholt werden.

Im Wesentlichen umfassen die Navigatoren die

- Navigation in Datenbeständen (Filter- und Sortierkriterien können festgelegt werden).
- Suche nach Schlüsselwerten.
- Persistenz-Verwaltung der Business-Objekte:
  - Erzeugung neuer Business-Objekte.
  - Löschung von Business-Objekten aus der Datenbank.
  - Speicherung der Änderungen an Business-Objekten in der Datenbank.

In der Regel gibt es je nach technischer Grundlage (ORM über VBSF, JDBC-`ResultSet`, Speicherliste) eine eigene Navigator-Klasse. Das Navigator-Objekt benutzt ein Manipulator-Objekt zur Verwaltung der Business-Objekte. Dieser Manipulator wird vom Navigator über verändernde Operationen (Einfügen, Löschen, Ändern usw.) informiert. Somit kann ein und dieselbe Navigator-Klasse für unterschiedliche Business-Objekte benutzt werden, der (business-) klassenspezifische Programmcode kommt lediglich in den Business- und den entsprechenden Manipulator-Klassen vor.

```
~~~java
nav.setFilterAndOrder(filter, order);
if (nav.open()) {
    try {
        while (nav.next()) {
            Object obj = nav.get();
```

```

        ...
    }
}
finally {
    nav.close();
}
}
~~~

```

#### #### Guide-Objekte

Guide-Objekte beschreiben den statischen Aufbau der Business-Objekte und deren Beziehungen untereinander. Es gibt grundsätzlich je nach technischer Implementierung (VBSF usw.) der Business-Objekte unterschiedliche Guide-Klassen, die alle von der Basisklasse `Guide` abgeleitet sind. Sie werden durch ein einheitliches Präfix gekennzeichnet, z.B. "`BsfGuide`" für die VBSF-spezifische Implementierung.

Aus den in den Guide-Objekten registrierten Informationen wird auch die Datenbankstruktur abgeleitet und somit kann das Erzeugen, Aktualisieren und Umkopieren von Datenbanken weitgehend automatisiert werden.

Es gibt verschiedene Info-Klassen zur Beschreibung der einzelnen Elemente:

- `FieldInfo`: beschreibt ein Feld.
- `ReferenceInfo`: beschreibt eine Referenz.
- `CollectionInfo`: beschreibt eine Collection.

```

~~~uml{align=center}
class Manipulator {
    getGuide(): Guide
    getName(): String
}
class Guide {
    createManipulator(Session): Manipulator
    getField(String): FieldInfo
    getReference(name): ReferenceInfo
    getCollection(name): CollectionInfo
}
class FieldDef {
    getName(): String
    getFieldType(): int
    getDataType(): int
    getDataClass(): Class<?>
    getSize(): int
    getDecimals(): int
    isAutoUniqueValue(): boolean
    isNullable(): boolean
}
class FieldInfo extends FieldDef {
    isPersistent(): boolean
}

```



```

    isUnsigned()
    isSummable()
    isPassword()
    isExternalKey()
    isExportable()
    isDuplicable()
    isSecondary()
    isAutoFilled()
    isRequired()
    isExtraField()
    isFormulaField()
    getDataHandlerType(): String
    getDataHandlerConstraints(): String
    getAccess(): int
}
class ReferenceInfo {
    getName(): String
    getRefClass(): Class<?>
    getRefFieldName(): String
    getRefType(): int
    isContained(): boolean
    isShareable(): boolean
    isPersistent(): boolean
    isAutoCreation(): boolean
    isAutoDeletion(): boolean
    isAutoReload(): boolean
    isEmptyStoring(): boolean
    isReplaceable(): boolean
    isUnique(): boolean
    isReferentialIntegrity(): boolean
    isCascadingDelete(): boolean
    isExportable(): boolean
    isDataDependent(): boolean
    isExtraReference(): boolean
    isFormulaReference(): boolean
}
class CollectionInfo {
    getName(): String
    getBusinessClass(): Class<?>
    getRefFieldName(): String
    getRefInfo(): ReferenceInfo
    getRefType(): int
    isOwnedCollection(): boolean
    isPersistent(): boolean
    isAutoDeletion(): boolean
    isDuplicable(): boolean
    isSecondary(): boolean
    isCheckForDuplicates(): boolean
    isCheckNotEmpty(): boolean
    isCheckMember(): boolean
    isExportable(): boolean

```

```
    getAccess(): int  
}
```

```
Guide --> Manipulator : erzeugt  
Guide "1" --> "*" FieldInfo : beinhaltet  
Guide "1" --> "*" ReferenceInfo : beinhaltet  
Guide "1" --> "*" CollectionInfo : beinhaltet  
~~~
```

### ### BSF/VBSF-Klassen

Derzeit werden alle persistenten Business-Objekte mit Hilfe der VBSF-Klassen in relationalen Datenbanken (RDBS) verwaltet.

Die wichtigsten Klassen sind:

#### #### BsfDatabasePool

Die Klasse `BsfDatabasePool` implementiert einen Pool von Datenbankverbindungen. Da der Aufbau einer neuen Verbindung zur Datenbank Zeit kostet und damit auch zusätzliche Ressourcen bindet, wird nicht für jede Anfrage eine neue Verbindung erstellt. Stattdessen wird eine Verbindung aus dem Pool verwendet. Pools verbessern so die Geschwindigkeit der Ausführung und reduzieren den Ressourcenverbrauch.

Der Datenbank-Pool wird über Änderungen an Einstellungen, Zusatzfeldern und Business-Objekten informiert und kann somit gecachte Informationen verwerfen oder gar die Verbindung schließen.

#### #### BsfDatabase

Die Klasse `BsfDatabase` repräsentiert die Verbindung (Connection) zur Datenbank. Die Hauptaufgaben sind:

- Transaktionsverwaltung: Transaktionen sind nicht verschachtelbar!
- Von der Datenbank können Collections zur Verwaltung der Business-Objekte angefordert werden (siehe `BsfCollection`).
- Objekt-Caching: jede Datenbank-Instanz hat einen eigenen Objekt-Cache, der in den Collections der jeweiligen Business-Klasse verwaltet wird. Das bedeutet, dass unterschiedliche Datenbank-Objekte immer unterschiedliche Objekt-Instanzen im Cache halten.

#### #### ObjectCollection/BsfCollection

Das Interface `ObjectCollection` definiert eine einheitliche Schnittstelle zur Verwaltung von Business-Objekten. Dabei wird die zugrunde liegende Datenquelle vollkommen abstrahiert. Persistente Business-Objekte (von BsoObject abgeleitet) werden über die `BsfCollection`-Klasse verwaltet. Sie implementiert ObjectCollection und bietet damit eine einheitliche Schnittstelle rund um die von VBSF gebotenen

Möglichkeiten. Dabei werden auch die im Guide eingestellten Caching-Optionen berücksichtigt. Die Funktionalität orientiert sich an den Standard-Collection-Klassen für objektorientierten Datenbank-Zugriff (ODMG). Die wichtigsten Funktionen sind:

- ``create()``:

Erzeugt und initialisiert ein Business-Objekt, welches daraufhin selbständig Daten verwalten kann. Es erhält Referenzen auf die Collection und die Datenbank, über welche es erzeugt wurde. Über die Datenbank ist es Business-Objekten möglich, Referenzen aufzulösen und Daten in Member-Collections zu verwalten. Falls es sich um Objekte einer Member-Collection handelt, wird automatisch die Referenz auf den Collection-Holder gesetzt.

- ``discardCreatedObject(Object obj)``:

Verwirft ein soeben erzeugtes Objekt. Der Aufruf sollte beim Verwerfen immer gemacht werden, um dem Objekt die Möglichkeit zu geben, evtl. angeforderte Ressourcen wieder abzugeben oder sonstige Aufräumarbeiten durchzuführen.

- ``add(Object obj)``:

Nimmt das neue Objekt in die Collection auf. In der Regel bewirkt dieser Aufruf das Speichern des neuen Objektes in der Datenbank. Eine Ausnahme stellen Owned-Objects dar, bei denen die Änderungen an den Objekten zunächst gecacht werden. Die Aktualisierung in der Datenbank erfolgt dann über den Collection-Holder. Owned-Objekte sind Teil eines Aggregats, dessen Gesamtzustand transaktionssicher und damit konsistent in der Datenbank verwaltet wird.

- ``update(Object obj)``:

Speichert die Änderungen am Objekt in der Datenbank. Im Falle von Owned-Objects werden die Änderungen zunächst gecacht.

- ``remove(Object obj)``:

Löscht das Objekt aus der Datenbank. Im Falle von Owned-Objects wird die Löschung zunächst im Speicher zwischengespeichert.

- ``reload(Object obj)``:

Lädt das Objekt erneut aus der Datenbank nach.

- ``add-|update-|remove-|reloadAll``: Varianten, um eine Liste von Objekten zu bearbeiten.

- Methoden um Objekte aus der Datenbank einzulesen:

- ``lookup(Object/long oid)``:

Diese Methode nimmt eine Sonderstellung ein. Sie sucht nach einem Objekt mit der angegebenen Objekt-ID. Nachdem die Objekt-ID das Objekt immer eindeutig identifiziert, kann bei der Suche nach dem Objekt ebenso eindeutig der Cache benutzt werden, was besonders zur Auflösung der Referenzen wichtig ist.

- ``getArray(CfgFilter filter, CfgOrder order)``:

Liefert ein Array mit allen Objekten, die den Abfragekriterien entsprechen. Die Abfrage kann u.U. vollkommen im Speicher (in memory query) abgearbeitet werden, wenn der Cache der Collection vollständig mit allen Objekten befüllt ist. Dies ist häufig bei Vorgabedaten der Fall.

- ``getEnumeration(CfgFilter filter, CfgOrder order)``:

Liefert einen Enumerator (Forward-Only-Cursor), mit welchem die Objekte sequentiell eingelesen werden können.

- ``getObject(CfgFilter filter, CfgOrder order)``:

Liefert das erste Objekt, das den Abfragekriterien entspricht.

- ``getUniqueObject(CfgFilter filter)``:

Gibt das eindeutige Objekt zurück, das den Abfragekriterien entspricht, oder null, wenn es mehrere entsprechende Objekte gibt.

- Methoden zur Kontrolle des Caches:

- ``isLoaded()``:

Ist der Cache mit allen Objekten der Collection gefüllt?

- ``load()``:

Lädt alle Objekte der Collection von der Datenbank in den Cache.

- ``discardAll()``:

Leert den Objekt-Cache.

- ``discardTree()``:

Leert den Objekt-Cache dieser Collection und aller hierarchisch untergeordneten Objekte (Member-Collections und Contained-References). Es wird auch (rekursiv) der Cache aller weiteren verworfenden Referenzen geleert. Dazu gehören die Controller-Referenzen (Owner und Controlling-Parent) und grundsätzlich alle Objekte, die entsprechend gekennzeichnet sind.

- ``discard(Object obj)``:

Verwirft die Referenz auf das Objekt (es wird aus dem Cache gelöscht, sofern es eingetragen war).

#### #### BsObject und (Bsf)Manipulator

``BsObject`` und ``(Bsf)Manipulator`` sind die Basisklassen für alle persistenten Business-Objekte, die auf VBSF aufsetzen. Beide Klassen bieten bereits Unterstützung für Zusatzfelder, d.h. man kann in der Datenbank Zusatzfelder definieren, die automatisch in der Anwendung (ohne zusätzliche Programmierung!) verwaltet werden.

Es gibt folgende Grundoperationen mit den jeweiligen Benachrichtigungsmethoden, in die man sich für Sonderbehandlungen einklinken kann:

| BsfCollection        | BsObject                 | Bemerkung                                                                                                                                                                                                                 |
|----------------------|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ----                 | -----                    |                                                                                                                                                                                                                           |
| create               | postCreate               | Das Objekt wurde im Speicher erzeugt                                                                                                                                                                                      |
| discardCreatedObject | cleanUp                  | Das neu erzeugte Objekt wird verworfen (DISCARD_CREATED_OBJECT)                                                                                                                                                           |
| add                  | pre/postUpdateImpl       | Das neu erzeugte Objekt wird gleich / wird gerade / wurde gespeichert (INSERT)                                                                                                                                            |
| update               | pre/postUpdateImpl       | Die Änderungen am Objekt werden gleich / werden gerade / wurden gespeichert (UPDATE)                                                                                                                                      |
| remove               | preDelete                | Der Status des Objekts wird gleich zur Löschung markiert. Über den Aufruf von readOldValues() wird der originale DB-Zustand verschiedener Daten eingelesen. Diese Methode sollte in der Regel nicht überschrieben werden. |
|                      | pre/postUpdateImpl       | Das Objekt wird gleich / wird gerade / wurde gelöscht (DELETE)                                                                                                                                                            |
| reload               | cleanUp                  | Das Objekt wird gleich nachgeladen (RELOAD)                                                                                                                                                                               |
|                      | postRefresh              | Das Objekt wurde gerade nachgeladen                                                                                                                                                                                       |
| COMMIT               | preCommitImpl            | Die Transaktion wird gleich abgeschlossen                                                                                                                                                                                 |
|                      | postCommitImpl           | Die Transaktion wurde erfolgreich abgeschlossen                                                                                                                                                                           |
|                      | cleanUp                  | Die Änderungen am Objekt wurden erfolgreich gespeichert (INSERT, UPDATE) oder das Objekt wurde erfolgreich aus der Datenbank gelöscht (DELETE)                                                                            |
|                      | transactionCompletedImpl | Alle Objekten haben zu diesem Zeitpunkt bereits die postCommitImpl-Benachrichtigung erhalten                                                                                                                              |
| ROLLBACK             | transactionAbortedImpl   | Die Transaktion wurde abgebrochen                                                                                                                                                                                         |

!!n

Es gilt zu beachten, dass ein Objekt nur einmal pro Transaktion geändert werden darf. Abhängige Objekte sollten also nur dann aktualisiert werden, wenn sie bisher keine Änderung aufweisen (siehe `isUnchanged()`) und/oder sie noch nicht in der Transaktion vorkommen (siehe `isInTransaction()`).

!!

Über folgende Methoden werden Collection- und Reference-Holder über Änderungen an untergeordneten Objekten benachrichtigt:

`collectionChanged::`

Ein oder mehrere Objekte einer Member-Collection wurden geändert. Der Collection-Holder kann daraufhin entsprechend reagieren (gecachte Daten verwerfen, (transiente) Flags für notwendige Neuberechnungen setzen usw.). Die Konstanten sind im `CollectionListener`-Interface definiert.

`referenceChanged::`

Eine (enthaltene) Referenz wurde geändert. Der Reference-Holder kann daraufhin entsprechend reagieren (replizierte Daten aktualisieren usw.). Die Konstanten sind im `CollectionListener`-Interface definiert.

Manipulatoren werden über die `workingNotification`-Methode über die Bearbeitung an Objekten informiert. Die wichtigsten Ereignisse sind:

| Ereignis                                                                                                     | Bemerkung                                                                                                                                                                                                                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OBJECT_CREATED                                                                                               | Das Objekt wurde erzeugt und zur weiteren Bearbeitung angemeldet ( <code>Navigator.addNew</code> )                                                                                                                                                                                                                                                                                                         |
| OBJECT_EDITING_STARTED                                                                                       | Die Bearbeitung des Objektes wurde gestartet ( <code>Navigator.edit</code> )                                                                                                                                                                                                                                                                                                                               |
| OBJECT_CHANGED                                                                                               | Das Objekt wurde geändert, befindet sich aber weiterhin in Bearbeitung ( <code>MnField.setValue</code> )                                                                                                                                                                                                                                                                                                   |
| OBJECT_UPDATE_CANCELLED                                                                                      | Die Bearbeitung wurde abgebrochen ( <code>Navigator.cancelUpdate</code> )                                                                                                                                                                                                                                                                                                                                  |
| OBJECT_ADDED                                                                                                 | Ein neues Objekt wurde gespeichert ( <code>Navigator.addNew/update</code> )                                                                                                                                                                                                                                                                                                                                |
| OBJECT_UPDATED                                                                                               | Die Änderungen am Objekt wurden gespeichert ( <code>Navigator.edit/update</code> )                                                                                                                                                                                                                                                                                                                         |
| OBJECT_DELETED                                                                                               | Das Objekt wurde gelöscht ( <code>Navigator.delete</code> )                                                                                                                                                                                                                                                                                                                                                |
| OBJECT_RELOADED                                                                                              | Das Objekt wurde nachgeladen ( <code>Navigator.reload</code> )                                                                                                                                                                                                                                                                                                                                             |
| OBJECT_ADDED_SUCCESSFULLY, OBJECT_ADDED_WITH_ERRORS, OBJECT_UPDATED_SUCCESSFULLY, OBJECT_UPDATED_WITH_ERRORS | Spezielle Benachrichtigungen bei mehrstufigen Aktualisierungen von Objekten. Bei Importen ist es zum Teil notwendig, Collection-Holder in der Datenbank zu speichern, bevor deren Kindobjekte importiert werden können. Am Ende wird dem Manipulator mitgeteilt, ob der Import des Objektes inklusive seiner untergeordneten Objekte in seiner Gesamtheit erfolgreich war oder (zum Teil) gescheitert ist. |

Die Manipulatoren haben in erster Linie die Aufgabe, die Oberfläche entsprechend zu aktualisieren. Die Business-Logik wird (in der Regel) nur von den Business-Objekten implementiert. In speziellen Fällen (Owned-Collections) übernehmen Manipulatoren auch das Zwischenspeichern der Feldwerte für das Rückgängigmachen der aktuellen Änderungen. Die Manipulatoren benachrichtigen ihren Parent (Collection- oder Reference-Holder) über Änderungen am Business-Objekt (analog zu den Collections der Business-Objekte). Die entsprechenden Methoden sind 'collectionChanged' und 'referencedObjectModified'.

!!n

Es gilt zu beachten, dass Business-Objekte und Manipulatoren unabhängig voneinander benachrichtigt werden. Die Business-Objekte haben keine Verbindung zu Manipulatoren.  
!!

### ### Beziehungen

Die Beziehungen zwischen Business-Objekten werden über die Referenzen modelliert. Über ReferenceInfo kann die Art der Referenz bestimmt werden:

| Art                | Bemerkung                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NORMAL             | "Normale" Referenz auf ein anderes Objekt.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| OWNER              | Referenz auf den Besitzer (Collection-Holder, Aggregation). Objekte, die einen Besitzer haben, haben keine eigenständige Persistenzverwaltung. Sie werden immer zusammen mit dem Besitzer gespeichert, gelöscht usw.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| CONTROLLING_PARENT | Referenz auf den Parent (Collection-Holder, Assoziation), welcher im Unterschied zur einfachen Parent-Beziehung (siehe PARENT) nicht nur die Persistenzverwaltung sondern auch die Bearbeitung der Objekte überwacht (Locking, Änderungsrechte usw. wie bei der Owner-Beziehung, aber eigenständige Persistenzverwaltung).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| PARENT             | Referenz auf den Parent (Collection-Holder, Assoziation). Objekte, die zu einem bestimmten Parent-Objekt gehören, haben eine eigenständige Persistenzverwaltung, welche aber unter der Kontrolle des Parents abläuft. Das bedeutet beispielsweise, dass neue Objekte nur unter der Kontrolle des Parents angelegt werden können und dieser im Gegensatz zu den sonstigen (NORMAL) Referenzen (normalerweise) auch nicht mehr ausgetauscht werden kann.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| CONTAINED          | Eine "im Besitzer enthaltene" Referenz (Aggregation). Enthaltene Objekte werden immer zusammen mit ihrem Besitzer (dem Reference-Holder) gespeichert, gelöscht usw. Bei den enthaltenen Referenzen können weiters folgende Angaben gemacht werden:<br>* setShareable: Legt fest, ob es sich um eine "gemeinsam benutzbare enthaltene" Referenz handelt. Solche Objekte können mehrere Besitzer haben. Demzufolge wird ein fehlgeschlagener Lösversuch nicht als Fehler interpretiert. +<br>N.B. Diese Art der Implementierung setzt referenzielle Integrität mit Lösabweisung voraus! +<br>Die Grundeinstellung ist false.<br>* setAutoCreation: Legt fest, ob beim Erzeugen des Besitzers automatisch auch ein neues "enthaltene" Objekt erzeugt werden soll. Die Grundeinstellung ist false.<br>* setAutoDeletion: Legt fest, ob beim Löschen des Besitzers automatisch auch das "enthaltene" Objekt gelöscht werden soll. Die Grundeinstellung ist true.<br>* setAutoReload: Legt fest, ob beim Nachladen des Besitzers automatisch auch das "enthaltene" Objekt nachgeladen werden soll. Die Grundeinstellung ist true.<br>* setEmptyStoring: Legt fest, ob leere |

"enthaltene" Objekte gespeichert werden sollen. Die Grundeinstellung ist true. |

Normalerweise werden die Beziehungen in der Datenbank mit referentieller Integrität angelegt. In Sonderfällen kann sie mit `'setReferentialIntegrity(false)'` deaktiviert werden.

Mit `'setCascadingDelete(true)'` kann das verschachtelte Löschen durch die Datenbank aktiviert werden. Im Normalfall werden Löschungen, die die referenzielle Integrität verletzen, von der Datenbank abgewiesen. Es muss beachtet werden, dass das automatische Löschen durch die Datenbank nur dann aktiviert werden darf, wenn keine besondere Business-Logik damit verknüpft ist. Andernfalls kann das automatische Löschen über `'CollectionInfo.setAutoDeletion(true)'` aktiviert werden. Das Löschen über Collections erfolgt über das Einlesen und anschließende Löschen jedes einzelnen Objektes, womit die Business-Logik voll zum Tragen kommt.

### ### Exception-Handling

Die Business-Objekte und die Datenbankzugriffsklassen (`'BsfDatabase'`, `'BsfCollection'` usw.) stellen die Methode `'handleException'` zur einheitlichen Behandlung von Ausnahmefehlern zur Verfügung. Sie sind so implementiert, dass die Ausnahme (`'Exception'`) ans Datenbank-Objekt weitergeleitet wird, welche wiederum die Bearbeitung an den austauschbaren MessageHandler delegiert. Nachdem die Business-Objekte selbst nicht wissen, ob sie im Rahmen einer grafischen Oberfläche interaktiv oder von einem Hintergrundprozess (Kommunikationsserver) bearbeitet werden, ist die zentrale Behandlung notwendig, damit im ersteren Fall die Anzeige in einer MessageBox und im zweiten Fall die Protokollierung in einer Log-Datei möglich ist.

### ### Programmiermuster

#### #### Guide und Zugriffsschlüssel registrieren

Dieses Beispiel zeigt die Registrierung eines Guide-Objektes und eines Zugriffsschlüssels für ein Business-Objekt.

Es werden Konstanten für sämtliche Namen benutzt, um die Programmpflege (spätere Refactorings) zu erleichtern.

So kann man immer schnell und eindeutig feststellen, an welchen Programmstellen auf Felder und sonstige Elemente einer bestimmten Business-Klasse zugegriffen wird.

~~~java

```
public class BsBild extends BsAutoID {

 // Name in der Ein- und Mehrzahl.
 public static final String NAME = "Bild";
 public static final String PLURAL_NAME = "Bilder";

 // Feldnamen
 public static final String FN_REF_BILDERGRUPPE = "ID_BildGrp";
```

```

public static final String FN_IMAGE_DATA = "ImgData";

// Namen für fiktive Felder
public static final String FN_IMAGE_INFO = "ImgInfo";
public static final String FN_IMAGE_WIDTH = "ImgWidth";
public static final String FN_IMAGE_HEIGHT = "ImgHeight";
public static final String FN_IMAGE_SIZE = "ImgSize";
public static final String FN_IMAGE_SIZE_KB = "ImgSizeKB";

// Namen für Referenzen
public static final String REF_BILDERGRUPPE = BsBildergruppe.NAME;

/** Name der Tabelle. */
public static final String TABLE_NAME = "gBild";

private BsfReference bildergruppe; // Bildergruppe
private BOABoolean selected; // Selektiert?
private BOAString code; // Code
private BOAString extCode; // Externer Code
private BsfImageField image; // das Bild

/** Initialisiert das Guide-Objekt. */
protected static void initializeGuide(BsfGuide g) {
 BsAutoID.initializeGuide(g);
 g.setManipulatorName(MnBild.class.getName());
 g.setResBundleName(BaseDataResBundle.class.getName());
 g.setModuleName(BaseConstants.MODULE_NAME);
 g.setTableName(TABLE_NAME);
 g.setSearchFields(FN_CODE);

 // Felder registrieren
 FieldInfo fi;
 fi = g.addField(createFieldInfo(FN_REF_BILDERGRUPPE, BsBildergruppe.class));
 fi.setAccess(Key.FIELD_READ);
 fi.setNullable(false);
 fi.setAutoFilled(true);

 fi = g.addField(createFieldInfo(FN_CODE, DataType.STRING, 25));
 fi.setDataHandler(TextHandler.TYPE, "UPPERSTRING,25");
 fi.setNullable(false);
 fi.setExternalKey(true);

 ...

 fi = g.addField(createFieldInfo(FN_IMAGE_DATA, DataType.BLOB));
 fi.setExternalName("Data");
 fi.setDataHandler(ImageHandler.TYPE, null);
 fi.setNullable(false);

 fi = g.addField(createFieldInfo(FN_IMAGE_WIDTH, DataType.INTEGER));
 fi.setExternalName("Width");

```



```

 fi.setAccess(Key.FIELD_READ);
 fi.setPersistent(false);

 ...

 // Referenzen registrieren
 g.addReference(BsBildergruppe.class, FN_REF_BILDERGRUPPE,
ReferenceInfo.PARENT)
 .setName(REF_BILDERGRUPPE);

 // Indizes registrieren
 CfgOrder order = new CfgOrder(FN_REF_BILDERGRUPPE, FN_CODE);
 order.setUnique(true);
 order.setCode("CODE");
 g.addIndex(order);

 order = new CfgOrder(FN_REF_BILDERGRUPPE, FN_CODE, CfgOrder.DESC);
 order.setUnique(true);
 order.setIndexPurpose(CfgOrder.IDX_REVERSE_ORDER);
 order.setCode("CODE_D");
 g.addIndex(order);
 }

 /** Registriert Hilfsklassen für das Business-Objekt. */
 public static void register(int[] dbTypes) {
 BsfGuide g = new BsfGuide(dbTypes, BsBild.class, NAME, PLURAL_NAME);
 initializeGuide(g);
 Guide.register(g);
 }

 /** Erzeugt (und registriert) den Zugriffsschlüssel für die Business-Klasse. */
 public static Key createKey(Key parent, boolean suppl, boolean regKeyPath) {
 return Guide.get(BsBild.class).createKey(parent, suppl, regKeyPath);
 }
}
~~~

```

#### #### Manipulator

Dieses Beispiel zeigt die Implementierung eines Manipulators mit den Konstanten für die Offsets der verschiedenen Elemente (Felder, Referenzen usw.). `getFieldValue` zeigt eine beispielhafte Nutzung des Parameters `fastOutput` für Bilder.

Mit `fastOutput=true` wird das bereits gerenderte und im Business-Objekt gecachte Bild geliefert, mit `fastOutput=false` die dem Bild zugrunde liegenden (und in der Datenbank gespeicherten) Daten als byte-Array. `setFieldValue` zeigt die Behandlung des Modus mit automatischen Ergänzungen und das Abweisen von eventuellen Wertänderungsversuchen von Feldern, auf die nur lesend zugegriffen werden kann.

```

~~~java
/*
 * MnBild.java 2004-04-28
 *
 * Copyright (c) Pro Data GmbH & ASA KG. All rights reserved.
 */

package it.prodata.gest.base;

import it.prodata.bsf.BsfManipulator;
import it.prodata.data.Guide;
import it.prodata.data.MnField;
import it.prodata.util.Session;

import java.util.List;

/**
 * Manipulator für Business-Klasse Bild.
 *
 * @author Andreas Rabensteiner
 * @since 2004-04-28
 */
public class MnBild extends BsfManipulator {

 // Konstanten für Referenzen-Offsets
 //private static final int RO_BILDERGRUPPE = 0;

 // Konstanten für Felder-Offsets
 private static final int FO_BILDERGRUPPE = 0;
 private static final int FO_BILDERGRUPPE_NAME = 1;
 private static final int FO_IMAGE_DATA = 2;
 private static final int FO_IMAGE_INFO = 3;
 private static final int FO_IMAGE_WIDTH = 4;
 private static final int FO_IMAGE_HEIGHT = 5;
 private static final int FO_IMAGE_SIZE = 6;
 private static final int FO_IMAGE_SIZE_KB = 7;

 private BsBild bild; // das gebundene Business-Objekt
 //private final int firstRef; // Index der ersten Referenz
 private final int firstField; // Index des ersten Feldes

 /** Konstruktor. */
 public MnBild(Guide guide, Session session) {
 super(guide, session);

 // Referenzen registrieren
 //firstRef = getCurrentReferenceCount();
 addReference(BsBild.FN_REF_BILDERGRUPPE);

 // Felder registrieren

```

```

 firstField = getCurrentFieldCount();
 addField(BsBild.FN_REF_BILDERGRUPPE);
 addNameSearchField(BsBild.FN_REF_BILDERGRUPPE, BsBildergruppe.FN_NAME);
 addField(BsBild.FN_IMAGE_DATA);
 addField(BsBild.FN_IMAGE_INFO);
 addField(BsBild.FN_IMAGE_WIDTH);
 addField(BsBild.FN_IMAGE_HEIGHT);
 addField(BsBild.FN_IMAGE_SIZE);
 addField(BsBild.FN_IMAGE_SIZE_KB);
 }

 @Override
 protected void attachObjectImpl(Object obj) {
 bild = (BsBild)obj;
 super.attachObjectImpl(obj);
 }

 /** Aktualisiert die Komponente(n) eines Feldes dieses Manipulators. */
 private void updateLocalField(int fieldOffset) {
 getField(firstField + fieldOffset).updateComponents();
 }

 @Override
 protected Object getFieldValue(int field, boolean fastOutput) {
 return switch (field - firstField) {
 case FO_BILDERGRUPPE, FO_BILDERGRUPPE_NAME -> bild.getBildergruppe();
 case FO_IMAGE_DATA -> fastOutput ? bild.getImageAsImageIcon() :
bild.getImageData();
 case FO_IMAGE_INFO -> bild.getImageInfo();
 case FO_IMAGE_WIDTH -> bild.getImageWidth();
 case FO_IMAGE_HEIGHT -> bild.getImageHeight();
 case FO_IMAGE_SIZE -> bild.getImageSize();
 case FO_IMAGE_SIZE_KB -> bild.getImageSizeKB();
 default -> super.getFieldValue(field, fastOutput);
 };
 }

 @Override
 protected void setFieldValue(int field, Object value, boolean autoCompletion) {
 switch (field - firstField) {
 case FO_IMAGE_DATA -> {
 bild.setImageData((byte[])value);
 if (autoCompletion) {
 updateLocalField(FO_IMAGE_INFO);
 updateLocalField(FO_IMAGE_WIDTH);
 updateLocalField(FO_IMAGE_HEIGHT);
 updateLocalField(FO_IMAGE_SIZE);
 updateLocalField(FO_IMAGE_SIZE_KB);
 }
 }
 case FO_BILDERGRUPPE, FO_BILDERGRUPPE_NAME,

```

```

 FO_IMAGE_INFO,
 FO_IMAGE_WIDTH,
 FO_IMAGE_HEIGHT,
 FO_IMAGE_SIZE,
 FO_IMAGE_SIZE_KB -> illegalFieldAccess();
 default -> super.setFieldValue(field, value);
 }
}

@Override
protected void addDefaultFields(List<MnField> fields) {
 super.addDefaultFields(fields);
 addField(fields, firstField + FO_BILDERGRUPPE);
 addField(fields, firstField + FO_IMAGE_DATA);
}
}

~~~

```

#### #### Initialisierung mit Defaultwerten

Wenn Business-Klassen das Interface VisualDataHandling implementieren, wird im Normalfall unmittelbar nach der Erzeugung von Objekten deren initializeDefaultValues-Methode aufgerufen:

```

~~~java
public class BsAdresse extends BsAutoID implements VisualDataHandling {
 ...

 @Override
 public void initializeDefaultValues(boolean prepareCopy) {
 assignUUID();
 if (!prepareCopy) {
 BaseSettings bs = getBaseSettings();
 int status = bs.getAdresseStatus();
 if (isValidStatus(status))
 setStatus(status);
 setSprache(bs.getAdresseSprache());
 ...
 }
 }
}
}

~~~

```

#### #### Automatische Ergänzungen

Innerhalb der grafischen Oberfläche und zum Teil auch bei Datenimporten und Zugriffen über REST-Schnittstelle erfolgt die Bearbeitung der Objekte im Modus mit automatischen Ergänzungen. Dabei können beim

Setzen von Feldern automatisch  
Werte für weitere Felder eingesetzt werden.

```
~~~java
public class BsAdresse {
 ...

 /** Liefert das Geschlecht (Männlich, Weiblich, Juristische Person). */
 public int getGeschlecht() {
 return geschlecht.getValue();
 }

 /** Setzt das Geschlecht (Männlich, Weiblich, Juristische Person). */
 public void setGeschlecht(int value) {
 setGeschlecht(value, false);
 }

 /**
 * Setzt das Geschlecht (Männlich, Weiblich, Juristische Person).
 * @param value Das Geschlecht.
 * @param completeDefaults Aktualisiert die Anrede und setzt im Falle einer
 * Person (siehe {@link #isPhysischePerson()}) die Rechtsform zurück.
 * @see #updateAnrede()
 */
 public void setGeschlecht(int value, boolean completeDefaults) {
 if (!isValidGeschlecht(value))
 value = UNKNOWN_GESCHLECHT;
 geschlecht.setValue(value);
 if (completeDefaults && value != UNKNOWN_GESCHLECHT) {
 updateAnrede();
 if (isPhysischePerson() && hasRechtsform())
 setRechtsform(null);
 }
 }
}
~~~
```

```
~~~java
public class MnAdresse extends BsManipulator {
 ...

 /** Liefert ein Feld dieses Manipulators. */
 private MnField getLocalField(int fieldOffset) {
 return getField(firstField + fieldOffset);
 }

 /** Aktualisiert die Komponente(n) eines Feldes dieses Manipulators. */
 private void updateLocalField(int fieldOffset) {
 getLocalField(fieldOffset).updateComponents();
 }
}
```

```

 /** Aktualisiert eine Referenz und ihre verbundenen Auswahlfelder dieses
Manipulators. */
 private void updateLocalReference(int refOffset) {
 getReference(firstRef + refOffset).update(true);
 }

 @Override
 protected void setFieldValue(int field, Object value, boolean autoCompletion) {
 switch (field - firstField) {
 ...
 case FO_GESCHLECHT -> {
 adresse.setGeschlecht(Numbers.intValue(value,
BsAdresse.UNKNOWN_GESCHLECHT), autoCompletion);
 if (autoCompletion) {
 updateLocalField(FO_PHYSISCHE_PERSON);
 updateLocalField(FO_JURISTISCHE_PERSON);
 updateLocalField(FO_NAME);
 updateLocalReference(RO_ANREDE);
 updateLocalField(FO_ANREDE_DOC);
 updateLocalField(FO_BRIEFANREDE);
 updateLocalReference(RO_RECHTSFORM);
 }
 }
 ...
 }
 }
}
~~~

```

Die Business-Logik wird in der Business-Klasse implementiert. Der Manipulator kennt die Zusammenhänge und aktualisiert eventuell angebundene Komponenten im Formular.

#### #### Referenzen

Referenzen werden in den Business-Objekten mit Hilfe der `BsReference`-Klasse verwaltet. BsReference greift über eine Collection auf die referenzierten Business-Objekte zu, die über die Methode `getReferenceCollection` vom Reference-Holder ermittelt wird. Damit hat der Reference-Holder die Möglichkeit, eine geeignete Collection zu liefern.

Dazu ein Beispiel von BsAdresse:

```

~~~java
public class BsAdresse {
 ...

 public void setLand(BsLand newRef, boolean completeDefaults) {
 if (!isReferencedLand(newRef)) { ②

```

```

 BsGemeinde g = getGemeinde();
 if (g != null && !g.isReferencedLand(newRef))
 setGemeinde(null);
 BsProvinz p = getProvinz();
 if (p != null && !p.isReferencedLand(newRef))
 setProvinz(null);
 BsRegion r = getRegion();
 if (r != null && !r.isReferencedLand(newRef))
 setRegion(null);
 }
 land.setValue(newRef);
 if (completeDefaults && newRef != null) {
 ...
 }
}

...

public boolean isReferencedProvinz(BsProvinz obj) {
 return provinz.isReferencedObject(obj);
}

public BsProvinz getProvinz() {
 return (BsProvinz)provinz.getValue();
}

public void setProvinz(BsProvinz newRef) {
 boolean changed = !isReferencedProvinz(newRef); ③
 if (changed) {
 BsGemeinde g = getGemeinde();
 if (g != null && !g.isReferencedProvinz(newRef))
 setGemeinde(null);
 }
 provinz.setValue(newRef);
 if (changed && newRef != null) { ④
 BsRegion newReg = newRef.getRegion(); // kann null sein (keine Regionen)
 if (!isReferencedRegion(newReg))
 setRegion(newReg);
 BsLand newLand = newRef.getLand(); // dürfte nie null sein
 if (newLand != null && !isReferencedLand(newLand))
 setLand(newLand);
 }
}

...

@Override
public BsfCollection getReferenceCollection(Class<?> refClass, String
refFieldName) { ①
 BsfCollection coll = null;
 if (BsRegion.class.isAssignableFrom(refClass)) {

```

```

 if (FN_REF_GEBURTSREGION.equals(refFieldName)) {
 BsLand l = getGeburtsland();
 if (l != null)
 coll = l.getRegionen();
 }
 else if (FN_REF_REGION.equals(refFieldName)) {
 BsLand l = getLand();
 if (l != null)
 coll = l.getRegionen();
 }
 }
 else if (BsProvinz.class.isAssignableFrom(refClass)) {
 if (FN_REF_GEBURTSPROVINZ.equals(refFieldName)) {
 BsLand l = getGeburtsland();
 if (l != null)
 coll = l.getProvinzen();
 }
 else if (FN_REF_PROVINZ.equals(refFieldName)) {
 BsLand l = getLand();
 if (l != null)
 coll = l.getProvinzen();
 }
 }
 else if (BsGemeinde.class.isAssignableFrom(refClass)) {
 if (FN_REF_GEBURTSGEMEINDE.equals(refFieldName)) {
 BsLand l = getGeburtsland();
 if (l != null)
 coll = l.getGemeinden();
 }
 else if (FN_REF_GEMEINDE.equals(refFieldName)) {
 BsLand l = getLand();
 if (l != null)
 coll = l.getGemeinden();
 }
 }
 return coll != null ? coll : super.getReferenceCollection(refClass,
refFieldName);
}

...
}
~~~

```

<1> Für Gemeinde, Provinz und Region wird die jeweilige Member-Collection vom eingesetzten Land zurückgegeben.

Dies hat zwei Vorteile:

- Provinzen und Regionen werden in den Member-Collection des Landes gecacht, d.h. beim ersten Zugriff werden gleich alle eingelesen, womit nur eine Abfrage an den Datenbankserver geht.
- Bei der Ermittlung der Referenzen über externe Schlüsselfelder (Code), wie es beim



Datenimport und beim Zugriff über die REST-Schnittstelle der Fall ist, können die Einträge eindeutig ermittelt werden. Regionen unterschiedlicher Länder könnten nämlich denselben Code haben. Dadurch, dass nur die Einträge des jeweiligen Landes berücksichtigt werden, wird auch verhindert, dass es zu inkonsistenten Referenzen kommt.

<2> Beim Einsetzen eines Landes werden eventuelle inkonsistente Referenzen auf Gemeinde, Provinz und Region zurückgesetzt.

N.B. Dies erfolgt unabhängig vom Parameter `'completeDefaults'`, um die Konsistenz der Daten in allen Fällen zu gewährleisten.

<3> Bei einem Wechsel der Provinz wird eine eventuell nicht zugehörige Gemeinde zurückgesetzt.

<4> Region und Land der Provinz werden bei Bedarf automatisch eingesetzt.

#### #### Member-Collections

Untersammlungen werden in den Business-Objekten mit Hilfe der `'BsfMemberCollection'`-Klasse verwaltet.

`'BsfMemberCollection'`-Instanzen erhalten über den Konstruktor eine Referenz auf den Collection-Holder, womit

Datenbankabfragen entsprechend eingeschränkt werden können. Der Collection-Holder wird über die Methode

`'collectionChanged'` über Änderungen an Collections benachrichtigt, worauf er bei Bedarf reagieren kann.

Dazu ein (vereinfachtes) Beispiel von `BsJournalbuchung`:

```
~~~java
public class BsJournalbuchung {
 ...
 // Namen für Collections
 public static final String COLL_JOURNALZEILEN = BsJournalzeile.NAME; ①
 ...
 private BsfMemberCollection journalzeilen; // Journalzeile-Collection ②
 ...
 private transient BigDecimal summeSoll; // Summe Soll ③
 private transient BigDecimal summeHaben; // Summe Haben
 ...

 protected static void initializeGuide(BsfGuide g) {
 ...
 // Collections registrieren ④
 g.addCollection(BsJournalzeile.getConcreteClass(),
BsJournalzeile.FN_REF_JOURNALBUCHUNG)
 .setName(COLL_JOURNALZEILEN);
 ...
 }
}
```

```

...

@Override
protected void init() {
 super.init();
 ...
 journalzeilen = new BsfMemberCollection(this,
BsJournalzeile.getConcreteClass()); ⑤
 ...
}

...

private void checkSummeSollHaben() { ⑩
 if (summeSoll == null) {
 /*summeSoll = */summeHaben = null;
 BsJournalzeile[] jza = getAllJournalzeilen(false);
 if (Fun.containsData(jza)) {
 for (var jz : jza) {
 summeSoll = BigDecimals.add(summeSoll, jz.getSoll());
 summeHaben = BigDecimals.add(summeHaben, jz.getHaben());
 }
 }
 if (summeSoll == null)
 summeSoll = BigDecimals.ZERO;
 }
}

public BigDecimal getSummeSoll() {
 checkSummeSollHaben();
 return BigDecimals.zeroToNull(summeSoll);
}

public BigDecimal getSummeHaben() {
 checkSummeSollHaben();
 return BigDecimals.zeroToNull(summeHaben);
}

...

public @NotNull BsfCollection getJournalzeilen() { ⑥
 return journalzeilen;
}

public BsJournalzeile[] getAllJournalzeilen(boolean sort) { ⑦
 return (BsJournalzeile[])journalzeilen.getAllNoExc(sort);
}

@Override
public BsfCollection getCollection(Class<?> bsClass, String refFieldName, boolean

```

```

create) { ⑧
 BsfCollection coll;
 if (bsClass == BsJournalzeile.getConcreteClass())
 coll = journalzeilen;
 else if (...)
 coll = ...
 else
 coll = super.getCollection(bsClass, refFieldName, create);
 return coll;
}

@Override
public void collectionChanged(Object source, int operation, Object element) throws
Exception {
 super.collectionChanged(source, operation, element);
 if (source == journalzeilen) {
 summeSoll = summeHaben = null; ⑨
 }
}
...
}
~~~

```

<1> Konstante für den Namen der Collection mit Präfix "COLL\_" (analog zu den Konstanten für die Feldnamen mit Präfix "FN\_").

<2> Deklaration des Members für die Collection.

<3> Member, die nicht in der Datenbank gespeichert werden, kennzeichnen wir per Konvention mit 'transient', obwohl das aus technischer Sicht nicht notwendig wäre.

<4> Registrierung der Collections im Guide-Objekt. Die Art der Beziehung (in diesem Fall handelt es sich um eine sogenannte Owned-Collection) wird aus der Information zur Referenz des Fremdschlüssel-Feldes FN\_REF\_JOURNALBUCHUNG ermittelt.

<5> Die Instanz der Collection wird in der 'init'-Methode erzeugt.

<6> Es wird eine public Getter-Methode zum Zugriff auf die Collection zur Verfügung gestellt.

<7> Häufig wird eine zusätzliche Methode zur Ermittlung aller Einträge der Collection als Array zur Verfügung gestellt.

Mit dem Parameter 'sort' kann angegeben werden, ob man ein sortiertes Array wünscht. Die Sortierung erfolgt

laut natürlicher Reihenfolge. Alle von BsObject abgeleiteten Klassen implementieren 'Comparable', die Default-Implementierung vergleicht DisplayName und Objekt-ID.

<8> Jede Collection muss in der Methode 'getCollection' berücksichtigt werden, da sich

verschiedene Programmstellen  
des Frameworks darauf stützen.

<9> Über die Methode 'collectionChanged' erhält der Collection-Holder eine Benachrichtigung über Änderungen an den Mitgliedern einer Collection. Hier werden die zwischengespeicherten Summen für Soll und Haben zurückgesetzt. An dieser Stelle sollten aus Effizienzgründen generell nie Daten eingelesen und/oder neu berechnet werden. Es sollen hingegen zwischengespeicherte Daten zurückgesetzt und/oder transiente Flags über die erfolgten Änderungen verwaltet werden, sodass die von der Änderung betroffenen Daten nur bei effektivem Bedarf aktualisiert werden müssen.

<10> Die Summen für Soll und Haben werden bei Bedarf neu berechnet und zwischengespeichert.

Im Manipulator werden Collections folgendermaßen verwaltet:

```
~~~java
public class MnJournalbuchung extends ... {
 ...
 // Konstanten für Collection-Offsets
 private static final int CLO_JOURNALZEILEN = 0; ①
 ...
 private BsJournalbuchung jourBuch; // das gebundene Business-Objekt
 private final int firstColl; // Index der ersten Collection ①
 ...

 public MnJournalbuchung(Guide guide, Session session) {
 super(guide, session);
 ...
 // Collections registrieren ②
 firstColl = getCurrentCollectionCount();
 addCollection(BsJournalzeile.getConcreteClass(),
BsJournalzeile.FN_REF_JOURNALBUCHUNG);
 ...
 }

 ...

 private void updateSummeSollHaben() { ⑤
 updateLocalField(FO_SUMME_SOLL);
 updateLocalField(FO_SUMME_HABEN);
 }

 ...

 @Override
 protected Navigator createCollectionNavigator(int coll) { ③
```

```

 return switch (coll - firstColl) {
 case CLO_XXX -> ...
 default -> super.createCollectionNavigator(coll);
 };
 };
}

@Override
public void collectionChanged(Object source, int operation, Object element) { ④
 super.collectionChanged(source, operation, element);
 if (isUpdateComponentEnabled()) {
 if (source instanceof MnJournalzeile || element instanceof BsJournalzeile)
 updateSummeSollHaben();
 updateLocalField(FO_UPDATE_ALL_NEEDED);
 }
}
}

~~~

```

<1> Es können optional Offset-Konstanten für die Collections verwaltet werden, um effizient darauf zugreifen zu können.

Alternativ kann auch mit dem Namen der Collections (COLL\_-Konstante der Business-Klasse) gearbeitet werden.

<2> Die Collections müssen im Manipulator registriert werden.

<3> Der Zugriff auf die Member einer Collection erfolgt über einen Navigator, der von der Methode 'createCollectionNavigator'

geliefert wird. Diese Methode muss (wie in unserem Fall) im Manipulator nicht überschrieben werden, wenn die

Business-Klasse eine entsprechende Collection über die Methode 'getCollection' liefert.

<4> Der Manipulator erhält bei Änderungen an Mitgliedern einer Collection eine entsprechende Benachrichtigung, worauf er mit

der Aktualisierung der betroffenen Elemente der Benutzeroberfläche reagiert, sofern diese Aktualisierung aktiviert

ist (siehe 'isUpdateComponentEnabled'). Der Collection-Holder hat die entsprechende Benachrichtigung unabhängig

vom Manipulator bereits über die Member-Collection erhalten, sodass sich die Aufgabe des Manipulators auf die Aktualisierung der Oberfläche beschränkt.

<5> Wird zur Aktualisierung der Komponenten aufgerufen, über welche die Summen dargestellt werden. Dabei erfolgt nur

dann ein Zugriff auf die Daten, wenn effektiv Komponenten für diese (berechneten) Felder angebunden sind. Damit

werden unnötige Datenzugriffe und Berechnungen vermieden.

#### Collections nur im Manipulator verwalten

Es kann Sinn machen, Untersammlungen von Business-Objekten nur über den Manipulator zu verwalten. Damit wird der Speicherbedarf der Business-Objekte reduziert, da keine `BsfMemberCollection`-Member angelegt werden. Dies macht prinzipiell immer dort Sinn, wo Untersammlungen nicht für die Implementierung einer speziellen Business-Logik benötigt werden und lediglich dem Benutzer für den Zugriff auf die entsprechenden Daten zur Verfügung gestellt werden sollen.

Dazu ein (vereinfachtes) Beispiel von MnKunde:

```
~~~java
public class MnKunde extends ... {

 // Namen für Collections
 public static final String COLL_POSTEN = BsPosten.NAME; ①
 ...

 /** Konstruktor. */
 public MnKunde(Guide guide, Session session) {
 super(guide, session);
 ...

 ci = new CollectionInfo(BsPosten.getConcreteClass(), BsPosten.FN_REF_KUNDE);
 ②
 ci.setName(COLL_POSTEN);
 addCollection(ci);
 ...
 }

 ...

 @Override
 protected Navigator createCollectionNavigator(int coll) { ③
 return switch (getCollection(coll).getName()) { ④
 ...
 case COLL_POSTEN -> {
 Navigator nav = createObjectNavigator(BsPosten.getConcreteClass());
 ⑤
 if (nav != null) {
 CfgFilter filter = CfgFilter.create(BsPosten.FN_REF_KUNDE, kunde);
 nav.addSystemFilter(filter); ⑥
 //nav.setProperty(Navigator.PROP_TYPE,
Navigator.TYPE_ABSOLUTE_POSITIONABLE); ⑦
 }
 yield nav;
 }
 ...
 default -> super.createCollectionNavigator(coll);
 };
 };
}
```

```

 }
 ...
}
~~~

```

<1> Die Konstante für den Namen der Collection (wie üblich mit Präfix "COLL\_") wird im Manipulator definiert  
(Collection steht nur hier zur Verfügung).

<2> CollectionInfo erzeugen und im Manipulator registrieren.

<3> createCollectionNavigator muss überschrieben werden. Es wird ein Navigator für die entsprechende Untersammlung zurückgegeben.

<4> Es kann eine switch-Variante mit Namen anstelle der Offsets (wie üblicherweise bei den Feldern) benutzt werden.

<5> Der Navigator wird über eine Hilfsmethode erzeugt. Die Implementierung der Basisklasse prüft, ob es sich um Betriebsdaten handelt und legt den Navigator gegebenenfalls über die entsprechende Member-Collection des Betriebes an.

<6> Die Datenmenge wird über einen für den Benutzer nicht sichtbaren Systemfilter eingeschränkt.

<7> Falls die erwartete Datenmenge relativ klein ist, kann man den Typ des Navigators auf "absolut positionierbar" ändern.

Damit werden bei einer Abfrage alle Daten in eine Speicherliste eingelesen. Im Grid steht dann eine Scrollbar anstelle der Navigationsschaltflächen zur Verfügung.

Es gibt eine Abweichung von diesem Muster für Fälle, wo auf Untersammlungen gefiltert wird. Das ORM-Framework benötigt hier Informationen zur Verknüpfung, um einen entsprechenden Filter (mit Join) umzusetzen. Dazu wird die Collection wie üblich im Guide registriert, der Zugriff auf die Daten aber nur über den Manipulator zur Verfügung gestellt.

Dazu ein (vereinfachtes) Beispiel von Bs/MnFaktLieferant:

```

~~~java
public class BsFaktLieferant extends ... {
 ...
 // Namen für Collections
 public static final String COLL_ARTIKEL = BsArtikellieferant.NAME; ①
 ...

 protected static void initializeGuide(BsfGuide g) {
 ...
 // Collections registrieren ②
 }
}

```

```

 g.addCollection(BsArtikellieferant.class,
BsArtikellieferant.FN_REF_LIEFERANT).setName(COLL_ARTIKEL);
 ...
 }
 ...
}
~~~

~~~java
public class MnFaktLieferant extends ... {
 ...
 /** Konstruktor. */
 public MnFaktLieferant(Guide guide, Session session) {
 super(guide, session);
 ...
 addCollection(BsArtikellieferant.class, BsArtikellieferant.FN_REF_LIEFERANT);
 ③
 ...
 }

 ...

 @Override
 protected Navigator createCollectionNavigator(int coll) {
 MnCollection mnColl = getCollection(coll);
 return switch (mnColl.getName()) {
 ...
 case BsFaktLieferant.COLL_ARTIKEL -> {
 Navigator nav =
createObjectNavigator(mnColl.getInfo().getBusinessClass());
 if (nav != null)
nav.addSystemFilter(CfgFilter.create(BsArtikellieferant.FN_REF_LIEFERANT, lieferant));
 yield nav;
 }
 ...
 default -> super.createCollectionNavigator(coll);
 };
 }
 ...
}
~~~

```

<1> Die Konstante für den Namen der Collection wird im Business-Objekt definiert.

<2> Informationen zur Collection werden im Guide-Objekt registriert.

<3> Für die Registrierung der Collection im Manipulator kann auf die im Guide-Objekt registrierten Informationen zurückgegriffen

werden (es muss kein CollectionInfo-Objekt erzeugt werden).

<4> In createCollectionNavigator muss ein entsprechend gefilterter Navigator für die Daten erzeugt werden.



Dieses Muster sollte bevorzugt implementiert werden, wenn davon auszugehen ist, dass auf Daten einer Untersammlung gefiltert wird, die man eigentlich nur im Manipulator verwalten möchte.

!!n

Man kann auch auf Daten einer Untersammlung filtern, deren Informationen nicht im Guide-Objekt registriert sind.

Allerdings muss das ORM-Framework hierfür eigens einen Manipulator erzeugen, um die benötigten Informationen zu ermitteln.

Das ist in Ausnahmefällen kein Problem, sollte aber für häufige Fälle aus Effizienzgründen vermieden werden.

!!

#### #### ConcreteClass-Muster

Ableitbare konkrete Business-Klassen werden mit dem ConcreteClass-Muster implementiert. Dabei wird die konkrete Klasse in einem statischen Member registriert, welcher in der Form `<Object>.getConcreteClass()` überall im Programm anstelle von `<Object>.class` benutzt wird.

```
~~~java
public class BsAdresse {
 ...

 // die konkrete Klasse für die Anwendung.
 private static Class<? extends BsAdresse> concreteClass = BsAdresse.class;

 ...

 /** Liefert die konkrete Klasse für diese Anwendung. */
 public static Class<? extends BsAdresse> getConcreteClass() {
 return concreteClass;
 }

 /** Setzt die konkrete Klasse für diese Anwendung. */
 public static void setConcreteClass(Class<? extends BsAdresse> cls) {
 assert concreteClass.isAssignableFrom(cls);
 concreteClass = cls;
 }
}
~~~
```

In der Applikation werden die konkreten Klassen noch vor den Guides registriert, damit alle Referenzen mit den korrekten (konkreten) Klassen registriert werden können.

```
~~~java
public class FibuApp extends ... {
 ...
}
```

```

@Override
public void setConcreteDatabaseClasses() {
 super.setConcreteDatabaseClasses();
 ...
 BsLand.setConcreteClass(BsFibuLand.class);
 BsAdresse.setConcreteClass(BsFibuAdresse.class);
 ...
}

@Override
public void registerDatabaseClasses() {
 super.registerDatabaseClasses();
 ...
 it.prodata.gest.fibu.data.BsFibuLand.register(DB_TYPES_FIRM);
 it.prodata.gest.fibu.data.BsFibuAdresse.register(DB_TYPES_FIRM);
 ...
}
}
~~~

```

Die Guide-Objekte werden nur von den konkreten Klassen registriert.

```

~~~java
public class BsAdresse {
 ...

 /** Initialisiert das Guide-Objekt. */
 protected static void initializeGuide(BsfGuide g) {
 ...
 }

 /** Registriert Hilfsklassen für abhängige Business-Objekte. */
 protected static void registerDependentClasses(int[] dbTypes) { ②
 BsAdressbeziehung.register(dbTypes);
 BsAdresseZuGruppe.register(dbTypes);
 ...
 }

 /** Erzeugt ein passendes Guide-Objekt. */
 protected static BsfGuide createGuide(int[] dbTypes) {
 return new BsfGuide(dbTypes, concreteClass, NAME, PLURAL_NAME);
 }

 /** Registriert Hilfsklassen für das Business-Objekt. */
 public static void register(int[] dbTypes) {
 if (concreteClass == BsAdresse.class) { ①
 BsfGuide g = createGuide(dbTypes);
 initializeGuide(g);
 Guide.register(g);
 }
 }
}

```

```

 registerDependentClasses(dbTypes); ②
 }
}
~~~

~~~java
public class BsFibuAdresse extends BsAdresse {
 ...

 /** Initialisiert das Guide-Objekt. */
 protected static void initializeGuide(BsfGuide g) {
 BsAdresse.initializeGuide(g);
 ...
 }

 /** Registriert Hilfsklassen für abhängige Business-Objekte. */
 protected static void registerDependentClasses(int[] dbTypes) { ②
 BsAdresse.registerDependentClasses(dbTypes);
 ...
 }

 /** Registriert Hilfsklassen für das Business-Objekt. */
 public static void register(int[] dbTypes) {
 BsAdresse.register(dbTypes); ③
 if (getConcreteClass() == BsFibuAdresse.class) { ①
 BsfGuide g = createGuide(dbTypes);
 initializeGuide(g);
 Guide.register(g);

 registerDependentClasses(dbTypes); ②
 }
 }
}
~~~

```

<1> Das Guide-Objekt wird nur von der konkreten Klasse registriert.

<2> Abhängige Klassen, die wiederum mit dem ConcreteClass-Muster implementiert sein könnten, werden idealerweise in einer getrennten registerDependentClasses-Methode registriert, die auch von den abgeleiteten Klassen aufgerufen werden kann.

<3> Man beachte den Aufruf der entsprechenden Methode der Basisklasse, welche u.U. weitere gemeinsame nicht abgeleitete Klassen oder Sonstiges registriert.

Das folgende Beispiel zeigt die Nutzung von <Object>.getConcreteClass() anstelle von <Object>.class.

BsAdresse ist eine ableitbare Klasse laut dem ConcreteClass-Muster.

BsBeziehungsart ist eine konkrete Klasse ohne Ableitungen.

```
~~~java
public class BsAdressbeziehung extends BsAutoID {
 ...

 /** Initialisiert das Guide-Objekt. */
 protected static void initializeGuide(BsfGuide g) {
 BsAutoID.initializeGuide(g);
 ...
 // Felder registrieren
 g.addField(createFieldInfo(FN_REF_ADRESSE, BsAdresse.getConcreteClass()));
 g.addField(createFieldInfo(FN_REF_BEZUGSADRESSE,
BsAdresse.getConcreteClass()));
 g.addField(createFieldInfo(FN_REF_BEZIEHUNGSART, BsBeziehungsart.class));
 ...
 }
}
~~~
```

### ### Zugriffsrechte

#### #### Zugriff auf Objekte

In Bezug auf die Business-Objekte werden folgende Zugriffsrechte verwaltet:

- Guide.access: die maximalen (technischen) Zugriffsrechte auf Business-Objekte (unabhängig vom Zugriffsschlüssel)
- Manipulator.maxAccess: maximale datenunabhängige Zugriffsstufe (Key)
- Manipulator/MnCollection.listAccess: maximale vom Collection- bzw. Reference-Holder abhängige Zugriffsstufe auf diese Objekte
- Manipulator.objAccess: maximale vom aktuellen Objekt abhängige Zugriffsstufe
- Manipulator.access: aktuelle, von außen steuerbare Zugriffsstufe (Änderungs-/Ansichtsdialog)

```
~~~java
 protected static void initializeGuide(Guide g) {
 g.setManipulatorClass(MnAdressvergleich.class);
 g.setResBundleName(RbAdresse.class.getName());
 g.setModuleName(BaseConstants.MODULE_NAME);
 g.setAccess(Key.READ_RIGHTS); ①
 ...
 }

 @Override
 protected void postFreezeSettings() {
 super.postFreezeSettings();
 if (!isAtLeastSupportUser())
 limitMaxAccess(Key.READ_RIGHTS); ②
 }
}
```

```

 }

 @Override
 protected void updateObjectAccess() {
 super.updateObjectAccess();
 if (konto.getArt() != BsKonto.KA_KONTO) {
 getCollection(firstColl + CLO_BILANZKONTO_KONTOZUORDNUNGEN)
 .limitListAccess(Key.READ_DELETE_RIGHTS); ③
 }
 }

 @Override
 protected void updateObjectAccess() {
 super.updateObjectAccess();
 if (!obj.isMemberOfCurrentBetrieb())
 limitObjectAccess(hasSpecialDataMaintenanceRights() ?
Key.READ_DELETE_RIGHTS : Key.READ_RIGHTS); ④
 }
}

```

~~~

<1> Nicht änderbares Objekt

<2> Nur Support-Benutzer dürfen Objekte ändern

<3> Die Objekte der Member-Collection des aktuell angeordneten Objekts dürfen nur gelesen oder gelöscht werden

<4> Das aktuell angeordnete Objekt gehört nicht zum aktuellen Betrieb und darf daher nicht geändert werden.

Im speziellen Datenpflegetmodus dürfen die Objekte von Support-Benutzern gelöscht werden.

#### #### Zugriff auf einzelne Elemente von Objekten

Die Zugriffsrechte auf die einzelnen Elemente von Business-Objekten werden in MnElement verwaltet:

- MnElement.maxAccess: maximale datenunabhängige Zugriffsstufe (Key)
- MnElement.access: momentane datenabhängige vom aktuellen Objekt selbst bestimmte Zugriffsstufe auf das Element
- MnCollection.listAccess: maximale vom Collection-Holder kontrollierte datenabhängige Zugriffsstufe (siehe auch oben)
- MnReference.listAccess: maximale vom Reference-Holder kontrollierte datenabhängige Zugriffsstufe (siehe auch oben)

~~~java

```

 @Override
 protected void postAccessInitialisation() {
 super.postAccessInitialisation();
 if (isDataSource() && !Key.canChangeObject(getMaxAccess()) ①
 && HotelBaseApp.hasExtraInventoryRights(getSession()))
 {
 // Der Benutzer darf den Extra-Bestand verwalten. Er benötigt hierfür

```

## Ändern-Rechte.

```
 setMaxAccess(Key.CHANGE_RIGHTS); ②
 getLocalField(FO_ANFANGSBESTAND).setMaxAccess(Key.FIELD_RIGHTS); ③
 getLocalField(FO_KORREKTUR).setMaxAccess(Key.FIELD_RIGHTS);
 getLocalField(FO_EINGANGSKORREKTUR).setMaxAccess(Key.FIELD_RIGHTS);
 getLocalField(FO_EINGANG).setMaxAccess(Key.FIELD_RIGHTS);
 getLocalField(FO_AUSGANG).setMaxAccess(Key.FIELD_RIGHTS);
 }
}

@Override
protected void updateObjectAccess() {
 super.updateObjectAccess();
 // Definitiv abgeschlossene Einträge dürfen nicht mehr verändert werden.
 if (meldung.isDefinitiv()) {
 limitObjectAccess(Key.CHANGE_RIGHTS); ④
 for (int i = 0, count = getCollectionCount(); i < count; i++)
 getCollection(i).limitListAccess(Key.READ_RIGHTS); ④
 }
}

@Override
protected void updateAccess() {
 super.updateAccess();
 boolean definitiv = meldung.isDefinitiv();
 int acc = definitiv ? Key.FIELD_READ : Key.FIELD_RIGHTS;
 int defFldIdx = firstField + FO_DEFINITIV;
 for (int i = 0, count = getFieldCount(); i < count; i++) {
 if (i != defFldIdx)
 getField(i).setAccess(acc); ④
 }
}
```

~~~

<1> Der Benutzer hat eigentlich keine Rechte, diese Daten zu ändern. Er hat aber das Sonderrecht, den Extra-Bestand zu verwalten.

<2> Noch vor dem Aufruf von postAccessInitialisation() wurde in freezeSettings() der Zugriff auf alle Elemente auf "nur Lesen" eingeschränkt. Es müssen daher explizit Ändern-Rechte vergeben werden, um die Bestandsfelder ändern zu können.

<3> Der Zugriff auf die für die Bestandsverwaltung benötigten Felder wird freigegeben.

<4> Definitive Meldungen dürfen nicht gelöscht werden. Die Member-Collections dürfen nicht geändert werden.

Alle Felder bis auf "Definitiv" dürfen nicht geändert werden.

## Implementierungshinweise:

- Manipulator.updateAccess() ruft Manipulator.updateObjectAccess() auf.
- Manipulator.updateObjectAccess() restauriert Manipulator.objAccess und listAccess sämtlicher Collections auf Key.MAX\_RIGHTS.

Abgeleitete Klassen müssen dann lediglich bei Bedarf Zugriffsrechte mittels

limitObjectAccess und limitListAccess begrenzen.

- Die Zugriffsrechte auf Felder und Kommandos werden in der Basisklasse nicht automatisch auf Key.MAX\_RIGHTS restauriert und müssen daher immer mittels setAccess bzw. setEnabled gesetzt werden.

## ## Ressourcen

Beim Programmieren werden häufig Ressourcen wie Files, Datenbankverbindungen usw. angefordert, die nach Verwendung wieder freigegeben werden müssen. In diesem Artikel werden die wichtigsten Richtlinien und Patterns hierfür beschrieben.

Diese Patterns gelten auch für Muster, wo Einstellungen temporär verstellt und am Ende wieder zurückgestellt werden.

### ### Einführendes Beispiel: Datenbank freigeben

Im folgenden Beispiel wollen wir die Anzahl der Analysen in der Datenbank ermitteln:

```
~~~java
private static long getAnalyseCount_LEAKY_DONT_DO_THIS(Session session) {
    BsfDatabase db = BsfDatabasePool.INSTANCE.getSystemDatabase(session);
    if (db == null)
        return -1;
    BsfCollection coll = db.getObjectCollection(BsAnalyse.class);
    return coll.getCountNoExc();
}
~~~
```

Die Methode liefert das richtige Ergebnis, ist aber falsch in Bezug auf die Ressourcenverwaltung.

Wir müssen nämlich die Datenbank mittels 'endUse()' freigeben, wenn wir sie nicht mehr verwenden.

Ansonsten bleiben diese Ressourcen unnötigerweise weiter gebunden.

Wenn sich das aufschauelt, kann das auch zu OutOfMemory-Fehlern führen.

Ein naiver Versuch das zu verbessern könnte so aussehen:

```
~~~java
private static long getAnalyseCount_LEAKY_DONT_DO_THIS(Session session) {
    BsfDatabase db = BsfDatabasePool.INSTANCE.getSystemDatabase(session);
    if (db == null)
        return -1;
    BsfCollection coll = db.getObjectCollection(BsAnalyse.class);
    long count = coll.getCountNoExc();
    db.endUse();
    return count;
}
~~~
```

Hier wird zwar `endUse()` aufgerufen, aber nur wenn alles glatt läuft. Bei Exceptions haben wir immer noch das Problem, dass die Methode zurückkehrt, ohne die Datenbank freizugeben. Zwar werden Exceptions in `getCountNoExc` abgefangen und behandelt, aber es könnte theoretisch eine Exception beim Behandeln auftreten, oder in `getObjectCollection`, oder ein `Error` statt einer Exception. Richtig ist also folgende Variante:

```
~~~java
private static long getAnalyseCount(Session session) {
    BsfDatabase db = BsfDatabasePool.INSTANCE.getSystemDatabase(session);
    if (db == null)
        return -1;
    try {
        BsfCollection coll = db.getObjectCollection(BsAnalyse.class);
        return coll.getCountNoExc();
    }
    finally {
        db.endUse();
    }
}
~~~
```

Wir fordern die Ressource (hier die Datenbank) an, benutzen sie in einem `try`-Block und geben sie am Ende in `finally` wieder frei. Dadurch ist garantiert, dass die Datenbank auch bei Exceptions korrekt freigegeben wird.

### Exceptions kommen vor

Das ist die erste und wichtigste Erkenntnis:

**\*\*Wenn es um Ressourcen geht, müssen wir stets auch an Exceptions denken.\*\***

Am besten sollten wir uns dabei gar nicht fragen: `[.line-through]"Kann dieser Code Exceptions werfen?"` sondern `"Was passiert mit meiner Ressource, falls dieser Code Exceptions wirft?"`.

Hier geht es nämlich nicht nur um "checked" Exceptions, die als `throws` deklariert oder abgefangen werden müssen, sondern auch um `RuntimeExceptions` wie `NullPointerException` oder `IllegalArgumentException`, die aufgrund von Programmierfehlern oder warum auch immer auftreten können, und zwar überall.

Klar kann man sagen, dass solche Programmierfehler nicht passieren sollten. Tatsache ist aber, dass sie passieren. Und wenn sie passieren, dann ist zwar die entsprechende Methode (im obigen Beispiel etwa `getObjectCollection` oder `getCount`) schuld an der Exception, aber nicht am Resource-Leak. Es ist nämlich nicht die Aufgabe dieser aufgerufenen Methoden,



die Ressource freizugeben, sondern die des Aufrufers. Wenn wir also durch so einen Programmierfehler eine Ressource leaken, dann ist dieser Leak selbst schon der nächste Programmierfehler, und den gilt es genauso zu vermeiden wie den ersten.

Älterer Code ist leider oft nicht nach dieser Richtlinie geschrieben. Das soll aber nicht als Muster für neuen Code gelten. Neuen Code sollten wir immer mit `try-finally` usw. absichern, und bestehende unsaubere Muster korrekt umbauen.

### ### Allgemeine Beispiele

#### #### try-with-resources

Falls die Ressource `AutoClosable` implementiert, sollte immer try-with-resources verwendet werden

#### ##### InputStream, OutputStream

```
~~~java
void writeTo(Path path, OutputStream out) throws IOException {
    try (InputStream in = Files.newInputStream(path)) {
        in.transferTo(out);
    }
}
~~~
```

#### ##### Streams von Files

Streams müssen normal nicht geschlossen werden, aber jene die von Methoden der `Files`-Klasse erzeugt werden schon:

```
~~~java
void printDirectoryTree(Path path) throws IOException {
    try (Stream<Path> paths = Files.walk(path)) {
        paths.forEach(System.out::println);
    }
}
~~~
```

#### ##### Transaktionen

```
~~~java
void doSomethingInTransaction() throws IOException {
    try (var tx = db.transaction()) {
        if (doSomething())
            tx.commit();
    }
}
~~~
```

Hier ist wichtig, dass keine Fehler "verschluckt" werden, sodass ``tx.commit()`` wirklich nur bei Erfolg ausgeführt wird.  
Am besten geht das meistens, wenn man mit Exceptions arbeitet und beispielsweise bei Methoden von ``BsfCollection`` nicht die ``*NoExc`` Varianten verwendet.

#### try-finally

##### ReentrantLock

```
~~~java
void doSomething() {
    lock.lock();
    try {
        // do something
    }
    finally {
        lock.unlock();
    }
}
~~~
```

##### Navigator.close

```
~~~java
void doSomething() {
    if (nav.open()) {
        try {
            while (nav.next()) {
                Object obj = nav.get();
                // do something
            }
        }
        finally {
            nav.close();
        }
    }
}
~~~
```

##### Manipulator

```
~~~java
void doSomething() {
    var man = coll.createManipulator();
    try {
        // do something
    }
    finally {
        man.releaseResources();
    }
}
```

```

    }
}
~~~

StdHttpClient

~~~java
void doSomething() {
    var httpClient = new StdHttpClient();
    try {
        // do something
    }
    finally {
        httpClient.freeResources();
    }
}
~~~

```

#### ##### Zustand restaurieren

Auch beim Restaurieren von Zustand soll try-finally verwendet werden:

```

~~~java
void doSomething() {
    MessageHandler oldMsgHandler = db.getMessageHandler();
    db.setMessageHandler(tempMessageHandler);
    try {
        // do something
    }
    finally {
        db.setMessageHandler(oldMsgHandler);
    }
}
~~~

```

#### #### try-catch

Wenn Ressourcen zurückgegeben werden, können sie natürlich nicht sofort mit finally freigegeben werden. Wenn allerdings das Anfordern der Ressource scheitert, sollen auch alle zugrundeliegenden Ressourcen wieder abgegeben werden. Dafür kann try-catch verwendet werden, um bei Exceptions diese Ressourcen zu schließen, bevor die Exception weitergeworfen wird. Man kann das als "halbes try-finally" sehen, welches die Ressource nur im Fehlerfall schließt.

```

~~~java
public DelimitedTextRecordSet(URL url, String enc) throws IOException {
    super();
    LineReader reader;
    InputStream in = url.openStream();
}

```

```

try {
    reader = LineReader.create(in, enc, DEFAULT_BUFFER_SIZE);
}
catch (UnsupportedEncodingException exc) {
    // Ungültige Zeichensatz-Kodierung -> wir schließen den InputStream wieder
    FileUtils.closeNoExc(in);
    throw exc;
}
init(reader);
this.url = url;
this.encoding = enc;
}
~~~

```

### ### Komponenten

#### #### Swing

In Swing-Komponenten kann `CloseListener.commitClosing` überschrieben werden, um Ressourcen zu schließen:

```

~~~java
public class MyComponent extends StdContainer {

    private BsBetrieb betrieb;

    public MyComponent(BsBetrieb betrieb) {
        this.betrieb = betrieb;
    }

    @Override
    public void commitClosing(CloseEvent event) {
        if (betrieb != null) {
            betrieb.getDatabase().endUse();
            betrieb = null;
        }
    }
}
~~~

```

#### #### Vaadin

In Vaadin muss `onDetach` überschrieben werden, um Ressourcen freizugeben. Dabei ist zu beachten, dass ein Aufruf von `onAttach` folgen kann, entweder weil die Komponente nur entfernt wurde, um sie irgendwo anders hinzuzufügen, oder weil Vaadin vom Framework aus alle Komponenten entfernt und wieder hinzufügt, um sich neu mit dem Client zu synchronisieren (der sogenannte Resync). Es gibt zwei Möglichkeiten, damit richtig umzugehen.

Die erste besteht darin, jegliche Ressourcen erst in `onAttach` anzufordern. Dadurch

ist es kein Problem, wenn die Komponente wieder attached wird - sie holt sich einfach die benötigten Ressourcen und kann wieder arbeiten:

```
~~~java
public class MyComponent extends Composite<VerticalLayout> {

    private ShortcutRegistration shortcutRegistration;

    @Override
    protected void onAttach(AttachEvent attachEvent) {
        shortcutRegistration = Shortcuts.addShortcutListener(this, this::onEnter,
Key.ENTER);
    }

    @Override
    protected void onDetach(DetachEvent detachEvent) {
        if (shortcutRegistration != null) {
            shortcutRegistration.remove();
            shortcutRegistration = null;
        }
    }

    private void onEnter() {
        // do something
    }
}
~~~
```

Als Variante können auch Attach- und DetachListener von außen zu einer Komponente hinzugefügt werden. So wird z.B. das Auto-Refresh in 'GridDialog' implementiert:

```
~~~java
var queue = man.getScopedQueue(context);
masterDetailView.addAttachListener(attachEvent -> {
    TimerTask timerTask = Fun.createTimerTask(() ->
queue.add(masterDetailView::refresh));
    Fun.getCommonTimer(true).schedule(timerTask, refreshRate, refreshRate);
    masterDetailView.addDetachListener(detachEvent -> {
        detachEvent.unregisterListener();
        timerTask.cancel();
    });
});
~~~
```

In diesem Beispiel arbeiten wir mit verschachtelten Listenern, damit der DetachListener direkt Zugriff auf den TimerTask erhält. Dabei müssen wir aufpassen, dass der DetachListener auch wieder entfernt wird, sonst wächst die Anzahl der DetachListener bei jedem Attach. Deshalb entfernt sich der DetachListener beim ersten

Aufruf selbst mittels ``detachEvent.unregisterListener()``.

Die zweite Möglichkeit besteht darin, das Freigeben der Ressourcen solange zu verzögern, bis feststeht, ob die Komponente nur durch einen Resync detached wurde und gleich wieder attached wurde oder nicht. Falls wir dann selbst die Komponente nie noch einmal explizit attachen, können wir uns sicher sein, dass sie detached bleibt. Hierfür kann die Hilfsmethode ``WebUtils.runWhenPermanent`` benutzt werden:

```
~~~java
public class MyComponent extends Composite<VerticalLayout> {

    private BsBetrieb betrieb;

    public MyComponent(BsBetrieb betrieb) {
        this.betrieb = betrieb;
    }

    @Override
    protected void onDetach(DetachEvent detachEvent) {
        WebUtils.runWhenPermanent(detachEvent, this::releaseResources);
    }

    private void releaseResources() {
        if (betrieb != null) {
            betrieb.getDatabase().endUse();
            betrieb = null;
        }
    }
}
~~~
```

### ### Spezielle Patterns

#### #### Loan Pattern

Man kann die Verwaltung der Ressource auch in eine Hilfsmethode auslagern und von der Verwendung trennen.

So könnte man z.B. die folgende Methode in `BsfDatabasePool` einführen:

```
~~~java
public <T> T useDatabase(Session session, int dbType, ThrowingFunction<? super
Bsfdatabase, ? extends T> fn) throws Exception {
    var db = getDatabase(session, dbType);
    if (db == null)
        return null;
    try {
        return fn.apply(db);
    }
}
```

```

        finally {
            db.endUse();
        }
    }
}
~~~

```

Die Verwendung könnte dann so aussehen:

```

~~~java
long count = BsfDatabasePool.INSTANCE.useDatabase(session, BsfApp.DB_TYPE_FIRM,
    db -> db.getObjectCollection(BsAdresse.getConcreteClass()).getCount()
);
~~~

```

#### #### Asynchrone Programmierung mit CompletableFuture

Bei `CompletableFutures` übernimmt `'whenComplete'` die Rolle von `'finally'`:

```

~~~java
CompletionStage<Void> doSomethingAsync() {
    var db = BsfDatabasePool.INSTANCE.getDatabase(session, BsfApp.DB_TYPE_FIRM);
    if (db == null)
        return CompletableFuture.completedFuture(null);
    return doSomethingAsyncWithDb(db)    // Liefert CompletionStage
        .whenComplete((r, exc) -> db.endUse());
}
~~~

```

Das reicht normalerweise aus, vor allem wenn man mit gut getesteten Bibliotheksfunktionen wie z.B. `HttpClient.sendAsync` zu tun hat. Ansonsten kann es aber passieren, dass z.B. `'doSomethingAsyncWithDb'` direkt eine synchrone Exception wirft, noch bevor eine `'CompletionStage'` zurückgegeben wird. Um das zu verhindern sollte folgendes Pattern benutzt werden:

```

~~~java
CompletionStage<Void> doSomethingAsync() {
    var db = BsfDatabasePool.INSTANCE.getDatabase(session, BsfApp.DB_TYPE_FIRM);
    if (db == null)
        return CompletableFuture.completedFuture(null);
    return CompletableFuture.completedFuture(db)
        .thenCompose(this::doSomethingAsyncWithDb) // Liefert CompletionStage
        .whenComplete((r, exc) -> db.endUse());
}
~~~

```

#### #### BsfDatabaseListener

Besonders bei Manipulatoren kann es vorkommen, dass sie gar nicht freigegeben werden, da sie oft keine wesentlichen Ressourcen verwalten. Wenn aber eine Manipulator-Implementierung trotzdem

sicherstellen will, dass Ressourcen zumindest irgendwann beim Freigeben der Datenbank abgegeben werden, kann dies über BsfDatabaseListener erreicht werden.

```
~~~java
public class MnCloudInstallation extends MnFirmDependentObject {
    // ...
    private BsfDatabaseListener databaseListener;
    private NotificationEventListener installationStateChangedListener;
    private NotificationEventListener installationInstanceStateChangedListener;
    private NotificationEventListener installationCloudConDeviceChangedListener;
    private NotificationEventListener installationBackupChangedListener;

    public MnCloudInstallation(Session session) {
        super(session);
        // ...
        registerAllListeners();
    }

    @Override
    public void releaseResources() {
        deregisterAllListeners();      // N.B. vor super-Aufruf, damit der
DatabaseListener deregistriert werden kann
        super.releaseResources();
    }

    /** Deregistriert alle Listener. */
    private void deregisterAllListeners() {
        if (databaseListener != null) {
            BsfDatabase db = getDatabase();
            if (db != null)
                db.removeBsfDatabaseListener(databaseListener);

TechnicalNotificationManager.deregisterEventListener(installationStateChangedListener)
;

TechnicalNotificationManager.deregisterEventListener(installationInstanceStateChangedL
istener);

TechnicalNotificationManager.deregisterEventListener(installationCloudConDeviceChanged
Listener);

TechnicalNotificationManager.deregisterEventListener(installationBackupChangedListener
);

        databaseListener = null;
        installationStateChangedListener = null;
        installationInstanceStateChangedListener = null;
        installationCloudConDeviceChangedListener = null;
        installationBackupChangedListener = null;
    }
}
```



```

    /*** Registriert alle Listener. */
    private void registerAllListeners() {
        BsfDatabase database = getDatabase();
        if (database != null) {
            // Über den DatabaseListener wird sichergestellt, dass alle Listener
            // spätestens beim Schließen der
            // Datenbank deregistriert werden.
            databaseListener = new BsfDatabaseListener() {
                @Override public void databaseClosed(BsfDatabase db) {
                    deregisterAllListeners();
                }

                @Override public void databaseUseTerminated(BsfDatabase db) {
                    deregisterAllListeners();
                }

                @Override public void databaseSessionChanged(BsfDatabase db) { }
            };
            database.addBsfDatabaseListener(databaseListener);

            initializeInstallationStateChangedListener();
            initializeInstallationInstanceStateChangedListener();
            initializeInstallationCloudConDeviceChangedListener();
            initializeInstallationBackupChangedListener();
        }
    }
}
~~~

```

#### #### BsfDatabase.beginUse

Bei BsfDatabase gibt es neben der Methode `endUse` auch `beginUse`. Es wird nämlich ein Benutzungszähler (`useCounter`) verwaltet, der von `beginUse` inkrementiert und von `endUse` dekrementiert wird. Die Verbindung wird erst an den Pool zurückgegeben, wenn der Zähler auf 0 geht.

Das ist nützlich für asynchrone Aufgaben, die am Ende die Ressource selbständig schließen, aber auch synchron ausgeführt werden können. Für die asynchrone Benutzung wird die Datenbank einfach übergeben.

~~~java

```

static class SomeDbTask extends DefaultSyncExecutableTask<Void> {

 private BsfDatabase db;

 SomeDbTask(BsfDatabase db) {
 this.db = db;
 }
}

```

```

@Override
protected Result<Void> executeImpl(ExecutionContext execCtx) {
 Result<Void> result = Result.ok();
 // ... do something with db
 return result;
}

@Override
public void releaseResources() {
 if (db != null) {
 db.endUse();
 db = null;
 }
 super.releaseResources();
}
}

void cmdSomeTaskInNonModalDialog(MessageContext context) {
 var db = BsfDatabasePool.INSTANCE.getDatabase(session, BsfApp.DB_TYPE_FIRM);
 if (db != null) {
 createExecutionController()
 .start(context, false, new SomeDbTask(db)); // false bedeutet nicht
modal
 }
}
}
~~~

```

Falls wir den Task aber synchron aufrufen und die Datenbank anschließend noch weiter benutzen wollen, müssen wir zuvor  
`beginUse` aufrufen, damit der Benutzungszähler nicht auf 0 geht:

```

~~~java
void doSomethingWithDb(ExecutionContext execCtx, BsfDatabase db) {
 // ... do something with db
 db.beginUse();
 Result<Void> someResult = new SomeDbTask(db).execute(execCtx);
 // ... do more stuff with db
}
~~~

```

Hier ist zu beachten, dass `beginUse` möglichst unmittelbar vor der Ausführung des Tasks aufgerufen wird.  
Ansonsten riskieren wir, dass es bis zur Ausführung zu einer Exception kommt, sodass der Task die Datenbank nie freigibt.  
Um ganz sicher zu gehen können wir das auch mit `try-catch` absichern:

```

~~~java
void doSomethingWithDb(ExecutionContext execCtx, BsfDatabase db) {
 // ... do something with db
}

```

```

db.beginUse();
try {
 // possible to do more stuff here
 Result<Void> someResult = new SomeDbTask(db).execute(execCtx);
}
catch (Exception e) {
 db.endUse();
 throw e;
}
// ... do more stuff with db
}
~~~

```

#### Wer schließt die Ressource?

Komplexe Patterns wie `beginUse` sind selten und meistens vermieden werden, wenn man sich an folgende Richtlinie hält:

**\*\*Wer die Ressource öffnet soll sie auch schließen\*\*.**

Nicht zufällig deckt sich das mit `try-with-resources`, wo die Ressource im Kopf geöffnet und automatisch beim Verlassen des Bodys geschlossen wird. Das hat nämlich folgende Vorteile:

- **\*\*Einfache Kontrolle:\*\*** Wenn ich sehe, dass eine Ressource angefordert wird, muss ich nicht (rekursiv) alle Methoden überprüfen, denen sie übergeben wird, sondern sollte noch in der selben Methode das Aufräumen finden.
- **\*\*Mehr Flexibilität:\*\*** Wenn eine Methode, die eine Ressource benutzt sie anschließend aufräumt, nimmt einem das die Möglichkeit, die Ressource anschließend weiter zu benutzen. Es gibt vielleicht Fälle, wo eine weitere Benutzung wenig Sinn macht, aber da hat man immer noch die anderen Vorteile.
- **\*\*Robuster:\*\*** Wenn ich mich darauf verlasse, dass eine Methode die Ressource am Ende abgibt, diese Methode aber nie aufgerufen wird weil vorher eine andere schon eine Exception wirft, dann bleibt die Ressource offen.

Folgendes sollte also vermieden werden:

```

~~~java
byte[] readAllBytesFrom_DONT_DO_THIS(InputStream in) throws IOException {
 try (in) { // Variante von try-with-resources ohne neue Variable
 return in.readAllBytes();
 }
}
~~~

```

Hier passiert an sich nichts schlimmes, aber die Verwaltung der Ressourcen wird insgesamt komplexer.

Außerdem wird der Aufrufer dazu verleitet, sich nicht selbst um das Schließen zu

kümmern:

```
~~~java
InputStream in = openInputStream();
byte[] bytes = readAllBytes_DONT_DO_THIS(in);
~~~
```

Hier ist nicht gleich ersichtlich, ob der `InputStream` sauber geschlossen wird. Stattdessen müssen wir alle Methoden kontrollieren, denen ein `InputStream` übergeben wird. Hier ist das noch relativ überschaubar, aber bei mehreren und verschachtelten Methoden kann das schnell unübersichtlich werden.

Außerdem riskieren wir so ein Leak, wenn noch Code zwischen dem Öffnen der Ressource und der Übergabe an die schließende Methode eingefügt wird.

### Ressourcen modellieren

#### `AutoCloseable`

Klassen, die `'AutoClosable'` (oder `'Closeable'`) implementieren, können mit `try-with-resources` benutzt werden.

```
~~~java
public class JsonReader implements AutoCloseable {

 private final Reader reader;

 public JsonReader(Reader reader) {
 this.reader = reader;
 }

 @Override
 public void close() throws IOException {
 reader.close();
 }
}
~~~
```

Die `close`-Implementierung sollte dabei idempotent sein. Das bedeutet, dass ein zweiter Aufruf keinen Effekt haben soll.

Im obigen Beispiel leiten wir den Aufruf einfach an den Reader weiter, der `'close'` ja auch idempotent implementiert.

#### `releaseResources`, `freeResources`

Unseren eigenen Klassen bieten oft eine Methode `'releaseResources'` oder `'freeResources'`, welche ähnlich wie `'AutoCloseable.close'` die Ressourcen der Klasse freigibt. Der Unterschied zwischen den beiden ist, dass ein Objekt nach

Aufruf von `'releaseResources'` nicht mehr benutzt werden kann, bei `'freeResources'` hingegen schon.

## ## CloseListener

### ### Motivation

In unserem Programm können stark verschachtelte Views entstehen, wie etwa ein Grid mit eingebettetem Formular, welches wiederum ein Grid beinhaltet, von dem das freistehende Formular geöffnet wurde, welches auch wieder ein Grid beinhaltet usw.

Auf jeder dieser Ebenen können Änderungen vorgenommen werden, die validiert und gespeichert werden müssen.

Wenn dann eine äußere Komponente geschlossen wird, z.B. weil ein Dialog mit "x" verlassen wird, oder wenn auf einen anderen Menüpunkt gewechselt wird, müssen alle verschachtelten

Komponenten auf Änderungen überprüft werden.

Falls es solche Änderungen gibt, wird für gewöhnlich eine Benutzerabfrage angezeigt, um zu entscheiden, ob die Änderungen gespeichert oder verworfen werden sollen.

Der folgende Screenshot veranschaulicht so ein Beispielszenario:

{align=center}

Hier befinden wir uns in den Sprints, welche die Backlog-Sammlung direkt einbettet.

Ein Backlog-Eintrag ist in einem freistehenden Formular geöffnet.

Dort gibt es die Sammlung der Tasks, wo wir dabei sind, einen neuen Eintrag zu erstellen.

Wenn wir nun von den Sprints wegnavigieren, indem wir beispielsweise auf "Home" klicken, müssen wir all diese Komponenten rekursiv durchgehen und auf Änderungen prüfen.

Um diese rekursive Benachrichtigung zu ermöglichen gibt es das `'CloseListener'` Interface.

### ### Übersicht

`'CloseListener'` ist ein Interface, welches Komponenten implementieren können, um sich beim Schließvorgang einzuklinken.

Vereinfacht dargestellt sieht das Interface folgendermaßen aus:

```
~~~java
```

```
public interface CloseListener extends EventListener {
```

```
 /** Ein Schließvorgang wurde angefordert. Liefert zurück, ob geschlossen werden darf oder nicht. */
```

```
 boolean startClosing(CloseEvent e);
```

```

 /** Wird aufgerufen, wenn startClosing true geliefert hat und definitiv
geschlossen wird. */
 default void commitClosing(CloseEvent e) {}

 /** Wird aufgerufen, wenn nicht geschlossen wird, obwohl startClosing true
geliefert hat. */
 default void abortClosing(CloseEvent e) {}
}
~~~

```

Das echte Interface ist mittlerweile asynchron, aber konzeptionell hat sich nichts verändert.

Alle `CloseListener`-Methoden erhalten als Argument ein `CloseEvent`, welches die Schließenanforderung repräsentiert.

Dieses beinhaltet einen `CloseMode`, welcher angibt, wie mit ungespeicherten Änderungen umzugehen ist: speichern (`SAVE`), verwerfen (`ABORT`) oder beim Benutzer nachfragen (`ASK`).

```

~~~java
public class CloseEvent extends EventObject {

 public enum CloseMode { ASK, SAVE, ABORT }

 private final CloseMode mode;

 public CloseEvent(Object source, CloseMode mode) {
 super(source);
 this.mode = mode;
 }

 public CloseMode getMode() {
 return mode;
 }
}
~~~

```

### ### Ablauf

Bei einem Schließvorgang sind meist mehrere `CloseListener` involviert. Bei erfolgreichem Schließen eines Dialogs läuft das folgendermaßen ab:

```

~~~uml{align=center}
hide footbox
actor Dialog as D
actor CloseListener1 as CL1
actor CloseListener2 as CL2
actor CloseListener3 as CL3

```

```

D -> CL1 : startClosing
D <-- CL1 : true
|||
D -> CL2 : startClosing
D <-- CL2 : true
|||
D -> CL3 : startClosing
D <-- CL3 : true
|||
D -> CL1 : commitClosing
D -> CL2 : commitClosing
D -> CL3 : commitClosing
~~~

```

Falls hingegen einer der 'CloseListener' bei 'startClosing' 'false' liefert, sieht das so aus:

```

~~~uml{align=center}
hide footbox
actor Dialog as D
actor CloseListener1 as CL1
actor CloseListener2 as CL2
actor CloseListener3 as CL3

D -> CL1 : startClosing
D <-- CL1 : true
|||
D -> CL2 : startClosing
D <-- CL2 : false
|||
D -> CL1 : abortClosing
~~~

```

Man beachte, dass nur von CloseListener1 'abortClosing' aufgerufen wird, weil nur dieser bei 'startClosing' 'true' zurückgegeben hat.

CloseListener2 hingegen hat selbst 'false' zurückgegeben und weiß somit schon, dass nicht geschlossen wird.

Von CloseListener3 wiederum wurde 'startClosing' gar nicht aufgerufen, weil bereits nach CloseListener2 feststand, dass nicht geschlossen wird.

Somit weiß CloseListener3 noch gar nicht, dass ein Schließvorgang eingeleitet wurde, und muss daher auch nichts abbrechen.

### ### Verteilung der CloseEvents

Ein Schließvorgang kann von jeder Komponente (typischerweise ein "Schließen"-Button) über 'Components.closeRequest(Component, CloseMode)' bzw.

'WebUtils.closeRequest(Component, CloseMode)' angefordert werden.

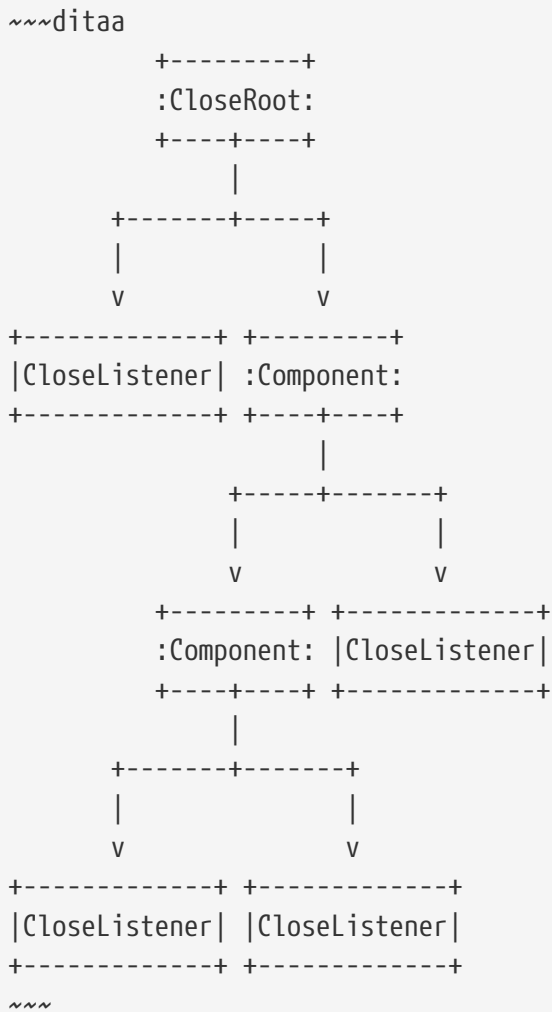
Dabei wird zunächst ausgehend von der angegebenen Komponente rekursiv über Parents die am nächsten 'CloseRoot' gesucht.

Das ist ein Interface, welches z.B. Dialoge implementieren, um zu definieren, dass sie selbst geschlossen werden können.

Von dort ausgehend werden rekursiv alle Kind-Komponenten benachrichtigt.  
 Für Kind-Komponenten, die nicht 'CloseListener' implementieren, werden rekursiv  
 wiederum deren Kinder überprüft.  
 Sobald eine Komponente gefunden wird, die 'CloseListener' implementiert, wird die  
 entsprechende Methode aufgerufen.

Die Kinder von CloseListener-Komponenten werden nicht automatisch benachrichtigt - bei  
 Bedarf muss das die Komponente selbst in ihrer Implementierung der CloseListener-  
 Methoden erledigen.

Das folgende Diagramm zeigt diesen Ablauf:



### ### Implementierung

Im Folgenden wird die 'CloseListener'-Implementierung des 'GuiObjectEditor's  
 gezeigt:

```

~~~java
@Override
public boolean startClosingSync(CloseEvent e) {
 if (objectEditor.isReleased())
 return true; // Der Editor wurde bereits freigegeben, der Dialog muss
 geschlossen werden

 boolean wasNew = objectEditor.isNewObject();

```



```

 boolean canStopEditing = objectEditor.stopEditing(e.getMode());
 boolean canCloseSuper = canStopEditing && super.startClosingSync(e);
 boolean canCloseListeners = canCloseSuper &&
CloseManager.fireStartClosing(listenerList, e);
 boolean canClose = canCloseListeners && objectEditor.startClosing(e);
 if (!canClose) {
 if (canCloseListeners)
 CloseManager.fireAbortClosing(listenerList, e);
 if (canCloseSuper)
 super.abortClosing(e);
 if (canStopEditing) {
 // Einer der CloseListener hat den Schließvorgang abgebrochen --> der
 // Dialog muss wieder in einen konsistenten Zustand gebracht werden.

objectEditor.restartEditing(objectEditor.isUpdateComponentsRequired(wasNew));
 }
 }
 return canClose;
 }
}

```

```

@Override
public void commitClosing(CloseEvent e) {
 super.commitClosing(e);
 CloseManager.fireCommitClosing(listenerList, e);
 dialogComponent = null;
 if (floatingForm != null) {
 floatingForm.detachComponents();
 floatingForm = null;
 }
 hm = null;
 objectEditor.commitClosing();
 objectEditor.releaseResources();
}

```

```

@Override
public void abortClosing(CloseEvent e) {
 super.abortClosing(e);
 CloseManager.fireAbortClosing(listenerList, e);
 objectEditor.abortClosing(e);
}
~~~

```

Wie bereits in `[(#_verteilung_der_closeevents)]` erwähnt, müssen sich die Methoden selbst darum kümmern, die Events an eventuelle enthaltene `CloseListener` weiterzugeben. Bei `'GuiObjectEditor'` betrifft das Kind-Komponenten (über `'super.startClosingSync'`), das freistehende Formular selbst, sowie eine Liste von `'CloseListener'`, die beim Editor programmtechnisch registriert wurden.

Dabei ist es wichtig, dass das Protokoll der Methoden wie in `[(#_ablauf)]` beschrieben eingehalten wird. Das betrifft vor allem `'startClosing'`.

Hier sehen wir, dass der Rückgabewert der internen `'startClosing'`-Aufrufe in getrennten Variablen festgehalten wird, damit am Ende klar ist, wessen `'abortClosing'`-Methode aufgerufen werden muss.

In `'commitClosing'` werden in der Swing Ressourcen freigegeben.

In `'abortClosing'` werden hier nur die enthaltenen Listener benachrichtigt.

Im Allgemeinen muss der Zustand vor `'startClosing'` restauriert werden, hier gibt es aber nichts zu tun.

Beim `'Grid'` hingegen ist das anders:

```
~~~java
@Override
public boolean startClosingSync(CloseEvent e) {
 boolean canClose = true;
 if (lifeState == LifeState.ACTIVE) {
 boolean canClosePane = dialogScrollPane == null ||
dialogScrollPane.startClosingSync(e);
 boolean canCloseSelf = canClosePane && prepareForClosing(e);
 if (canCloseSelf && CloseManager.fireStartClosing(listenerList, e))
 lifeState = LifeState.CLOSING_STARTED;
 else {
 canClose = false;
 if (canClosePane && dialogScrollPane != null)
 dialogScrollPane.abortClosing(e);
 }
 }
 return canClose;
}

@Override
public void commitClosing(CloseEvent e) {
 if (lifeState.isVital()) {
 lifeState = LifeState.COMMIT_CLOSING_STARTED;
 if (dialogScrollPane != null)
 dialogScrollPane.commitClosing(e);
 CloseManager.fireCommitClosing(listenerList, e);
 // ... gibt Ressourcen frei
 }
}

@Override
public void abortClosing(CloseEvent e) {
 if (lifeState == LifeState.CLOSING_STARTED) {
 if (dialogScrollPane != null)
 dialogScrollPane.abortClosing(e);
 CloseManager.fireAbortClosing(listenerList, e);
 lifeState = LifeState.ACTIVE;
 }
}
```

~~~

Hier wird über einen Enum `'LifeState'` festgehalten, ob das Grid gerade aktiv, beim Schließen, oder schon geschlossen ist.

Dieser Zustand ändert sich in `'startClosing'` auf `'CLOSING_STARTED'`, aber nur wenn `'true'` zurückgegeben wird.

In `'abortClosing'` wird er wieder auf `'ACTIVE'` zurückgestellt, und in `'commitClosing'` auf `'COMMIT_CLOSING_STARTED'`.

Hier sieht man, warum `'abortClosing'` und die Einhaltung des Protokolls wichtig ist.

### ### Freigabe von Ressourcen

In Swing haben wir die volle Kontrolle über die Dialoge und können gewährleisten, dass `'CloseListener'` immer durchlaufen werden, bevor Dialoge geschlossen werden.

Im Web können wir das nicht, da ein Benutzer z.B. einfach den Browser schließen kann. Der Server kann zwar darauf reagieren, aber den Schließvorgang nicht mehr unterbinden. In so einem Fall werden `'CloseListener'` gar nicht aufgerufen, sondern nur der `'onDetach'`-Hook von Vaadin.

Daher wird das Freigeben von Ressourcen in Swing in `'commitClosing'` erledigt, in Vaadin hingegen in `'onDetach'`.

### ### Asynchrones Modell

Eigentlich ist `'CloseListener'` bzw. `'startClosing'` mittlerweile asynchron.

Anstatt eines `'boolean'`-Wertes wird nämlich eine `'PendingCloseInitiation'` zurückgegeben,

ein Wrapper einer `'CompletionStage<Boolean>'`.

Weil aber nach wie vor viele Implementierungen synchron sind, gibt es ein abgeleitetes Interface `'Sync'`, welche `'startClosing'` per Default-Methode auf einen synchronen Hook mappts:

~~~java

```
public interface CloseListener extends EventListener {

 PendingCloseInitiation startClosing(CloseEvent e);

 interface Sync extends CloseListener {

 boolean startClosingSync(CloseEvent e);

 @Override
 default PendingCloseInitiation startClosing(CloseEvent e) {
 return PendingCloseInitiation.of(startClosingSync(e));
 }
 }
}
```

~~~

`'PendingCloseInitiation'` bietet nützliche Methoden, um mehrere Aufrufe zu verketteten und auf deren Ausgang zu reagieren. Die wichtigsten sind:

`onSuccess` reagiert auf einen Erfolg (`true`):

```
~~~java
listener.startClosing(event)
    .onSuccess() -> handleSuccess());
// Äquivalent zu
if (listener.startClosingSync(event))
    handleSuccess();
~~~
```

`onFailure` reagiert auf eine Abweisung (`false`):

```
~~~java
listener.startClosing(event)
    .onFailure() -> handleFailure());
// Äquivalent zu
if (!listener.startClosingSync(event))
    handleFailure();
~~~
```

`thenCompose` verkettet zwei asynchrone Aufrufe von `startClosing`:

```
~~~java
PendingCloseInitiation canClose = listener1.startClosing(event)
    .thenCompose() -> listener2.startClosing());
// Äquivalent zu
boolean canClose = listener1.startClosingSync(event)
    && listener2.startClosingSync(event);
~~~
```

### ### SilentMode

Es gibt Fälle, wo Komponenten programmtechnisch geschlossen werden und zwar versuchen sollen zu speichern, aber keine Benutzerabfragen anzeigen dürfen.

Wenn z.B. ein Update ansteht, werden alle Benutzer abgemeldet, damit das Programm heruntergefahren werden kann.

Dabei soll so gut wie möglich alles gespeichert werden, damit niemand Daten verliert. Aber es sollen keine Abfragen kommen, da viele Benutzer vielleicht gerade gar nicht arbeiten, sodass nie eine Entscheidung getroffen werden und der Updateprozess hängen bleiben würde.

Für solche Fälle gibt es den `SilentMode`, welcher angibt, ob Benutzerabfragen angezeigt werden dürfen, ob versucht werden soll zu speichern, und wie bei Validierungsfehler vorgegangen werden soll:

```
~~~java
public enum SilentMode {

    NONE,
    DISCARD,
    STORE_OR_DISCARD,
    STORE_OR_STOP;
}
```

```

    public boolean isSilent() {
        return this != NONE;
    }

    public boolean isStoreChangesSilently() {
        return this == STORE_OR_DISCARD || this == STORE_OR_STOP;
    }

    public boolean isDiscardChangesSilently() {
        return this == DISCARD || this == STORE_OR_DISCARD;
    }
}
~~~

```

Es wird bei Bedarf ein entsprechender `'SilentMode'` in die Session eingesetzt und ein `'CloseEvent'` mit `'CloseMode.ABORT'` geworfen.

`'CloseListener'`, die auf den `'SilentMode'` nicht reagieren, führen einen gewöhnlichen Abbruch durch.

Die anderen ermitteln den `'SilentMode'` aus der Session und müssen beides, `'CloseMode'` und `'SilentMode'` betrachten, um zu verstehen, wie vorgegangen werden soll.

Dazu gibt es einige Hilfsmethoden, z.B. `'CloseMode.canSaveChanges(SilentMode)'`, welche angibt, ob gespeichert werden soll, und dafür beide in Betracht zieht.

```

~~~java
//...
SilentMode silentMode = session.getSilentMode();
boolean saveChanges = closeMode.canSaveChanges(silentMode);
boolean discardChangesSilently = silentMode.isDiscardChangesSilently();
//...
~~~

```

// NB: tabsize und source-highlighter:coderay werden von AsciiDocFX nicht unterstützt. Sie sollten daher beim Generieren der Dokumente (PDF, HTML) als Kommandozeilen-Parameter angegeben werden.

```
//:tabsize: 4
```

```
//:source-highlighter: coderay
```

## ## Nebenläufige Aufgaben

### ### Einleitung

Neben der direkten Datenbearbeitung bietet das Programm die Möglichkeit, komplexe Operationen wie z.B. Importe oder Exporte durchzuführen oder Berichte zu generieren.

Viele solcher Operationen können potentiell lange andauern und sollten daher nebenläufig ausgeführt werden, sodass der Benutzer auch weiterhin mit dem Programm interagieren und eventuell die Operation abbrechen kann.

Umgekehrt soll auch die Operation ihren aktuellen Status dem Benutzer rückmelden.

Das Kommando "Gültigkeit überprüfen" z.B. zeigt dem Benutzer während der Ausführung den aktuellen Status, einen Counter der bisher verarbeiteten Datensätze und einen User-Log mit den gescheiterten Validierungen an. Über einen Button kann das Kommando auch abgebrochen werden.

{align=center}

Das Package `it.prodata.util.concurrent` bildet ein kleines Framework, welches das Implementieren und Ausführen solcher Operationen erleichtert und vereinheitlicht.

Primär bietet es folgende Features:

- Einheitliche Kommunikation zwischen Ausführungslogik und Aufrufer.
- Nebenläufige Aufgaben können sowohl in einem Dialog überwacht als auch unabhängig von der Oberfläche ausgeführt werden.
- Zur ressourcenschonenden Ausführung können Thread-Pools verwendet werden.

### ### Überblick

```
~~~uml{align=center}
interface ExecutableTask {
    +{abstract} getSettings()
    +{abstract} execute(ExecutionContext)
    +{abstract} executeAsync(ExecutionContext, Executor)
    +onCancel(ExecutionContext, boolean)
    +{abstract} releaseResources()
}
class Result {
    +getStatus()
    +getData()
    +getFailure()
}
class ExecutionSettings {
    -title
    -managingStatusText
    -managingCounter
    -managingProgress
    -managingLog
    -abortable
    ... weitere Einstellungen
}
class ExecutionContext {
    +getSettings()
    +addCompleteListener(CompleteListener)
    +taskStarted(ExecutableTask)
    +taskCompleted(ExecutableTask, Result)
    +cancel(boolean)
    +isCancelled()
}
interface StatusMonitor {
```

```

    +{abstract} getLogWriter()
    +{abstract} getUserLogWriter()
    +{abstract} setStatusText(String)
    +{abstract} setProgress(int)
    +{abstract} setCounter(int)
    +{abstract} setInfo(Object)
}
interface ExecutionController {
    +setAutoStarting(boolean)
    +setAutoClosing(boolean)
    +setLogViewSize(int, int)
    ... (weitere Einstellungen)
    +start(Object, boolean, ExecutableTask<?>)
}

ExecutableTask    -d-> ExecutionContext : wird ausgeführt in
ExecutableTask    -r-> Result : liefert
ExecutableTask    -d-> ExecutionSettings : liefert bevorzugte
ExecutionContext -d-> ExecutionSettings : liefert effektive
ExecutionContext -d-> StatusMonitor : hält
ExecutionController -d-> ExecutionSettings : berücksichtigt
ExecutionController -l-> ExecutionContext : erstellt
ExecutionController -d-> StatusMonitor : setzt
ExecutionController -u-> ExecutableTask : führt aus
~~~

```

Eine nebenläufige Aufgabe wird als `ExecutableTask` definiert. Die Methode `ExecutableTask.execute(ExecutionContext)` führt die Aufgabe in einem angegebenen Ausführungskontext aus. Dieser `ExecutionContext` bildet ein Bindeglied zwischen Aufrufer und Task, indem er folgende Funktionen übernimmt:

- Er verwaltet `ExecutionSettings` die angeben, welche Formen der Interaktion zwischen Task und Aufrufer unterstützt werden.
- Für Rückmeldungen über den aktuellen Ausführungsstatus hält er einen `StatusMonitor`.
- Er verwaltet die laufenden Tasks, welche sich über die Methoden `taskStarted()` bzw. `taskCompleted()` bei ihm melden müssen, um sie beispielsweise über einen Abbruchwunsch informieren zu können.

Damit entkoppelt der `ExecutionContext` den Task von der Oberfläche: Der Task kann über den Kontext mit der Oberfläche kommunizieren, ist aber andererseits auch unabhängig von einer Oberfläche ausführbar.

Um einen `ExecutableTask` nebenläufig, also in einem anderen Thread auszuführen, wird `ExecutableTask.executeAsync(ExecutionContext, Executor)` mit einem entsprechenden `Executor` aufgerufen. Diese Methode liefert eine `CompletionStage` eines `Result`-Objekts zurück. Ein `Result` repräsentiert das Ergebnis eines Tasks anhand eines `int` Status sowie optionalen Ergebnis-Daten und einen eventuell aufgetretenen Ausnahmefehler.

Zum Öffnen eines Dialogs, der die Ausführung eines Tasks kontrolliert, wird ein `ExecutionController` verwendet.

Anhand der bevorzugten `ExecutionSettings` des Tasks werden im Dialog nur Steuerelemente angeboten, die der Task unterstützt.

Die Einstellungen werden auch in einen neu erzeugten `ExecutionContext` übernommen.

Dieser ist über einen `StatusMonitor`, der bei Rückmeldungen die entsprechenden Komponenten aktualisiert, mit der Oberfläche verdrahtet.

Außerdem registriert der `ExecutionController` einen `CompleteListener`, um über den Ausgang der Operation informiert zu werden.

### ### AbstractExecutionController

Manche Tasks müssen Parameter vom Benutzer abfragen oder in einer speziellen Form Rückmeldung geben, sodass ein spezieller Dialog erforderlich ist.

Ein Beispiel hierfür ist das Kommando "Artikelsuche".

Über ein Suchfeld kann der Benutzer einen Suchtext angeben.

Der Task sucht nach passenden Artikeln und zeigt das Ergebnis in einem Grid an.

{align=center}

Mit der Klasse `AbstractExecutionController` steht eine Basisklasse für solche spezielle Dialoge zur Verfügung.

Ableitungen müssen die Methode `createTask()` implementieren, welche den Task erstellt, der bei `start()` ausgeführt werden soll.

Zum Ausführen von alternativen Tasks kann `startTask(ExecutableTask)` verwendet werden.

Nach Beendigung des Tasks wird `onCompleteLater()` im Dispatch-Thread aufgerufen, um abschließende Arbeiten durchzuführen bzw. die Oberfläche zu aktualisieren.

```
~~~uml{align=center}
abstract class AbstractExecutionController<D> {
    +start(): boolean
    #{abstract} createTask(): ExecutableTask<?, D>
    #onCompleteLater(ExecutionContext,\n Result<D>): void
}
class ArtikelSucheAdmin {
    #createTask()\n : ExecutableTask<?, List<BsObject>>
    #onCompleteLater(ExecutionContext,\n Result<List<BsObject>>): void
}
-class ArtikelSucheTask {
    #executeAsync(ExecutionContext, Executor)\n :
CompletionStage<Result<List<BsObject>>>
}
interface ExecutableTask<R, D> {
    +{abstract} executeAsync(ExecutionContext, Executor)\n :
CompletionStage<Result<D>>
}
```



```

class Result<D> {
    +getStatus(): int
    +isSuccess(): boolean
    +getData(): D
    +getFailure(): Throwable
}

```

```

ArtikelSucheAdmin -u-|> AbstractExecutionController : <<bind>\nD -> <List<BsObject>>

```

```

ArtikelSucheTask -u-|> ExecutableTask : <<bind>\nD -> <List<BsObject>>

```

```

ArtikelSucheAdmin -r-> ArtikelSucheTask : erstellt/startet

```

```

ArtikelSucheTask --> Result : liefert

```

```

ArtikelSucheAdmin --> Result : aktualisiert Grid anhand von

```

```

~~~

```

Die ermittelten Artikel werden als Ergebnisdaten in ein 'Result'-Objekt gepackt und zurückgegeben.

Solche 'Result'-Daten sind allgemein als ein generischer Typ 'D' deklariert, sodass wir in unserem Fall 'List<BsObject>' dafür festlegen können.

Auch 'ArtikelSucheAdmin' ist so deklariert, dass er nur Tasks erzeugen kann, die Ergebnisdaten vom Typ 'List<BsObject>' liefern können.

Daher erhält die Methode 'onCompleteLater()' bereits ein entsprechend typisiertes 'Result'-Objekt und kann ohne Casts sowohl auf Status als auch auf Daten zugreifen.

### ### Synchrone und asynchrone Tasks

#### #### Hintergrund

Der Aufruf eines **\*\*synchrone\*\*** Tasks kehrt erst zurück, wenn die gesamte Operation beendet wurde, und liefert dann ein entsprechendes 'Result'-Objekt als Ergebnis.

Falls der Task von einem externen Ereignis wie z.B. einer HTTP-Antwort abhängt, wartet er in seinem Thread auf dieses, damit er es in seinem Ergebnis berücksichtigen kann.

Der Vorteil des synchronen Modells ist die Einfachheit: Für den Aufrufer genügt ein einfacher Methodenaufruf, um den Task durchzuführen und das Ergebnis zu ermitteln.

Das folgende UML-Sequenzdiagramm zeigt die Ausführung von zwei synchronen Tasks, die beide einen REST-Service kontaktieren:

```

~~~uml{width=70% align=center}
hide footbox
actor Caller
box "Executor"
    participant "Thread1" as T1
    participant "Thread2" as T2

```

```

end box
participant REST

Caller -> T1 : execute(syncTask1)
activate T1

Caller -> T2 : execute(syncTask2)
activate T2

T1 -> REST : GET

T2 -> REST : GET

T1 <-- REST : 200 OK

T2 <-- REST : 200 OK

Caller <-- T1 : Result.OK
deactivate T1

Caller <-- T2 : Result.OK
deactivate T2
~~~

```

Dieses Beispiel veranschaulicht aber auch den Nachteil des synchronen Modells: Ein Thread, der auf ein Ereignis wartet, kann während dieser Zeit keine anderen Berechnungen durchführen.

Somit besetzt jeder synchrone Task einen Thread, auch wenn dieser nur wartet. Falls sehr viele solche Tasks gleichzeitig gestartet werden, kann dadurch der Speicher knapp werden.

**\*\*Asynchrone\*\*** Tasks vermeiden dieses Problem, indem sie asynchrone Ereignisse auch asynchron verarbeiten.

Sie warten nicht in ihrem Thread auf das Ereignis, sondern geben lediglich eine Rückruffunktion an, die das Ereignis behandelt sobald es eintrifft. Somit ist der Thread sofort frei für andere Aufgaben.

Um den Vorteil des asynchronen Modells zu verdeutlichen, zeigt das folgende Diagramm, wie eine Ausführung des vorherigen Beispiels mit asynchronen Tasks aussehen könnte.

Im Unterschied zur synchronen Variante reicht hier ein einziger Ausführungsthread, um beide Aufgaben durchzuführen.

```

~~~uml{width=55% align=center}
hide footbox
actor Caller
box "Executor"
    participant "Thread1" as T1
end box
participant REST

```

```
Caller -> T1 : execute(asyncTask1)
activate T1
```

```
T1 -> REST : GET
```

```
Caller <-- T1 : return
deactivate T1
```

```
Caller -> T1 : execute(asyncTask2)
activate T1
```

```
T1 -> REST : GET
```

```
Caller <-- T1 : return
deactivate T1
```

```
T1 <-- REST : 200 OK
activate T1
```

```
Caller <-- T1 : Result.OK
deactivate T1
```

```
T1 <-- REST : 200 OK
activate T1
```

```
Caller <-- T1 : Result.ok()
deactivate T1
```

```
~~~
```

Natürlich kommt es hier auf das genaue Timing an, ob der Thread für den zweiten Task bereits wiederverwendet werden kann oder nicht.

In jedem Fall ist der Thread früher für weitere Aufgaben frei, was bei hoher Last einen wesentlichen Unterschied machen kann.

Asynchrone Task sind jedoch nicht immer effizienter als synchrone Tasks, sondern nur falls sie auch von asynchronen Ereignissen abhängen.

Andernfalls muss der Ausführungsthread des Tasks sowieso nie warten und wird daher auch bei synchronen Tasks optimal genutzt.

Der Einfachheit halber sollte in so einem Fall eine synchrone Variante bevorzugt werden.

#### #### Umsetzung

Um sowohl ein synchrones als auch ein asynchrones Modell für Tasks zu unterstützen, liefert die Methode `'ExecutableTask.execute()'` nur einen generischer Typ `'<R>'`.

Für jedes der beiden Modelle existiert ein abgeleitetes Interface, welches den konkreten Rückgabetyt festlegt:

- Ein `'SyncExecutableTask'` ist ein synchroner `'ExecutableTask'`, dessen `'execute()'` Methode direkt ein `'Result'`-Objekt liefert.

- Ein `'AsyncExecutableTask'` ist ein asynchroner `'ExecutableTask'`, dessen `'execute()'` Methode eine `'CompletionStage'` liefert, die bei Beendigung mit dem entsprechenden `'Result'`-Objekt abgeschlossen wird.

Jeder `'ExecutableTask'` kann mithilfe der Methoden `'toSync()'` bzw. `'toAsync()'` in einen `'SyncExecutableTask'` bzw. `'AsyncExecutableTask'` konvertiert werden. Bei einer Konversion von synchron nach asynchron wird das Ergebnis als `'CompletionStage'` verpackt. Im umgekehrten Fall wird auf das Ergebnis der `'CompletionStage'` gewartet, um es anschließend direkt zurückzugeben.

Eine `'CompletionStage'` repräsentiert eine Stufe einer potentiell asynchronen Berechnung und kann mit Funktionen und anderen `'CompletionStage'`s zu einer asynchronen Pipeline verkettet werden.

Dies ermöglicht ein asynchrones Programmiermodell, welches dem synchronen sehr nahe kommt. Zum Vergleich stellt der folgende Codeausschnitt die beiden Modelle anhand einer sequentiellen Ausführung von Tasks gegenüber.

In beiden Fällen startet der zweite Task erst, wenn der erste beendet wurde und nur falls er Erfolg hatte.

```

~~~java
/** Synchrone Variante. */
<D> Result<D> executeSequentially(ExecutionContext execCtx,
    SyncExecutableTask<D> task1, SyncExecutableTask<D> task2)
{
    Result<D> result = task1.execute(execCtx);
    return result.isError()
        ? result
        : task2.execute(execCtx);
}

/** Asynchrone Variante. */
<D> CompletionStage<Result<D>> executeSequentially(ExecutionContext execCtx,
    AsyncExecutableTask<D> task1, AsyncExecutableTask<D> task2)
{
    return task1.execute(execCtx)
        .thenCompose( ①
            result -> result.isError()
                ? CompletableFuture.completedFuture(result) ②
                : task2.execute(execCtx)
        );
}
~~~

```

<1> Verkettet eine `'CompletionStage'` mit einer Funktion zu einer neuen `'CompletionStage'`.

<2> Verpackt das Ergebnis als `'CompletionStage'`.

Der wesentliche Unterschied liegt darin, dass die asynchrone Variante das Ergebnis des ersten Tasks nicht direkt behandelt, sondern in einer Funktion, die an eine Verkettungs-Methode von `'CompletionStage'` übergeben wird.

Hier verwenden wir die Methode `'thenCompose()'`, welche eine neue `'CompletionStage'` liefert, die einer Verkettung der vorherigen mit einer

weiteren asynchronen Operation entspricht.

Diese asynchrone Operation wird als Funktion angegeben, die das Ergebnis der vorherigen `CompletionStage` erhält und ein neues, asynchrones Ergebnis als `CompletionStage` liefert.

Ähnlich wie bei `Stream.flatMap()` wird das Ergebnis der Funktion in das Ergebnis der Verkettung "verflacht".

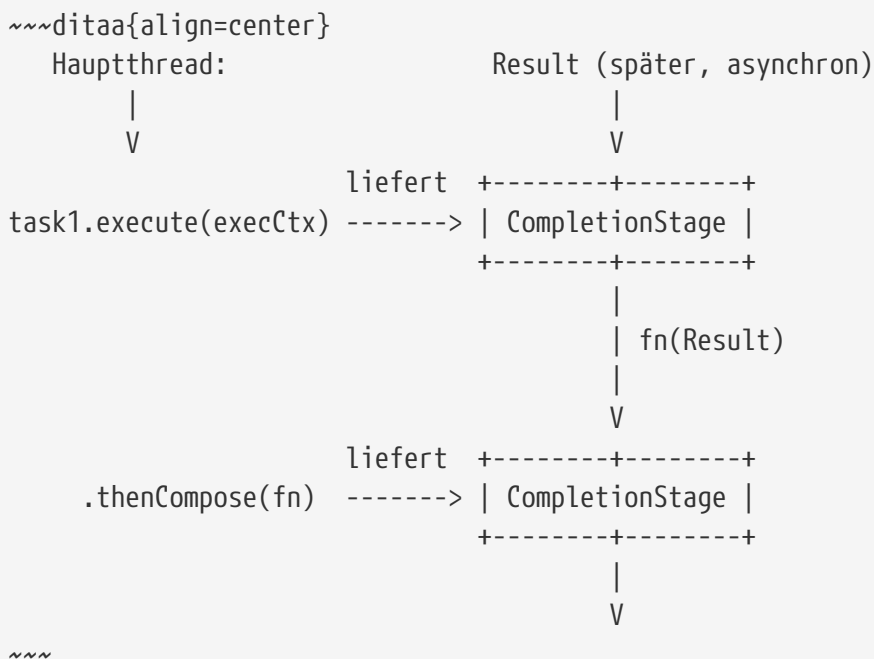
In unserem Beispiel repräsentiert die vorherige `CompletionStage` das Ergebnis des ersten Tasks und verkettet es mit einer Funktion, die das asynchrone Ergebnis des zweiten Tasks liefert, außer der erste Task wurde fehlerhaft beendet. In diesem Fall wird das fehlerhafte Ergebnis mithilfe von `CompletableFuture.completedFuture()` verpackt und zurückgegeben.

Das folgende Diagramm veranschaulicht die Ausführung des asynchronen Beispiels. Der erste Task wird im Haupt-Thread gestartet und liefert eine `CompletionStage` für sein Ergebnis.

Im selben Thread hängt `thenCompose()` eine Funktion an die erste `CompletionStage` an und liefert eine neue `CompletionStage` zurück, welche dann das zweite Ergebnis repräsentiert und als Ergebnis der Methode `executeSequentially()` zurückgegeben wird.

Das konkrete Ergebnis, sprich das `Result`-Objekt, steht aber noch nicht fest.

Erst später, wenn der erste Task beendet wurde, wird die entsprechende `CompletionStage` damit abgeschlossen und an die nächste Operation weitergereicht.



Zusätzlich zu den Interfaces existieren abstrakte Basisklassen für beide Varianten, welche bereits den Kontext über Start bzw. Beendigung des Tasks benachrichtigen, sodass sich Ableitungen auf die eigentliche Aufgabe konzentrieren können.

Über die Methode `taskResumed()` können asynchrone Tasks den Kontext auch darüber informieren, wenn sie asynchrone Arbeiten wiederaufnehmen.

Außerdem bieten die Basisklassen die Möglichkeit, den Ausführungskontext zu

konfigurieren und einen speziellen 'MessageHandler' einzusetzen, falls es sich um den Haupt-Task des Kontexts handelt.

```
~~~uml{align=center}
interface ExecutableTask<R, D> {
    +{abstract} getSettings(): ExecutionSettings
    +{abstract} execute(ExecutionContext): R
    +onCancel(ExecutionContext): void
    +{abstract} releaseResources(): void
    +{abstract} toSync(): SyncExecutableTask
    +{abstract} toAsync(): AsyncExecutableTask
}
abstract class AbstractExecutableTask<R, D> extends ExecutableTask {
    +getSettings(): ExecutionSettings
    #registerMessageHandler(\n ExecutionContext): void
    #configureExecutionContext(\n ExecutionContext): void
    #getResultData(int, Throwable)\n : Result<D>
    #toResult(int, Throwable)\n : Result<D>
    +releaseResources(): void
}
interface SyncExecutableTask<D>
interface AsyncExecutableTask<D>

abstract class DefaultSyncExecutableTask<D> extends AbstractExecutableTask implements
SyncExecutableTask {
    +execute(ExecutionContext)\n : Result
    #{abstract} executeImpl(ExecutionContext)\n : Result
}
abstract class DefaultAsyncExecutableTask<D> extends AbstractExecutableTask implements
AsyncExecutableTask {
    +execute(ExecutionContext)\n : CompletionStage<Result>
    #{abstract} executeImpl(ExecutionContext)\n : CompletionStage<Result>
}

class ExecutionContext {
    +taskStarted(ExecutableTask<?, ?>): void
    +taskResumed(ExecutableTask<?, ?>): void
    +taskCompleted(ExecutableTask<?, ?>, Result<?>): void
    +canBeConfiguredBy(ExecutableTask<?>): void
}

SyncExecutableTask -u-> ExecutableTask : <<bind>>\nR -> Result
AsyncExecutableTask -u-> ExecutableTask : <<bind>>\nR -> CompletionStage<Result>

DefaultSyncExecutableTask --> ExecutionContext : benachrichtigt bei\n Start &
Beendigung
DefaultAsyncExecutableTask --> ExecutionContext : benachrichtigt bei\n Start &
Beendigung
~~~
```

### ### Zusätzliche Hilfsklassen

#### #### ExecutableTaskAdapter

Ein `ExecutableTaskAdapter` ist ein `ExecutableTask`, der Methodenaufrufe an einen gekapselten Task delegiert.

Dies erlaubt es, das Verhalten eines bestehenden Tasks nach Wünschen anzupassen, indem er in eine Ableitung von `ExecutableTaskAdapter` verpackt wird, wobei einzelne Methoden entsprechend überschrieben werden.

Für maximale Flexibilität ist der Typ des gekapselten Tasks generisch und unabhängig vom Ergebnistyp des Adapters. Dies erlaubt es, einen Task eines bestimmten Interfaces an ein anderes Ziel-Interface "anzupassen".

So nutzen auch die Konversions-Methoden zwischen synchronen und asynchronen Tasks jeweils einen speziellen Adapter:

```
~~~uml{align=center}
set namespaceSeparator none
interface ExecutableTask<R, D> {
    +{abstract} getSettings(): ExecutionSettings
    +{abstract} execute(ExecutionContext): R
    +onCancel(ExecutionContext): void
    +{abstract} releaseResources(): void
    +{abstract} toSync(): SyncExecutableTask<D>
    +{abstract} toAsync(): AsyncExecutableTask<D>
}
abstract class ExecutableTaskAdapter<T extends ExecutableTask<?, ?>, R, D> {
    #task: T
    +getSettings(): ExecutionSettings
    +onCancel(ExecutionContext): void
    +releaseResources(): void
}
interface SyncExecutableTask<D> {
    +toSync()\n : SyncExecutableTask<D>
    +toAsync()\n : AsyncExecutableTask<D>
}
interface AsyncExecutableTask<D> {
    +toSync()\n : SyncExecutableTask<D>
    +toAsync()\n : AsyncExecutableTask<D>
}
class SyncExecutableTask.AsyncAdapter<D> {
    +execute(ExecutionContext)\n : CompletionStage<Result<D>>
}
class AsyncExecutableTask.SyncAdapter<D> {
    +execute(ExecutionContext)\n : Result<D>
}
ExecutableTaskAdapter .u.|> ExecutableTask
SyncExecutableTask -u-> SyncExecutableTask.AsyncAdapter : toAsync()
SyncExecutableTask.AsyncAdapter .d.|> AsyncExecutableTask
SyncExecutableTask.AsyncAdapter -u-|> ExecutableTaskAdapter: <<bind>>\nT ->
SyncExecutableTask<D>,\nR -> CompletionStage<Result<D>>
```

```

AsyncExecutableTask -u-> AsyncExecutableTask.SyncAdapter : toSync()
AsyncExecutableTask.SyncAdapter .d.|> SyncExecutableTask
AsyncExecutableTask.SyncAdapter -u-> ExecutableTaskAdapter: <<bind>>\nT ->
AsyncExecutableTask<D>,\nR -> Result<D>
~~~

```

#### #### ExecutableTaskWrapper

Ein `ExecutableTaskWrapper` ist ein `ExecutableTaskAdapter` mit identischem Quell- und Ziel-Interface.

Im Unterschied zum Adapter kann ein Wrapper dadurch auch die `execute()` Methode an den gekapselten Task delegieren.

Dies kann sinnvoll sein, um vor oder oder nach Ausführung des gekapselten Tasks eine bestimmte Methode aufzurufen.

Für Wrapper von synchronen und asynchronen Tasks wird jeweils eine konkrete Inner-Class bereitgestellt:

```

~~~uml{align=center}
set namespaceSeparator none
interface SyncExecutableTask<D> {
    +toSync()\n : SyncExecutableTask<D>
    +toAsync()\n : AsyncExecutableTask<D>
}
interface AsyncExecutableTask<D> {
    +toSync()\n : SyncExecutableTask<D>
    +toAsync()\n : AsyncExecutableTask<D>
}
abstract class ExecutableTaskAdapter<T extends ExecutableTask<?, ?>, R, D> {
    #task: T
    +getSettings(): ExecutionSettings
    +onCancel(ExecutionContext): void
    +releaseResources(): void
}
abstract class ExecutableTaskWrapper<R, D> {
    +execute(ExecutionContext): R
}
class ExecutableTaskWrapper.Sync<D> {
}
class ExecutableTaskWrapper.Async<D> {
}

```

```

ExecutableTaskWrapper -u-> ExecutableTaskAdapter: <<bind>>\nT -> ExecutableTask<?, ?>
ExecutableTaskWrapper.Sync -u-> ExecutableTaskWrapper: <<bind>>\nR -> Result<D>
ExecutableTaskWrapper.Async -u-> ExecutableTaskWrapper: <<bind>>\nR ->
CompletionStage<Result<D>>
ExecutableTaskWrapper.Sync .u.|> SyncExecutableTask
ExecutableTaskWrapper.Async .u.|> AsyncExecutableTask
~~~

```

#### #### MultiSyncExecutableTask



Während ein `ExecutableTaskAdapter` und dessen Ableitungen einen einzelnen Task wrappen, gibt es auch Klassen zum Wrappen einer Liste mehrerer Tasks. Diese führen alle Tasks in der Liste aus und verknüpfen die Teilergebnisse aufeinanderfolgender Tasks mittels `ExecutableTask.getFinalStatus`, um das Endergebnis zu ermitteln.

Ein `MultiSyncExecutableTask` ist ein `SyncExecutableTask`, der eine Liste von anderen `SyncExecutableTask`'s nacheinander ausführt.

Das folgende UML-Sequenzdiagramm zeigt, wie ein `MultiSyncExecutableTask` eine Liste von drei Tasks ausführt, die jeweils einen REST-Service kontaktieren.

```
~~~uml{width=70% align=center}
```

```
hide footbox
```

```
actor Caller
```

```
box "MultiSyncExecutableTask"
```

```
    participant "Task1" as T1
```

```
    participant "Task2" as T2
```

```
    participant "Task3" as T3
```

```
end box
```

```
participant REST
```

```
Caller -> T1 : execute()
```

```
activate T1
```

```
T1 -> REST : GET
```

```
...500 ms...
```

```
T1 <-- REST : 200 OK
```

```
T1 --> T2 : Result.ok()
```

```
T1 -> T2 : execute()
```

```
deactivate T1
```

```
activate T2
```

```
T2 -> REST : GET
```

```
...500 ms...
```

```
T2 <-- REST : 200 OK
```

```
T2 --> T3 : Result.ok()
```

```
T2 -> T3 : execute()
```

```
deactivate T2
```

```
activate T3
```

```
T3 -> REST : GET
```

```
...500 ms...
```

```
T3 <-- REST : 200 OK
```

```
Caller <-- T3 : Result.ok()
```

```
deactivate T3
```

```
~~~
```

Jeder Task wartet in seinem Thread auf das Ergebnis der REST-Abfrage, bevor der nächste Task startet.  
Bei einem Round-Trip von 500 Millisekunden wie im Diagramm angedeutet, dauert die Ausführung aller Tasks mindestens 1,5 Sekunden und besetzt während dieser Zeit den Thread.

#### #### MultiAsyncExecutableTask

Ein `MultiAsyncExecutableTask` ist ein `AsyncExecutableTask`, der eine Liste von anderen `AsyncExecutableTask`'s ausführt.

Im Unterschied zu `MultiSyncExecutableTask` gibt es hier zwei Ausführungsvarianten: **verkettet** und **nebenläufig**.

Bei der verketteten Variante wird ein Task erst gestartet, nachdem der vorherige beendet wurde.

Das Beispiel mit den drei Tasks und dem REST-Service würde folgendermaßen aussehen:

```
~~~uml{width=70% align=center}
```

```
hide footbox
```

```
actor Caller
```

```
box "MultiAsyncExecutableTask"
```

```
    participant "Task1" as T1
```

```
    participant "Task2" as T2
```

```
    participant "Task3" as T3
```

```
end box
```

```
participant REST
```

```
Caller -> T1 : execute()
```

```
activate T1
```

```
T1 -> REST : GET
```

```
Caller <-- T1 : return
```

```
deactivate T1
```

```
...500 ms...
```

```
T1 <-- REST : 200 OK
```

```
activate T1
```

```
T1 --> T2 : Result.ok()
```

```
T1 -> T2 : execute()
```

```
deactivate T1
```

```
activate T2
```

```
T2 -> REST : GET
```

```
deactivate T2
```

```
...500 ms...
```

```
T2 <-- REST : 200 OK
```

```
activate T2
```

```
T2 --> T3 : Result.ok()
```

```
T2 -> T3 : execute()
```

```

deactivate T2
activate T3

T3 -> REST : GET
deactivate T3
...500 ms...
T3 <-- REST : 200 OK
activate T3

Caller <-- T3 : Result.ok()
deactivate T3
~~~

```

Die verkettete Ausführung ähnelt der synchronen: Auch hier werden die Tasks strikt nacheinander ausgeführt, wodurch wieder eine Gesamtdauer von mindestens 1,5 Sekunden zustandekommt.

Der Unterschied liegt jedoch darin, dass die Tasks nach dem Senden der Abfrage vorzeitig zurückkehren und damit zwischenzeitlich den Thread für andere Aufgaben freigeben können.

Die nebenläufige Ausführung macht sich genau diese Eigenschaft asynchroner Tasks zunutze und startet den nächsten Task sofort nachdem der vorherige zurückkehrt, und nicht erst bei dessen Beendigung.

In unserem Beispiel bedeutet das, dass alle drei Tasks ihre Anfragen an den REST-Service senden, ohne auf die Antwort einer vorherigen Anfrage zu warten:

```

~~~uml{width=70% align=center}
hide footbox
actor Caller
box "MultiAsyncExecutableTask"
    participant "Task1" as T1
    participant "Task2" as T2
    participant "Task3" as T3
end box
participant REST

Caller -> T1 : execute()
activate T1

T1 -> REST : GET

T1 -> T2 : execute()
deactivate T1
activate T2

T2 -> REST : GET

T2 -> T3 : execute()
deactivate T2
activate T3

```

```

T3 -> REST : GET

Caller <-- T3 : return
deactivate T3

...500 ms...

T1 <-- REST : 200 OK
activate T1

T1 --> T2 : Result.ok()
deactivate T1

T2 <-- REST : 200 OK
activate T2

T2 --> T3 : Result.ok()
deactivate T2

T3 <-- REST : 200 OK
activate T3

Caller <-- T3 : Result.ok()
deactivate T3
~~~

```

Durch das gleichzeitige Senden der drei Abfragen muss der Round-Trip zum Server nur ein anstatt drei Mal abgewartet werden, wodurch der Task bereits nach einer halben Sekunde beendet werden kann.

Interessanterweise werden hierfür aber nicht unbedingt mehrere Threads benötigt.

Die Anfragen werden sogar garantiert alle im selben Thread gesendet, während es bei den Antworten darauf ankommt, ob diese gleichzeitig oder versetzt eintreffen.

Generell führt eine nebenläufige Ausführung mehrerer Tasks zu einer Performancesteigerung gegenüber der synchronen oder verketteten Variante, ohne dafür merklich mehr Ressourcen zu beanspruchen.

Ein Nachteil ist in diesem Fall die erhöhte Last auf den REST-Service. Außerdem wird der Thread bei verketteter Ausführung bereits abgegeben, sobald der erste Task gestartet ist, und nicht erst nach dem Start des letzten.

Die synchrone Variante ist vor allem dann sinnvoll, wenn die Tasks ihre Arbeit komplett selbst erledigen und nicht von externen Ereignissen abhängen.

```

// NB: tabsize und source-highlighter:coderay werden von AsciidocFX nicht unterstützt.
// Sie sollten daher beim Generieren der Dokumente (PDF, HTML) als Kommandozeilen-
// Parameter angegeben werden.
//:tabsize: 4

```

```
//:source-highlighter: coderay
```

```
Blockieren in Vaadin
```

```
Einleitung
```

In unserer Codebasis gibt es unzählige Stellen, an denen der Benutzer über einen Dialog zu einer Entscheidung gezwungen wird. Dabei wird ein synchrones Programmiermodell verwendet: Die Methode, die den Dialog öffnet, kehrt erst zurück, wenn der Dialog durch treffen einer Entscheidung geschlossen wurde. Erst dann wird der nachfolgende Code weiter ausgeführt.

In Vaadin sind einige Tricks nötig, um dieses Programmiermodell zu unterstützen. Um sie zu verstehen, müssen wir uns zunächst das Zugriffsmodell in Vaadin anschauen. Grundsätzlich hält der Server eine Kopie des DOM-Baums im Browser, und Änderungen daran werden hin- und her-synchronisiert.

Das folgende Diagram zeigt den groben Ablauf für serverseitige Logik:

```
~~~uml{align=center}
hide footbox
actor Browser as B
box "Server"
    participant "Vaadin" as V
end box

B -> V : Request
activate V

V -> V : lock
activate V
V -> V : Change UI
V -> V : unlock
deactivate V

B <-- V : Response
deactivate V
~~~
```

Eine Benutzerinteraktion im Browser triggert einen Request an den Server. Bei einem Klick auf einen Button wird z.B. der entsprechende Listener auf dem Server aufgerufen, welcher dann Änderungen an der serverseitigen Kopie der UI vornehmen kann. Am Ende sammelt Vaadin diese Änderungen und schickt sie an den Browser.

Änderungen an der UI dürfen nur mit einem Lock auf die VaadinSession vorgenommen werden, damit keine Konflikte

entstehen. Vaadin-Listener werden automatisch mit diesem Lock ausgeführt.

### ### Nicht blockierende Dialoge

Von Vaadin aus gesehen sollten Benutzerabfragen normalerweise nicht blockierend umgesetzt werden:

```
~~~uml{align=center}
hide footbox
actor Browser as B
box "Server"
    participant "Vaadin\nwith lock" as V
end box
```

```
B -> V : Request
activate V
```

```
V -> V : Show Dialog
B <-- V : Response
deactivate V
```

```
B -> V : Decision
activate V
V -> V : Close Dialog
B <-- V : Response
deactivate V
```

~~~

Das funktioniert sowohl direkt mit Vaadin-Listenern als auch mit 'CompletionStage'.

~~~java

```
var bDelete = new Button("Delete",
    e -> yesNoAsync("Wirklich Löschen") // nicht blockierend, mit CompletionStage-
Antwort
    .thenAccept(ok -> {
        if (ok)
            deleteInDb(); // Behandlung der Entscheidung, wird erst bei
nächstem Request aufgerufen
    }) // der Listener selbst kehrt sofort zurück
);
~~~
```

Entscheidend ist, dass nicht blockierend auf die Antwort gewartet wird. Stattdessen wird beim ersten Request nur der Dialog angezeigt und eine entsprechende Antwort gesendet, der Request ist damit abgeschlossen. Die Entscheidung des Benutzers einfach ein weiterer Request, der wie jeder andere behandelt wird.

### ### Blockieren mit Lock

Was aber passiert nun, wenn wir nach Anzeige des Dialogs blockierend auf die Antwort warten?

~~~java

```
var bDelete = new Button("Delete", e -> {
 if (yesNo("Wirklich Löschen")) // blockierend, mit boolean-Antwort
 deleteInDb(); // Behandlung der Entscheidung
});
~~~
```

Es entsteht ein Deadlock!

Das folgende Diagramm zeigt warum:

```
~~~uml{align=center}
hide footbox
actor Browser as B
box "Server"
 participant "Vaadin\nwith lock" as V
end box
```

```
B -> V : Request
activate V
```

```
V -> V : Show Dialog
note right of V
 Dialog würde erst durch Response
 an den Browser geschickt werden
end note
```

```
V -> V : Block until answer
activate V
```

... deadlock ...

```
V -> V : Close Dialog
B x-- V : Response
deactivate V
~~~
```

Es gibt insgesamt zwei Probleme. Zum einen wird der Dialog gar nicht im Browser angezeigt, weil Vaadin damit wartet, bis die Behandlung des Requests abgeschlossen ist. Dazu kommt es aber nie, da wir ja blockierend auf eine Antwort des Benutzers warten, was wiederum ewig dauert, da der Benutzer den Dialog nie zu Gesicht bekommt und somit auch nicht antworten kann.

Selbst wenn der Dialog im Browser richtig angezeigt werden würde, käme das Problem

beim Verarbeiten der Antwort zustande. Diese benötigt nämlich immer den Lock, bekommt ihn aber nicht, da wir bereits mit dem Lock auf die Antwort warten.

### ### Blockieren im Hintergrund-Thread

Um an gewissen Stellen trotzdem blockieren zu können, verlagern wir die Abarbeitung der Logik an einen anderen Thread.

Hierfür verwenden wir eine Queue um zu garantieren, dass nicht mehrere Aufgaben gleichzeitig ausgeführt werden, und dass die Reihenfolge eingehalten wird.

```
~~~uml{align=center}
!pragma teoz true
hide footbox
actor Browser as B
box "Server"
 participant "Vaadin\nwith lock" as V
 participant "Queue" as Q
end box
```

```
B -> V : Request
activate V
```

```
V -> Q : execute
activate Q
B <-- V : Response
deactivate V
Q -> Q : lock
activate Q
note over Q : custom logic
Q -> B : Show Dialog
Q -> Q : unlock
deactivate Q
```

```
B -> V : Decision
activate V
& V -> Q : Decision
B <-- V : Response
deactivate V
```

```
Q -> Q : lock
activate Q
B <- Q : Close Dialog
note over Q : custom logic
Q -> Q : unlock
deactivate Q
deactivate Q
```

~~~



Der Vaadin-Listener, der den ursprünglichen Request behandelt, stellt also nur seine Logik in die Queue, beendet sich selbst sofort wieder und gibt damit auch den Lock frei. Die Queue wird in einem Hintergrund-Thread mit Lock auf die VaadinSession abgearbeitet. Während ein Task in der Queue blockierend auf eine Benutzerentscheidung wartet, wird der Lock automatisch freigegeben (eigentlich nicht ganz automatisch, siehe `[_web_completable_future]`). Dadurch ist es möglich, sowohl Änderungen an der UI vorzunehmen (erfordert Lock), als auch blockierend auf Benutzerentscheidungen zu warten (funktioniert nur ohne Lock).

~~~java

```
var bDelete = new Button("Delete", e -> {
 // Gesamte Logik in Queue ausführen
 BlockingTaskQueue.guarded(bDelete).add(() -> handleDelete());
});

void handleDelete() {
 if (yesNo("Wirklich Löschen")) // blockierend, mit boolean-Antwort
 deleteInDb(); // Behandlung der Entscheidung
}
~~~
```

Über diese Strategie ist es also prinzipiell möglich, blockierend auf Benutzerabfragen zu warten. Eine Schwierigkeit jedoch bleibt, nämlich muss die Ausführung einer blockierenden Methode bereits außerhalb dieser Methode an den Hintergrund-Thread verlagert werden. Oft geschieht dies "auf Verdacht", bzw. zur Sicherheit falls die auszuführende Methode blockieren muss. So werden beispielsweise Kommandos immer über die Queue ausgeführt, da hier oft blockierende Dialoge verwendet werden. Dasselbe gilt für die Aktualisierung von Feldern, wobei dort blockierende Dialoge seltener aber doch vorkommen, z.B. bei Validierungen.

Für manche Fälle gibt es hier spezielle Unterstützung. Beispielsweise kann man über die Hilfsmethode `'wrapListener'` einen Listener so wrappen, dass die gesamte Logik in der Queue ausgeführt wird:

~~~java

```
var bDelete = new Button("Delete", BlockingTaskQueue.wrapListener(this, e -> {
    if (yesNo("Wirklich Löschen"))
        deleteInDb();
}));
~~~
```

Das erste Argument ist der sogenannte Lifecycle-Guard, und wird im folgenden Abschnitt beschrieben.

Guards

Wie bereits erwähnt geben wir während dem Blockieren den Lock der VaadinSession ab, um Deadlocks zu verhindern.

Das bedeutet aber auch, dass sich der Zustand während dem Blockieren stark verändern kann. So kann es z.B. sein, dass die aktuelle Komponente bereits vom DOM entfernt wurde und all ihre Ressourcen freigegeben hat, etwa weil der Benutzer den Browser geschlossen hat.

Für diese Fälle unterstützt die BlockingTaskQueue einen "Lifecycle-Guard", welcher die Tasks möglichst davor schützt, in so einem Zustand ausgeführt zu werden. Der Guard selbst ist im Allgemeinen eine Funktion, die true liefert, falls der Task gerade ausgeführt werden kann, ansonsten false.

Es kann aber auch eine "Lifecycle-Owner" Komponente angegeben werden. In diesem Fall wird der Task nur ausgeführt, falls der Owner inzwischen nicht detach wurde.

Schaun wir uns dazu ein vollständiges Beispiel an:

```
~~~java
class AddressDeleteView extends VerticalLayout {

    private BsAdresse addr;

    MyComponent(BsAdresse addr) {
        this.addr = addr;
        add(new Button("Delete", BlockingTaskQueue.wrapListener(this, e -> {
            if (yesNo("Wirklich Löschen"))
                deleteInDb();
        })));
    }

    private void deleteInDb() {
        addr.getCollection.removeNoExc(addr);
    }

    @Override
    protected onDetach(DetachEvent event) {
        if (addr != null) {
            addr.getDatabase().endUse();
            addr = null;
        }
    }
}
~~~
```

Um den Eintrag zu löschen, brauchen wir eine offene Datenbankverbindung. Wenn der Benutzer die View aber verlässt, müssen wir die Verbindung wieder abgeben, sonst bleibt sie ewig in Verwendung. Das kann aber passieren, während wir auf die Antwort des `yesNo` Aufrufs warten, oder theoretisch auch bevor wir die Frage überhaupt stellen.

Hier kommt der Guard ins Spiel. Er sorgt dafür, dass der Task nur ausgeführt wird, falls die `AddressDeleteView` noch attacht ist. Der Guard wird auch nochmal geprüft, wenn `yesNo` zurückkehrt. Falls er false liefert, wird der Task nicht weiter ausgeführt (es wird eine CancellationException geworfen - Methoden wie `yesNo` fangen diese ab und liefern false).

WebCompletableFuture

Das "automatische" Freigeben des Locks wird in der Klasse `WebCompletableFuture` realisiert, unserer eigenen Ableitung von `CompletableFuture`.

Im Web wird also vorzugsweise `WebCompletableFuture` anstelle von `CompletableFuture` verwendet, damit die Dinge wie oben beschrieben funktionieren.

Wenn hingegen auf ein herkömmliches `CompletableFuture` gewartet wird, oder andere blockierende Aufufe gemacht werden, wird der Lock NICHT automatisch abgegeben und es kommt zu Deadlocks.

~~~java

```
var bWorks = new Button("Works", BlockingTaskQueue.wrapListener(this, e -> {
    var cf = new WebCompletableFuture<Boolean>();
    // ... show dialog that completes cf
    if (cf.join()) // releases Lock while blocking
        doSomething();
}));
var bDoesntWork = new Button("Doesn't work", BlockingTaskQueue.wrapListener(this, e -> {
    var cf = new CompletableFuture<Boolean>();
    // ... show dialog that completes cf
    if (cf.join()) // doesn't release Lock while blocking
        doSomething();
}));
```

~~~

Quiz

Überlege, welche der folgenden ClickListener zu Deadlocks führen und welche nicht. Hier wartet `yesNo` blockierend auf eine Antwort vom Benutzer, `yesNoAsync` wartet asynchron, und `doSomething` führt irgend eine Operation durch, die auf

nichts wartet.

~~~java

```
var b1 = new Button("b1", e -> {
    yesNoAsync("...").thenAccept(yes -> {
        if (yes)
            doSomething();
    });
});

var b2 = new Button("b2", e -> {
    yesNoAsync("...").thenAccept(yes -> {
        if (yes && yesNo("Really?"))
            doSomething();
    });
});

var b3 = new Button("b3", BlockingTaskQueue.wrapListener(this, e -> {
    if (yesNo("..."))
        doSomething();
}));

var b4 = new Button("b4", BlockingTaskQueue.wrapListener(this, e -> {
    if (yesNo("...") && yesNo("Really?"))
        doSomething();
}));

var b5 = new Button("b5", BlockingTaskQueue.wrapListener(this, e -> {
    yesNoAsync("...").thenAccept(yes -> {
        if (yes)
            doSomething();
    });
}));

var b6 = new Button("b6", BlockingTaskQueue.wrapListener(this, e -> {
    yesNoAsync("...").thenAccept(yes -> {
        if (yes && yesNo("Really?"))
            doSomething();
    });
}));

var b7 = new Button("b7", e -> {
    yesNoAsync("...").thenAccept(yes -> {
        if (yes) {
            BlockingTaskQueue.guarded(this).add(() -> {
                if (yesNo("Really?"))
                    doSomething();
            });
        }
    });
});
```

```
});  
~~~
```

Hinweis: Hinsichtlich Deadlocks muss man bei Verkettungen von CompletionStages überlegen, wie die vorherige CompletionStage beendet wird. Bei `'yesNoAsync("...").thenAccept(yes -> { ... })'` z.B. wird die CompletionStage vom ClickListener des "OK"-Buttons beendet, also wird auch `'thenAccept'` von diesem Listener ausgeführt.

Lösungen:

1. Kein Deadlock, weil alles asynchron ist.
2. Deadlock, weil `'yesNo'` ohne Queue aufgerufen wird.
3. Kein Deadlock, Standard-Verwendung der Queue.
4. Kein Deadlock, Standard-Verwendung der Queue (in der Queue darf mehrfach blockiert werden).
5. Kein Deadlock, in der Queue darf asynchron gearbeitet werden.
6. Deadlock, weil wir in `'thenAccept'` nicht mehr in der Queue sind.
7. Kein Deadlock, weil wir beim Blockieren in der Queue sind.

// NB: tabsize und source-highlighter:coderay werden von AsciiDocFX nicht unterstützt. Sie sollten daher beim Generieren der Dokumente (PDF, HTML) als Kommandozeilen-Parameter angegeben werden.

```
//:tabsize: 4
```

```
//:source-highlighter: coderay
```

## Vaadin Update

### Neue Version einsetzen

In der `_pom.xml_` Datei des Root-Projektes (`_Ombis_`) muss der Property `_vaadin.version_` die neue Version zugewiesen werden.

Beispiel:

```
~~~xml
```

```
<vaadin.version>23.0.8</vaadin.version>
```

```
~~~
```

Anschließend muss die Datei `_g14-grid-connector.js_` aktualisiert werden. Dabei handelt es sich um eine gepatchte Version von `_vaadin-grid-connector.js_`.

Um sie zu aktualisieren muss zunächst die ungepatchte Datei der jeweiligen Version kopiert werden. Diese findet man auf [<https://github.com/vaadin/flow-components/blob/23.0.8/vaadin-grid-flow-parent/vaadin-grid-flow/src/main/resources/META-INF/resources/frontend/gridConnector.js>](<https://github.com/vaadin/flow-components/blob/23.0.8/vaadin-grid-flow-parent/vaadin-grid-flow/src/main/resources/META-INF/resources/frontend/gridConnector.js>)[Github] (Den Link auf die entsprechende Version ändern).

Anschließend können unsere Änderungen mithilfe des Patches `._/g14-grid-connector.js.patch_` auf die Datei übertragen werden.

Ob weitere Anpassungen notwendig sind hängt von der Vaadin-Version ab und kann meist dem Changelog entnommen werden.

### ### Generierte Dateien neu erzeugen

Nachdem die Vaadin Version aktualisiert wurde, müssen die automatisch generierten Dateien aktualisiert werden. Dazu muss zunächst das Script `_bin/scripts/remove-generated-files.js_` ausgeführt werden:

~~~

```
C:\Path\To\Ombis\bin\scripts>node remove-generated-files.js
```

~~~

Anschließend müssen die Dateien anhand eines Builds aller Applikationen wieder erzeugt werden:

~~~

```
build ALL notFailOnError -DskipWeb -DskipDependencyCheck
```

~~~

Mit der Angabe ``-T4`` kann der Build-Prozess auf 4 Threads (Zahl kann angepasst werden) aufgeteilt werden. Da das Build relativ viel CPU-Leistung verbraucht wird das System dadurch jedoch potentiell eingebremst.

Nach dem Build muss ein ``mvn clean`` ausgeführt werden, damit Vaadin lokal wieder im Developer-Modus startet. In IntelliJ muss anschließend die Run-Configuration ``Vaadin prepareFrontend`` ausgeführt werden.

### ### Jar für Runtime aktualisieren

Auch im Projekt ``OmbisDependencies`` muss die neue Version in `_vaadin-all/pom.xml_` eingetragen werden.

Anschließend kann über das entsprechende ``build`` Kommando das neue `_vaadin-all.jar_` erzeugt werden.

Dieses muss anschließend im Ombis-Projekt unter `_lib_` eingefügt werden.

## ## Checklisten

### ### Allgemein

- [ ] **\*\*Feldänderungen\*\*** sollten vermieden werden, da diese Auswirkungen auf Rest und andere Schnittstellen haben. All unsere Web und App Projekt müssen auf diese Felder überprüft werden, da keine Fehler ausgeworfen werden, wenn das Feld nicht mehr gefunden werden kann. Am besten werden alle Projekte mit der Textsuche auf das betreffende Feld durchsucht.

### ### Was tun wenn Vaadin nicht mehr funktioniert?

1. 'File' / 'Invalidate caches...' / 'Clear file system cache and Local History' und 'Clear downloaded shared indexes'
2. 'Maven' (rechter Bildschirmrand) / 'Execute Maven Goal' (kursives m) / `mvn clean`

3. 'Maven' (rechter Bildschirmrand) / 'Reload All Maven Projects' (Refresh Symbol)
4. 'Build' / 'Rebuild Project'

### ### Commit

- [ ] Es muss einen **\*\*Backlog\*\*** zum Commit geben (Ausnahmen:
  - Weiterdrehen von Versionen,
  - Ausbessern von Tippfehlern und
  - Änderungen die keine Auswirkungen auf das Endprodukt haben,wie Ergänzungen von Kommentaren oder Aktualisierung der Datenbankprototypen (Firm.xml))
- [ ] **\*\*Art, Bereich, Thema und Produkte\*\*** im Backlog kontrollieren
- [ ] **\*\*Lösung\*\*** eintragen (ein für den Supporter verständlichen Text)
- [ ] **\*\*SVN\*\*** aktualisieren
- [ ] evtl. **\*\*Portierungen\*\*** eintragen
- [ ] **\*\*Arbeitszeiten und evtl. Tasks\*\*** im BL überprüfen und abschließen
- [ ] **\*Daily Stunden abziehen**
- [ ] **\*\*Status\*\*** auf ABZUNEHMEN setzen
- [ ] **\*\*In Bearbeitung\*\*** jemanden für die Code Review einsetzen
- [ ] Eventuelle Testdatenbanken aus dem Backlog löschen

### #### Konvention Commit-Kommentar

Formal muss ein SVN Commit wie folgt aussehen, wobei die Backlogbezeichnung optional ist:

- [Bereich] - [Backlognummer] - Zwischen-Commit - [Backlogbezeichnung]
- in einer neuen Zeile darunter soll beschrieben werden was gemacht wurde
  - folgende Informationen können in weiteren neuen Zeilen hinzugefügt werden:
    - BUGFIX
    - Nachtrag R[SVN-Nummer]
    - Code Review R[SVN-Nummer]

### **\*\*Beispiel 1\*\***

AgroQs - 19446

- Bodenarten in Kontrollestelle Stammdaten implementiert und beim Export an SQK berücksichtigt.

### **\*\*Beispiel 2\*\***

Fakt - 19411 - Detailverkauf-Beleg Verteilung von Filiale zu Hauptstelle differenter Preis bei Pfandzeile

- Feld PreisDW als sekundäres Feld gekennzeichnet

### **\*\*Beispiel 3\*\***

Vaadin - 19250 - Zwischen-Commit - Controller für Datenbearbeitung:

- DefaultNavDataProvider: Optimierung bei der Datenabfrage und Absicherung gegen eventuelles (theoretisches) Fehlen eine Limit-Angabe.

- Vaadin-Service wird jetzt Master-abhängig registriert.

#### **\*\*Beispiel 4\*\***

- Vaadin - 19250 - Zwischen-Commit - Controller für Datenbearbeitung:
- Code-Review R79687

#### **### Versionen**

Folgende Punkte sind zu berücksichtigen, wenn neue Jars oder Apks erstellt werden.  
Für genauere Informationen siehe <<"JarsErstellen.adoc">>.

#### **#### Alpha**

**\*\*Wann?\*\*** Wird jeden Tag bis zu 2 Mal automatisiert erstellt, ansonsten wie folgt vorgehen:

IMPORTANT: Werden immer aus dem Trunk erstellt!

- [ ] `buildPD CI\_BUILD` für Alpha Jars (\_Ombis\bin\\_)
- [ ] `buildPDWeb all\_apps ci\_build` für Alpha-Zips (\_Ombis\web\tools\\_)
- [ ] `node androidBuild.js stock ci\_build` für Alpha-APKs von Ombis Stock (\_Ombis\Mobils\tools\\_)
- [ ] `node androidBuild.js stockhelper ci\_build` für Alpha-APKs von Ombis Stockhelper (\_Ombis\Mobils\tools\\_)
- [ ] Kontrolle des Jars/Zips/APKs unter \_SupportSW\01\_Jars\04\_AlphaJars\_

#### **#### Service-Release**

**\*\*Wann?\*\*** Bei Bedarf, darf nur korrigierte Fehler oder Punkte mit Serviceklasse EXPEDITE enthalten.

IMPORTANT: Werden immer aus dem Brunch erstellt!

#### **##### Ombis**

- [ ] Programmierer kurz informieren und alles alles notwendige auf den Branche portieren
- [ ] Kontrolle ob alle betreffenden Punkte eine Portierung eingetragen haben
- [ ] {Jenkins} starten (erhöht Versionsnummer pro Jar auf den nächsten Buchstaben, erstellt die gewünschten Jars, legt diese in die nötigen Ordner ab und versendet die Sprint-Mail, bzw eine Mail an die eingetragenen Adressen, wenn das Build schief läuft).

#### **##### Alternative**

- [ ] Sicherheitshalber eigenen Workspace öffnen um ggf. Konflikte auszuschließen
- [ ] Konstanten im Branch um Buchstabe erweitern bzw. Buchstabe weiterdrehen und committen
- [ ] Branch aktualisieren
- [ ] {Jenkins} starten (`buildPD CI\_BUILD` im Branch ausführen, um das Service-Release-Jar zu erstellen)
- [ ] Kontrolle der automatisch eingetragenen Produktversion in Ombis
- [ ] Kontrolle Mail ob alle betreffende Punkte aufscheinen
- [ ] Kontrolle des Jars unter \_SupportSW\01\_Jars\01\_NewJars\_ auf Versions- und SVN-Nummer



#### ##### Zips

- [ ] Betroffene Backlogs portieren
- [ ] Konstanten im Branch um Buchstabe erweitern bzw. Buchstabe weiterdrehen und committen
- [ ] Kontrolle ob 'releaseThisVersion' bei den gewünschten Webportalen bereits auf true ist. (Sonst im Branch und Trunk setzen und committen.)
- [ ] {Jenkins} starten (Zips werden mit 'buildPDWeb ci\_build projekt1 projekt2 ...' bzw. wenn für alle Zips mit 'buildPDWeb ci\_build all\_apps' im Release-Branch unter \_web\tools\_ erstellt. Die Projekt-Namen erhält man mit 'buildPDWeb help').
- [ ] Kontrolle der automatisch eingetragenen Produktversion in Ombis
- [ ] Kontrolle Mail ob alle betreffende Punkte aufscheinen
- [ ] Kontrolle des Zips unter \_SupportSW\01\_Jars\01\_NewJars\_

#### ##### APKs

- [ ] Betroffene Backlogs portieren
- [ ] Konstanten im Branch um Buchstabe erweitern bzw. Buchstabe weiterdrehen und committen
- [ ] Kontrolle ob 'releaseThisVersion' bei den gewünschten APKs bereits auf true ist. (Sonst im Branch und Trunk setzen und committen.)
- [ ] {Jenkins} starten (APKs werden mit 'node androidBuild.js stock/stockhelper ci\_build' im Release-Branch unter \_/tools/\_ erstellt.).
- [ ] Kontrolle der automatisch eingetragenen Produktversion in Ombis
- [ ] Kontrolle Mail ob alle betreffende Punkte aufscheinen
- [ ] Kontrolle des APKs unter \_SupportSW\01\_Jars\01\_NewJars\_

#### #### RC

**\*\*Wann?\*\*** am Ende jeden Sprints am letzten Dienstag.

- [ ] Kontrolle ob es Punkte mit Status TEST gibt, diese abklären, ansonsten darf kein RC erstellt werden
- [ ] Programmierer kurz informieren
- [ ] {Jenkins} starten

#### ##### Alternative

- [ ] SVN UPDATE
- [ ] @ in Ombis, Ombis/web und OmbisMobile mit RC ersetzen und Service-Release Branches erstellen:
  - [ ] Ombis: @ in pom.xml bei Konstanten mit RC ersetzen (auch Hotel und Badminton)
  - [ ] Ombis/web: @ in productInfo.json Dateien der Projekte auf RC drehen (Ersetze \_"revision": "@\_ mit \_"revision": "RC\_)
  - [ ] OmbisMobile: @ in build.gradle Dateien auf RC drehen (Ersetze \_def apkRevision = "@\_ mit \_def apkRevision = "RC\_)

Am Ende werden zwei gleichnamige Releasebranches für Ombis und OmbisMobile erstellt. Im RepoBrowser von Tortoise kann man sich die Situation auch gut ansehen. Jeweils unter Ombis und OmbisMobile gibt es einen Ordner \_branches\_ mit den bisherigen Releasebranches.

TIP: Erstellung des Branch ohne Commit im Trunk:

1. Rechtsklick auf SVN-Ordner Ombis oder OmbisMobile (trunk) wo man die @s mit

RC ersetzt hat

2. Subversion Aktion: "Branch/tag..."
3. To path: ist dann der Pfad zum neuen Branch
4. Log Message einfach Ombis/Hotel XXYRC
5. in der Auswahl Create copy from die Option Working Copy auswählen

- [ ] im TaskTracking den neuen Release-Branch unter Entwicklungszweige eintragen und beim Entwicklungszweig den Sprint eintragen. Muss nur einmal eingetragen werden, weil die ReleaseBranches von Ombis und OmbisMobile den selben Namen haben.
- [ ] Auf {Jenkins} \*\*ReleaseBuilds\*\* starten (aus dem Branch heraus mit buildPD Jars erstellen. buildPD checkt selbstständig was für eine Version erstellt wird.)
- Im Master-Branch (Trunk) Konstanten anpassen:
  - [ ] Ombis im pom.xml Konstanten erhöhen und auf @ setzen
  - [ ] Ombis/web in den productInfo.json Dateien -> "\_ombisVersion\_" anpassen, "\_revision\_" auf @ setzen und "\_releaseThisVersion\_" auf 'false'
  - [ ] OmbisMobile in build.gradle Dateien -> "\_OmbisMajor\_" und "\_OmbisMinor\_" anpassen, "\_apkRevision\_" auf @ setzen, "\_versionId\_" anpassen und "\_releaseThisVersion\_" auf 'false'
- [ ] Konstanten für Masterdatenbank weiterdrehen (über IntelliJ mit DevelopDatabaseVersionsUpdater)
- [ ] Kontrolle des Jars unter \_SupportSW\01\_Jars\01\_RcJars\_ auf Versions- und SVN-Nummer

Nach erfolgreichem(!!!) Erstellen der Jars:

- [ ] Auf {Jenkins} \*\*OmbisWebReleaseBuilds\*\* starten (werden RC-Zips mit 'buildPDWeb ci\_build all\_apps' im Release-Branch unter \_web\tools\_ erstellt werden.)
- [ ] Auf {Jenkins} \*\*OmbisMobileReleaseBuilds\*\* starten (muss auch überprüft werden ob neue APKs zur Verfügung gestellt werden müssen.)

#### Final

**\*\*Wann?\*\*** nach Sprintende am Freitagnachmittag.

- [ ] Programmierer kurz informieren
- [ ] Kontrolle ob alle BLs, welche bereits im neuen Sprint erledigt wurden und für die Finale Version portiert wurden. Ansonsten erscheint der BL nicht in der Mail. Mail wird automatisch nach erfolgreichem Build versandt.
- [ ] {Jenkins} starten

##### Alternative

- [ ] in den beiden Releasebranches:
  - [ ] Ombis: RC aus pom.xml entfernen (außer bei bis auf Hotel und Badminton)
  - [ ] Ombis/web: RC aus productInfo.json Dateien entfernen
  - [ ] OmbisMobile: RC aus build.gradle Dateien entfernen
- [ ] {Jenkins} \*\*ReleaseBuilds\*\* starten (buildPD Final um die Jars zu erstellen)
- [ ] Kontrolle der automatisch eingetragenen Produktversion in Ombis
- [ ] Kontrolle Mail ob alle betreffende Punkte aufscheinen
- [ ] Kontrolle der Jars unter \_SupportSW\01\_Jars\01\_NewJars\_ auf Versions- und SVN-Nummer
- [ ] Alte ServiceRelease-Jars und Final-Jars vom Ordner \_SupportSW\01\_Jars\01\_NewJars\_ in \_SupportSW\01\_Jars\05\_OldJars\_ verschieben. (Im Fakt-Ordner trifft das nicht zu, Help-Desk macht das selbst)
- [ ] Eventuell, falls nicht automatisch erfolgt: Mail Einzeldruck aus Sprint mit

Version **\*\*OMBIS\*\*** und Versionsnummer.

Nach erfolgreichem(!!!) Erstellen der Jars:

- [ ] {Jenkins} **\*\*OmbisWebReleaseBuilds\*\*** starten (können die Final-Zips mit ``buildPDWeb ci_build all_apps`` im Release-Branch unter `_web\tools_` erstellt werden.)
- [ ] {Jenkins} **\*\*OmbisMobileReleaseBuilds\*\*** starten (Neue APK(s) erstellen, falls es Android-seitig Änderungen gibt, welche noch nicht in einer Version enthalten sind (siehe hierzu `<<"AndroidProjekt.adoc">>`)

#### Mail händisch versenden (nur in Ausnahmefällen)

Die Sprint-Mail wird in mit buildPD Final automatisch versendet, sollte sie ausnahmsweise händisch versendet werden müssen, muss dies aus dem Sprint heraus gemacht werden, in welchem der BL portiert wurde. Der Titel im Druckdialog muss angepasst werden und wenn es sich um ein Servicerelease handelt, muss die SVN der letzten Version eingetragen werden, sowie der Release Branche.

## Copyright Update

### Alle Jahre wieder

Der einfachste Weg mit dem Weiterdrehen des Copyright-Jahrs anzufangen ist, in den Commits des Vorjahres zur gleichen Zeit nach derselben Routine zu suchen.

|                    |       |
|--------------------|-------|
| Relevante Commits  |       |
| Copyright bis 2017 | 62777 |

#### Genereller Ablauf

1. In der IDE in den Java-Quellen mit einem Filesearch nach folgenden Zeichenfolgen suchen.

```
`-YYYY Pro Data`
`-YYYY ASA`
```

Die Ergebnisse kurz durchschauen, verifizieren ob es Res-Bundles sind und ersetzen. Weiters soll noch die Anzahl überprüft werden. Eventuell neue Files berücksichtigen.

2. Die Startbildschirme anpassen! Diese befinden sich unter ``/images/splashlogos``

Die konkret durchzuführenden Operationen sind in der dort vorhandenen ``readme.txt`` Datei beschrieben.

3. Im Projekt für den Laucher (=Exe-Datei) muss in der entsprechender `.rc` Datei ebenfalls das Copyright-Bis-Jahr auf den aktuellen Wert gesetzt werden. Anschließend müssen natürlich die Exe-Dateien neu erstellt werden.